

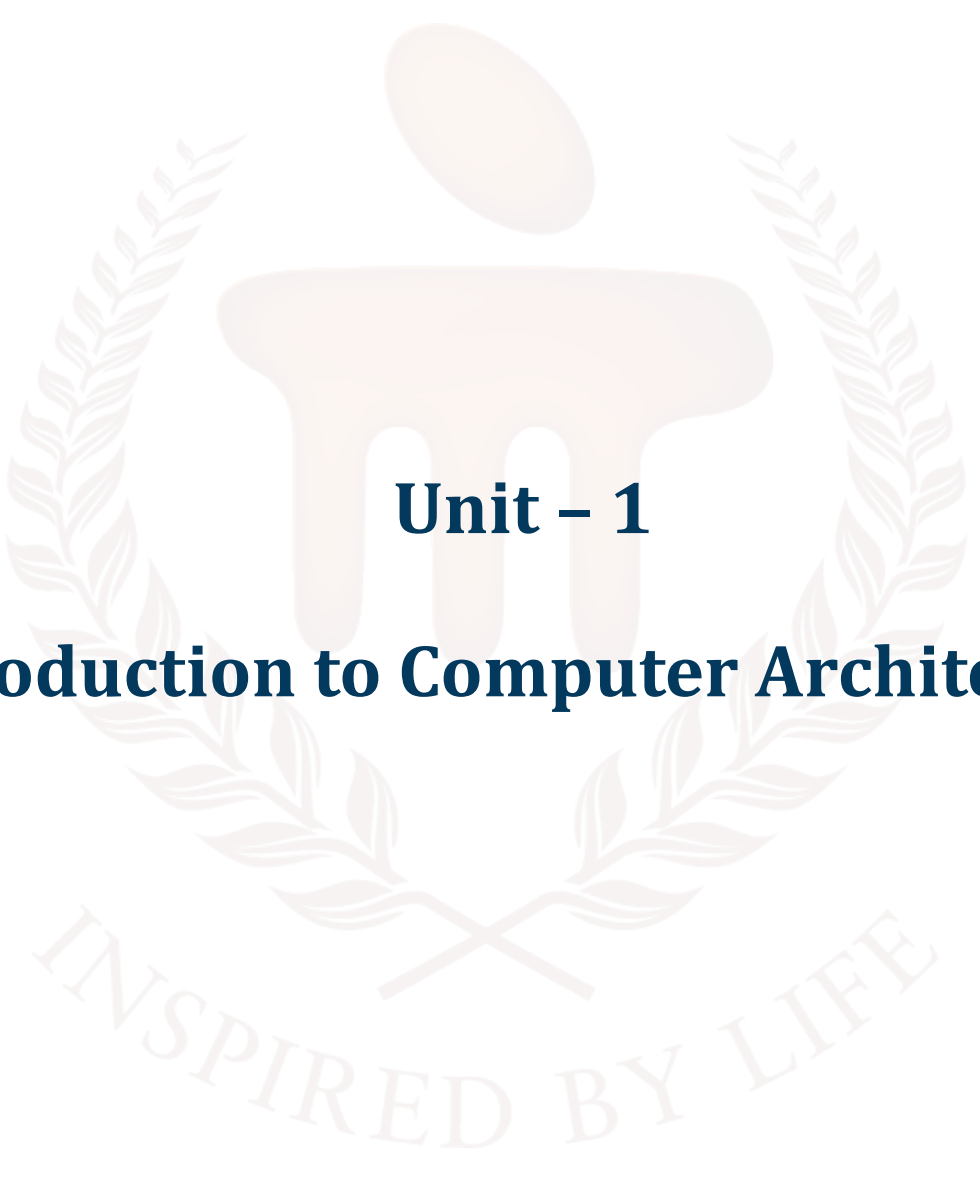
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 1

Introduction to Computer Architecture

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Computer Architecture	-	-	6 - 12
2.1	Types of Computer Architecture	1, 2	1	
3	Block Diagram of a Computer	3, 4	2	13 - 19
4	Computer Memory Section	5	-	20 - 23
4.1	Computer Memory Works	-	3	
5	Input/Output Section	-	-	24 - 28
5.1	Components of the Input/Output Section	6	-	
5.2	Functions of the Input/Output Section	-	-	
5.3	Types of Input/Output Operations	-	-	
5.4	How the I/O Section Works	-	-	
5.5	Examples of I/O in Action	-	4	
6	CPU	-	-	29 - 33
6.1	History of CPU	7	-	
6.2	Components of the CPU	-	-	
6.3	Functions of the CPU	8	-	
6.4	Types of CPUs	-	5	
7	Summary	-	-	34
8	Glossary	-	-	35 - 36
9	Terminal Questions	-	-	37
10	Answers	-	-	38 - 39
11	References	-	-	40

1. INTRODUCTION

Computer architecture is a fundamental area of study that explores computer systems' design, structure, and operation.

This unit covers the essential components that constitute a computer, focusing on the block diagram of a computer system, the memory section, the input/output (I/O) section, and the Central Processing Unit (CPU). A thorough understanding of these core elements is fundamental for computer science, information technology, or electronics studies, as they form the basis for hardware and software systems.

The unit introduces the block diagram of a computer, which visually represents the central functional units and their interconnections within the system. This diagram typically includes the input unit, memory section, CPU, and output unit. Each of these components serves a distinct and critical role in the functioning of the computer. The input unit is the primary interface between the user and the computer, allowing data entry and instructions through devices such as keyboards, mice, or scanners. This input is then translated into a machine-readable format, typically binary code.

Further, the unit explains how the output unit presents the processed data in a human-readable format using devices such as monitors and printers. Understanding the roles and interactions of these components provides a comprehensive perspective on computer operation. This foundational knowledge supports the study of more advanced topics in computer engineering and technology, establishing a solid base for continued academic and professional development.

1.1. Objectives

After studying this unit, you should be able to:

- Define the basic components of computer architecture.
- Discuss the organisation of the different parts of the computer.
- Explain the roles of the Arithmetic Logic Unit (ALU) and Control Unit.
- Analyse the interaction between memory hierarchy and CPU performance.
- Illustrate the data input, processing, storage, and output process.
- Compare different types of memory and input/output devices.



2. COMPUTER ARCHITECTURE

Computer architecture is the conceptual design and fundamental operational structure of a computer system. It specifies how a computer's hardware and software components interact to process data and execute instructions, forming the essential framework for all types of computing devices, from smartphones and laptops to servers and supercomputers.

The central processor unit (CPU), memory units (RAM and storage), input/output (I/O) devices, and the communication channels (buses and networks) that connect these components are the main components that make up computer architecture.

There are three primary aspects of computer architecture:

1. **System Design:** This covers the physical hardware components, including the CPU, memory controllers, data paths, and peripheral devices. It also involves the organisation of these components and how they communicate with each other, affecting the overall performance and capabilities of the system.
2. **Instruction Set Architecture (ISA):** The ISA is the set of basic commands and data types that the CPU can understand and execute. It acts as the interface between the hardware and the software, allowing programmers to write code that the machine can process. Different processors may have different ISAs, such as x86 or ARM.
3. **Microarchitecture:** This refers to the detailed implementation of the ISA within a specific processor or family of processors. It defines how instructions are executed, how data flows within the CPU, and how resources like registers and caches are managed.

Computer architecture is crucial because it directly impacts a system's performance, efficiency, cost, and scalability. A well-designed architecture enables reliable, secure, and high-speed computing, which is essential for modern applications such as artificial intelligence, cloud computing, gaming, and scientific simulations. It also influences energy consumption, heat generation, and the ability to upgrade or expand the system in the future.

Classic examples of computer architecture include the Von Neumann and Harvard architectures. The Von Neumann architecture uses a single memory space for both instructions and data, which simplifies design but can lead to bottlenecks. The Harvard architecture, on the other hand, separates memory for instructions and data, allowing for faster and more efficient processing in certain applications.

2.1. Types of Computer Architecture

The definition of computer architecture is not very juicy and hence doesn't contain deepness. It is the type of computer architecture that makes the topic interesting and deep.

In this section, we will see different types of computer architecture that will help you in both academic and professional life.

Type 1: Von-Neumann Architecture

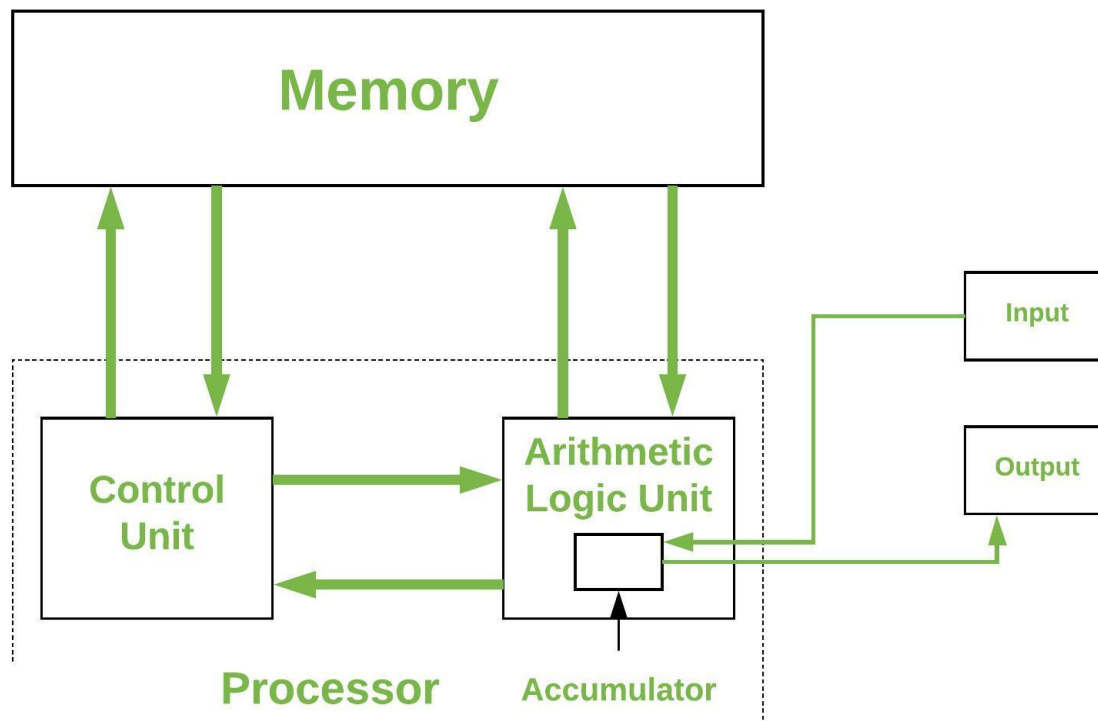


Figure 1.1: Von-Neumann Architecture

Figure 1.1 illustrates the basic structure of the Von Neumann architecture, which is a foundational model for most modern computers. This architecture was proposed by mathematician and physicist John von Neumann in the 1940s. Let's break down the main components shown in the diagram:

1. Memory

- **Role:** Stores both data and program instructions in the same memory unit.
- **Connection:** Memory is connected to both the Control Unit and the Arithmetic Logic Unit (ALU), allowing both to fetch or store data and instructions as needed.

2. Processor

The processor in this diagram is divided into two main parts:

- **Control Unit**
 - **Function:** controls how the processor operates. After retrieving and decoding instructions from memory, it instructs the ALU on what actions to take.
 - **Connection:** Communicates with both memory and the ALU.
- **Arithmetic Logic Unit (ALU)**
 - **Function:** Performs all arithmetic and logical operations (such as addition, subtraction, AND, OR).
 - **Accumulator:** A special register within the ALU that temporarily holds data during calculations.
 - **Connection:** Interacts with memory, the control unit, and input/output units.

3. Input/Output (I/O)

- **Input:** Devices or mechanisms through which data and instructions enter the computer system (e.g., keyboard, mouse).
- **Output:** Devices through which processed data is sent out of the system (e.g., monitor, printer).
- **Connection:** The ALU communicates with both input and output units to receive data for processing and to send results out.

Working Principle

- Both instructions and data are stored in the same memory.
- The control unit fetches instructions from memory, decodes them, and coordinates with the ALU to execute operations.
- The ALU performs calculations or logical operations, often using the accumulator for temporary data storage.
- Input devices supply data, and output devices display or transmit results.

Type 2: Harvard Architecture

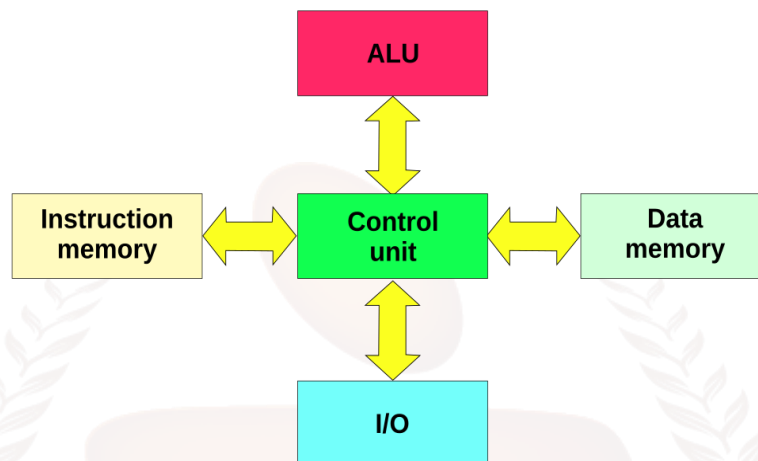


Figure 1.2: Harvard Architecture

Figure 1.2 represents a basic computer architecture model, showing the main components and how they interact. The structure is similar to the Harvard architecture, as it separates instruction memory and data memory.

Components Mentioned in the Figure

1. Control Unit (Centre, Green Box)

- Role: Acts as the brain of the computer. It manages and coordinates all operations by directing the flow of data between the different components.
- In Figure 1.2, located at the centre, connecting to all other blocks.

2. ALU (Arithmetic Logic Unit) (Top, Red Box)

- Role: Performs all arithmetic (addition, subtraction, etc.) and logical (AND, OR, NOT, etc.) operations.
- In Figure 1.2, the control unit is connected above, indicating that the control unit sends data and instructions to the ALU for processing.

3. Instruction Memory (Left, Yellow Box)

- Role: Stores the program instructions that the computer needs to execute.
- In Figure 1.2: Shown to the left of the control unit. The control unit fetches instructions from here.

4. Data Memory (Right, Light Green Box)

- Role: Stores the data that is being used and manipulated by the program.
- In Figure 1.2: Shown to the right of the control unit. The control unit reads from and writes to this memory during program execution.

5. I/O (Input/Output) (Bottom, Blue Box)

- Role: Handles communication with external devices, such as keyboards, displays, printers, and storage devices.
- In Figure 1.2, Data flow to and from input/output devices is managed by the control 3 unit, which is situated above it.

How the Components Work Together

- The control unit fetches instructions from instruction memory.
- It decodes these instructions and, if necessary, retrieves or stores data from/to data memory.
- For operations like calculations, the control unit sends data to the ALU, which processes it and returns the result.
- The I/O block allows the system to receive input from or send output to external devices, all coordinated by the control unit.

Type 3: Modified Harvard Architecture

Imagine your brain could read a book and write an essay at the exact same time—no waiting, no switching tasks! That's the magic behind the Modified Harvard Architecture in computers.

Main Features

- **Double Lanes for Data and Instructions:**

Just like a highway with separate lanes for cars and trucks, this architecture gives instructions and data their own dedicated paths. The CPU can fetch instructions and access data at the same time, making everything run faster and smoother.

- **Flexible Memory Use:**

Unlike the strict Harvard model, where instructions and data live in totally separate houses, the modified version lets them share space when needed. This means the CPU can treat instruction memory as data memory if it wants—super handy for things like lookup tables and constants!

- **Split Caches for Speed:**

Modern processors often use separate “caches” (tiny, super-fast memory areas) for instructions and data. This keeps the CPU from getting bogged down, letting it zip through tasks like a race car.

Working Principle

Here’s how it all comes together:

1. **Simultaneous Fetching:**

The CPU can grab an instruction to execute and fetch or write data at the same time—no waiting in line! This parallel action is a big reason why modern chips are so speedy.

2. **Instruction Memory as Data:**

Need to use a constant or a pre-written table? No problem! The CPU can reach into instruction memory and treat it like data, saving space and time.

3. **Unified Main Memory:**

While the CPU uses separate caches for instructions and data, the main memory can be shared. This means the system gets the best of both worlds: speed from the Harvard approach and flexibility from the Von Neumann style.

4. **Special Moves:**

Some CPUs even have special instructions to make accessing data in instruction memory extra efficient—like having a secret shortcut in your favorite video game.

Type 4: RISC (Reduced Instruction Set Computing)

Think of RISC like doing things step-by-step in the simplest way possible. This helps computers work faster and use less energy.

Main Features

- **Simple Instructions:**

RISC uses a small number of easy instructions. Each instruction does one simple job, like adding two numbers or moving data.

- **Same Size Instructions:**

All instructions are the same length. This makes it easier and faster for the computer to read and understand them.

- **Lots of Registers:**

Registers are small, fast storage spots inside the CPU. RISC has many of these, so it can keep important data close by without going to slower memory.

- **Load and Store:**

Only special instructions read from or write to memory. All other operations happen inside the CPU using registers.

- **Works Well with Pipelining:**

The CPU can work on many instructions at once by breaking the process into steps, like an assembly line.

How It Works

1. **Break Big Tasks Into Small Steps:**

Instead of doing complicated things in one go, RISC breaks them into simple instructions.

2. **One Instruction at a Time:**

Each instruction finishes in one clock cycle, making the CPU very fast.

3. **Use Registers Often:**

The CPU keeps data in registers to avoid slow memory access.

4. **Process Many Instructions at Once:**

Thanks to pipelining, the CPU can start working on a new instruction before finishing the last one.

5. **Easy for Compilers:**

Because instructions are simple, programs can be easily turned into fast machine code.

SELF-ASSESSMENT QUESTIONS - 1

True or False:	
1	In the Von Neumann architecture, both data and program instructions are stored in the same memory unit.
2	The Harvard architecture uses a single memory space for both instructions and data.
3	RISC (Reduced Instruction Set Computing) uses complex instructions that perform multiple operations in a single step.
Fill in the Blanks:	
4	In the Modified Harvard Architecture, the CPU can fetch an instruction and access _____ at the same time, improving performance.

3. BLOCK DIAGRAM OF COMPUTER

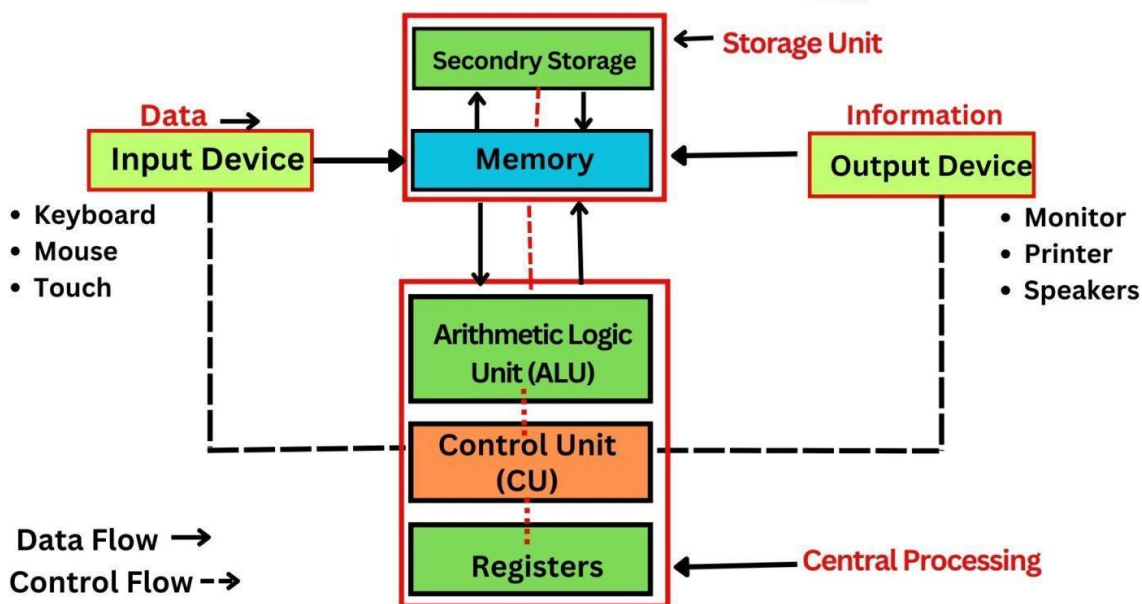


Figure 1.3: Block Diagram of a Computer

Figure 1.3 provided is a comprehensive block diagram of a computer system, visually representing the core components and their intricate interactions. This diagram serves as a foundational tool for understanding how a computer processes data, executes instructions, and produces meaningful output for the user. It breaks down the system into distinct units—input, processing, storage, and output—while illustrating the flow of data and control signals between them.

A. Computer System Architecture

A computer system is an intricate configuration of physical parts that work together to carry out computations. By dividing the system into functional units—input devices, the central processing unit (CPU), memory, secondary storage, and output devices—the block diagram reduces this complexity. Arrows that show the direction of data flow (solid lines) and control flow (dashed lines) are used to illustrate the roles and interactions of each unit. While the other parts manage data input, storage, and output generation, the central processing unit (CPU), sometimes known as the computer's heart, coordinates the entire operation by carrying out commands. This diagram shows a high-level overview of how various components cooperate to convert unprocessed input into meaningful information.

B. Input Devices: The Gateway for User Interaction

Description:

The main way that people communicate with computers is through input devices, which allow them to send commands, data, or processing instructions. Since they act as the input point for all user-generated data, these devices are necessary to start any computing task.

Examples (as shown in Figure 1.3):

- **Keyboard:** A widely used input device that allows users to type text, numbers, symbols, and commands. It is essential for tasks like writing documents, entering data, or issuing software commands.
- **Mouse:** A pointing device that enables users to navigate graphical user interfaces by moving a cursor, clicking on icons, or selecting options. It is particularly useful in modern operating systems with visual interfaces.
- **Touch:** This term refers to touch-sensitive devices such as touchscreens or touchpads, which allow users to interact directly with the screen by tapping, swiping, or pinching. These devices are common in smartphones, tablets, and some laptops.

Role in the System:

Input devices convert user actions—such as typing, clicking, or touching—into digital signals that the computer can interpret. These signals, represented as data, are sent to the memory or directly to the CPU for further processing. Figure 1.3 shows an arrow labelled "Data" flowing from the input devices to the memory, indicating the direction of this data transfer. Without input devices, the computer would have no means of receiving user instructions, making them a critical component of the system.

C. Central Processing Unit (CPU): The Brain of the Computer

The Central Processing Unit (CPU) is the core component responsible for executing instructions and processing data within the computer system. Often described as the "brain" of the computer, the CPU handles all the computational work required to run programs and perform tasks. In Figure 1.3, the CPU is subdivided into three key subcomponents: the Arithmetic Logic Unit (ALU), the Control Unit (CU), and Registers, each with a distinct role in the processing workflow.

C.1 Arithmetic Logic Unit (ALU): The Computational Core

Description:

One essential component of the CPU that handles all logical and mathematical operations needed for program execution is the Arithmetic Logic Unit (ALU). It is the part that manages the decision-making and "number crunching" procedures.

Functions:

- **Arithmetic Operations:** Basic mathematical operations including addition, subtraction, multiplication, and division are all carried out by the ALU. For instance, it can multiply values during a calculation or determine the total of two numbers.
- **Logical Operations:** The ALU also handles logical operations, including comparisons (e.g., determining if one number is greater than another) and bitwise operations (e.g., AND, OR, NOT). These operations are essential for decision-making in programs, such as evaluating conditions in an "if" statement.

Role in the System:

After processing data from memory or registers in accordance with the given instructions, the ALU returns the processed data to memory or registers for storage. From basic calculations to intricate algorithms, it plays a crucial role in carrying out the computational activities that underpin all software processes.

C.2 Control Unit (CU): The Orchestrator of Operations

Description:

The Control Unit (CU) acts as the coordinator within the CPU, managing the flow of data and instructions to ensure that all components work together seamlessly. It does not perform calculations but instead directs the operations of the CPU and other parts of the system.

Functions:

- **Instruction Fetching:** The CU retrieves instructions from memory, where they are stored as part of a program.
- **Instruction Decoding:** It deciphers these instructions to ascertain what has to be done, such as transferring data or carrying out a calculation.

- **Execution Coordination:** To carry out the decoded instructions, the CU transmits control signals to the ALU, registers, memory, and other parts. For instance, it may tell the memory to save a result or tell the ALU to add two numbers.

Role in the System:

By controlling the flow of information and commands, the CU makes sure the computer runs efficiently. Dashed arrows labelled "Control Flow," which depict the CU sending signals to the ALU, registers, and memory, are used to represent this in Figure 1.3. The efficient operation of programs and the general operation of the computer depend on this synchronisation.

C.3 Registers: High-Speed Temporary Storage

Description:

Registers are small, high-speed storage units located within the CPU, designed to hold temporary data during processing. They are the fastest form of storage in the computer, much faster than memory or secondary storage, due to their proximity to the CPU.

Functions:

- **Temporary Storage:** Data, instructions, or intermediate results that the CPU needs to access rapidly while processing are stored in registers. For instance, they could store the address of the subsequent instruction to be retrieved or the operands for an ALU operation.
- **Examples of Registers:** The Program Counter (PC) keeps track of the next instruction to fetch, the Instruction Register (IR) holds the current instruction being performed, and the Accumulator stores the results of calculations that are still in progress.

Role in the System:

Registers provide the CPU with immediate access to critical data, reducing the time it takes to fetch information from slower memory. They act as a buffer between the ALU and memory, ensuring that the CPU can operate at maximum speed. The diagram shows registers as part of the CPU, interacting closely with the ALU and CU during processing.

D. Memory: The Primary Storage for Active Data

Description:

While the computer is operating, data and instructions are temporarily stored in memory, often known as primary storage or RAM (Random Access Memory). It is volatile, which means that when

the computer is turned off, its contents are gone.

Functions:

- **Data Storage:** The data that is received from input devices is stored in memory so that the CPU can process it.
- **Instruction Storage:** It stores the instructions for the programs that the CPU is running at the moment so that the CU can retrieve them when necessary.
- **Result Storage:** The outcomes of data processing by the CPU are frequently written back to memory before being transmitted to secondary storage or output devices.

Role in the System:

Between the CPU and secondary storage, memory acts as a quick-access bridge. It guarantees that the CPU can quickly obtain the information and commands it needs to do jobs effectively. Data goes both ways, as seen by the bidirectional arrows between memory and the CPU in Figure 1.3. The CPU reads data from memory and writes processed results back to it. Secondary storage and memory work together to retrieve data as needed for operation.

E. Secondary Storage: The Long-Term Data Repository

Description:

Non-volatile storage devices that keep data even while the computer is off are referred to as secondary storage. Secondary storage is intended for long-term data retention, in contrast to memory, which is only used temporarily.

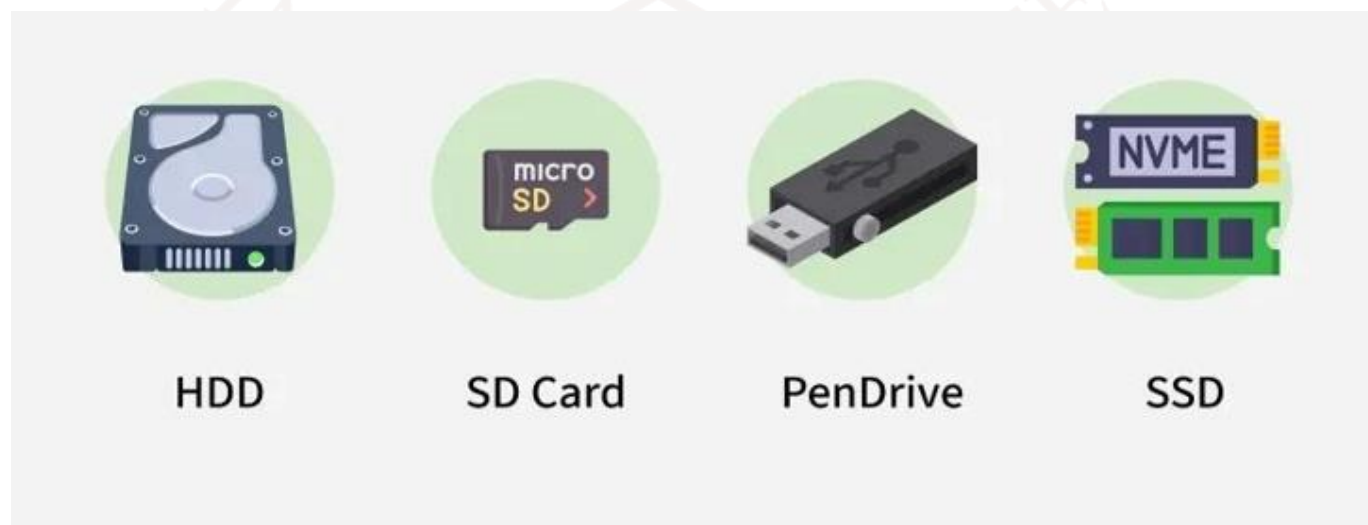


Figure 1.4: Secondary Storage

Examples:

Figure 1.4 Shows Secondary Storage in which various things comes like:

- **Hard Disk Drives (HDD):** Conventional storage devices that store vast volumes of data, including user files, operating systems, and programs, on spinning discs.
- **Solid-State Drives (SSD):** Flash memory-based solid-state drives (SSDs) are faster substitutes for HDDs that provide more durability and faster data access.
- **USB Drives:** USB drives are small, portable storage devices that allow data to be stored and moved between computers.

Functions:

- **Permanent Storage:** Secondary storage holds data that needs to be preserved, such as software programs, documents, photos, and videos.
- **Data Transfer:** It transfers data to memory when the CPU needs to process it. For example, when a user opens a document, the file is copied from secondary storage to memory for faster access.

Role in the System:

Secondary storage provides a large-capacity, permanent storage solution for the computer. While it is slower than memory, it can store vastly more data, making it essential for keeping files and programs accessible over time. The Figure 1.4 shows secondary storage connected to memory, with an arrow indicating that data flows from secondary storage to memory before being processed by the CPU.

F. Output Devices: Presenting Results to the User

Description:

Output devices are responsible for presenting the processed data to the user in a human-readable or perceivable form. They translate the digital results of the CPU's computations into formats that users can understand, such as visuals, text, or sound.

Examples:

- **Monitor:** The primary output device for visual information, displaying text, images, videos, and graphical interfaces on a screen. It allows users to see the results of their actions, such as a document being edited or a webpage being browsed.

- **Printer:** A device that produces physical copies of digital data, such as documents, photos, or reports, on paper. It is useful for creating tangible outputs that users can share or archive.
- **Speakers:** Output devices that produce sound, such as music, voice recordings, or system alerts. They are essential for multimedia applications and user notifications.

Role in the System:

Output devices complete the computing cycle by delivering the results of the CPU's processing to the user. They receive data from the CPU (via memory) and convert it into a format suitable for human perception, as indicated by the arrow labelled "Information" in the Figure. Without output devices, users would have no way to interact with or benefit from the computer's computations.

SELF-ASSESSMENT QUESTIONS - 2

True or False:	
5	The Arithmetic Logic Unit (ALU) is responsible for performing both arithmetic and logical operations within the CPU.
6	Secondary storage devices, such as hard drives and SSDs, lose all stored data when the computer is powered off.
7	Output devices, like monitors and printers, are used to present processed information to the user in a human-readable form.
Fill in the Blanks:	
8	The _____ is often called the "brain" of the computer because it executes instructions and processes data.

4. COMPUTER MEMORY SECTION

Computer memory is just like the human brain. It is used to store data/information and instructions. It is a data storage unit or a data storage device where data is to be processed and instructions required for processing are stored. It can store both the input and output.

- It's faster than secondary memory (e.g., hard drives).
- It is usually volatile, meaning it loses data when power is turned off.
- A computer needs to run; a computer can't operate without primary memory.

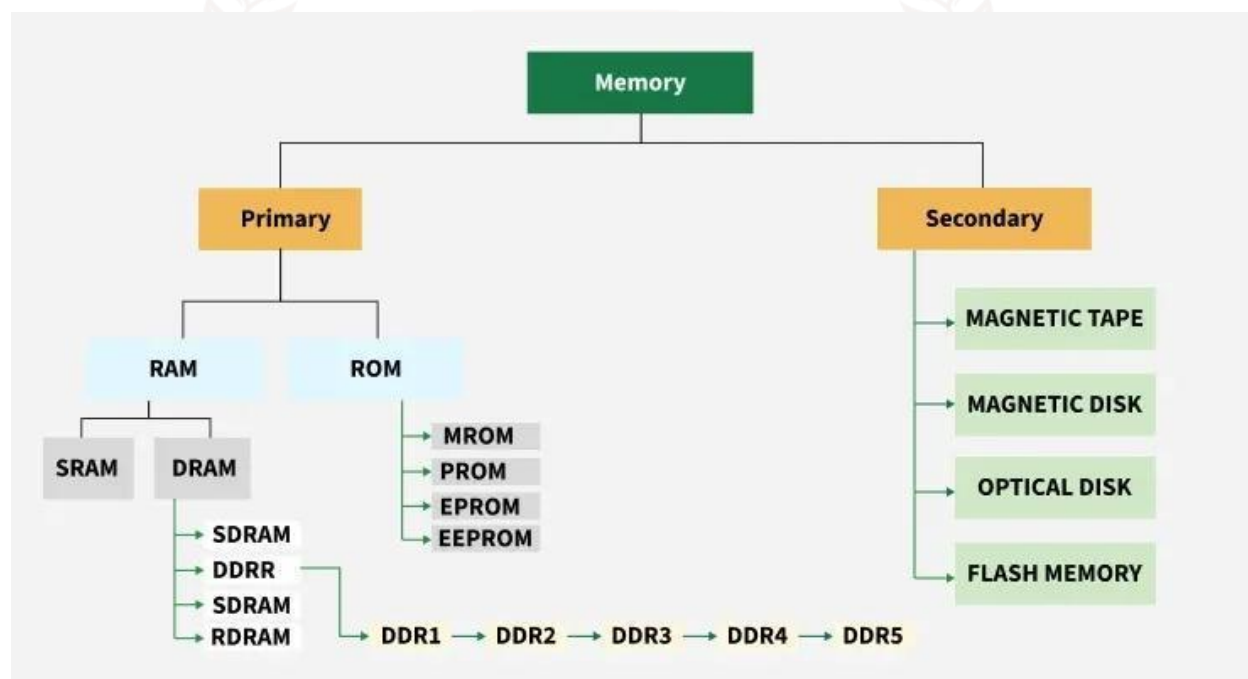


Figure 1.5: Computer Memory Section

Figure 1.5 is a flowchart categorising types of computer memory. At the top, it labels "Memory" as the main category, which splits into two primary branches: Primary and Secondary.

- **Primary Memory:**
 - **RAM (Random Access Memory):**
 - **SRAM (Static RAM):** Faster and more expensive, used in cache memory.
 - **DRAM (Dynamic RAM):** Slower and cheaper, needs constant refreshing, used in main memory.
 - **SDRAM (Synchronous DRAM):** Synchronised with the system clock for faster performance.

- **DDR (Double Data Rate):** Transfers data on both rising and falling clock edges.
- **DDR1, DDR2, DDR3, DDR4, DDR5:** Successive generations of DDR, each improving speed and efficiency.
- **RDRAM (Rambus DRAM):** A high-speed memory type, less common today.
- **ROM (Read-Only Memory):**
 - **MROM (Masked ROM):** Pre-programmed during manufacturing, non-editable.
 - **PROM (Programmable ROM):** Can be programmed once after manufacturing.
 - **EPROM (Erasable Programmable ROM):** Can be erased using UV light and reprogrammed.
 - **EEPROM (Electrically Erasable Programmable ROM):** Can be erased and reprogrammed electrically.
- **Secondary Memory:**
 - **Magnetic Tape:** Sequential access storage, used for backups.
 - **Magnetic Disk:** Includes hard drives, stores data magnetically.
 - **Optical Disk:** Includes CDs/DVDs, uses laser technology for reading/writing.
 - **Flash Memory:** Non-volatile, used in USB drives and SSDs.

The diagram visually organises these memory types, showing their hierarchical relationships and classifications.

4.1. Computer Memory Works

Any computing system needs computer memory since it allows for data processing, retrieval, and storage. It serves as a location to temporarily or permanently store data and instructions that the computer's processor needs in order to perform operations. Primary memory and secondary memory are the two main categories of memory, each of which has a specific function in a computer's operation.

4.1.1. Primary Memory: The Working Space

The central processing unit (CPU) of the computer has direct access to primary memory, often known as main memory. It is built for speed to meet the demands of the CPU and is volatile, meaning that when the system is turned off, its data is lost. Random Access Memory (RAM) and Read-Only Memory (ROM) are the two fundamental memory types.

The CPU's primary workspace is RAM. When a program is running, it stores information and commands that the processor needs to access fast. RAM is further separated into two categories: dynamic RAM (DRAM) and static RAM (SRAM). Because it doesn't need to be refreshed frequently, SRAM is speedier and more dependable; nonetheless, it is costly and usually utilised in limited amounts, like cache memory.

Although DRAM is the primary system memory and is less expensive, it must be updated often to preserve data integrity. There are subtypes of DRAM, such as DDR (Double Data Rate) memory, which have improved in speed, bandwidth, and energy efficiency across the generations DDR1, DDR2, DDR3, DDR4, and DDR5. SDRAM (Synchronous DRAM) also syncs with the system clock for quicker performance. For example, DDR5, the most recent standard as of 2025, is perfect for contemporary applications like gaming and AI tasks since it offers faster data transfer rates and less power consumption than DDR4.

The other primary memory type, ROM, is non-volatile, which means that data is stored there even after the system is turned off. The BIOS (Basic Input/Output System) or firmware, which contains the first boot-up instructions for the computer, is stored in ROM. Programmable ROM (PROM) can be programmed once after manufacturing, Erasable Programmable ROM (EPROM) can be erased using UV light and reprogrammed, MROM (Masked ROM) is programmed during manufacturing and cannot be changed, and EEPROM (Electrically Erasable Programmable ROM) can be erased and rewritten electrically, providing greater flexibility.

4.1.2. Secondary Memory: Long-Term Storage

Long-term data storage is accomplished by secondary memory, which is non-volatile. The CPU cannot access it directly like it can primary memory; instead, data must be moved to RAM in order to be processed. Flash memory, optical discs, magnetic tapes, and magnetic discs are examples of secondary memory devices.

Because of their sequential access, magnetic tapes are among the earliest types of secondary storage and are mostly utilised for backups and archiving. Although they are slow to retrieve, they are economical for storing massive volumes of data. Hard disc drives (HDDs) and other magnetic discs store data on rotating platters covered in magnetic material. They are slower than solid-state devices (SSDs), but they provide faster access than cassettes.

Laser technology is used by optical discs, such as CDs and DVDs, to read and write data. Although they are robust and portable, their storage capacity is not as large as that of more recent models.

Because of its speed, robustness, and lack of moving parts, flash memory—found in USB devices and SSDs—has emerged as the most popular type of secondary storage. Because SSDs use flash memory to enable quicker read/write speeds and improved durability, they have largely supplanted HDDs in current PCs.

4.1.3. How Memory Works Together

A computer's ability to function depends on how its primary and secondary memory interact. The BIOS, which is kept in ROM, loads the operating system into RAM from secondary memory (such as an SSD) and initialises the hardware when your computer boots up. The CPU can swiftly access the operating system's data and instructions once they are in RAM. Applications and data are also stored in RAM for quick access when you open them. Although this is far slower than physical RAM, the system may use some of the secondary memory as virtual memory (such as a swap file on an SSD) if RAM fills up.

Programs and documents that must be stored for all time are written back to secondary memory. For instance, a file is transferred from RAM to an SSD or HDD when it is saved. The computer maintains a balance between speed (via primary memory) and storage capacity (through secondary memory) thanks to this interaction.

SELF-ASSESSMENT QUESTIONS - 3

True or False:	
9	RAM is a type of primary memory that is volatile and loses its data when the computer is turned off.
Fill in the Blanks:	
10	The two main types of primary memory are _____ and ROM.
11	Secondary memory is _____, meaning it retains data even after the computer is powered off.

5. INPUT/OUTPUT SECTION

The hardware, software, and interfaces that control the flow of data into and out of a computer system are all included in the input/output (I/O) section. It serves as a link between external devices like keyboards, displays, printers, and storage devices and internal processing components like the CPU and memory. The I/O section makes sure the computer can process data or instructions from devices or users and provide results that are useful.

5.1. Components of the Input/Output Section

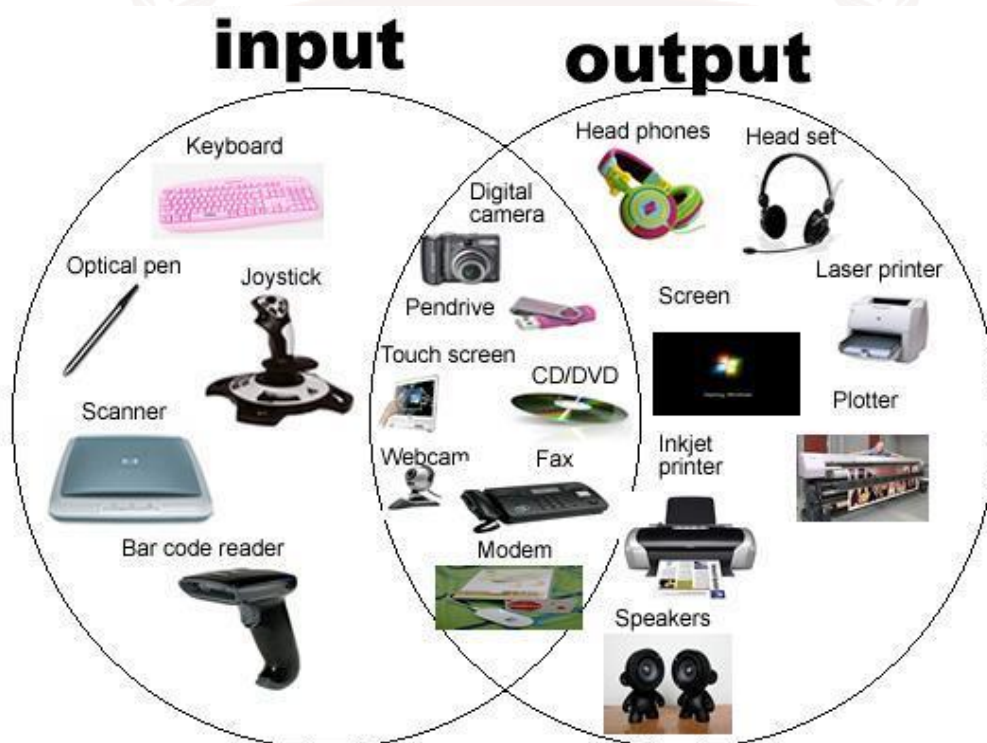


Figure 1.6: Input/Output Section

The I/O section consists of several key components which shown in Figure 1.6 also:

1. Input Devices:

- These are hardware devices that allow users or external systems to send data to the computer. Common input devices include:
 - Keyboard: Allows users to input text and commands.
 - Mouse: Facilitates pointing, clicking, and navigation on graphical user interfaces (GUIs).
 - Microphone: Captures audio input for voice commands or recordings.
 - Scanner: Converts physical documents or images into digital data.

- Webcam: Captures video or images for input.
- Sensors: Devices like touchscreens, motion detectors, or temperature sensors provide environmental data.
- Network Interfaces: Allow data input from networks or the internet.

2. Output Devices:

- These devices display or deliver the processed data from the computer to the user or other systems. Common output devices include:
 - Monitor: Displays visual output, such as text, images, or videos.
 - Printer: Produces physical copies of digital documents or images.
 - Speakers: Output audio, such as music or voice feedback.
 - Projectors: Display visual content on large surfaces for presentations.
 - Network Interfaces: Send processed data to other systems or devices over a network.

3. I/O Interfaces:

- These are the hardware and software components that connect input/output devices to the computer's CPU and memory. Examples include:
 - Ports: Physical connectors like USB, HDMI, or Ethernet ports.
 - Controllers: Specialised chips (e.g., USB controller, graphics card) that manage data transfer between the CPU and I/O devices.
 - Device Drivers: Software that enables the operating system to communicate with specific I/O devices.
 - Buses: Data pathways (e.g., PCIe, SATA) that transfer data between the CPU, memory, and I/O devices.

4. I/O Software:

- The software layer that manages I/O operations, including:
 - Operating System I/O Subsystem: Handles device communication, buffering, and error handling.
 - Firmware: Low-level software embedded in I/O devices to control their operation.
 - Application Software: Programs that interact with I/O devices, such as word processors or media players.

5.2. Functions of the Input/Output Section

The I/O section performs several critical functions to ensure the seamless operation of a computer system:

1. Data Input:
 - Accepts raw data or commands from users or external devices.
 - Converts analog signals (e.g., sound from a microphone) into digital data that the computer can process, if necessary.
2. Data Output:
 - Delivers processed data in a human-readable or machine-readable format.
 - Converts digital data into analog signals (e.g., video output to a monitor) when required.
3. Data Transfer Management:
 - Manages the flow of data between the CPU, memory, and I/O devices.
 - Handles data buffering to match the speed differences between fast processors and slower I/O devices.
4. Error Handling:
 - Detects and corrects errors during data transfer, such as corrupted data or device malfunctions.
5. Device Communication:
 - Ensures compatibility between the computer and a wide range of devices through standardised protocols and interfaces.
6. Interrupt Handling:
 - Manages interrupts, which are signals from I/O devices requesting CPU attention (e.g., when a key is pressed or a file is read from a disk).

5.3. Types of Input/Output Operations

I/O operations can be categorised based on how data is transferred between the computer and devices:

1. Programmed I/O:
 - The CPU directly controls the I/O operations by continuously checking the status of the device (polling).
 - Suitable for simple, low-speed devices but inefficient for high-speed or complex systems.

2. Interrupt-Driven I/O:

- The device sends an interrupt signal to the CPU when it is ready to send or receive data.
- More efficient than programmed I/O as it frees the CPU to perform other tasks while waiting for I/O operations.

3. Direct Memory Access (DMA):

- Allows high-speed devices (e.g., hard drives, network cards) to transfer data directly to or from memory without involving the CPU for each byte.
- Used for large data transfers to improve performance.

4. Memory-Mapped I/O:

- I/O devices are mapped to specific memory addresses, allowing the CPU to communicate with devices as if they were memory locations.
- Common in modern systems for simplicity and efficiency.

5.4. How the I/O Section Works

The I/O section operates through a coordinated process involving hardware and software:

1. Input Process:

- An input device (e.g., keyboard) captures data (e.g., a keypress).
- The device sends the data to the I/O controller via a port or bus.
- The controller processes the data and may generate an interrupt to notify the CPU.
- The CPU, guided by the operating system and device drivers, retrieves the data and stores it in memory for processing.

2. Output Process:

- Data is processed by the CPU and sent to the target output device's I/O controller (such as a monitor).
- The controller converts the data (such as pixel data for a display) into a format that the device can use.
- The output device shows the user the data (text on a screen, for example).

3. Buffering and Caching:

- The system use caches and buffers (temporary storage spaces) to store data while it is being sent in order to manage performance discrepancies between the CPU and I/O devices.

4. Error Handling and Synchronisation:

- By identifying faults and retransmitting data as needed, the I/O component guarantees data integrity.
- Mechanisms for synchronisation guarantee that input and output processes take place in the right sequence.

5.5. Examples of I/O in Action

1. Typing a Document:

- Input: A USB controller transmits keyboard keystrokes to the CPU.
- Processing: The text is stored in memory once the CPU decodes the keystrokes.
- Output: A monitor displays the text.

2. Printing a Photo:

- Input: A mouse or touchscreen is used to choose a digital image.
- Processing: The CPU uses a WiFi or USB connection to transfer the picture data to the printer.
- Output: A hard copy of the picture is created by the printer.

3. Streaming a Video:

- Input: Data is received from a network interface (e.g., Wi-Fi).
- Processing: The CPU decodes the video stream.
- Output: The audio is played through speakers, while the video is seen on a monitor.

SELF-ASSESSMENT QUESTIONS - 4

True or False:	
12	A printer is an output device that produces a physical copy of digital documents or images.
Fill in the Blanks:	
13	Devices like the keyboard and mouse are called _____ devices because they allow users to send data and instructions to the computer.
14	The hardware and software components that connect input/output devices to the CPU and memory are called I/O _____.

6. CPU

The majority of the processing, computation, and decision-making activities that power your device's operation are handled by the Central Processing Unit (CPU), which acts as the brain of the computer. The CPU carefully carries out the required instructions to keep everything operating efficiently, whether you're streaming a movie, playing a game, or working on a school assignment.

The CPU is housed in a dedicated socket on the motherboard, the central circuit board that links all computer components. Key tasks of the CPU include:

- Performing mathematical operations (such as addition or multiplication).
- Executing applications and games.
- Coordinating interactions between the keyboard, mouse, and display.

6.1. History of CPU

The history of the CPU spans centuries, with key breakthroughs shaping modern computing. Here's a concise timeline for students in Figure 1.7:

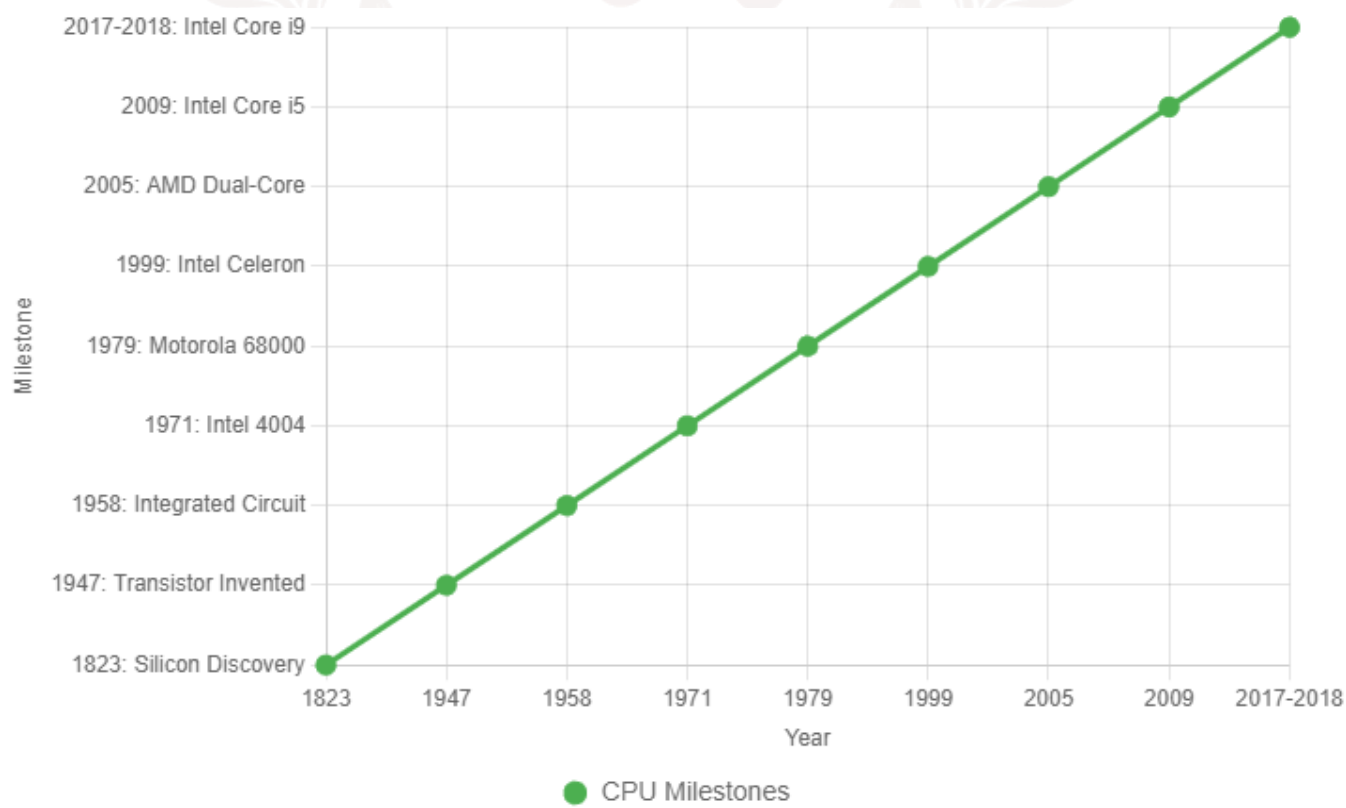


Figure 1.7: History of CPU

- 1823: Baron Jons Jakob Berzelius discovered silicon, the foundational material for modern CPUs.
- 1947: John Bardeen, Walter Brattain, and William Shockley invented the transistor, a critical component enabling compact, efficient CPUs.
- 1958: Jack Kilby and Robert Noyce developed the integrated circuit, packing multiple transistors onto a single chip.
- 1971: Intel introduced the Intel 4004, the first microprocessor (a single-chip CPU), kickstarting the personal computer revolution.
- 1979: Motorola launched the Motorola 68000, a robust CPU powering early computers and gaming consoles.
- 1999: Intel released the Celeron processors, boosting speed and affordability for personal computers.
- 2005: AMD debuted the dual-core processor, enabling CPUs to multitask efficiently.
- 2009: Intel unveiled the Core i5, a quad-core processor that significantly enhanced computing performance.
- 2017-2018: Intel launched the Core i9, a high-performance CPU for desktops and laptops, pushing processing power to new heights.

6.2. Components of the CPU

The Central Processing Unit (CPU) is made up of several key components that work together to process instructions and make a computer function. Here's a concise overview of the main components for students:

- **Arithmetic Logic Unit (ALU):** The Arithmetic Logic Unit (ALU) is capable of carrying out logical and mathematical operations, including comparisons like greater than and less than, as well as addition, subtraction, multiplication, and division.
- **Control Unit (CU):** Coordinating the movement of information and commands between the CPU, memory, and other hardware elements, the Control Unit (CU) functions as a traffic director.
- **Registers:** Registers are tiny, incredibly quick storage spaces inside the CPU that are used to temporarily store processed data and instructions.
- **Cache Memory:** The CPU's cache memory is a tiny, fast memory that speeds up processing by storing frequently used data.

- Clock: The clock controls the CPU's operating speed, which is expressed in cycles per second (hertz, or GHz in more recent CPUs).
- Instruction Decoder: Program instructions are interpreted and converted into a format that the CPU can use by an instruction decoder.

These components work together to fetch, decode, and execute instructions, enabling the CPU to handle tasks like running apps, performing calculations, and managing hardware interactions.

6.3. Functions of the CPU

The CPU's main job is to process instructions from programs. It does this through a process called the Fetch-Decode-Execute-Store cycle:

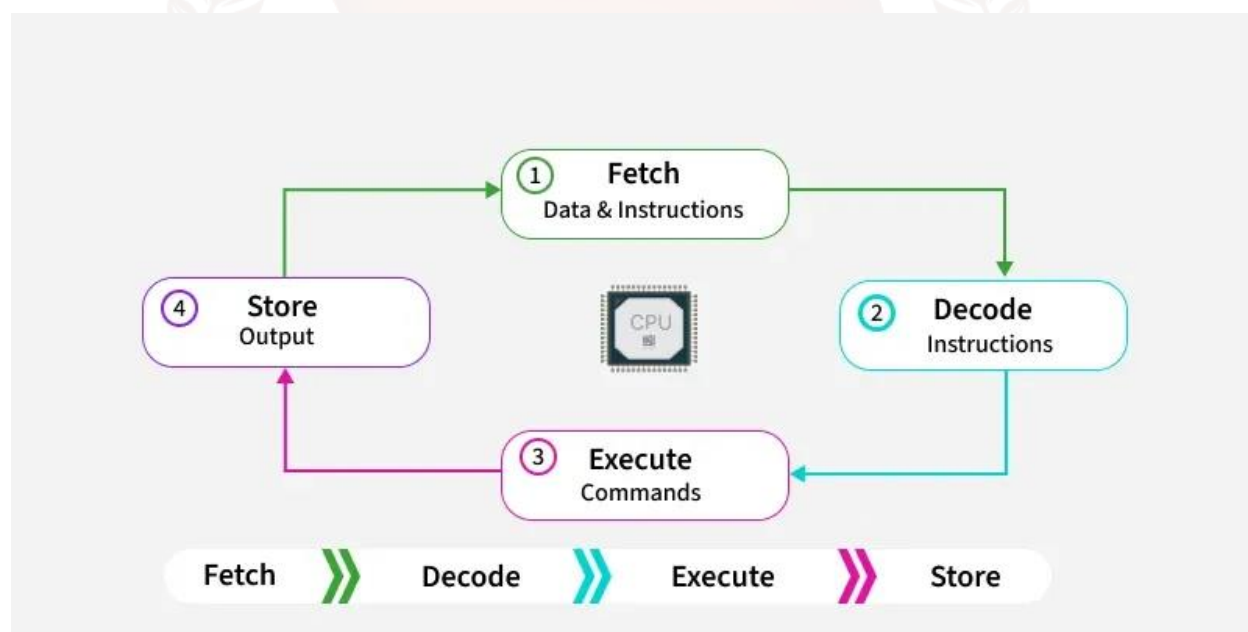


Figure 1.8: Functions of the CPU

Figure 1.8 illustrates the basic functions of a CPU, often referred to as the "Fetch-Decode-Execute Cycle" or "Instruction Cycle." This cycle explains how a CPU processes instructions from a computer program. Here's a simple breakdown for students:

1. **Fetch (Data & Instructions):**
 - The CPU starts by retrieving (or "fetching") an instruction and any needed data from the computer's memory (RAM). This step is like the CPU picking up a task it needs to work on.
2. **Decode (Instructions):**
 - The CPU interprets the fetched instruction to understand what action is required. For example, it figures out if the instruction is to add two numbers or display something on the

screen. The Instruction Decoder (a CPU component) helps with this translation.

3. Execute (Commands):

- The CPU carries out the instruction. This could involve performing a calculation (using the Arithmetic Logic Unit), moving data, or interacting with other hardware like the keyboard or screen. The Control Unit ensures everything happens in the right order.

4. Store (Output):

- After executing the instruction, the CPU saves the result back to memory or sends it to an output device (like a screen). For example, if the task was to add two numbers, the sum is stored for future use.

The arrows in the diagram show that this process is a continuous cycle: after storing the output, the CPU goes back to fetch the next instruction, and the cycle repeats. This loop allows the CPU to handle all the tasks needed to run apps, games, or any computer program.

6.4. Types of CPUs

CPUs come in various types, designed for different devices, performance needs, and architectures. Here's a concise overview for students:

- **Desktop CPUs:** Built for personal computers, these CPUs prioritise performance for tasks like gaming, video editing, and multitasking. Examples include Intel Core i7 and AMD Ryzen 7.
- **Mobile CPUs:** Found in smartphones and tablets, these are smaller, energy-efficient CPUs optimised for battery life and heat management. Examples include ARM-based chips like the Apple A18 (used in iPhones) and Qualcomm Snapdragon.
- **Laptop CPUs:** A balance between performance and power efficiency, designed for portability. They often have lower power versions of desktop CPUs, like Intel Core i5 U-series or AMD Ryzen 5 mobile processors.
- **Server CPUs:** Designed for data centres and servers, these CPUs handle heavy workloads, multitasking, and reliability for tasks like web hosting or cloud computing. Examples include Intel Xeon and AMD EPYC.
- **Embedded CPUs:** Used in specific devices like smart appliances, cars, or IoT gadgets. These are simple, low-power CPUs tailored for dedicated tasks, often ARM-based.
- **RISC vs. CISC CPUs:**
 - **RISC (Reduced Instruction Set Computing):** Uses simple, fast instructions (e.g., ARM processors in mobiles).

- CISC (Complex Instruction Set Computing): Handles more complex instructions (e.g., x86 processors in desktops).
- Multi-Core CPUs: Contain multiple cores (like dual-core, quad-core, or octa-core) on a single chip, allowing parallel task processing. Most modern CPUs, like Intel Core i9 or AMD Ryzen 9, are multi-core.

SELF-ASSESSMENT QUESTIONS - 5

True or False:	
15	The Control Unit (CU) in the CPU is responsible for performing mathematical calculations like addition and multiplication.
Fill in the Blanks:	
16	The process by which the CPU retrieves, interprets, carries out, and then stores instructions is called the _____ cycle.
17	CPUs that contain more than one processing unit on a single chip are called _____ CPUs.

7. SUMMARY

Computer architecture examines the design and functionality of computer systems, focusing on the Central Processing Unit (CPU), memory, and input/output (I/O) sections. The CPU, the system's core, includes the Arithmetic Logic Unit (ALU) for performing mathematical and logical operations, the Control Unit (CU) for managing data flow, and registers for rapid data storage. The CU fetches, decodes, and executes instructions, while registers and cache memory enhance processing efficiency. The CPU operates via the Fetch-Decode-Execute-Store cycle, ensuring seamless task execution.

Memory is categorised into primary and secondary types. Primary memory, such as volatile RAM (including SRAM, DRAM, and DDR variants) and non-volatile ROM (MROM, PROM, EPROM, EEPROM), enables fast data access for the CPU. Secondary memory, encompassing magnetic tapes, disks, optical disks, and flash memory (e.g., SSDs), provides long-term, non-volatile storage. Data moves from secondary storage to RAM for processing, balancing speed and capacity.

The I/O section facilitates communication between the computer and external devices. Input devices, like keyboards and mice, convert user actions into digital signals, while output devices, such as monitors and printers, present processed data. I/O operations, including programmed I/O, interrupt-driven I/O, and Direct Memory Access (DMA), ensure efficient data transfer, supported by interfaces like USB.

Architectural designs, such as Von Neumann and Harvard, dictate component interactions. Von Neumann uses a single memory for data and instructions, potentially causing bottlenecks, whereas Harvard separates them for faster processing. The Modified Harvard Architecture offers flexibility and speed, while RISC employs simple instructions for efficiency, and CISC handles complex ones for versatility. These architectures optimise performance across devices, from laptops to servers, influencing speed, scalability, and energy efficiency, forming a critical foundation for computer science studies.

8. GLOSSARY

ALU (Arithmetic Logic Unit)	- A CPU component that performs mathematical calculations and logical operations, such as addition or comparisons.
Control Unit (CU)	- The CPU part that directs data flow and instruction execution, coordinating activities between memory, ALU, and I/O devices.
Registers	- Small, high-speed storage within the CPU for holding temporary data or instructions during processing.
Primary Memory	- Fast, volatile storage (e.g., RAM, ROM) directly accessed by the CPU for active data and program instructions.
Secondary Memory	- Non-volatile storage (e.g., HDDs, SSDs) for long-term data retention, slower than primary memory.
Input Devices	- Hardware (e.g., keyboard, mouse) that sends user data or commands to the computer.
Output Devices	- Hardware (e.g., monitor, printer) that presents processed data in a user-readable format.
Von Neumann Architecture	- A design where data and instructions share the same memory, simplifying structure but potentially causing bottlenecks.
Harvard Architecture	- A design with separate memory for instructions and data, enabling faster parallel processing.
Modified Harvard Architecture	- A hybrid design allowing simultaneous instruction and data access with flexible memory sharing.
RISC (Reduced Instruction Set Computing)	- A CPU design using simple, uniform instructions for faster execution and efficiency.
CISC (Complex Instruction Set Computing)	- A CPU design with complex instructions for versatile but slower processing.

Instruction Set Architecture (ISA)	- The set of commands a CPU can execute, serving as the interface between hardware and software.
Microarchitecture	- The specific implementation of an ISA, defining how instructions are processed within a CPU.
Data Flow	- The movement of data between input devices, memory, CPU, and output devices during processing.
Control Flow	- The coordination of operations via signals from the Control Unit to ensure proper instruction execution.
Cache Memory	- High-speed memory within the CPU for storing frequently accessed data to reduce access time.
Clock	- A CPU component that regulates processing speed, measured in cycles per second (e.g., GHz).
Instruction Decoder	- A CPU component that interprets program instructions for execution.
Bus	- A communication pathway transferring data between CPU, memory, and I/O devices.

9. TERMINAL QUESTIONS

1. What is computer architecture?
2. Name the three primary aspects of computer architecture.
3. How do Von Neumann and Harvard architectures differ from each other?
4. What are the main components of the Von Neumann architecture?
5. Discuss the role of the Control Unit in a computer.
6. What is the function of the Arithmetic Logic Unit (ALU)?
7. Outline the significance of input devices in a computer system.
8. Explain the concept of 'pipelining' in RISC architecture.
9. How does the Modified Harvard Architecture improve performance?
10. What is the role of memory in the block diagram of a computer?

10. ANSWERS

Self-Assessment Questions

1. True
2. False
3. False
4. data
5. True
6. False
7. True
8. Central Processing Unit (CPU)
9. True
10. RAM
11. non-volatile
12. True
13. input
14. interfaces
15. False
16. Fetch-Decode-Execute (or Instruction) cycle
17. Multi-core

Terminal Questions Answers

1. Computer architecture is the conceptual design and fundamental operational structure of a computer system. It specifies how a computer's hardware and software components interact to process data and execute instructions, forming the essential framework for all types of computing devices. Refer to Section 2 for more details.
2. The three primary aspects are:
 - System Design
 - Instruction Set Architecture (ISA)
 - Microarchitecture Refer to Section 2 for more details.
3. Von Neumann architecture uses a single memory space for both instructions and data, which can lead to bottlenecks. Harvard architecture separates memory for instructions and data, allowing

for faster and more efficient processing in certain applications. Refer to Section 2.1 for more details.

4. The main components are:
 - Memory (stores both data and program instructions)
 - Processor (divided into Control Unit and Arithmetic Logic Unit)
 - Input/Output (I/O) units Refer to Section 2.1 for more details.
5. The Control Unit directs the operation of the processor by fetching instructions from memory, decoding them, and instructing the ALU on what operations to perform. Refer to Section 2.1 for more details.
6. The ALU performs all arithmetic (addition, subtraction, etc.) and logical (AND, OR, NOT, etc.) operations within the processor. Refer to Section 2.1 for more details.
7. Input devices are essential for user interaction, allowing users to provide data or commands to the computer, which are then converted into digital signals for processing. Refer to Section 3 for more details.
8. Pipelining allows the CPU to work on several instructions at once by breaking the process into steps, like an assembly line, improving processing speed. Refer to Section 2.1 for more details.
9. It allows the CPU to fetch instructions and access data simultaneously using separate paths, and can share instruction memory as data memory when needed, enhancing speed and flexibility. Refer to Section 2.1 for more details.
10. Memory stores both data and program instructions, serving as a central hub for data exchange between the CPU, input, and output devices. Refer to Section 3 for more details.

11. REFERENCES

1. Hennessy, J. L., & Patterson, D. A. (2017). Computer Architecture: A Quantitative Approach. Morgan Kaufmann.
2. Patterson, D. A., & Hennessy, J. L. (2012). Computer organization and design: the hardware/software interface, 4th ed. (Rev. ed. of: Computer organization and design / John L. Hennessy, David A. Patterson. 1998.). In Elsevier eBooks.
<http://dspace.uniten.edu.my/handle/123456789/14251>
3. (PDF) Computer Architecture Sixth Edition a quantitative approach PDFDrive com | Saad Farooq - academia.edu. (n.d.-a).
https://www.academia.edu/44613414/computer_architecture_sixth_edition_a_quantitative_approach_pdfdrive_com
4. Towards a reference architecture for future industrial internet of things networks | IEEE conference publication | IEEE xplore. (n.d.-b). <https://ieeexplore.ieee.org/document/9610707/>
5. Search. (n.d.). Pearson. <https://www.pearson.com/store/p/computer-organization-and-architecture-designing-for-performance/P100000253068>

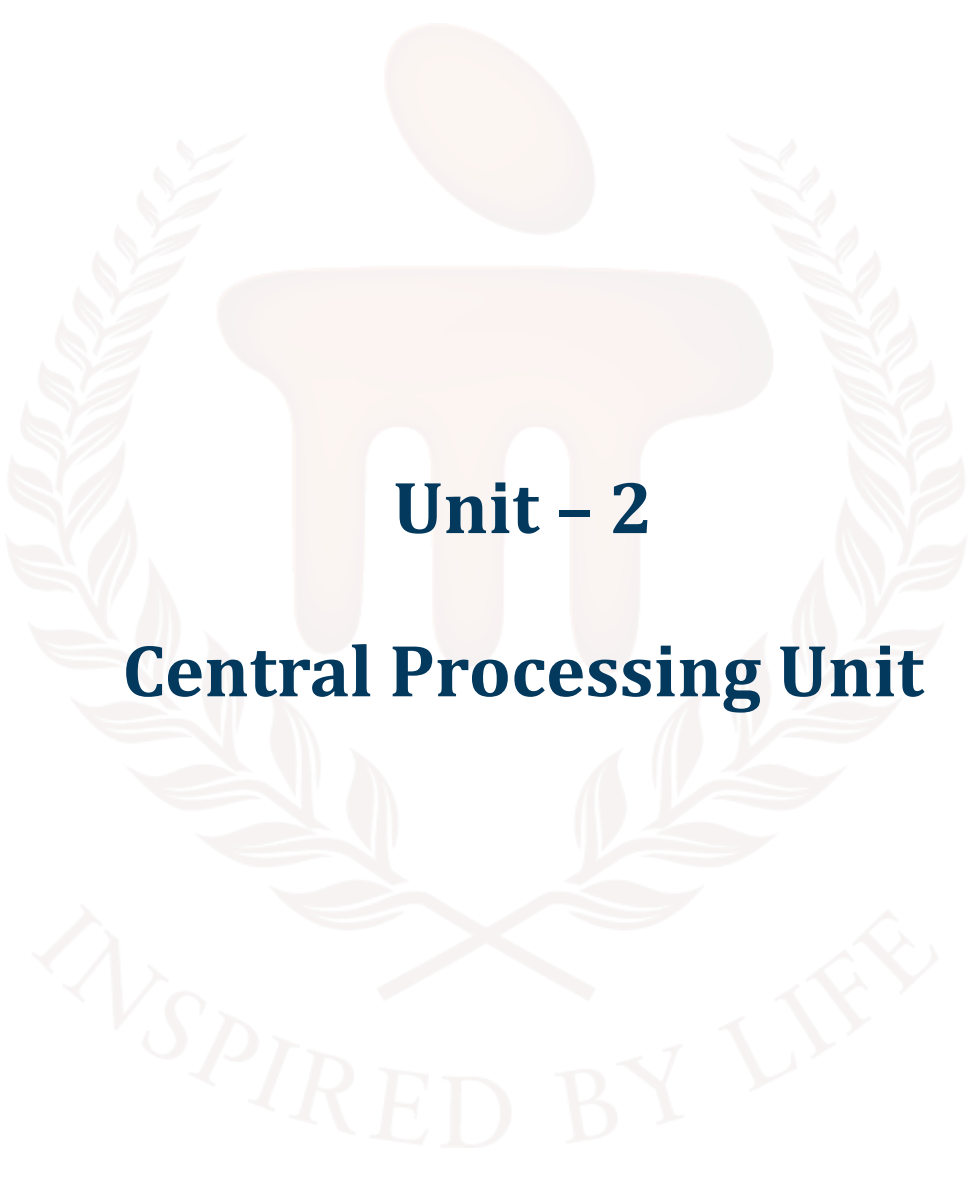
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 2

Central Processing Unit

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 6
1.1	Objectives	-	-	
2	Registers	1	-	7 - 11
2.1	How the Components Work Together	-	-	
2.2	Register Size	-	1	
3	Arithmetic Unit	-	-	12 - 17
3.1	Brief history and evolution	2	-	
3.2	Core functions of an Arithmetic Unit	-	-	
3.3	Components of an Arithmetic Unit	3	-	
3.4	Working Principle	-	2	
4	Instruction Handling Areas	-	-	18 - 22
4.1	The Anatomy of an Instruction	-	-	
4.2	Different Instruction Formats	-	-	
4.3	CPU Components Behind Instruction Handling	4	-	
4.4	The Instruction Cycle	-	-	
4.5	Types of Instructions	-	-	
	Addressing Modes	-	3	
5	Stack	5	-	23 - 32
5.1	Core Operations of a Stack	6	-	
5.2	Stack Implementation Techniques	-	-	
5.3	Stack Overflow and Underflow	7	-	
5.4	Applications of Stacks in Computing	-	-	
5.5	Stacks in Computer Architecture	-	4	
6	Summary	-	-	33
7	Glossary	-	-	34 - 35

8	Terminal Questions	-	-	36
9	Answers	-	-	37 - 38
10	References	-	-	39



1. INTRODUCTION

In the previous unit, we learnt about computer architecture which explores the design, structure, and operation of computer systems. You also learnt about essential components that make up a computer, focusing on the block diagram of a computer, the memory section, the input/output (I/O) section, and the Central Processing Unit (CPU).

The Central Processing Unit (CPU), widely regarded as the cornerstone unit of any computing system, serves as the primary hub for processing instructions that enable a computer to perform tasks ranging from simple calculations to complex data manipulations. Often likened to the brain of a computer, the CPU operates as a highly coordinated unit, integrating several critical components to execute programs with precision and speed.

This unit also explains the essential elements that define the CPU's functionality: registers, the arithmetic unit, instruction handling areas, and stacks.

Registers act as high-speed, temporary storage locations within the CPU, holding data, addresses, and instructions for immediate access during processing. The arithmetic unit, a vital subcomponent, is responsible for executing numerical computations and logical operations, enabling the CPU to solve mathematical problems and make decisions.

Instruction handling areas manage the critical processes of fetching, decoding, and executing instructions, ensuring seamless program execution. Stacks provide an efficient mechanism for managing memory, particularly for handling function calls, local variables, and return addresses, thereby maintaining the flow of complex programs.

By exploring these components in detail, this unit elucidates how the CPU, as a unified processing unit, orchestrates their interactions to deliver the computational power that underpins modern computer systems. This exploration lays the groundwork for understanding the principles of computer architecture and the intricate design of processing units that drive technological innovation.

1.1. Objectives

After studying this unit, you should be able to:

- Define the role and structure of the Central Processing Unit (CPU) in a computer system.
- Explain the functions of registers in data storage and processing.
- Discuss the operations performed by the arithmetic unit within the CPU.
- Analyse the processes involved in instruction handling areas for program execution.



2. REGISTERS

The CPU is the brain of your computer, working at lightning speed to process information. But even the fastest brain needs a place to keep important stuff handy—enter CPU registers! These are tiny, ultra-fast memory units built right inside the processor, acting as the CPU's personal notepad for quick calculations and decisions.

Why are registers so important? Well, fetching data from RAM is like walking across the room to grab a book, while accessing the hard drive is like driving to the library. Registers, on the other hand, are like having sticky notes right on your desk—instant access! This immediate availability is crucial because the CPU needs to work with numbers, addresses, and instructions as fast as possible to keep everything running smoothly.

Here's how registers help the CPU stay on top of its game:

- **Data Handling:** General-purpose registers are the CPU's scratchpad, holding numbers and information needed for calculations and logic.
- **Memory Navigation:** Address registers act like GPS, pointing the CPU to the exact spot in memory where data lives.
- **Keeping Track:** Special registers, like the program counter, keep tabs on which instruction comes next, making sure nothing gets lost in the shuffle.
- **Status Updates:** Control and status registers are the CPU's dashboard, lighting up with flags and signals that guide its next move.

Even though RAM is much faster than your computer's storage, it's still not speedy enough for the CPU's split-second needs. That's why registers are the go-to for immediate data access, with cache memory serving as a helpful middleman—faster than RAM, but still a step behind registers.

In short, CPU registers are the secret sauce behind your computer's speed and efficiency. They're the reason your programs run smoothly, your games don't lag, and your device keeps up with your every command. Without them, the CPU would be stuck waiting, and nobody likes a slowpoke computer.

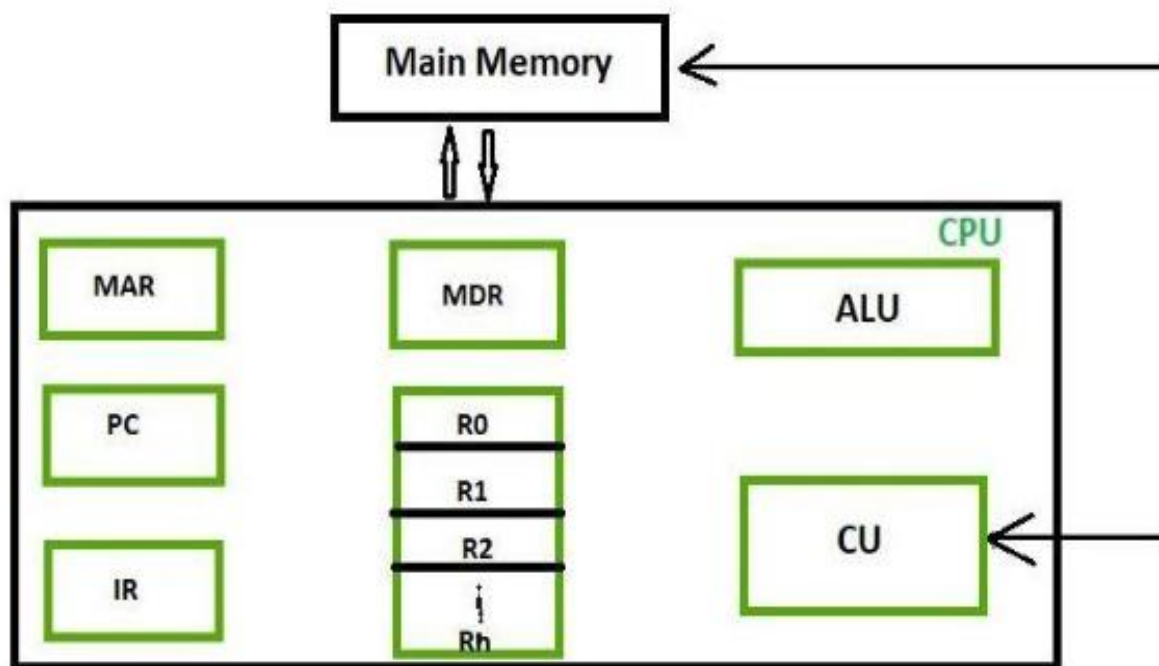


Figure 1: Registers

Let's break down each part and see how they function together to execute instructions and process data.

1. Main Memory (RAM)

Main Memory is where the computer temporarily stores programs and data that are currently in use. When you run an application, it is loaded from permanent storage (like a hard drive or SSD) into main memory so the CPU can access it quickly. Main memory is volatile, meaning its contents are lost when the computer is turned off.

2. Inside the CPU: Key Components

The CPU is the core processing unit of the computer, often referred to as the "brain" of the system. Inside the CPU, there are several specialised components, each with a unique role:

a. MAR (Memory Address Register)

- **Purpose:** The MAR holds the address in main memory where data is to be read from or written to.
- **Function:** Whenever the CPU needs to access a specific memory location, it places the address of that location into the MAR.

b. MDR (Memory Data Register)

- **Purpose:** The MDR temporarily stores data that is being transferred to or from main memory.

- **Function:** When data is read from memory, it is placed in the MDR before being used by the CPU. When data is written into memory, it is placed in the MDR before being sent to the main memory.

c. PC (Program Counter)

- **Purpose:** The PC keeps track of the address of the next instruction to be executed.
- **Function:** After an instruction is fetched, the PC is updated to point to the next instruction, ensuring the CPU processes instructions in the correct order.

d. IR (Instruction Register)

- **Purpose:** The IR holds the current instruction that the CPU is decoding and executing.
- **Function:** Once an instruction is fetched from memory, it is loaded into the IR for decoding and execution by the control unit.

e. General Purpose Registers (R0, R1, R2, ..., Rh)

- **Purpose:** These registers serve as temporary storage for data and intermediate results during processing.
- **Function:** Registers like R0, R1, R2, up to Rh, are used to hold operands for calculations, store results, and facilitate quick data manipulation without needing to access main memory.

f. ALU (Arithmetic Logic Unit)

- **Purpose:** The ALU is responsible for performing all arithmetic (addition, subtraction, multiplication, division) and logical (AND, OR, NOT, XOR) operations.
- **Function:** When the CPU needs to perform a calculation or comparison, the ALU does the actual work, using data stored in the registers.

g. CU (Control Unit)

- **Purpose:** The CU directs the operation of the processor.
- **Function:** It interprets instructions from the IR and generates control signals to coordinate the activities of the ALU, registers, and memory. The CU ensures that all parts of the CPU work together smoothly.

2.1. How the Components Work Together

Step 1: Fetching an Instruction

- The Program Counter (PC) contains the address of the next instruction.
- This address is sent to the Memory Address Register (MAR).
- The CPU requests the instruction from Main Memory at that address.

- The instruction is loaded from memory into the Memory Data Register (MDR).
- The instruction is then transferred from the MDR to the Instruction Register (IR).

Step 2: Decoding the Instruction

- The Control Unit (CU) reads the instruction from the IR.
- It decodes the instruction to determine the operation to perform and which data or registers are involved.

Step 3: Executing the Instruction

- If the instruction involves computation, the necessary data is loaded into the General Purpose Registers (R0, R1, etc.).
- The Arithmetic Logic Unit (ALU) performs the required operation (like addition or comparison).
- The result is stored back in a register or prepared to be written to memory.

Step 4: Storing the Result

- If the result needs to be saved in main memory, the address is placed in the MAR, and the data is placed in the MDR.
- The data is then written from the MDR to the specified location in Main Memory.

Step 5: Moving to the Next Instruction

- The Program Counter (PC) is incremented to point to the next instruction, and the cycle repeats.

2.2. Register Size

The size of a CPU register is measured in bits. A bit is the smallest data unit in computing, representing a binary value of either 0 or 1. Registers can be 8-bit, 16-bit, 32-bit, 64-bit, or even larger in modern processors. The register size indicates how many bits of data the CPU can process or transfer at one time.

For example, an 8-bit register can hold 8 bits of data, equivalent to 1 byte, while a 64-bit register can hold 64 bits, or 8 bytes.

Common Register Sizes and Their Evolution

- **8-bit Registers:** Early microprocessors, such as the Intel 8080 and Zilog Z80, used 8-bit registers. These processors could handle data 8 bits at a time, which limited their processing power and memory addressing to relatively small amounts.
- **16-bit Registers:** The Intel 8086 and 80286 processors introduced 16-bit registers, doubling the amount of data processed per cycle compared to 8-bit CPUs. This allowed for more efficient processing and larger memory addressing capabilities.
- **32-bit Registers:** With the arrival of processors like the Intel 80386, 32-bit registers became standard. These registers could handle 4 bytes of data at once and address up to 4 gigabytes (GB) of memory, which was a significant leap in computing power.
- **64-bit Registers:** Modern processors, including Intel's Core series and AMD's Ryzen, utilise 64-bit registers. These registers can process 8 bytes of data simultaneously and theoretically address up to 18 exabytes of memory, vastly expanding the potential for complex computations and large memory usage.

SELF-ASSESSMENT QUESTIONS - 1

True or False:	
1	Registers are slower than RAM but faster than hard drives.
2	The Program Counter (PC) register holds the address of the next instruction to be executed by the CPU.
Fill in the Blanks:	
3	The register that temporarily stores data being transferred to or from main memory is called the _____.
4	The size of a CPU register, which indicates how many bits of data the CPU can process at one time, is also known as the _____.

3. ARITHMETIC UNIT

The Arithmetic Unit (AU) is a crucial component of a computer's Central Processing Unit (CPU) responsible for performing all arithmetic operations on binary numbers. As a part of the broader Arithmetic Logic Unit (ALU), the AU specifically handles mathematical calculations such as addition, subtraction, and sometimes multiplication and division. These operations form the foundation of most computational tasks.

The AU receives binary inputs from the CPU's registers, executes the specified arithmetic operation, and produces the result in binary form. It also generates important status flags like carry, zero, and overflow, which help the CPU make decisions during program execution. By efficiently performing these calculations, the Arithmetic Unit plays a vital role in enabling the CPU to process data and execute instructions accurately and swiftly.

In essence, the Arithmetic Unit is the mathematical engine within the CPU, enabling computers to perform the essential arithmetic tasks required for all kinds of computing processes.

3.1. Brief history and evolution

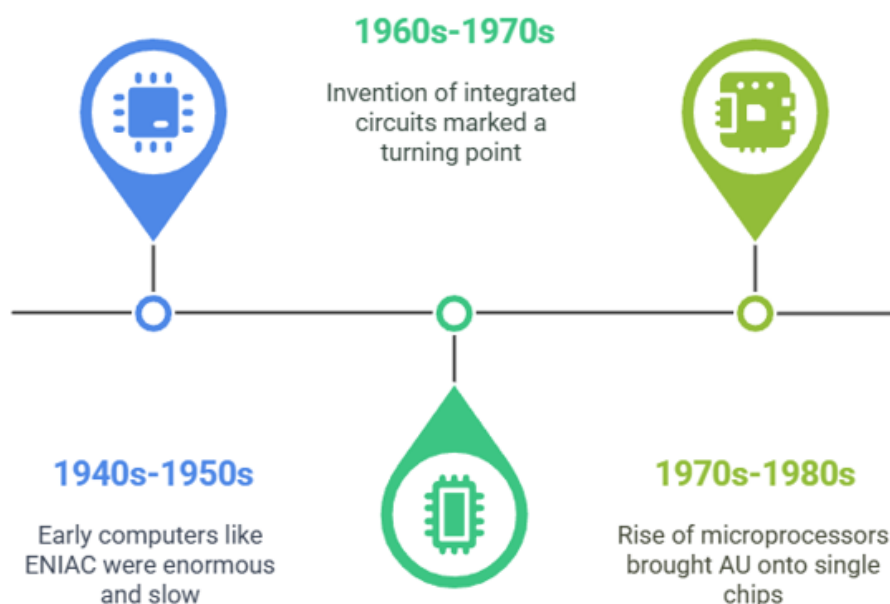


Figure 2: Evolution of Computing Technology

- The Arithmetic Unit (AU) is the heart of all mathematical processing in a computer, and its evolution tells a fascinating story of technological progress.
- In the early days of computing during the 1940s and 1950s, machines like the ENIAC were enormous and slow, performing arithmetic operations bit by bit in a serial manner. These early computers often required manual rewiring to switch between different operations, making them cumbersome and inefficient.
- The 1960s and 1970s marked a turning point with the invention of integrated circuits. This breakthrough allowed arithmetic units to shrink dramatically in size while increasing in speed. Chips such as the Fairchild 3800 could perform 8-bit arithmetic operations efficiently, and innovations like the “bit-slice” design enabled engineers to build scalable and flexible arithmetic units by combining multiple chips.
- The rise of microprocessors in the 1970s and 1980s brought the AU onto single chips, making computers more compact and affordable. Early microprocessors had limited word sizes and required multiple cycles for complex operations, but advances in transistor technology soon enabled wider, faster, and more capable arithmetic units.

3.2. Core functions of an Arithmetic Unit

The Arithmetic Unit (AU) is a vital component of a computer’s processor, primarily responsible for executing all numerical calculations required during program execution. It acts as the mathematical engine of the CPU, performing a variety of arithmetic operations on binary data. Here are the core functions that define the AU’s role in computing:

Addition and Subtraction

At the heart of the AU is its ability to perform addition and subtraction. These are the most fundamental arithmetic operations, allowing the processor to combine or reduce values as needed. Whether calculating sums, differences, or adjusting addresses, these operations are essential for nearly all computational tasks.

Multiplication and Division

Many arithmetic units are also designed to handle multiplication and division. While multiplication can often be implemented as repeated addition, modern AUs typically have dedicated circuits to perform these operations more efficiently. Division, especially with floating-point numbers, may be more complex and sometimes handled by a separate floating-point unit (FPU), but basic integer division is often part of the AU’s capabilities.

Increment and Decrement

The AU can quickly increase (increment) or decrease (decrement) a value by one. These simple operations are crucial for loops, counters, and pointer arithmetic, enabling efficient control flow and data management in programs.

Comparison Operations

Beyond pure arithmetic, the AU can compare two values to determine relational conditions such as equal to, greater than, or less than. These comparisons generate status flags that influence decision-making processes like branching and conditional execution in software.

Bitwise Shift Operations

The AU also performs bitwise shift operations, moving bits left or right within a binary number. Shifting is useful for multiplying or dividing by powers of two, data encoding, and optimising certain calculations.

3.3. Components of an Arithmetic Unit

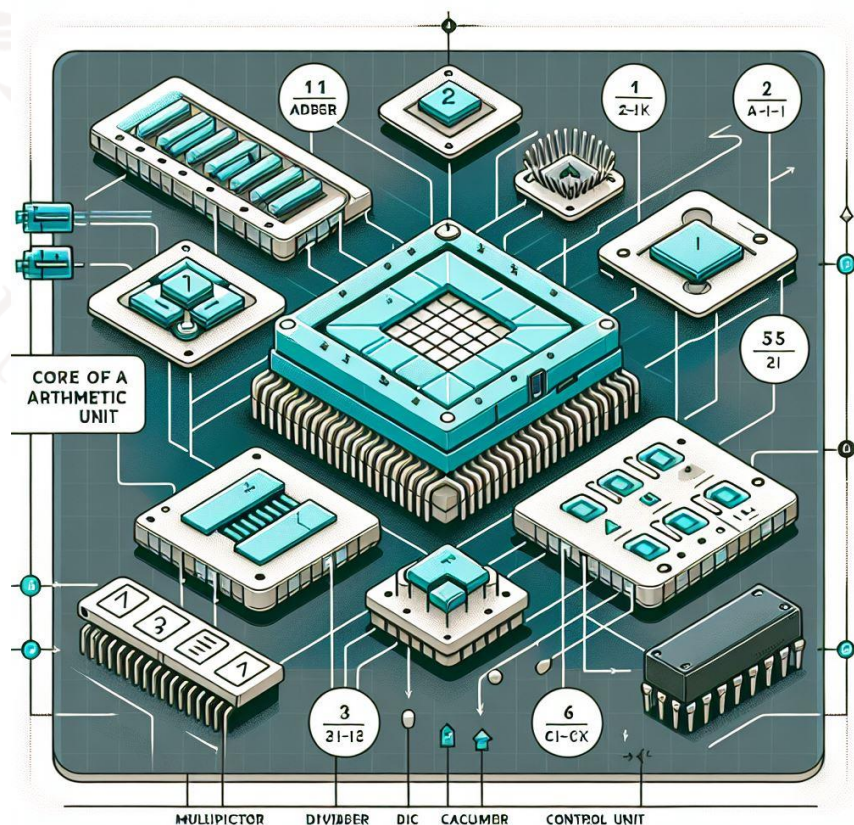


Figure 3: Components of AU

The Arithmetic Unit (AU) is a computer's processor powerhouse that performs all the essential mathematical operations. But what exactly makes up this incredible unit? Let's take a closer look at the key components that work together seamlessly to execute arithmetic tasks with lightning speed and precision.

Adders: The Heart of Calculation

At the core of the AU lies the adder, a digital circuit specially designed to add binary numbers. From simple half-adders, which add two single bits, to full-adders that handle carry bits, these circuits combine to form complex parallel adders capable of adding multi-bit numbers simultaneously. Addition is fundamental—not only for summing numbers but also as the basis for other arithmetic operations like subtraction and multiplication.

Subtractors: The Flip Side of Addition

Subtraction in the AU is cleverly handled using subtractors, often implemented through the concept of 2's complement arithmetic. Instead of building separate subtraction circuits, the AU transforms subtraction into addition by taking the complement of the number and adding it. This elegant technique simplifies hardware design and speeds up calculations.

Multipliers and Dividers: Tackling Complex Operations

While addition and subtraction are straightforward, multiplication and division are more complex and require dedicated components. Multipliers use repeated addition or advanced algorithms to quickly calculate products, while dividers perform iterative or algorithmic processes to find quotients and remainders. In many modern processors, these units are optimised for speed, enabling rapid execution of complex mathematical tasks.

Accumulator: The Temporary Workspace

The accumulator is a special register that temporarily holds intermediate results during multi-step calculations. Think of it as a workspace where the AU stores partial answers before finalising the result. This component is crucial for chaining operations efficiently, reducing the need to access slower memory locations repeatedly.

Registers: Fast Storage for Operands and Results

Within the AU, registers serve as small, ultra-fast storage units that hold the operands (input values) and the results of arithmetic operations. By keeping data close to the processing circuits, registers minimise delays and boost overall computational speed.

Control Circuits: The AU's Conductor

Just like a conductor leads an orchestra, the control circuits coordinate the activities inside the AU. They interpret instructions from the CPU's control unit, manage the flow of data between components, and ensure that the right operations happen at the right time. Without these control signals, the AU's components wouldn't work in harmony.

Status Flags / Condition Codes: The Outcome Indicators

After every arithmetic operation, the AU updates a set of status flags—special indicators that provide important information about the result. These include flags for zero (indicating a result of zero), carry (overflow beyond the number's capacity), overflow (when the result exceeds the range of representable numbers), and sign (positive or negative result). These flags are essential for decision-making in programs, influencing branching and conditional execution.

3.4. Working Principle

The Arithmetic Unit (AU) is a crucial part of a computer's processor that handles all the basic math operations like addition, subtraction, multiplication, and division. It works behind the scenes to make sure calculations are done quickly and accurately. Here's a step-by-step explanation of how the AU works in a simple way:

Receiving Inputs and Instructions

The AU starts by receiving two main things: the numbers it needs to work on (called operands) and instructions from the control unit. The control unit tells the AU which arithmetic operation to perform, such as adding two numbers or subtracting one from another.

Deciding Which Operation to Perform

Once the AU knows the instruction, it selects the right circuit inside itself to carry out the operation. For example, if it needs to add numbers, it activates the adder circuit; if it needs to subtract, it uses a method called two's complement to simplify the process.

Performing the Calculation

The AU then performs the actual calculation using digital circuits designed for arithmetic. For addition, it adds the binary numbers bit by bit. For subtraction, it converts the number to its two's complement and adds it. Multiplication and division are usually done through repeated addition or subtraction, or with special circuits in more advanced processors.

Storing the Result and Updating Flags

After the calculation is complete, the AU stores the result in a special register called the accumulator or another register inside the CPU. It also updates status flags — these are small indicators that tell the processor important information about the result, such as whether the result is zero, if there was an overflow, or if a carry was generated during addition.

Sending the Result for Further Use

Finally, the AU sends the result back to the CPU's registers or memory so that other parts of the computer can use it for further processing or storage. This entire process happens very quickly, often within a single clock cycle, allowing your computer to perform complex tasks efficiently.

SELF-ASSESSMENT QUESTIONS - 2

True or False:	
5	The Arithmetic Unit (AU) is responsible for performing all arithmetic operations on binary numbers within the CPU.
6	Status flags generated by the Arithmetic Unit are not important for program execution.
Fill in the Blanks:	
7	The _____ is a special register that temporarily holds intermediate results during multi-step calculations in the Arithmetic Unit.
8	Subtraction in the Arithmetic Unit is often implemented using _____ arithmetic, which simplifies hardware design and speeds up calculations.

4. INSTRUCTION HANDLING AREAS

At its core, a computer is a machine that follows instructions. These instructions are detailed “recipes” or “commands” that tell the computer exactly what to do. Instructions are the fundamental building blocks of computation, whether it’s adding two numbers, moving data from one place to another, or deciding which part of a program to run next.

These instructions are written in machine language, a binary code that the computer’s hardware understands directly. But how does a computer know which instruction to execute next? How does it understand what each instruction means? That’s where the process of instruction handling comes into play—a carefully orchestrated sequence that ensures every instruction is fetched, interpreted, and executed correctly.

4.1. The Anatomy of an Instruction

Imagine you’re following a recipe in a cookbook. Each step tells you what action to perform (like “chop” or “mix”) and what ingredients to use (like “onions” or “flour”). Similarly, every computer instruction has two main parts:

- **Opcode (Operation Code):** This is the “verb” of the instruction, telling the CPU what operation to perform. Examples include ADD, SUBTRACT, LOAD, STORE, or JUMP.
- **Operands:** These are the “nouns” or “ingredients” that the operation works on. They might be data values, memory addresses, or registers.

4.2 Different Instruction Formats

Instructions come in different shapes and sizes depending on the computer’s design and architecture. Some common formats include:

- **Three-address instructions:** These specify two source operands and one destination operand. For example, ADD A, B, C means add the contents of A and B, then store the result in C. This format is very expressive but requires more bits to encode.
- **Two-address instructions:** Here, one operand acts as both a source and destination, e.g., ADD A, B adds A and B, storing the result back in B.
- **One-address instructions:** Often used with an implicit accumulator register, e.g., ADD A adds the contents of A to the accumulator.

- **Zero-address instructions:** Used in stack-based architectures where operands are implicitly on top of the stack.

The choice of instruction format impacts the complexity of the CPU, the size of programs, and the ease of programming.

4.3 CPU Components Behind Instruction Handling

The CPU (Central Processing Unit) is the brain of your computer, and it relies on several key components to handle instructions efficiently:

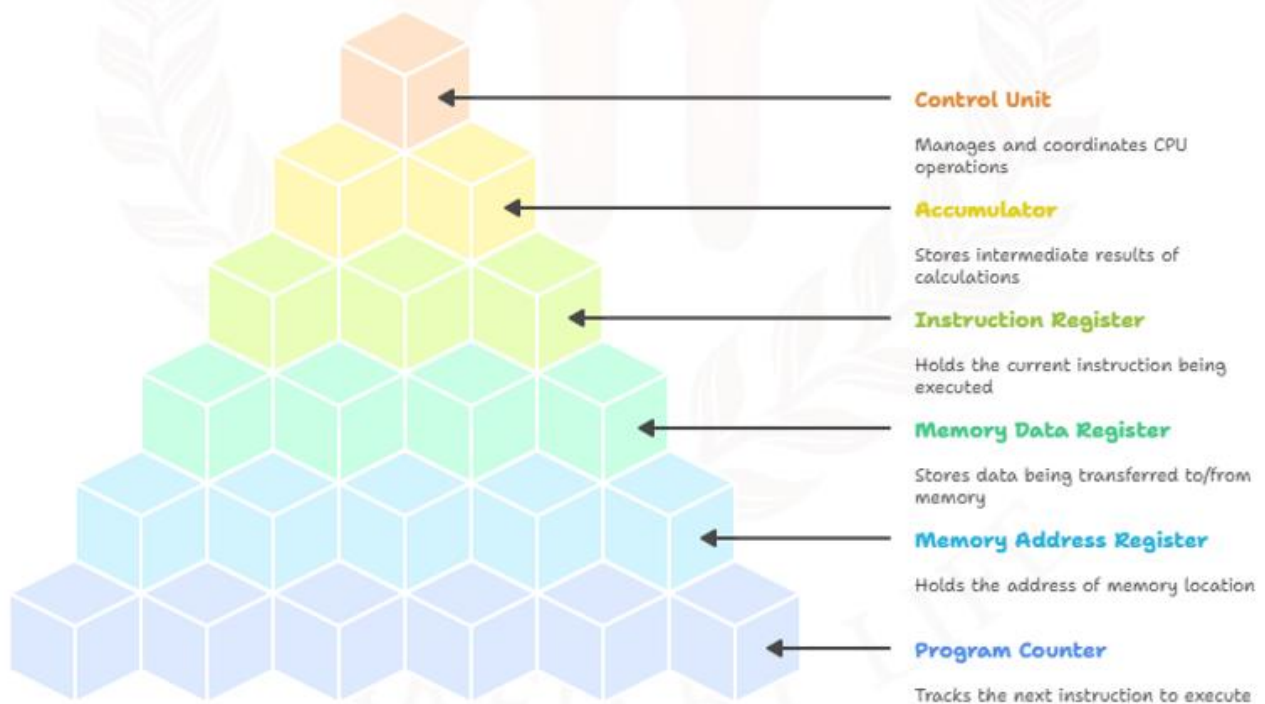


Figure 4: CPU Instruction Handling

- **Program Counter (PC):** Think of this as the CPU's bookmark, holding the address of the next instruction to fetch.
- **Memory Address Register (MAR):** Temporarily holds the memory address from which data or instructions are to be fetched or stored.
- **Memory Data Register (MDR):** Temporarily holds the data fetched from or to be written into memory.
- **Instruction Register (IR):** Holds the current instruction being decoded and executed.

- **Accumulator (ACC):** A special register that stores intermediate results, especially during arithmetic and logic operations.
- **Control Unit (CU):** The mastermind that decodes instructions and generates the control signals needed to coordinate all parts of the CPU.

Each of these components plays a crucial role, working in harmony to keep the instruction handling process smooth and efficient.

4.4. The Instruction Cycle

Instruction handling is not a one-time event—it's a continuous, rhythmic process called the fetch-decode-execute cycle. This cycle repeats over and over, billions of times per second, letting your computer perform complex tasks seamlessly.

Fetch

The cycle begins by fetching the next instruction from memory. The PC holds the address of this instruction. This address is copied to the MAR, which sends it to memory. The instruction stored at that memory location is fetched into the MDR, then transferred to the IR. Meanwhile, the PC is incremented to point to the next instruction, readying the CPU for the next fetch.

Decode

Once the instruction is in the IR, the Control Unit springs into action. It decodes the opcode to determine what operation needs to be performed and identifies any operands. This step is like reading and understanding a recipe before cooking.

Execute

Finally, the CPU executes the instruction. This might involve performing arithmetic or logic operations in the ALU (Arithmetic Logic Unit), moving data between registers and memory, or changing the flow of the program (like jumping to a new instruction). The results are stored as needed, and the cycle begins anew.

This elegant dance happens at incredible speeds, enabling your computer to multitask and run sophisticated software effortlessly.

4.5. Types of Instructions

A computer's instruction set is like a toolkit, packed with different types of instructions designed to handle every possible task:

- **Data transfer:** Instructions are responsible for moving data between memory and registers, using commands like LOAD, STORE, and MOVE.
- **Arithmetic instructions:** Handle mathematical operations such as addition, subtraction, multiplication, and division, allowing the CPU to perform calculations.
- **Logic instructions:** Manage bitwise operations like AND, OR, NOT, and XOR, which are crucial for decision-making and comparisons.
- **Control flow instructions:** alter the sequence of execution, enabling the implementation of loops, conditionals, and function calls through commands like JMP, BRANCH, CALL, and RET.
- **System instructions:** Oversee system-level tasks such as handling interrupts and managing input/output operations, using commands like INT, IN, OUT, and HLT.

Having a rich and well-designed instruction set allows the CPU to be versatile and powerful.

4.6. Addressing Modes

When an instruction needs data, it must know where to find it. This is where addressing modes come in—they define how the operands of an instruction are interpreted. Some common addressing modes include:

- **Immediate Addressing:** The operand is directly embedded in the instruction itself. For example, in LOAD #5, the value 5 is part of the instruction and is loaded directly into a register.
- **Direct Addressing:** The instruction contains the actual memory address of the operand. For instance, LOAD 1000 means the CPU will fetch the data stored at memory location 1000.
- **Indirect Addressing:** The instruction refers to a memory location that holds the address of the operand. So, LOAD (1000) means the CPU will go to address 1000, find another address there (say 2000), and then load the data from 2000.
- **Register Addressing:** The operand is located in a CPU register. For example, LOAD R1 means the data in register R1 is used directly.
- **Indexed Addressing:** This mode combines a base address with an offset (often stored in an index register) to access elements in arrays or tables. For example, LOAD BASE+INDEX allows flexible access to sequential data structures.

SELF-ASSESSMENT QUESTIONS - 3

True or False:	
9	The opcode in a computer instruction specifies the operation that the CPU should perform.
10	In the instruction cycle, the Program Counter (PC) holds the actual instruction that is currently being executed.
Fill in the Blanks:	
11	A _____ instruction format is commonly used in stack-based architectures, where operands are implicitly on top of the stack.
12	The _____ is the CPU component responsible for decoding instructions and generating control signals to coordinate the activities of the processor.

5. STACK

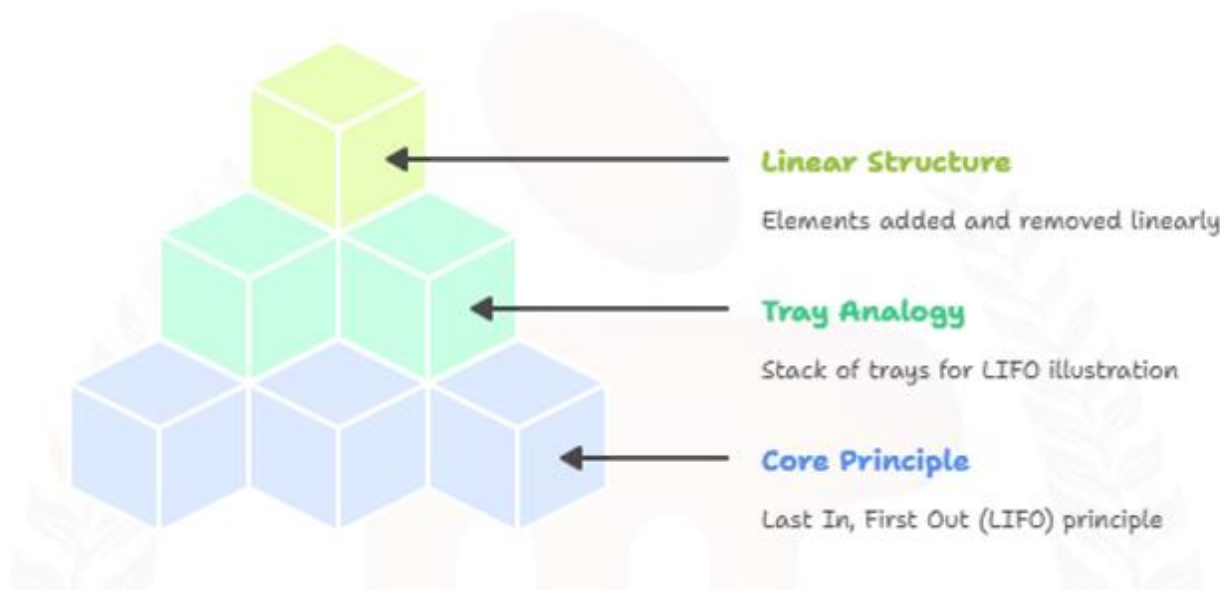


Figure 5: Stack data structure hierarchy

A stack is a linear data structure that follows the Last In, First Out (LIFO) principle. This means the last element added is the first one to be removed, just like a stack of trays in a cafeteria, where you can only take the top tray or add a new one on top.

Stacks are abstract data types, meaning their behaviour is defined by the operations you can perform on them, not by how they are implemented. The LIFO order ensures that data is accessed in a strict sequence, which is incredibly useful for a wide variety of computational tasks.

5.1. Core Operations of a Stack

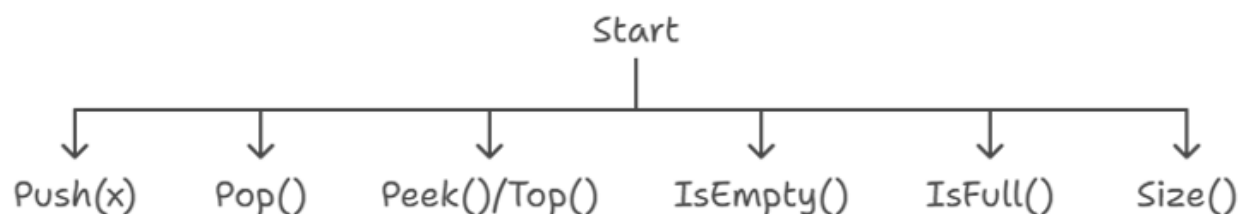


Figure 6: Stack Operations

A stack supports several fundamental operations:

- **Push(x)**: Add element x to the top of the stack.
- **Pop()**: Remove and return the element at the top of the stack.
- **Peek()/Top()**: Return the top element without removing it.
- **IsEmpty()**: Check if the stack is empty.
- **IsFull()**: Check if the stack is full (for fixed-size stacks).
- **Size()**: Return the number of elements in the stack.

These operations are designed to be efficient—typically $O(1)$ time complexity—making stacks fast and predictable.

5.2. Stack Implementation Techniques

Array-Based Stack

The simplest way to implement a stack is with an array and an integer top that tracks the index of the current top element. This method is fast and straightforward but requires you to know the maximum possible size in advance.

```
#include <iostream>
```

```
#define MAX 100 // Maximum size of the stack
```

```
class Stack {
```

```
private:
```

```
    int arr[MAX];
```

```
    int top;
```

```
public:
```

```
    Stack() {
```

```
        top = -1;
```

```
    }
```

```
// Push an element onto the stack
```

```
void push(int value) {
```

```
    if (top >= MAX - 1) {
```

```
        std::cout << "Stack Overflow\n";
```

```
        return;
```



```
}  
arr[++top] = value;  
std::cout << value << " pushed to stack\n";  
}  
  
// Pop an element from the stack  
void pop() {  
    if (top < 0) {  
        std::cout << "Stack Underflow\n";  
        return;  
    }  
    std::cout << arr[top--] << " popped from stack\n";  
}  
  
// Peek at the top element  
int peek() {  
    if (top < 0) {  
        std::cout << "Stack is Empty\n";  
        return -1;  
    }  
    return arr[top];  
}  
  
// Check if the stack is empty  
bool isEmpty() {  
    return top < 0;  
}  
  
// Display all elements  
void display() {  
    if (top < 0) {  
        std::cout << "Stack is Empty\n";  
        return;  
    }  
}
```

```
}  
std::cout << "Stack elements: ";  
for (int i = 0; i <= top; i++)  
    std::cout << arr[i] << " ";  
std::cout << "\n";  
}  
};  
  
int main() {  
    Stack s;  
    s.push(10);  
    s.push(20);  
    s.push(30);  
    s.display();  
    s.pop();  
    s.display();  
    std::cout << "Top element is: " << s.peek() << "\n";  
    return 0;  
}
```

Linked List-Based Stack

A linked list-based stack uses nodes where each node points to the next. This allows the stack to grow and shrink dynamically.

```
#include <iostream>  
using namespace std;
```

```
// Node structure for the linked list
```

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
// Stack class using linked list
```

```
class Stack {
private:
    Node* top;

public:
    Stack() {
        top = nullptr;
    }

    // Push operation
    void push(int value) {
        Node* newNode = new Node();
        newNode->data = value;
        newNode->next = top;
        top = newNode;
        cout << value << " pushed to stack\n";
    }

    // Pop operation
    void pop() {
        if (top == nullptr) {
            cout << "Stack Underflow\n";
            return;
        }
        Node* temp = top;
        cout << top->data << " popped from stack\n";
        top = top->next;
        delete temp;
    }

    // Peek operation
    int peek() {
        if (top == nullptr) {
```

```
        cout << "Stack is Empty\n";
        return -1;
    }
    return top->data;
}

// Check if stack is empty
bool isEmpty() {
    return top == nullptr;
}

// Display stack elements
void display() {
    if (top == nullptr) {
        cout << "Stack is Empty\n";
        return;
    }
    Node* temp = top;
    cout << "Stack elements: ";
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << "\n";
}

// Destructor to clean up memory
~Stack() {
    while (top != nullptr) {
        Node* temp = top;
        top = top->next;
        delete temp;
    }
}
```

```
}  
};  
  
// Main function to demonstrate stack operations  
int main() {  
    Stack s;  
    s.push(10);  
    s.push(20);  
    s.push(30);  
    s.display();  
    s.pop();  
    s.display();  
    cout << "Top element is: " << s.peek() << "\n";  
    return 0;  
}
```

Dynamic Arrays

Languages like Python and Java provide dynamic arrays (lists, ArrayLists) that can be used as stacks, automatically resizing as needed.

5.3. Stack Overflow and Underflow

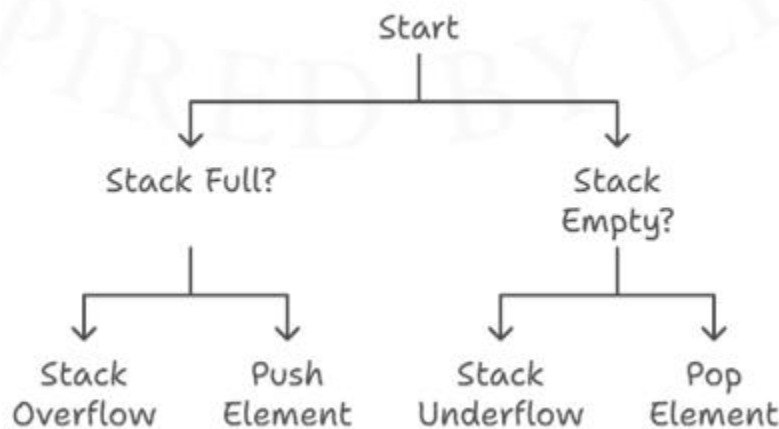


Figure 7: Stack Overflow and Underflow

- **Stack Overflow:** Occurs when you try to push an element onto a full stack. In an array-based stack, this means exceeding the allocated size.
- **Stack Underflow:** Happens when you try to pop from an empty stack.

Both are critical conditions that must be handled to prevent program crashes or unpredictable behaviour.

5.4. Real-World Analogies and Everyday Uses

Stacks aren't just a computer science concept—they're everywhere in daily life:

- **Plates in a Cafeteria:** You add and remove plates from the top.
- **Undo/Redo in Editors:** Each action is pushed onto an undo stack; undo pops the last action, redo pushes it onto another stack.
- **Browser History:** Each page visit is a push; clicking "Back" is a pop.

5.5. Applications of Stacks in Computing

Function Calls and the Call Stack

When a program calls a function, it needs to remember where to return after the function finishes. This is managed by the call stack. Each function call pushes a new frame onto the stack, containing the return address, parameters, and local variables. When the function returns, its frame is popped off, and execution resumes at the correct place.

Recursive function calls rely heavily on the call stack, as each recursive call pushes a new frame.

Expression Evaluation and Syntax Parsing

Stacks are essential for evaluating mathematical expressions, especially those in postfix (Reverse Polish Notation) or for converting infix expressions (like $A + B * C$) to postfix.

- **Balanced Parentheses:** Stacks help check if parentheses in expressions are balanced.
- **Syntax Parsing:** Compilers use stacks to parse and validate code syntax.

Backtracking Algorithms

In algorithms like maze solving, puzzles (e.g., Sudoku), and searching, stacks are used to remember previous states. If a path leads to a dead end, the algorithm pops back to the previous state and tries a new path.

Memory Management

Stacks are used to manage local variables and function calls in most programming languages. Each thread in a program typically has its own stack.

Depth-First Search (DFS)

DFS in graphs and trees uses a stack (explicitly or via recursion) to keep track of nodes to visit.

5.6. Stacks in Computer Architecture

The Stack Pointer and Stack Frames

In hardware, stacks are managed using a stack pointer (SP) register that points to the top of the stack in memory. When a value is pushed, the SP is adjusted (usually decremented in downward-growing stacks) and the value is written to the new SP location. When a value is popped, the SP is incremented, and the value is read.

Each function call creates a stack frame containing the return address, parameters, and local variables. Stack frames are pushed and popped as functions are called and return.

Stack Direction

Some architectures have stacks that grow towards higher memory addresses (upwards), while others grow towards lower addresses (downwards). The choice affects how the stack pointer is updated and how memory is managed.

Hardware Support for Stacks

Some CPUs have dedicated instructions for stack operations, such as PUSH, POP, CALL, and RET. These instructions automatically adjust the stack pointer and manage stack frames.

Security and Stack Overflow

Stack overflows can be exploited by attackers to overwrite return addresses and inject malicious code (buffer overflow attacks). Modern CPUs and operating systems use techniques like stack canaries and non-executable stacks, as well as address space layout randomisation (ASLR), to mitigate these risks.

SELF-ASSESSMENT QUESTIONS - 4

True or False:	
13	A stack follows the Last In, First Out (LIFO) principle, meaning the last element added is the first one to be removed.
14	In a stack, both push and pop operations can occur at any position within the stack.
Fill in the Blanks:	
15	The operation that removes and returns the top element of a stack is called _____.
16	A stack overflow occurs when you try to push an element onto a stack that is already _____.



6. SUMMARY

The Central Processing Unit (CPU) is the powerhouse of any computer, orchestrating the execution of instructions and facilitating seamless data processing. Its internal architecture is a marvel of engineering, integrating specialised components that work in harmony to deliver high-speed computation.

Registers, acting as the CPU's ultra-fast scratchpad, enable immediate data access, which is crucial for maintaining rapid workflow and minimising delays. The arithmetic unit, embedded within the processor, is dedicated to performing essential mathematical and logical operations, serving as the computational engine for all tasks.

Instruction management mechanisms ensure that programs are executed in a logical, ordered manner, while stacks play a pivotal role in managing memory for complex program flows, such as function calls and returns.

Modern CPUs are distinguished by their ability to process large volumes of data simultaneously, enabled by advances in register size and design. This evolution has transformed computing, allowing for more sophisticated applications and greater efficiency. The interplay between memory, control units, and processing elements forms the backbone of computer architecture, shaping how software and hardware interact.

Understanding these core principles is fundamental for grasping how computers achieve their remarkable speed, reliability, and versatility in an ever-evolving technological landscape.

7. GLOSSARY

Arithmetic Logic Unit (ALU)	- A core component of the CPU responsible for performing arithmetic (addition, subtraction, etc.) and logical (AND, OR, NOT, XOR) operations.
Arithmetic Unit (AU)	- A subcomponent of the ALU dedicated specifically to performing mathematical calculations such as addition, subtraction, multiplication, and division.
Bit	- The smallest unit of data in computing, representing a binary value of 0 or 1.
Cache Memory	- A small, high-speed memory layer between the CPU and main memory that stores frequently accessed data for quicker retrieval.
Central Processing Unit (CPU)	- The main processing hub of a computer, often called the “brain,” responsible for executing instructions and managing all operations.
Control Unit (CU)	- The part of the CPU that interprets instructions from memory and directs the operation of the processor’s components.
General Purpose Registers	- Small, fast storage locations within the CPU used for temporarily holding data, addresses, or intermediate results during processing.
Instruction Register (IR)	- A register that holds the current instruction being decoded and executed by the CPU.
Main Memory (RAM)	- Volatile memory used to temporarily store programs and data that are currently in use by the computer.
MAR (Memory Address Register):	- A register that holds the address in main memory where data is to be read from or written to.
MDR (Memory Data Register)	- A register that temporarily stores data being transferred to or from main memory.
Program Counter (PC)	- A register that keeps track of the address of the next instruction to be executed by the CPU.

Register Size	- The number of bits a register can hold, determining how much data the CPU can process at one time (e.g., 8-bit, 16-bit, 32-bit, 64-bit).
Registers	- Tiny, ultra-fast memory units inside the CPU used for immediate data access, storing data, addresses, and instructions temporarily.
Stack	- A memory structure used to manage function calls, local variables, and return addresses, supporting complex program execution flows.
Status Flags	- Indicators set by the CPU (such as zero, carry, or overflow) to signal the outcome of operations and guide subsequent actions.

8. TERMINAL QUESTIONS

1. What is the primary function of CPU registers?
2. Explain how address registers assist the CPU.
3. Explain the role of the Memory Address Register (MAR).
4. Why is the Memory Data Register (MDR) important?
5. Identify the role of the Program Counter (PC) during instruction execution.
6. Describe the function of the Instruction Register (IR).
7. Explain the role of general-purpose registers.
8. Define how register size impacts CPU performance.
9. Explain the main responsibility of the Arithmetic Logic Unit (ALU).
10. What is the role of the Arithmetic Unit (AU) for CPU operations?
11. What status flags does the Arithmetic Unit generate, and why are they important?
12. Explain the evolution of the Arithmetic Unit in improving computing.
13. Define adders and subtractors in the context of the Arithmetic Unit.
14. List the comparison operations and their importance in the Arithmetic Unit.

9. ANSWERS

Self-Assessment Questions

1. False
2. True
3. Memory Data Register (MDR)
4. word size
5. True
6. False
7. accumulator
8. two's complement
9. True
10. False
11. zero-address
12. Control Unit (CU)
13. True
14. False
15. pop
16. full

Terminal Questions Answers

1. CPU registers serve as ultra-fast, temporary storage locations within the processor, holding data, addresses, and instructions for immediate access during processing.
Refer to Section 2 for more details.
2. Address registers act like a GPS for the CPU, pointing to the exact spot in memory where data is stored or needs to be accessed.
Refer to Section 2 for more details.
3. The MAR holds the address in main memory where data is to be read from or written to, enabling accurate memory navigation.
Refer to Section 2 for more details.
4. The MDR temporarily stores data being transferred to or from main memory, acting as a buffer during data movement.

Refer to Section 2 for more details.

5. The PC keeps track of the address of the next instruction to be executed, ensuring the CPU processes instructions in the correct order.

Refer to Section 2 for more details.

6. The IR holds the current instruction that the CPU is decoding and executing, allowing the control unit to interpret and act on it.

Refer to Section 2 for more details.

7. General-purpose registers temporarily store data and intermediate results during processing, enabling quick data manipulation without accessing main memory.

Refer to Section 2 for more details.

8. Register size, measured in bits, determines how much data the CPU can process at one time, influencing processing power and memory addressing capability.

Refer to Section 2.2 for more details.

9. The ALU performs all arithmetic (addition, subtraction, multiplication, division) and logical (AND, OR, NOT, XOR) operations required by the CPU.

Refer to Section 2 and Section 3 for more details.

10. The AU executes numerical calculations such as addition, subtraction, multiplication, and division, forming the mathematical engine of the CPU.

Refer to Section 3 for more details.

11. The AU generates status flags like carry, zero, and overflow, which help the CPU make decisions during program execution.

Refer to Section 3 for more details.

12. Advances in integrated circuits and microprocessors made arithmetic units smaller, faster, and more capable, enabling modern computers to perform complex operations efficiently.

Refer to Section 3.1 for more details.

13. Adders are digital circuits that add binary numbers, while subtractors often use 2's complement arithmetic to transform subtraction into addition, simplifying hardware design.

Refer to Section 3.3 for more details.

14. Comparison operations allow the AU to determine relational conditions (equal, greater, less), generating status flags that influence branching and conditional execution in software.

Refer to Section 3.2 for more details.

10. REFERENCES

1. Wright, G., & Hanna, K. T. (2025, May 16). What is an arithmetic logic unit (ALU) and how does it work? WhatIs. <https://www.techtarget.com/whatis/definition/arithmetic-logic-unit-ALU>
2. Hennessy, J. L., & Patterson, D. A. (2017). Computer Architecture: A Quantitative Approach. Morgan Kaufmann.
3. Patterson, D. A., & Hennessy, J. L. (2012). Computer organization and design: the hardware/software interface, 4th ed. (Rev. ed. of: Computer organization and design / John L. Hennessy, David A. Patterson. 1998.). In Elsevier eBooks.
<http://dspace.uniten.edu.my/handle/123456789/14251>
4. (PDF) Computer Architecture Sixth Edition a quantitative approach PDFDrive com | Saad Farooq - academia.edu. (n.d.-a).
https://www.academia.edu/44613414/computer_architecture_sixth_edition_a_quantitative_approach_pdfdrive_com

BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105
COMPUTER ORGANIZATION AND
ARCHITECTURE



Unit – 3

Micro Operations

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Register Transfer	1, 2, 3	1	6 - 13
2.1	Symbolic Notation and Syntax	-	-	
2.2	Key Concepts of Register Transfer Language (RTL)	-	-	
2.3	Register Transfer in Digital Design	-	-	
2.4	Basic symbols of RTL	-	-	
2.5	Register Transfer Operations	-	-	
2.6	Advantages & Disadvantages	-	-	
3	Bus and Memory Transfer	-	2	14 - 21
3.1	Bus Transfer	-	-	
3.2	Memory Transfer	-	-	
4	Arithmetic Micro Operations	-	3	22 - 29
4.1	Types of Arithmetic Micro Operations	-	-	
4.2	Rules for Arithmetic Micro Operations	-	-	
5	Summary	-	-	30
6	Glossary	-	-	31
7	Terminal Questions	-	-	32
8	Answers	-	-	33 - 34
9	References	-	-	35

1. INTRODUCTION

In the previous unit, you learned about the basic structure and function of the Central Processing Unit (CPU). In this unit, we explore its internal components in more detail. Registers are small, high-speed storage units within the CPU that temporarily hold data, instructions, and addresses during processing. The Arithmetic Logic Unit (ALU) performs essential mathematical and logical operations, enabling the CPU to carry out computations. Instruction handling areas manage the flow of instructions through the fetch-decode-execute cycle, ensuring that each command is processed efficiently. Additionally, stacks are used to manage function calls and local variables, providing a structured way to store and retrieve data during program execution. Together, these components form the core of the CPU's processing capabilities.

In this unit, you will learn the term micro operations, which refers to the most basic operations performed within a computer's central processing unit (CPU) that manipulate data stored in registers. Micro operations are essential building blocks that enable the execution of complex machine instructions by breaking them down into simpler, manageable steps. Each micro operation involves elementary tasks such as transferring data between registers, performing arithmetic calculations, or executing logical operations. These operations are controlled by the CPU's control unit, which sequences them to implement higher-level instructions efficiently. Understanding micro operations is fundamental to grasping how a computer processes instructions at the hardware level.

There are several key types of micro operations. Register transfer micro operations involve moving binary data from one register to another without altering the data itself. These transfers are crucial for internal data handling and are described using Register Transfer Language (RTL), a symbolic notation that represents data flow between registers. Bus and memory transfer micro operations manage data movement between registers and memory or across a shared communication pathway called a bus. These operations are vital for fetching instructions and data from memory and storing results back into memory. Lastly, arithmetic micro operations perform basic calculations such as addition, subtraction, increment, and decrement using arithmetic circuits like adders and subtractors.

You will learn how these micro operations work individually and together to execute instructions within a CPU. You will explore the hardware mechanisms that enable data transfers and arithmetic processing, including the use of multiplexers, logic gates, and control signals. Additionally, you will understand how micro operations contribute to instruction cycles—fetch, decode, and execute—and how they facilitate efficient and precise control of the CPU's internal processes. By mastering micro

1.1. Objectives

- Define micro-operations in computer architecture.
- Explain register transfer, bus, and memory transfer micro-operations.
- Outline the arithmetic micro-operations and their implementation.
- Explain Register Transfer Language (RTL) to represent data movement.
- Discuss how the control unit sequences and manages micro-operations



2. REGISTER TRANSFER

Register Transfer refers to the process of moving data from one register to another within a digital computer system. Registers are small, fast storage elements inside the CPU that temporarily hold data and instructions during processing. Register transfer operations are fundamental to the internal functioning of a computer, as they enable the CPU to manipulate and route data efficiently between its various components.

Computer architects use Register Transfer Language (RTL) to describe these operations clearly and concisely. RTL is a symbolic notation that specifies how data moves between registers and what operations are performed on that data. For example, an RTL statement can represent the action of adding the contents of two registers and storing the result in a third register. This standardised approach makes it easier to design, analyse, and communicate the internal operations of digital systems.

2.1 Symbolic Notation and Syntax

Registers are usually denoted by uppercase letters, sometimes with subscripts or numbers to distinguish between different registers (e.g., R1, R2, PC for Program Counter, MAR for Memory Address Register). The basic syntax for data transfer is:

$$R2 \leftarrow R1$$

This means the contents of register R1 are transferred to register R2, replacing whatever was previously stored in R2.

Microoperation Notation

Microoperations are expressed using the left-arrow symbol ($\leftarrow$) to indicate the direction of data movement. Multiple microoperations can be listed on a single line, separated by commas, to indicate that they occur simultaneously:

$$R3 \leftarrow R1 + R2, R1 \leftarrow R1 + 1$$

This means R3 receives the sum of R1 and R2, and R1 is incremented by 1, both in the same clock cycle.

Control Functions

Conditional transfers are specified using a control signal followed by a colon:

P: R2 $\leftarrow$ R1

This means the transfer from R1 to R2 occurs only if the control signal P is true (logic high). The colon separates the control condition from the operation, making the execution dependent on the control signal.

2.2 Key Concepts of Register Transfer Language (RTL)

- Register Transfer Language (RTL) provides a clear and symbolic way to describe the flow of data and microoperations between registers within a digital system.
- RTL is primarily used to model and represent synchronous circuits controlled by clock signals by specifying how data moves and what operations are performed at the register level.
- It allows designers to define data transfers and logical or arithmetic operations at the register level, making it easier to specify, analyse, and verify digital hardware behaviour before physical implementation.
- RTL descriptions are commonly written using hardware description languages (HDLs) such as Verilog or VHDL, serving as a bridge between high-level system specifications and the actual physical devices.
- By abstracting the design to the register-transfer level, RTL enables efficient documentation, simulation, and optimisation of complex digital systems, facilitating the transition from conceptual design to hardware realisation.

2.3 Register Transfer in Digital Design

- Describes the nature and behaviour of hardware at the register-transfer level, focusing on data flow between registers and the operations performed on that data.
- Was historically used to model data flow using registers and combinational logic, serving as a bridge between high-level algorithmic descriptions and physical hardware implementation.

- Facilitates the translation of high-level hardware designs to the gate level, enabling synthesis and optimisation for digital circuits.
- Allows simulation and validation of hardware behaviour, supporting cycle-accurate testing and debugging before physical implementation.
- Forms the foundation for digital circuit development using hardware description languages (HDLs) like Verilog and VHDL.

2.4 Basic symbols of RTL

Table 1: Basic symbols of RTL

Symbol	Description	Example
Letters and Numbers	Denotes a Register	MAR, R1, R2
()	Denotes a part of the register	R1(8-bit) R1(0-7)
<-	Denotes a transfer of information	R2 <- R1

,	Specify two micro-operations of Register Transfer	$R1 \leftarrow R2$ $R2 \leftarrow R1$
:	Denotes conditional operations	$P : R2 \leftarrow R1$ if $P=1$
Naming Operator ($:=$)	Denotes another name for an already existing register/alias	$Ra := R1$

2.5 Register Transfer Operations

There are different types of register transfer operations:

Simple Transfer - $R2 \leftarrow R1$

simple transfer operation, denoted as $R2 \leftarrow R1$, where the contents of register $R1$ are copied into register $R2$. Importantly, this process does not alter the original value in $R1$, ensuring that the source register remains unchanged after the transfer. Such a transfer is unconditional, meaning it occurs without any prerequisite conditions or checks. Register transfer operations like these are essential for coordinating data movement and processing within various digital systems, providing the foundation for more complex computational tasks.

Conditional Transfer

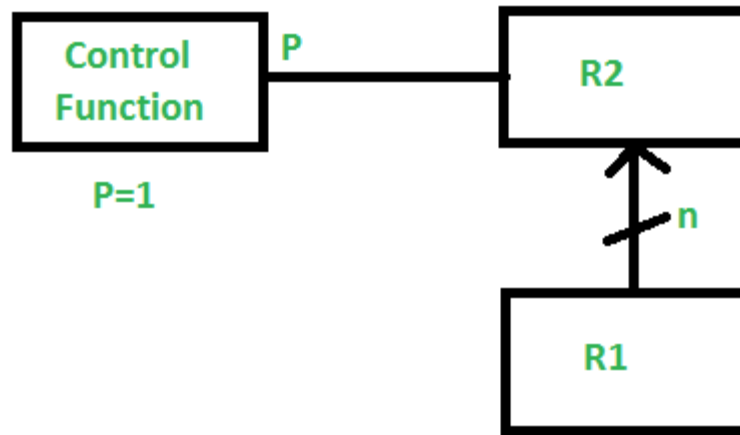
$$P : R2 \leftarrow R1$$

$$n = \text{no of bits}$$

Figure 3-1 Conditional Transfer

Figure 3-1 indicates that if $P=1$, then the content of $R1$ is transferred to $R2$. It is a unidirectional operation.

Simultaneous Operations:

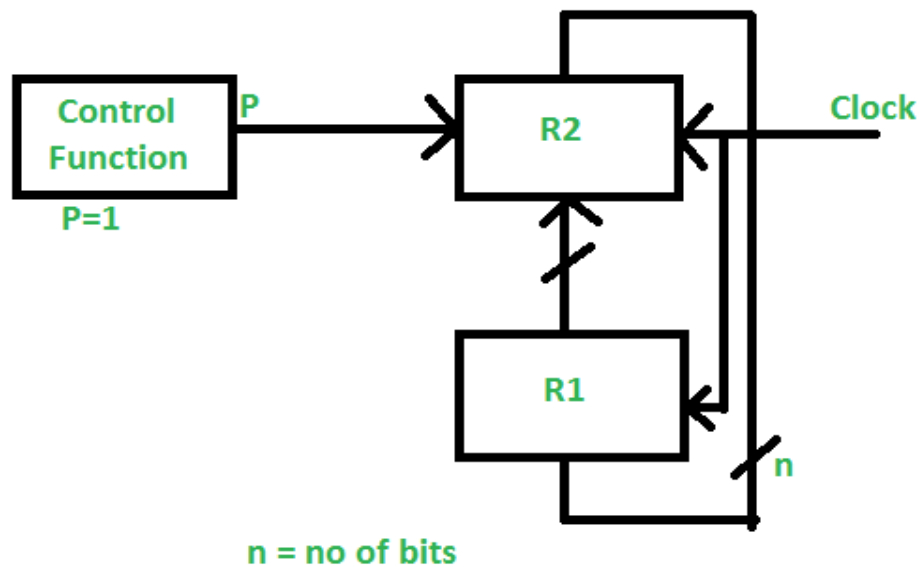
$$P : R2 \leftarrow R1, R1 \leftarrow R2$$

$$n = \text{no of bits}$$

Figure 3-2 Simultaneous Operations

In Figure 3-2, If 2 or more operations are to occur simultaneously then they are separated with comma (,).

If the control function $P=1$, then load the content of R1 into R2 and at the same clock, load the content of R2 into R1.

2.6 Advantages & Disadvantages

Advantages

- **Enables efficient and systematic hardware design:** RTL provides a structured way to describe the flow of data and operations within a digital system, allowing designers to create hardware that is both efficient and well-organised.
- **Allows early simulation and detection of design errors:** By modelling the system at the register transfer level, designers can simulate the design before actual hardware implementation. This helps in identifying and correcting errors early in the development process, reducing the risk of costly revisions.
- **Bridges conceptual descriptions and gate-level hardware implementation:** RTL acts as an intermediary between high-level architectural concepts and detailed gate-level hardware, making it easier to translate abstract ideas into concrete hardware components.
- **Facilitates reuse of design components and modules:** RTL descriptions are often modular, allowing designers to reuse proven components and modules across different projects, which saves time and ensures consistency in hardware design.
- **Provides a clear framework for conducting timing analysis on digital systems:** Since RTL explicitly defines the sequence and timing of data transfers and operations, it offers a solid foundation for performing accurate timing analysis and ensuring that the system meets its performance requirements.

Disadvantages

- **Debugging RTL designs can be challenging and time-consuming:** Errors at the register transfer level can be difficult to trace and fix, often requiring a deep understanding of both the design and the underlying hardware operations.
- **Poorly optimised RTL may lead to inefficient and unnecessarily large hardware constructions:** If RTL code is not carefully written and optimised, it can result in hardware

that uses more resources than necessary, leading to increased area, power consumption, and potentially lower performance.

- **Hardware-oriented reasoning is required, making it less accessible for those without hardware knowledge:** RTL is inherently focused on hardware concepts, which can make it challenging for individuals with primarily software backgrounds to fully understand and utilise.
- **Synthesis results are dependent on the capabilities and quality of specific design tools:** The effectiveness of converting RTL descriptions into actual hardware can vary depending on the synthesis tools used. Different tools may produce different results, impacting performance and efficiency.
- **Offers a low level of abstraction compared to high-level design methods, making large designs harder to manage:** RTL operates at a lower abstraction level than many modern design methodologies, which can make managing and understanding complex or large-scale systems more

SELF-ASSESSMENT QUESTIONS - 1

True or False:	
1	Register Transfer Language (RTL) is used to describe asynchronous circuits in digital systems.
Fill in the Blanks:	
2	The symbol ____ in RTL denotes a conditional transfer operation.
Multiple Choice Questions	
3	What does the RTL statement $R3 \leftarrow R1 + R2$, $R1 \leftarrow R1 + 1$ indicate? a) Sequential addition and increment b) Simultaneous addition and increment c) Conditional transfer only d) Memory transfer operation
True or False:	
4	A simple register transfer operation alters the contents of the source register.
Fill in the Blanks:	

5	In RTL, the notation $P: R2 \leftarrow R1$ means the transfer occurs only if the control signal _____ is true.
---	--



3. BUS AND MEMORY TRANSFER

3.1 Bus Transfer

A bus in digital systems is a group of parallel wires or lines that can carry multiple bits of data simultaneously. In the context of micro operations, a bus acts as a shared data highway that connects different registers within the CPU. The primary advantage of using a bus is that it eliminates the need for direct, point-to-point wiring between every pair of registers, which would be highly complex and impractical as the number of registers increases.

For example, if there are n registers and each register must communicate with every other register, direct connections would require $n(n-1)$ data lines. This quickly becomes unmanageable as n grows. A common bus structure allows all registers to connect to the same set of lines, greatly simplifying the hardware design and making the system more scalable and maintainable.

General Syntax

The general syntax for a bus transfer micro operation is:

Destination Register $\leftarrow$ Source Register

This means:

The contents of the Source Register are transferred to the Destination Register using the bus.

Examples:

Register-to-Register Transfer

$R2 \leftarrow R1$

This means the contents of register R1 are transferred to register R2 via the bus.

Register-to-Memory Transfer

$M[AR] \leftarrow DR$

This means the contents of the Data Register (DR) are transferred to the memory location specified by the Address Register (AR).

Memory-to-Register Transfer

$DR \leftarrow M[AR]$

This means the contents of the memory location specified by AR are transferred to the Data Register (DR) via the bus.

Explanation

- The arrow ($\leftarrow$) indicates the direction of data flow.
- The left side of the arrow is the destination (where data is loaded).
- The right side of the arrow is the source (where data comes from).
- The bus is implied in the operation; it is not always explicitly mentioned in the syntax.

3.1.1 Implementation of Bus Transfers

Using Multiplexers

One popular method for implementing a bus is to use multiplexers. Suppose there are four registers—A, B, C, and D—each 4 bits wide. Each bit line of the bus is connected to a 4-to-1 multiplexer. The select lines of the multiplexers determine which register's output is placed onto the bus.

For example:

- If the select lines are set to 10, then the contents of register C are placed onto the bus.
- The register that is to receive the data (say, register D) has its load signal activated so it can copy the data from the bus.

This approach is efficient and easy to expand if more registers are added to the system.

Using Tri-State Buffers

Another common approach is to use tri-state buffers. Each register output is connected to the bus through a tri-state buffer, which can be in one of three states: logic 0, logic 1, or high impedance (effectively disconnected). Only one register's buffer is enabled at a time, ensuring that only one register drives the bus and preventing data conflicts.

For instance, if register B needs to transfer its data to register A, the control unit enables the tri-state buffer for register B, placing its data on the bus. Simultaneously, the load signal for register A is activated, allowing it to read the data from the bus.

Step-by-Step Example of Bus Transfer Micro Operation

Suppose we want to transfer the contents of register C to register D:

1. **Enable Register C:** The control unit activates the select lines (in the case of multiplexers) or enables the tri-state buffer for register C, placing its data onto the bus.
2. **Load Register D:** The load signal for register D is activated, allowing it to copy the data from the bus.
3. **Result:** Register D now contains the data that was in register C, while register C remains unchanged.

Symbolically:

$D \leftarrow C$

This operation is performed via the common bus, even though the bus is not explicitly mentioned.

Concrete Example with Binary Data

Let's say:

- Register C contains 1101 (binary)
- Register D contains 0000 (binary)

After the bus transfer operation $D \leftarrow C$:

- Register D will contain 1101
- Register C remains at 1101

3.1.2 Control of Bus Transfers

The control unit of the CPU is responsible for generating the necessary control signals to manage bus transfers. These include:

- Select signals for multiplexers or enable signals for tri-state buffers to choose the source register.
- Load signals for the destination register to capture the data from the bus.
- Timing signals to synchronise the transfer, usually with the system clock.

Proper timing is crucial. The data must be placed on the bus before the destination register loads it, and only one register should drive the bus at any moment to avoid bus contention.

3.1.3 Advantages of Bus Transfer Micro Operations

- **Reduced Hardware Complexity:** Using a common bus reduces the number of wires and connections needed between registers, making the design simpler and more cost-effective.
- **Scalability:** Adding more registers is straightforward; each new register simply connects to the existing bus.

- **Flexibility:** Any register can transfer data to any other register, provided the correct control signals are used.
- **Efficient Data Movement:** Bus transfers allow for quick and organised data movement, which is essential for instruction execution.
- **Ease of Maintenance:** A bus-based system is easier to troubleshoot and expand compared to a system with direct connections between every register.

3.1.4 Applications of Bus Transfer Micro Operations

Bus transfer micro operations are not limited to register-to-register transfers. They are also used for:

- **Register to Memory Transfers:** Moving data from a register to a memory location via the bus.
- **Memory to Register Transfers:** Loading data from memory into a register through the bus.
- **I/O Transfers:** Communicating with input/output devices using the bus as a shared pathway.

For example, to load data from memory into a register:

$$DR \leftarrow M[AR]$$

Here, the data from the memory location addressed by the address register (AR) is placed on the bus and loaded into the data register (DR).

3.2 Memory Transfer

Memory transfer refers to the process by which binary information is either retrieved (read) from memory or stored (written) into memory. In digital systems, memory is structured as an array of storage locations, each uniquely identified by an address. Accessing or modifying data in memory requires the use of two special-purpose registers:

- **Address Register (AR or MAR):** This register holds the address of the memory location to be accessed.
- **Data Register (DR or MDR):** This register temporarily holds the data being transferred to or from memory.

The transfer of data between these registers and memory is governed by specific control signals and forms a fundamental part of instruction execution.

3.2.1 Types of Memory Transfer Operations

1. Memory Read Operation

A memory read operation involves transferring data from a specified memory location into a register, typically the Data Register (DR). The sequence for a read operation is as follows:

1. The address of the required memory location is loaded into the Address Register (AR).
2. A read control signal is issued to the memory unit.
3. The memory unit places the contents of the specified address onto the data bus.
4. The data is then loaded into the Data Register (DR).

Syntax:

$$DR \leftarrow M[AR]$$

This notation indicates that the contents of the memory location specified by AR are transferred to DR.

Example:

Suppose the address register contains the value 1050. The operation to read the contents of memory location 1050 into the data register is symbolically written as:

$$DR \leftarrow M[1050]$$

This means the data stored at memory address 1050 is loaded into the data register.

2. Memory Write Operation

A memory write operation transfers data from a register (usually DR or another data register) into a specified memory location:

1. The address of the target memory location is loaded into the Address Register (AR).
2. The data to be written is placed in the Data Register (DR).
3. A write control signal is issued to the memory.
4. The contents of DR are written into the memory location specified by AR.

Syntax:

$$M[AR] \leftarrow DR$$

This notation signifies that the contents of DR are written into the memory location specified by AR.

Example:

Suppose the address register contains the value 2050, and the data register holds the data to be written. The operation to write the contents of the data register into memory location 2050 is symbolically written as:

$$M[2050] \leftarrow DR$$

This means the data in the data register is stored at memory address 2050.

3.2.2 Hardware Implementation of Memory Transfer

The hardware required for memory transfer consists of several key components:

- Address Register (AR): Supplies the address to the memory unit.
- Data Register (DR): Holds the data being read from or written to memory.
- Control Logic: Generates the read and write control signals.
- Data Bus: Transfers data between memory and registers.

Read Operation Example:

- The address is loaded into AR.
- The read control signal is activated.
- The memory places the data from the specified address onto the data bus.
- The data is loaded into DR on the next clock pulse.

Write Operation Example:

- The address is loaded into AR.
- The data to be written is placed into DR.
- The write control signal is activated.
- The data from DR is stored in the memory location specified by AR.

3.2.3 Additional Syntax Examples

Here are further examples of memory transfer syntax and their meanings:

Table 2: Examples of some additional syntax

$DR \leftarrow M[AR]$	Read data from the memory address in AR into DR
$M[AR] \leftarrow DR$	Write data from DR into the memory address in AR
$DR \leftarrow M[2500]$	Read data from memory address 2500 into DR
$M[3000] \leftarrow DR$	Write data from DR into memory address 3000

Illustrative Scenario:

Suppose a CPU must fetch an instruction from memory address 4000:

$IR \leftarrow M[4000]$

Here, the instruction at address 4000 is loaded into the Instruction Register (IR).

3.2.4 Importance and Applications

Memory transfer operations are crucial for several reasons:

- **Instruction Fetching:** Retrieving instructions from memory into the instruction register.
- **Data Manipulation:** Moving operands between memory and registers for processing.
- **Storing Results:** Writing computation results back to memory.
- **I/O Operations:** Facilitating data exchange between memory and input/output devices.

Efficient memory transfer is essential for system performance, as it directly affects how quickly the CPU can fetch instructions and data, and how fast results can be stored or retrieved.

Memory transfer micro operations are fundamental to the operation of digital computers. They enable the CPU to interact with memory, fetching instructions, retrieving and storing data, and supporting all higher-level operations. By using registers such as AR and DR, along with control signals and buses, memory transfers are executed efficiently, ensuring the smooth functioning of the entire system. A thorough understanding of these operations, including their syntax and practical examples, is essential for anyone studying computer architecture and organisation, as they form the foundation for more advanced computational processes.

SELF-ASSESSMENT QUESTIONS - 2

True or False:	
6	A common bus requires $n(n-1)$ data lines to connect n registers
Fill in the Blanks:	
7	The RTL syntax ____ represents a memory read operation from the address in AR to DR.
Multiple Choice Questions	
8	Which component selects the source register's data in a bus transfer using multiplexers? a) Control unit b) Tri-state buffer c) Select lines d) Data register
True or False:	
9	Tri-state buffers allow multiple registers to drive the bus simultaneously without conflicts
Fill in the Blanks:	
10	In a memory write operation, the ____ register holds the address of the target memory location.

4. ARITHMETIC MICRO OPERATIONS

In the study of computer organisation and architecture, micro operations are the most fundamental actions performed on the data stored within the registers of a computer's central processing unit (CPU). These operations are the building blocks of all instruction execution at the hardware level. Micro operations are typically classified into four main categories: register transfer, arithmetic, logic, and shift micro operations. Among these, arithmetic micro operations are paramount, as they provide the essential mathematical functions required for all forms of data processing, both simple and complex.

Arithmetic micro operations are defined as the basic operations that perform arithmetic calculations—such as addition, subtraction, increment, decrement, and complement—on binary data stored in registers. These operations are at the heart of all computational processes in a digital computer.

The significance of arithmetic micro operations lies in their ubiquity and necessity:

- They enable the execution of arithmetic instructions such as ADD, SUB, INC, and DEC, which are fundamental to all computational tasks.
- They are indispensable for data manipulation, address calculation, loop control, and program flow.
- They directly impact the performance, efficiency, and capability of the CPU, as most computational activities ultimately rely on arithmetic processing.
- They serve as the basis for more complex operations, including multiplication, division, and floating-point arithmetic, which are constructed from sequences of basic arithmetic micro operations.

4.1 Types of Arithmetic Micro Operations

Arithmetic micro operations can be broadly categorised as follows:

1. Addition

Addition involves summing the contents of two registers and storing the result in a destination register.

Syntax:

$$R3 \leftarrow R1 + R2$$

This means the contents of register R1 are added to the contents of register R2, and the result is stored in register R3.

Example:

If R1 contains the binary value 0101 (decimal 5) and R2 contains 0011 (decimal 3), then after the operation:

$$R3 \leftarrow R1 + R2$$
$$R3 = 0101 + 0011 = 1000 \text{ (decimal 8)}$$

2. Subtraction

Subtraction involves deducting the contents of one register from another.

Syntax:

$$R3 \leftarrow R1 - R2$$

This means the contents of register R2 are subtracted from R1, and the result is stored in R3.

Example:

If R1 = 1001 (decimal 9) and R2 = 0011 (decimal 3), then:

$$R3 \leftarrow R1 - R2$$
$$R3 = 1001 - 0011 = 0110 \text{ (decimal 6)}$$

3. Increment

Increment increases the value of a register by one.

Syntax:

$$R1 \leftarrow R1 + 1$$

This operation is commonly used for loop counters, address incrementation, and similar tasks.

Example:

If R1 = 0111 (decimal 7), after increment:

$$R1 \leftarrow R1 + 1$$

$R1 = 1000$ (decimal 8)

4. Decrement

Decrement decreases the value of a register by one.

Syntax:

$$R1 \leftarrow R1 - 1$$

This operation is used for loop control, stack pointer adjustment, and so forth.

Example:

If $R1 = 0100$ (decimal 4), after decrement:

$$R1 \leftarrow R1 - 1$$

$R1 = 0011$ (decimal 3)

5. Complement Operations

- One's Complement: Inverts all bits of a register.
 - Syntax: $R2 \leftarrow \neg R2$
 - Example: If $R2 = 1010$, then after the operation, $R2 = 0101$.
- Two's Complement: Forms the negative of a binary number (by inverting all bits and adding one).
 - Syntax: $R2 \leftarrow \neg R2 + 1$
 - Example: If $R2 = 0001$ (decimal 1), then after the operation, $R2 = 1111$ (decimal -1 in two's complement form).

6. Add with Carry

In some operations, especially in multi-word arithmetic, it is necessary to add two registers along with a carry input.

Syntax:

$$R3 \leftarrow R1 + R2 + C_{in}$$

Where C_{in} is the carry input from a previous operation.

7. Subtract with Borrow

Similarly, subtraction may involve a borrow input.

Syntax:

$$R3 \leftarrow R1 - R2 - Bin$$

Where Bin is the borrow input.

8. Transfer (Move) Operation

While not strictly arithmetic, transfer operations are sometimes included when combined with arithmetic actions.

Syntax:

$$R2 \leftarrow R1$$

This simply copies the contents of $R1$ into $R2$.

4.2 Rules for Arithmetic Micro Operations

The execution of arithmetic micro operations is governed by several important rules and conventions:

1. Operand Sourcing:

- Operands must be available in the source registers before the operation is executed.
- If operands are not present, they must be loaded from memory or another register.

2. Result Storage:

- The result of an arithmetic micro operation is always stored in a destination register.
- The destination register may be one of the source registers or a separate register.

3. Bit Width Consistency:

- All participating registers must have the same bit width for the operation to be valid.
- Mismatched register sizes can lead to undefined results or hardware errors.

4. Overflow and Carry Handling:

- Arithmetic operations may produce results that exceed the bit width of the registers.
- The ALU sets status flags (carry, overflow, zero, sign) to indicate these conditions.

- Programmes or subsequent micro operations may use these flags for branching or error handling.

5. Atomicity:

- Each arithmetic micro operation is considered atomic at the hardware level; it completes in a single clock cycle or as defined by the micro-instruction.

6. Order of Execution:

- When multiple micro operations are specified, they are executed in the order given, unless explicitly stated otherwise.

7. Control Signal Generation:

- The control unit must generate appropriate signals to select the desired operation in the ALU and to enable the correct registers for reading and writing.

8. No Side Effects:

- Arithmetic micro operations should not modify registers other than those specified as destinations, unless explicitly required by the operation (e.g., updating status flags).

4.3 Detailed Examples of Arithmetic Micro Operations

Example 1: Addition

Suppose R1 = 00001101 (decimal 13) and R2 = 00000111 (decimal 7).

Operation:

$$R3 \leftarrow R1 + R2$$

Calculation:

```
00001101
+ 00000111
-----
00010100 (decimal 20)
```

So, R3 will contain 00010100 (decimal 20).

Example 2: Subtraction Using Two's Complement

Suppose R1 = 00001010 (decimal 10) and R2 = 00000101 (decimal 5).

Operation:

$R3 \leftarrow R1 - R2$

Calculation:

- Two's complement of R2 (00000101): invert bits $\rightarrow$ 11111010, add 1 $\rightarrow$ 11111011.
- Add to R1: $00001010 + 11111011 = 00000101$ (ignoring carry out).

So, R3 will contain 00000101 (decimal 5).

Example 3: Increment

Suppose R1 = 00001111 (decimal 15).

Operation:

$R1 \leftarrow R1 + 1$

Calculation:

$00001111 + 1 = 00010000$ (decimal 16)

So, R1 will contain 00010000.

Example 4: Decrement

Suppose R1 = 00010000 (decimal 16).

Operation:

$R1 \leftarrow R1 - 1$

Calculation:

$00010000 - 1 = 00001111$ (decimal 15)

So, R1 will contain 00001111.

Example 5: One's Complement

Suppose R2 = 10101010.

Operation:

$R2 \leftarrow \neg R2$

Calculation:

 $\neg 10101010 = 01010101$

So, R2 will contain 01010101.

Example 6: Two's Complement

Suppose R2 = 00000101 (decimal 5).

Operation:

 $R2 \leftarrow \neg R2 + 1$

Calculation:

- One's complement: 11111010
- Add 1: $11111010 + 1 = 11111011$ (decimal -5 in two's complement)

So, R2 will contain 11111011.

SELF-ASSESSMENT QUESTIONS - 3

True or False:	
11	Arithmetic micro operations are executed by the CPU's control unit.
Fill in the Blanks:	
12	The operation $R1 \leftarrow R1 + 1$ is an example of a ____ micro operation.
Multiple Choice Questions	
13	What is the result of the operation $R2 \leftarrow \neg R2$ if R2 initially contains 1010? a) 0101 b) 1111 c) 1010 d) 0000
True or False:	
14	Arithmetic micro operations must use registers with the same bit width to avoid errors.

Fill in the Blanks:

15 The operation $R3 \leftarrow R1 - R2$ represents a ____ micro operation.

Multiple Choice Questions

16 What happens when an arithmetic operation exceeds the register's bit width?

- a) The result is discarded
- b) The ALU sets status flags
- c) The operation is aborted
- d) The control unit halts



5. SUMMARY

- Micro operations are the elemental actions executed within a CPU to manipulate register data, forming the foundation for complex instruction execution in computer architecture. This unit examines register transfer, bus and memory transfer, and arithmetic micro operations, which are critical for understanding CPU functionality.
- Register Transfer entails transferring binary data between registers using Register Transfer Language (RTL), a formal notation (e.g., $R2 \leftarrow R1$) that specifies data flow. RTL supports simple, conditional, and simultaneous transfers, controlled by signals to ensure precise execution.
- It facilitates hardware design by modelling synchronous circuits, enabling simulation and optimisation for implementation in languages like Verilog or VHDL. While RTL enhances design efficiency and error detection, it poses challenges in debugging and requires careful optimisation to avoid inefficient hardware.
- Bus and Memory Transfer operations manage data movement via a shared bus or between registers and memory. A bus, implemented with multiplexers or tri-state buffers, simplifies connections by serving as a common data pathway. Memory transfers, such as reading ($DR \leftarrow M[AR]$) or writing ($M[AR] \leftarrow DR$), utilise address (AR) and data (DR) registers. These operations support instruction fetching and data manipulation, reducing hardware complexity and improving scalability.
- Arithmetic Micro Operations perform calculations like addition, subtraction, increment, decrement, and complement on register data. Executed by the ALU, these atomic operations, governed by strict rules, ensure consistent bit widths and handle overflows via status flags. Collectively, these micro operations enable efficient instruction execution, underpinning advanced studies in computer systems.

6. GLOSSARY

Arithmetic Micro Operations	- CPU operations like addition, subtraction, increment, decrement, and complement for mathematical computations on register data.
Bus	- A shared pathway of parallel wires for transferring data between registers, memory, or I/O devices.
Control Signal	- Signal from the control unit to manage the micro-operation execution.
Control Unit	- The CPU component that sequences and coordinates micro operations.
Data Register (DR/MDR)	- Register holding data during memory transfers.
Memory Transfer	- Operations to read from or write to memory using address and data registers.
Micro Operations	- Basic CPU actions manipulating register data, including register transfer, arithmetic, logic, and shift.
Multiplexer	- Hardware selects one register's data for the bus transfer.
Register	- High-speed CPU storage for temporary data or instructions.
Register Transfer	- Moving binary data between registers using RTL.
Register Transfer Language (RTL)	- Notation (e.g., $R2 \leftarrow R1$) for describing data movement and operations.
Tri-State Buffer	- Device enabling register connection to the bus, preventing data conflicts.

7. TERMINAL QUESTIONS

1. Explain micro-operations and their role in CPU instruction execution.
2. When is Register Transfer Language (RTL) used in digital system design?
3. Where do the CPU register transfer operations primarily occur?
4. Name the RTL symbol that represents data transfer between registers.
5. Which component generates control signals for register transfer operations?
6. Why does a common bus reduce hardware complexity in CPUs?
7. What is a multiplexer's role in bus transfers?
8. When are tri-state buffers used in bus transfer operations?
9. Where are memory transfer operations critical in CPU architecture?
10. Which control signals enable a memory read operation?
11. Who ensures proper sequencing of memory write operations?
12. Why are arithmetic micro operations vital for CPU performance?
13. Distinguish between one's complement and two's complement operations?
14. When does the ALU set status flags during arithmetic operations?
15. What are the advantages of RTL aid hardware design, and what are its challenges?

8. ANSWERS

Self-Assessment Questions

1. False
2. Colon (:)
3. b) Simultaneous addition and increment
4. False
5. P
6. False
7. $DR \leftarrow M[AR]$
8. c) Select lines
9. False
10. Address Register (AR)
11. False
12. Increment
13. a) 0101
14. True
15. Subtraction
16. b) The ALU sets status flags

Terminal Questions Answers

Answer 1: Micro operations are fundamental CPU actions manipulating register data, enabling complex instruction execution. For more details, refer to Section 1.

Answer 2: RTL is used in hardware design to model synchronous circuits and specify data flow. For more details, refer to Section 2.2.

Answer 3: Register transfer operations occur in CPU registers, facilitating data movement. For more details, refer to Section 2.

Answer 4: The left-arrow ($\leftarrow$) symbolises data transfer in RTL (e.g., $R2 \leftarrow R1$). For more details, refer to Section 2.1.

Answer 5: The CPU's control unit generates control signals for register transfers. For more details, refer to Section 2.5.

Answer 6: A common bus uses a shared pathway, reducing direct connection complexity. For more details, refer to Section 3.1.

Answer 7: A multiplexer selects a source register's data for bus transfer. For more details, refer to Section 3.1.1.

Answer 8: Tri-state buffers connect registers to the bus, enabling one at a time to avoid conflicts. For more details, refer to Section 3.1.1.

Answer 9: Memory transfers are critical for instruction fetching and data manipulation. For more details, refer to Section 3.2.4.

Answer 10: Read control signals transfer data from memory to the Data Register. For more details, refer to Section 3.2.1.

Answer 11: The control unit sequences memory write operations. For more details, refer to Section 3.2.2.

Answer 12: Arithmetic micro operations enable calculations for data processing and program flow. For more details, refer to Section 4.

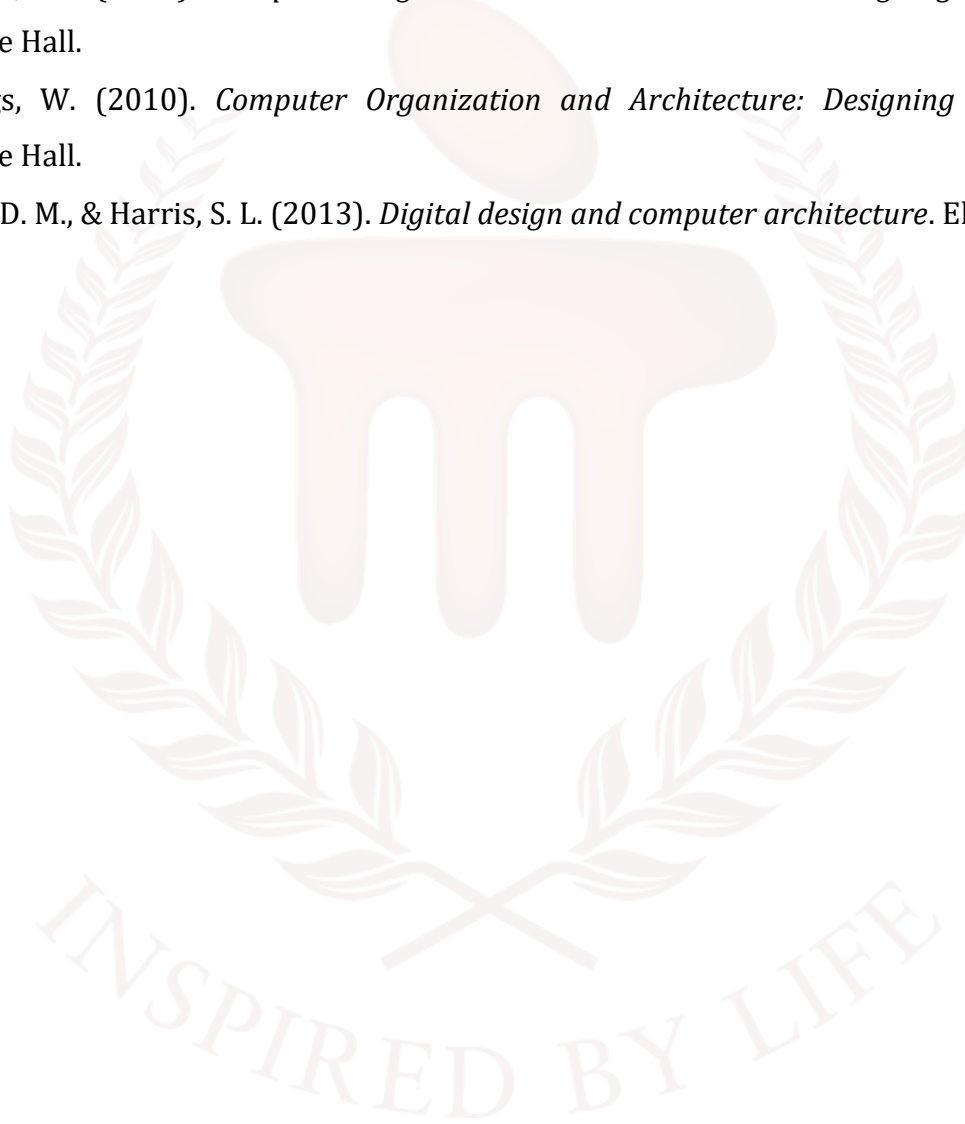
Answer 13: One's complement inverts bits; two's complement inverts and adds 1. For more details, refer to Section 4.1.

Answer 14: ALU sets flags (carry, overflow) when results exceed bit width. For more details, refer to Section 4.2.

Answer 15: RTL aids design and simulation but is hard to debug and optimise. For more details, refer to Section 2.6.

9. REFERENCES

- Mano, M. M. (1982). *Computer System Architecture*. Prentice Hall.
- Stallings, W. (2010). *Computer Organization and Architecture: Designing for Performance*. Prentice Hall.
- Stallings, W. (2010). *Computer Organization and Architecture: Designing for Performance*. Prentice Hall.
- Harris, D. M., & Harris, S. L. (2013). *Digital design and computer architecture*. Elsevier.



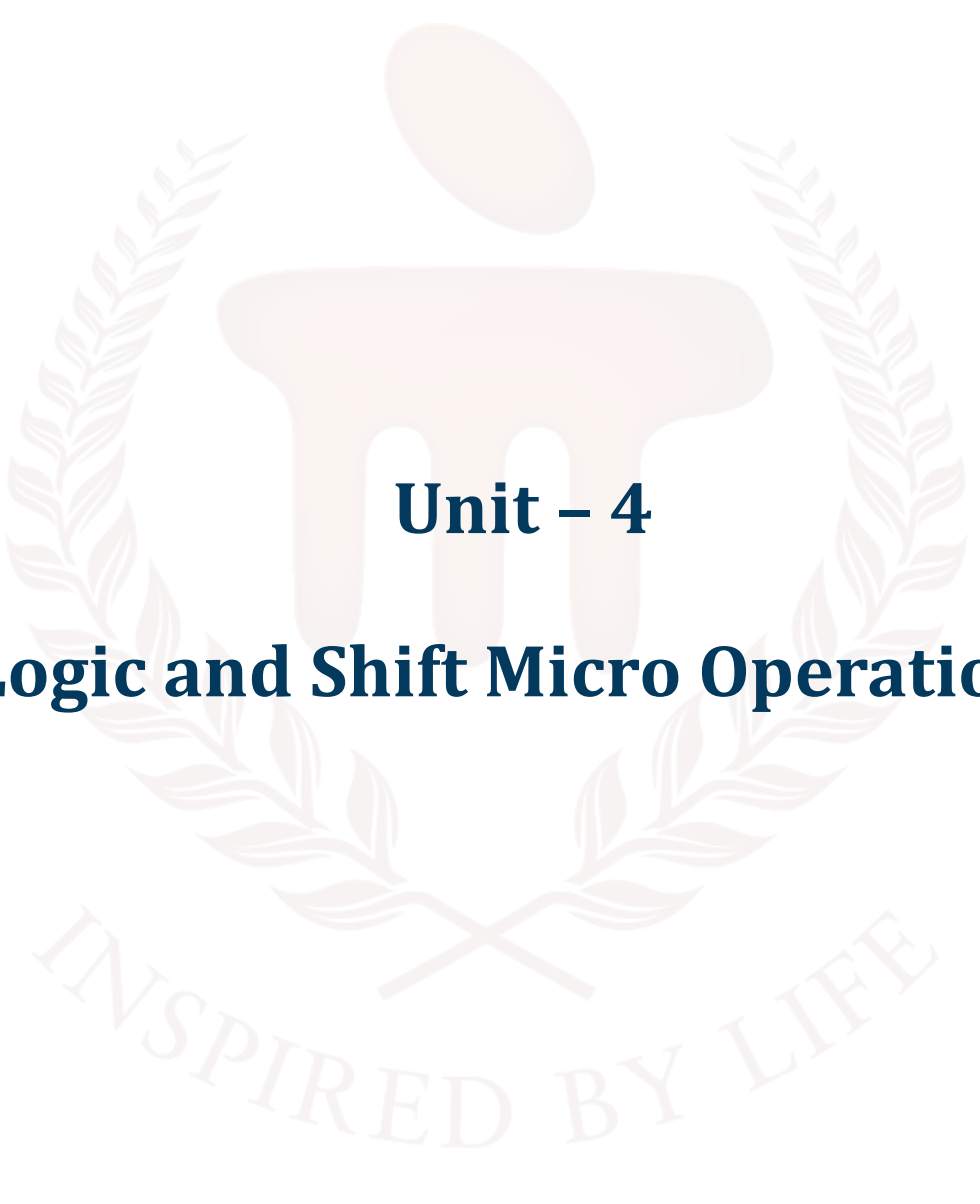
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 4

Logic and Shift Micro Operations

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 7
1.1	Objectives	-	-	
2	Logic Micro operations	1, 2, 3, 4, 5	1	8 - 18
2.1	Types of Logic Micro Operations	-	-	
2.2	Syntax, Rules	-	-	
2.3	Special Logic Micro Operations (Advanced Bit Manipulation)	-	-	
2.4	Applications of Logic Micro Operations	-	-	
2.5	Practice Examples and Detailed Explanations	-	-	
3	Shift Micro operations	6, 7, 8, 9, 10, 11	2	19 - 27
3.1	Logical Shift	-	-	
3.2	Arithmetic Shift	-	-	
3.3	Circular Shift	-	-	
3.4	Advantages & Disadvantages	-	-	
3.5	Applications	-	-	
4	Arithmetic Logic Shift Unit	11, 12	3	28 - 33
4.1	Core Functionalities of the ALSU	-	-	
4.2	ALSU Circuit Implementation	-	-	
4.3	Operation Selection and Functional Table	-	-	
4.4	Advantages of ALSU Integration	-	-	
4.5	Typical Applications	-	-	
5	Summary	-	-	34
6	Glossary	-	-	35
7	Terminal Questions	-	-	36

8	Answers	-	-	37 -39
9	References	-	-	40



1. INTRODUCTION

In the previous unit, you learned about the basic structure and function of the Central Processing Unit (CPU). Building on that foundation, this unit introduces micro operations, which are the fundamental actions performed by the CPU at the hardware level. These operations are categorised into register transfer, bus and memory transfer, and arithmetic micro operations. Register transfer operations involve moving data between registers within the CPU. For example, the operation $R2 \leftarrow R1$ copies the contents of register R1 into R2. Though simple, these operations are essential for organising data before and after arithmetic or logical tasks. Bus and memory transfers deal with moving data between the CPU and memory or I/O devices. A bus is a set of parallel lines that carry data, addresses, and control signals across different parts of the computer. Using a common bus reduces hardware complexity, improves scalability, and allows flexible data movement. For instance, to write data from a register to memory, the CPU loads the target address into the Address Register (AR), places the data in the Data Register (DR), and issues a write control signal. The memory then stores the data at the specified address, represented as $M[AR] \leftarrow DR$. Reading data from memory involves loading the address into AR, issuing a read signal, and transferring the data into DR. These operations are controlled by the CPU's control unit, which generates the necessary signals to manage the flow of data. Bus transfers are also used for I/O operations, allowing the CPU to communicate with external devices using the same shared pathway. Arithmetic micro operations perform basic mathematical tasks such as addition, subtraction, increment, and decrement. These are executed by the Arithmetic Logic Unit (ALU) and are essential for processing numerical data. For example, $R3 \leftarrow R1 + R2$ adds the contents of R1 and R2 and stores the result in R3, while $R3 \leftarrow R1 - R2$ performs subtraction. Increment and decrement operations modify register values by one, such as $R1 \leftarrow R1 + 1$ or $R1 \leftarrow R1 - 1$. One's complement inverts all bits in a register ($R2 \leftarrow \neg R2$), and two's complement negates a binary number by inverting the bits and adding one ($R2 \leftarrow \neg R2 + 1$). More advanced operations like add with carry ($R3 \leftarrow R1 + R2 + C_{in}$) and subtract with borrow ($R3 \leftarrow R1 - R2 - B_{in}$) are used in multi-word arithmetic, where carry or borrow bits from previous operations must be considered. Micro operations follow specific rules to ensure correct execution. Operands must be available in source registers before execution, and results are stored in destination registers. All participating registers must have the same bit width to avoid errors. The ALU sets status flags such as carry, overflow, zero, and sign to indicate special conditions, which can be used for branching or error handling. Each micro operation is atomic, completing in one clock cycle or as defined by the micro-instruction. When multiple operations are specified, they execute in the given order unless explicitly stated otherwise. The

control unit generates signals to select the desired ALU operation and enable the correct registers for reading and writing. Importantly, micro operations should not modify any registers other than those specified as destinations, except when updating status flags. These rules and operations form the foundation of how CPUs execute instructions and manage data at the hardware level.

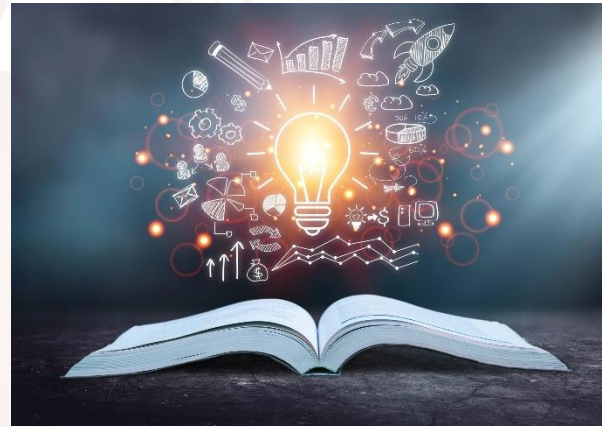
In this unit, you will learn about logic and shift micro operations, fundamental concepts in computer organisation and digital systems that underpin the functionality of modern computing architectures. These micro operations are essential building blocks in the design and operation of digital circuits, enabling processors to perform arithmetic, logical, and data manipulation tasks efficiently. By understanding these operations, you gain insight into how computers process and manipulate binary data at the hardware level, forming the basis for more complex computations in processors and memory systems.

Logic micro operations involve the manipulation of binary data using logical functions such as AND, OR, NOT, and XOR. These operations are performed bit-by-bit on binary variables, typically stored in registers, to achieve tasks like data comparison, masking, and selective bit modification. For instance, the AND operation can be used to clear specific bits in a register, while the XOR operation is crucial for tasks like error detection and data encryption. Logic micro operations are integral to arithmetic logic units (ALUs), which serve as the computational core of a CPU, executing instructions that involve decision-making, data filtering, or bit-level transformations. Understanding these operations allows you to appreciate how computers handle conditional operations and data processing at the lowest level.

Shift micro operations, on the other hand, involve the movement of bits within a register or memory location, either to the left or right, to achieve tasks like multiplication, division, or data alignment. These operations include logical shifts, arithmetic shifts, and circular shifts, each serving distinct purposes. Logical shifts fill vacated bit positions with zeros, making them useful for unsigned number manipulation, while arithmetic shifts preserve the sign bit for signed numbers, enabling efficient multiplication or division by powers of two. Circular shifts, also known as rotations, cycle bits within the register, which is valuable in cryptographic algorithms and data serialisation. Shift operations are critical in optimising computational efficiency, reducing the need for complex arithmetic circuits.

1.1. Objectives

- Define logic micro operations
- Explain their role in manipulating binary data using logical functions like AND, OR, NOT, and XOR.
- Determine the purpose and applications of logic micro operations in arithmetic logic units (ALUs).
- Identify different types of shift micro operations.
- Demonstrate how to perform shift micro operations
- Apply logic and shift micro operations to design and analyse basic digital circuits in computer architecture.



2. LOGIC MICRO OPERATIONS

Logic micro-operations are defined as operations that perform bitwise logical functions on the contents of registers. Unlike arithmetic micro-operations, which interpret the register contents as numerical values and perform operations such as addition or subtraction, logic micro-operations treat each bit within a register as an independent binary variable.

The operation is conducted in parallel across all bits, with the result for each bit determined solely by the corresponding bits of the operand registers.

The primary objective of logic micro-operations is to facilitate the manipulation of individual bits or groups of bits within a register, thereby enabling a range of functionalities such as masking, setting, clearing, toggling, and testing of bits.

These operations are foundational to the implementation of control logic, status flag management, data encryption, error detection and correction, as well as digital signal processing.

2.1 Types of Logic Micro Operations

There are four canonical types of logic micro-operations, each corresponding to a fundamental binary logic function.

Table 1: Types of Logic Micro Operations

Operation	Symbol	Description	Formal Syntax
AND	$\wedge$	Bitwise AND	$A \leftarrow A \wedge B$
OR	$\vee$	Bitwise OR	$A \leftarrow A \vee B$
XOR	$\oplus$	Bitwise Exclusive OR	$A \leftarrow A \oplus B$
NOT	$\neg$	Bitwise Complement	$A \leftarrow \neg A$

Each operation is formally defined below, accompanied by its truth table, operational rules, and illustrative examples.

AND Micro-operation

The AND micro-operation performs a bitwise logical AND between corresponding bits of two registers, denoted as A and B. The result bit is 1 if and only if both corresponding bits in A and B are 1; otherwise, the result is 0.

Formal Syntax:

$A \leftarrow A \wedge B$

Table 2: Truth Table

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Example:

Let $A = 1010$, $B = 1100$

$A \leftarrow A \wedge B$

Result: $1010 \wedge 1100 = 1000$

Explanation:

- The operation is performed bitwise:
 - $1 \wedge 1 = 1$ (leftmost bit)
 - $0 \wedge 1 = 0$
 - $1 \wedge 0 = 0$
 - $0 \wedge 0 = 0$

Thus, the result is 1000.

OR Micro-operation

The OR micro-operation performs a bitwise logical OR between corresponding bits of two registers. The result bit is 1 if at least one of the corresponding bits in A or B is 1; otherwise, the result is 0.

Formal Syntax:

$A \leftarrow A \vee B$

Table 3: Truth Table

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

Example:

Let $A = 1010$, $B = 1100$

$A \leftarrow A \vee B$

Result: $1010 \vee 1100 = 1110$

Explanation:

- Bitwise operation:

- $1 \vee 1 = 1$
- $0 \vee 1 = 1$
- $1 \vee 0 = 1$
- $0 \vee 0 = 0$

Thus, the result is 1110.

XOR (Exclusive OR) Micro-operation

The XOR micro-operation, also known as Exclusive OR, performs a bitwise comparison between corresponding bits of two registers. The result bit is 1 if the corresponding bits are different; otherwise, it is 0.

Formal Syntax:

$A \leftarrow A \oplus B$

Table 4: Truth Table

A	B	$A \oplus B$
0	0	0
0	1	1

1	0	1
1	1	0

Example:

Let A = 1010, B = 1100

$A \leftarrow A \oplus B$

Result: $1010 \oplus 1100 = 0110$

Explanation:

- Bitwise operation:

$$1 \oplus 1 = 0$$

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

$$0 \oplus 0 = 0$$

Thus, the result is 0110.

NOT (Complement) Micro-operation

The NOT micro-operation, also referred to as the complement operation, inverts each bit of the register. Every 0 becomes 1, and every 1 becomes 0.

Formal Syntax:

$A \leftarrow \neg A$

Table 5: Truth Table

A	$\neg A$
0	1
1	0

Example:

Let A = 1010

$A \leftarrow \neg A$

Result: $\neg 1010 = 0101$

Explanation:

- Bitwise inversion:
 - 1 becomes 0
 - 0 becomes 1
 - 1 becomes 0
 - 0 becomes 1

Thus, the result is 0101.

2.2 Syntax, Rules

- Two-operand operations:
 $A \leftarrow A \text{ op } B$
where op is one of AND ($\wedge$), OR ($\vee$), XOR ($\oplus$).
- Single-operand operation:
 $A \leftarrow \neg A$
(for the NOT operation).

Rules for Logic Micro-operations

- Both operand registers must be of identical bit-width.
- Operations are performed in parallel across all bits.
- The result is stored in the destination register, typically the first operand.

Example in RTL

AND: $A \leftarrow A \wedge B$

OR: $A \leftarrow A \vee B$

XOR: $A \leftarrow A \oplus B$

NOT: $A \leftarrow \neg A$

2.3 Special Logic Micro Operations (Advanced Bit Manipulation)

Logic micro-operations can be combined or sequenced to perform advanced bit manipulation tasks, which are essential for operations such as selective setting, clearing, complementing, masking, and inserting data.

Selective-Set Operation

Purpose:

Sets bits in register A to 1 wherever the corresponding bits in register B are 1.

Syntax:

$A \leftarrow A \vee B$

Example: $A = 1010$ $B = 1100$ $A \leftarrow A \vee B$ **Result:** 1110**Explanation:**

- Bits in A corresponding to 1s in B are set to 1; other bits remain unchanged.

Selective-Complement Operation**Purpose:**

Complements bits in A wherever the corresponding bits in B are 1.

Syntax: $A \leftarrow A \oplus B$ **Example:** $A = 1010$ $B = 1100$ $A \leftarrow A \oplus B$ **Result:** 0110**Explanation:**

- Bits in A corresponding to 1s in B are toggled; others remain unchanged.

Selective-Clear Operation**Purpose:**

Clears (sets to 0) bits in A wherever the corresponding bits in B are 1.

Syntax: $A \leftarrow A \wedge \neg B$ **Example:** $A = 1010$ $B = 1100$ $\neg B = 0011$ $A \leftarrow A \wedge \neg B$ **Result:** 0010**Explanation:**

- Bits in A corresponding to 1s in B are cleared to 0.

Mask Operation

Purpose:

Clears bits in A wherever the corresponding bits in B are 0, effectively isolating or “masking” desired bits.

Syntax:
$$A \leftarrow A \wedge B$$
Example:
$$A = 1010$$
$$B = 1100$$
$$A \leftarrow A \wedge B$$
Result: 1000**Explanation:**

- Only bits in A where B is 1 are retained.

Insert Operation**Purpose:**

Inserts a new value into a group of bits in a register.

Steps:

1. Mask the bits to be replaced (clear them).
2. OR the result with the new bits to be inserted.

Example:

Insert 1001 into the left half of A ($A = 01101010$):

- **Step 1:** Mask: 00001111 (AND operation)
 $01101010 \wedge 00001111 = 00001010$
- **Step 2:** Insert value: 10010000 (OR operation)
 $00001010 \vee 10010000 = 10011010$

Explanation:

- The left half of A is cleared, and the new value is inserted.

2.4 Applications of Logic Micro Operations

Logic micro-operations are indispensable in a multitude of applications within computer systems, including but not limited to:

Bit Manipulation

- Setting, clearing, or toggling specific bits in registers (e.g., flags, status registers, control words).

Masking

- Isolating or extracting specific bits from a word, such as accessing subfields within instruction formats or data packets.

Conditional Logic

- Implementing hardware-level decision-making, enabling or disabling features based on current conditions.

Data Packing and Unpacking

- Efficiently storing multiple small values in a single word and extracting individual values from a packed word.

Cryptography

- Bitwise operations are fundamental to many encryption and hashing algorithms, ensuring data security and integrity.

Error Detection and Correction

- Parity checks, cyclic redundancy checks (CRC), and other error-detection techniques rely on logic micro-operations to compute check bits and detect errors.

Digital Signal Processing

- Logic operations are employed in DSP algorithms for filtering, modulation, and error correction.

2.5 Practice Examples and Detailed Explanations

Example 1: AND Micro-operation

Given: $A = 1101$, $B = 1011$

$A \leftarrow A \wedge B$

$1101 \wedge 1011 = 1001$

Explanation:

- $1 \wedge 1 = 1$
- $1 \wedge 0 = 0$
- $0 \wedge 1 = 0$
- $1 \wedge 1 = 1$

Result: 1001

Example 2: OR Micro-operation

$A \leftarrow A \vee B$

$$1101 \vee 1011 = 1111$$

Explanation:

- $1 \vee 1 = 1$
- $1 \vee 0 = 1$
- $0 \vee 1 = 1$
- $1 \vee 1 = 1$

Result: 1111

Example 3: XOR Micro-operation

$$A \leftarrow A \oplus B$$

$$1101 \oplus 1011 = 0110$$

Explanation:

- $1 \oplus 1 = 0$
- $1 \oplus 0 = 1$
- $0 \oplus 1 = 1$
- $1 \oplus 1 = 0$

Result: 0110

Example 4: NOT Micro-operation

$$A \leftarrow \neg A$$

$$\neg 1101 = 0010$$

Explanation:

- 1 becomes 0
- 1 becomes 0
- 0 becomes 1
- 1 becomes 0

Result: 0010

Example 5: Selective-Set

$$A = 1010, B = 1100$$

$$A \leftarrow A \vee B = 1110$$

Explanation:

- Bits in A corresponding to 1s in B are set to 1.

Example 6: Selective-Clear

A = 1010, B = 1100

$\neg B = 0011$

$A \leftarrow A \wedge \neg B = 0010$

Explanation:

- Bits in A corresponding to 1s in B are cleared to 0.

Example 7: Mask

A = 1010, B = 1100

$A \leftarrow A \wedge B = 1000$

Explanation:

- Only bits in A where B is 1 are retained.

Example 8: Insert Operation

Insert 1001 into the left half of A (A = 01101010):

- Mask: 00001111 (AND operation)
 $01101010 \wedge 00001111 = 00001010$
- OR with 10010000:
- $00001010 \vee 10010000 = 10011010$

Explanation:

The left half of A is cleared, and the new value is inserted.

SELF-ASSESSMENT QUESTIONS - 1

True or False:	
1	Logic micro-operations treat each bit in a register as an independent binary variable, unlike arithmetic micro-operations, which interpret bits as numerical values.(T/F)
Multiple Choice Questions	
2	Which logic micro-operation produces a 1 only when both input bits are 1? a) OR b) XOR

	c) AND d) NOT
Fill in the Blanks:	
3	The _____ micro-operation is used to toggle bits in a register where corresponding bits in another register are 1, using the operation $A \leftarrow A \oplus B$
True or False:	
4	The NOT micro-operation requires two operand registers to perform bit inversion.(T/F)
Multiple Choice Questions	
5	What is the result of the OR micro-operation on A = 1010 and B = 1100? a) 1000 b) 1110 c) 0110 d) 1010

3. SHIFT MICRO OPERATIONS

Shift micro-operations constitute fundamental procedures within computer architecture, facilitating the movement of bits within a register either to the left or to the right. These operations are integral to various computational processes, including serial data transmission, binary multiplication and division, and the efficient manipulation of data when combined with arithmetic and logical operations. By enabling bitwise shifts, these micro-operations enhance the speed and flexibility of data processing within the central processing unit.

There are three main types of shift micro-operations:

- Logical Shift
- Arithmetic Shift
- Circular Shift

3.1 Logical Shift

A logical shift is a fundamental bitwise operation involving moving all bits within a binary value to either the left or right. During this process, the shifted bits introduce zeros into the positions vacated. This operation is extensively utilised in binary arithmetic and data processing, particularly when handling data as a sequence of bits rather than as signed numerical values.

Types of Logical Shift Micro-operations

Logical Left Shift:

Each bit in the register is shifted from one position to the left in this operation. The least significant bit (LSB) is filled with zero, while the most important bit (MSB) is discarded. The left shift operator is commonly denoted by the symbol `<<`.

Syntax:

The general syntax for the left shift is

"shift-expression" `<<` k

Note: For unsigned numbers, every time we shift a number towards the left by 1 bit it multiplies that number by 2.

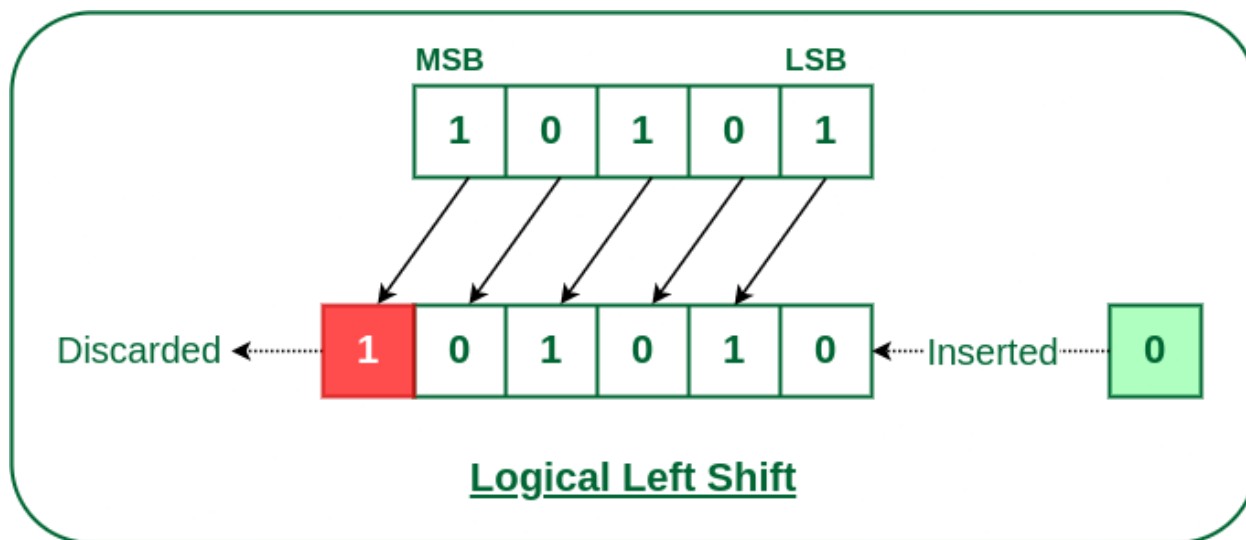


Figure 4-1 Logical Left Shift

Usage: Used in the multiplication of unsigned binary numbers.

Example: Let's take an 8-bit unsigned binary number 01010011 (which is 83 in decimal). If we perform a logical shift left in Figure 4-1:

- Before Shift: 01010011 (83 in decimal)
- After Logical Shift Left: 10100110 (166 in decimal)

2. Logical Right Shift

In a logical right shift operation, each bit within the register is shifted one position to the right. The least significant bit (LSB) is discarded, while a zero is inserted into the vacated most significant bit (MSB) position. The right shift operation is typically represented by the double right arrow symbol ($\gg$).

The general syntax for the right shift operation is as follows:

`shift_expression >> k`

Note: For unsigned numbers, each right shift by one bit effectively divides the value by two, discarding any remainder.

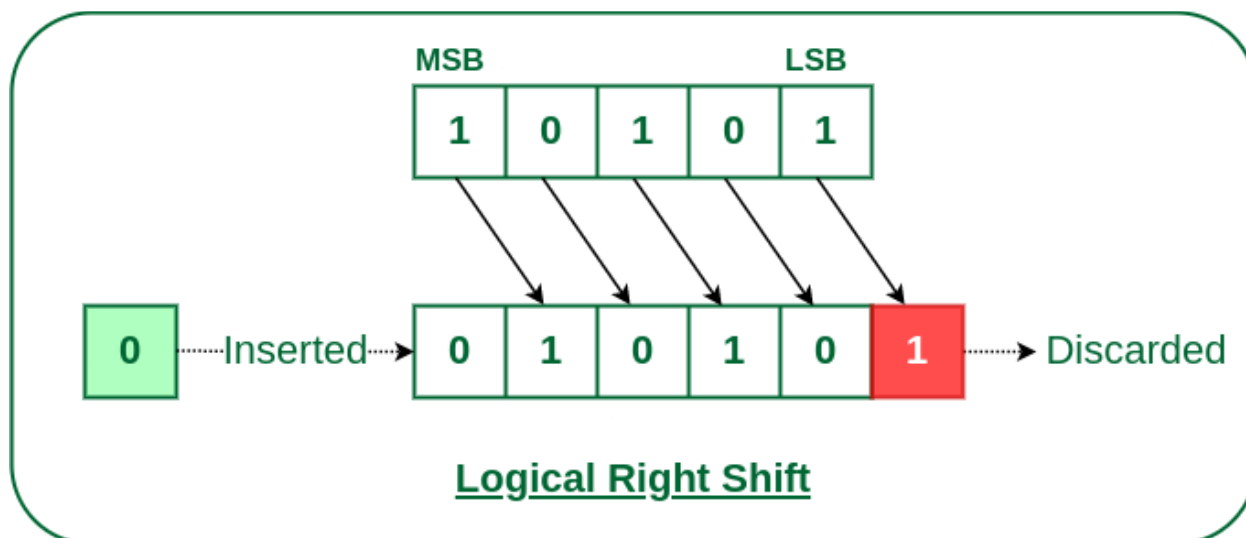


Figure 4-2 Logical Right Shift

Usage: Used in division of unsigned binary numbers.

Example: Let's take the same 8-bit binary number 01010011 (83 in decimal). If we perform a logical shift right in Figure 4-2:

- Before Shift: 01010011 (83 in decimal)
- After Logical Shift Right: 00101001 (41 in decimal)

3.2 Arithmetic Shift

The arithmetic shift micro-operation shifts a signed binary number either to the left or to the right. There are two primary forms of this operation:

Arithmetic Left Shift

In an arithmetic left shift, each bit in the binary number is moved one position to the left. The least significant bit (LSB) is filled with zero, and the most significant bit (MSB) is discarded, similar to a logical left shift. However, the arithmetic left shift is specifically interpreted as a multiplication by two for signed integers, preserving the sign of the number in two's complement representation.

Shift Left Arithmetic

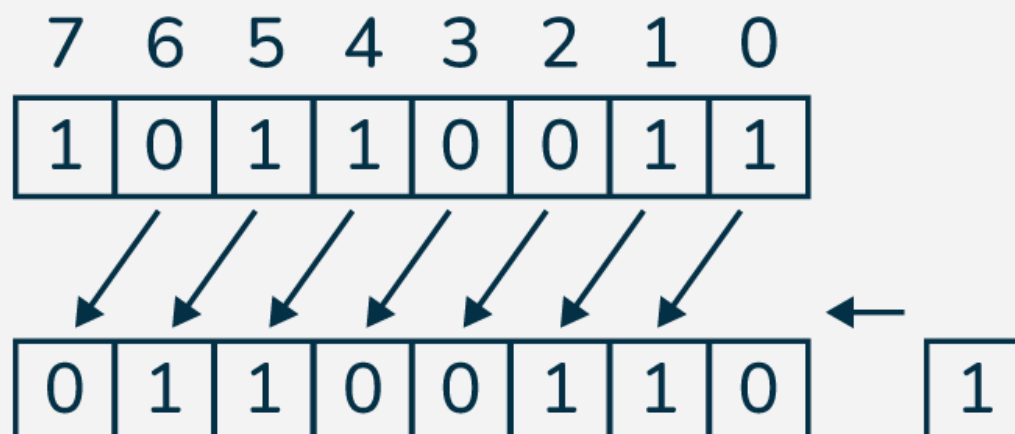


Figure 4-3 Arithmetic Left Shift

Usage:

Arithmetic left shifts are used to multiply signed binary numbers by powers of two.

Example:

Consider a signed 8-bit binary number in two's complement form in Figure 4-3:

Before shift: 11111111 (which represents -1 in decimal)

After arithmetic left shift: 11111110 (which represents -2 in decimal)

This operation effectively doubles the value of the signed binary number while maintaining its sign, making it a fundamental operation in arithmetic computations and digital logic design.

Arithmetic Right Shift

In an arithmetic right shift operation, each bit of the binary number is shifted one position to the right. The least significant bit (LSB) is discarded, and the vacated most significant bit (MSB) is filled with the value of the original MSB, effectively replicating the sign bit. This process, known as sign extension, preserves the sign of the number in two's complement representation.

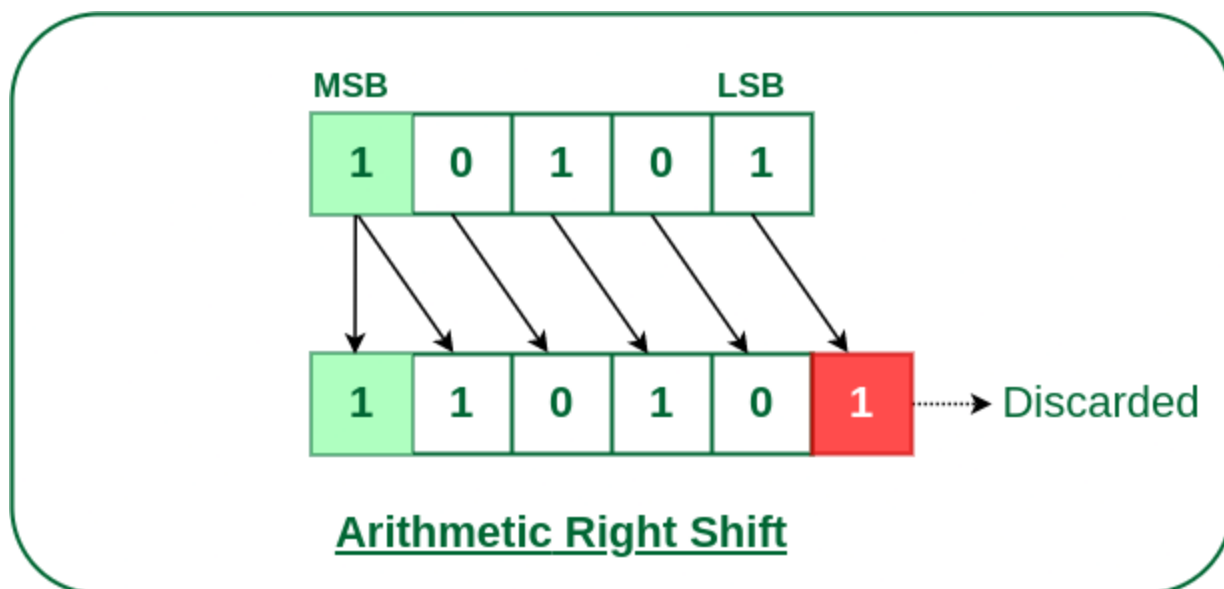


Figure 4-4 Arithmetic Right Shift

Usage:

The arithmetic right shift is utilised to divide signed binary numbers by powers of two, while maintaining the correct sign of the result.

Example:

Consider a signed 8-bit binary number in two's complement form in Figure 4-4:

Before shift: 11111111 (which represents -1 in decimal)

After arithmetic right shift: 11111111 (still -1 in decimal)

In this example, the sign bit (1) is preserved after the shift, ensuring the result remains consistent with the original value.

3.3 Circular Shift

A circular shift, also known as a bitwise rotation, is an operation in which the bits within a register are rotated around both ends, ensuring that no information is lost in the process. Unlike logical or arithmetic shifts, the bits that are shifted out at one end are reintroduced at the opposite end, maintaining the original bit pattern in a different sequence.

Circular Left Shift

In a circular left shift, each bit in the register is moved one position to the left. The most significant bit (MSB), which is shifted out, is then reintroduced into the least significant bit (LSB) position. This operation effectively rotates the bit sequence, preserving all original information but altering the order of the bits.

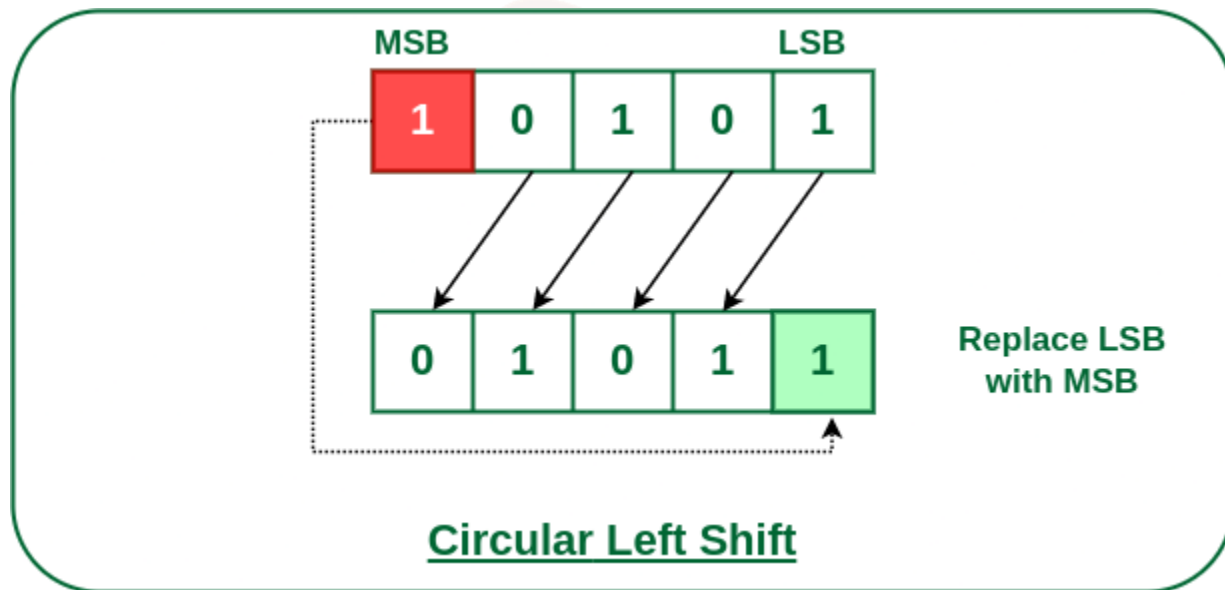


Figure 4-5 Circular Left Shift

Usage: Used in data encryption algorithms and certain arithmetic operations, where the shift should be cyclic.

Example: Let's take an 8-bit binary number 11010011. If we perform a circular shift left:

- Before Shift: 11010011
- After Circular Shift Left: 10100111

Circular Right Shift

In this micro shift operation, each bit in the register is shifted to the right one by one. After shifting, the MSB becomes empty, so the value of the LSB is filled in there.

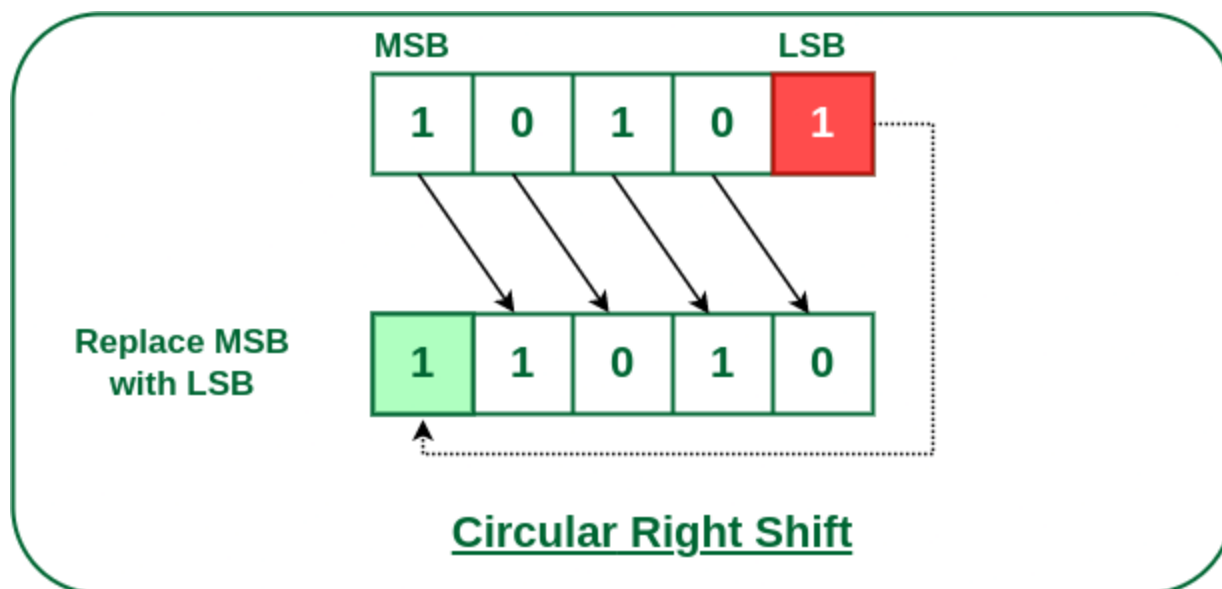


Figure 4-6 Circular Right Shift

Usage: Used in situations where rotation of bits is required, such as in encryption algorithms.

Example: Let's take the same 8-bit binary number 11010011. If we perform a circular shift right:

- Before Shift: 11010011
- After Circular Shift Right: 11101001

3.4 Advantages & Disadvantages

Advantages

- **Efficient Arithmetic:** Shift micro-operations facilitate rapid multiplication and division by powers of two, thereby enhancing computational performance.
- **Bitwise Manipulation:** Shifts are indispensable for operations such as encryption, data compression, and error detection, where manipulation of individual bits is required.
- **Resource Efficiency:** These operations demand minimal hardware resources, resulting in faster execution and reduced power consumption.
- **Memory Optimisation:** By optimising memory access patterns, shift micro-operations can contribute to overall system efficiency.
- **Direct Hardware Implementation:** Shift operations are natively supported in hardware, ensuring low-latency and resource-efficient execution.

Disadvantages

- **Limited Applicability:** Shift micro-operations are primarily suitable for arithmetic involving powers of two and for bitwise manipulations; they are not appropriate for complex arithmetic operations.
- **Potential Data Loss:** Improper management of shift operations can result in the loss of significant bits, potentially leading to data corruption.
- **Increased Hardware Complexity:** Excessive use of shift operations may introduce additional hardware complexity and elevate power consumption, particularly in large-scale systems.
- **Context Switching Overhead:** Managing shift operations across multiple tasks in multitasking environments can introduce additional processing delays.
- **Error Susceptibility:** Incorrect implementation of shift micro-operations can lead to issues such as overflow, underflow, or unintended modification of the sign bit in signed numbers.

3.5 Applications

- **Serial Data Transfer:** Shift micro-operations are fundamental for the serial transfer of data within registers, allowing data to be moved bit by bit, either to the left or right.
- **Arithmetic Operations:** They are used in arithmetic computations, such as multiplication and division by powers of two. Arithmetic left shifts multiply signed binary numbers by two, while arithmetic right shifts divide signed numbers by two, all while preserving the sign bit.
- **Bitwise Manipulation:** Shift operations are essential for bitwise tasks such as encryption, data compression, and error detection, where individual bits need to be manipulated efficiently.
- **Circular Data Rotation:** Circular (or rotate) shifts are used to rotate bits within a register without loss of information, which is useful in algorithms that require bit rotation, such as certain cryptographic functions and digital signal processing.
- **Efficient Hardware Implementation:** Shift micro-operations are directly supported by hardware, enabling fast execution with minimal resource usage, which is critical for performance-sensitive applications.
- **Memory Access Optimisation:** They can optimise memory access patterns and support operations like address calculation in array indexing and buffer management..

SELF-ASSESSMENT QUESTIONS - 2

Fill in the Blanks:	
6	In a logical left shift, the least significant bit is filled with a _____, and the most significant bit is discarded.
True or False:	
7	Arithmetic right shifts preserve the sign bit of a signed number by replicating it in the most significant bit position. (T/F)
Multiple Choice Questions	
8	Which shift micro-operation is most suitable for cryptographic algorithms requiring bit rotation without data loss? a) Logical Shift b) Arithmetic Shift c) Circular Shift d) Selective Shift



4. ARITHMETIC LOGIC SHIFT UNIT

The Arithmetic Logic Shift Unit (ALSU) is a critical component of a computer's central processing unit (CPU), serving as the backbone for executing a diverse set of micro-operations essential to data processing. The ALSU extends the capabilities of the traditional Arithmetic Logic Unit (ALU) by integrating arithmetic, logical, and shift operations into a single, cohesive hardware unit. This integration streamlines processing tasks, enhances efficiency, and reduces hardware complexity within the CPU. By handling binary data stored in registers, the ALSU performs operations such as addition, subtraction, bitwise AND, OR, XOR, and bit shifts, making it indispensable for computational and control tasks in modern computer systems.

The ALSU's ability to perform these operations in a single clock cycle, coupled with its combinational circuit design, ensures high-speed data manipulation. This paper explores the structure, functionalities, implementation, and applications of the ALSU, providing a detailed analysis of its role in computer architecture and its significance in enabling efficient processing.

4.1 Core Functionalities of the ALSU

The ALSU supports three primary types of operations: arithmetic, logical, and shift. Each category serves distinct purposes in data processing, enabling the CPU to handle a wide range of computational and control tasks.

Arithmetic Operations

Arithmetic operations form the foundation of numerical computations in a computer system. The ALSU is capable of performing operations such as:

- **Addition:** Adds two binary numbers, incorporating a carry-in bit for multi-bit operations.
- **Subtraction:** Computes the difference between two numbers, handling borrow bits as needed.
- **Increment:** Increases a number by one, useful for loop counters and address calculations.
- **Decrement:** Decreases a number by one, often used in iterative processes.

These operations are essential for tasks ranging from basic calculations to complex algorithm execution. The ALSU manages carry and borrow bits to ensure accurate multi-bit arithmetic, making it a vital component for numerical processing.

Logic Operations

Logical operations manipulate data at the bit level, enabling tasks such as masking, setting, clearing, or toggling specific bits. The ALSU supports the following bitwise operations:

- **AND:** Performs a bitwise AND, producing a 1 only when both input bits are 1. Used for masking specific bits.
- **OR:** Performs a bitwise OR, producing a 1 if either input bit is 1. Useful for setting bits.
- **XOR:** Performs a bitwise exclusive OR, producing a 1 if the input bits differ. Commonly used for toggling bits.
- **NOT:** Complements a binary word, flipping each bit from 0 to 1 or vice versa.

These operations are critical for control processing, data manipulation, and decision-making within programs, such as evaluating conditions or modifying register contents.

Shift Operations

Shift operations reposition bits within a binary word, enabling efficient data manipulation. The ALSU supports two types of shifts:

- **Logical Shift:**
 - **Logical Shift Right (shr):** Shifts all bits one position to the right, inserting a zero into the most significant bit (MSB). The least significant bit (LSB) is discarded.
 - **Logical Shift Left (shl):** Shifts all bits one position to the left, inserting a zero into the LSB. The MSB is discarded.

Logical shifts are primarily used for unsigned data and are effective for tasks like bit manipulation or fast multiplication/division by powers of two.

- **Arithmetic Shift:**
 - **Arithmetic Shift Right:** Similar to logical shift right, but preserves the MSB (sign bit) to maintain the sign of signed numbers. This is crucial for signed integer arithmetic.
 - **Arithmetic Shift Left:** Identical to logical shift left, effectively multiplying the number by two. However, overflow detection is necessary to ensure valid results.

Shift operations are widely used for tasks such as scaling numbers, aligning data, or extracting specific bits from a word.

4.2 ALSU Circuit Implementation

One-Stage ALSU Design

The ALSU is implemented as a cascaded structure, with each stage processing one bit of the input data. For an n-bit word, the ALSU comprises n identical stages, each handling a single bit from two source registers (A and B).

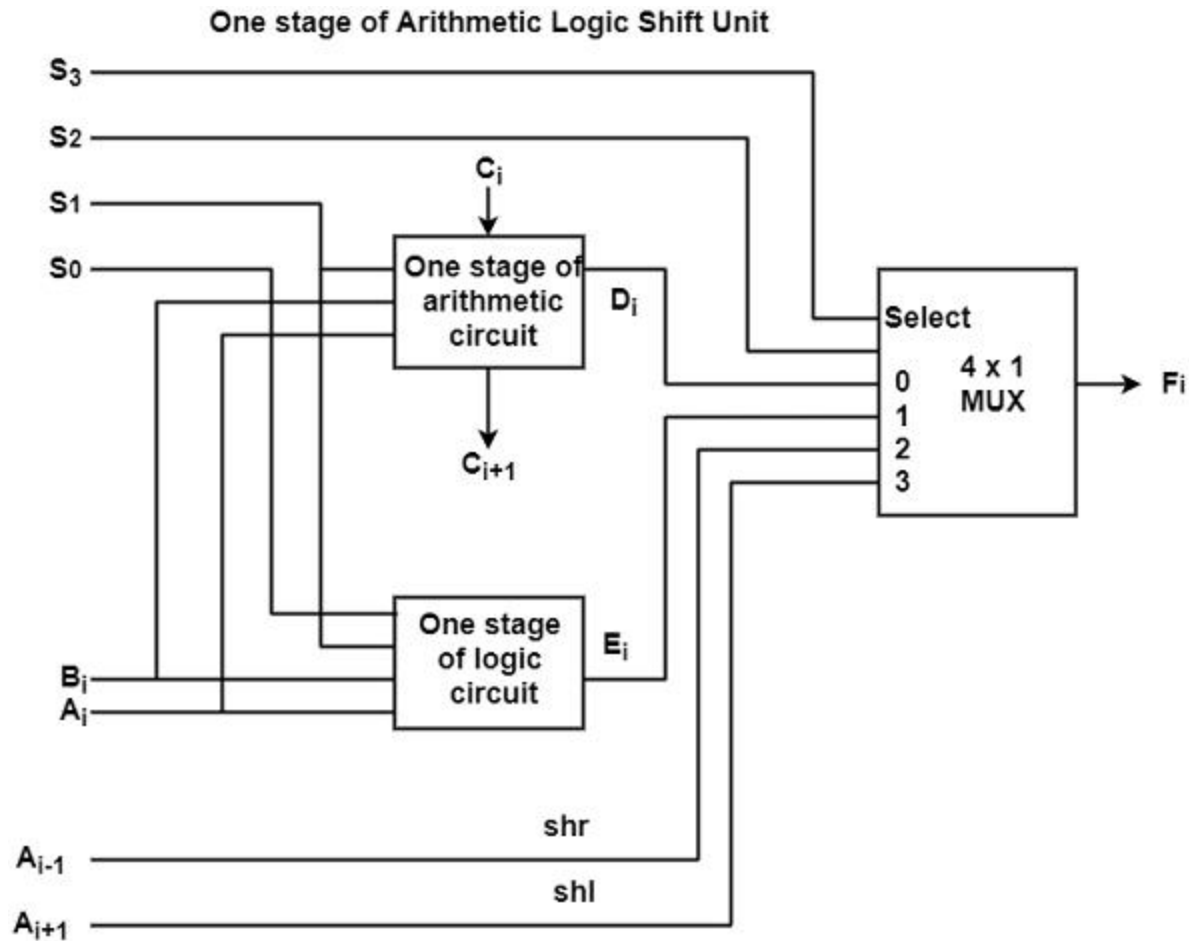


Figure 4-7 One-Stage ALSU Design

Each stage includes(in Figure 4-7):

- **Arithmetic Circuit:** Performs operations like addition, subtraction, increment, and decrement, incorporating carry-in and carry-out signals.
- **Logic Circuit:** Executes bitwise operations (AND, OR, XOR, NOT) on the input bits.
- **Shift Circuit:** Handles logical and arithmetic shifts, adjusting bit positions as required.
- **4x1 Multiplexer:** Selects the output from one of the above circuits based on selection inputs (S₃, S₂, S₁, S₀).

The selection variables determine which circuit's output is forwarded to the destination register, ensuring the correct operation is performed.

Cascading and Carry Handling

For multi-bit operations, the ALSU stages are cascaded, with the carry-out of one stage connected to the carry-in of the next. This configuration enables proper carry propagation for arithmetic operations, ensuring accurate results for additions and subtractions across multiple bits. The carry-

in signal is relevant only for arithmetic operations and is ignored for logical and shift operations, as these do not involve carry propagation.

4.3 Operation Selection and Functional Table

Selection Variables

The ALSU uses five control variables—S3, S2, S1, S0, and Cin (carry-in)—to determine the operation to be performed. The combination of S3 and S2 specifies the operation type:

- S3S2 = 00: Arithmetic operations
- S3S2 = 01: Logical operations
- S3S2 = 10 or 11: Shift operations

The specific operation within each category is further defined by S1, S0, and Cin, as detailed in the functional table.

Functional Table Overview

The ALSU supports 14 distinct operations, categorised as follows:

Table 6: Functional Table

S3	S2	S1	S0	Cin	Operation	Function
0	0	0	0	0	Transfer A	$F = A$
0	0	0	0	1	Increment A	$F = A + 1$
0	0	0	1	0	Addition	$F = A + B$
0	0	0	1	1	Add with carry	$F = A + B + 1$
0	0	1	0	0	Subtract with borrow	$F = A + B'$
0	0	1	0	1	Subtraction	$F = A + B' + 1$
0	0	1	1	0	Decrement A	$F = A - 1$
0	0	1	1	1	Transfer A	$F = A$
0	1	0	0	X	AND	$F = A \wedge B$
0	1	0	1	X	OR	$F = A \vee B$
0	1	1	0	X	XOR	$F = A \oplus B$
0	1	1	1	X	Complement A	$F = A'$

1	0	X	X	X	Shift right	$F = \text{shr } A$
1	1	X	X	X	Shift left	$F = \text{shl } A$

This table outlines the ALSU's versatility, covering eight arithmetic operations, four logical operations, and two shift operations.

4.4 Advantages of ALSU Integration

The integration of arithmetic, logical, and shift operations into a single ALSU offers several advantages:

- **Centralisation:** By consolidating multiple functions into one unit, the ALSU reduces hardware redundancy, simplifying CPU design and lowering manufacturing costs.
- **Efficiency:** The combinational design ensures operations are completed in a single clock cycle, maximising throughput and performance.
- **Flexibility:** The ALSU's ability to handle a wide range of operations makes it adaptable to diverse processing needs, from numerical computations to bit-level manipulations.

These advantages make the ALSU a cornerstone of modern CPU architecture, enabling efficient and versatile data processing.

4.5. Typical Applications

The ALSU's versatility supports a wide range of applications:

- **Arithmetic Calculations:** Performs all basic mathematical operations required by programs, from simple additions to complex algorithmic computations.
- **Bitwise Data Manipulation:** Facilitates tasks like masking, setting, or clearing bits, which are crucial for data processing and control operations.
- **Fast Multiplication/Division:** Shift operations enable rapid multiplication or division by powers of two, optimising performance in algorithms.
- **Control and Status Operations:** Logical operations are used to manipulate status flags and control bits, supporting decision-making and system control.

The Arithmetic Logic Shift Unit (ALU) is a vital component of modern CPUs, combining arithmetic, logical, and shift operations into a single, efficient hardware unit. Its combinational circuit design ensures high-speed processing, completing operations in a single clock cycle. By supporting a diverse set of micro-operations, the ALU enables rapid and flexible data manipulation, making it indispensable for computational and control tasks.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
9	What operation does the ALSU perform when $S3S2 = 00$, $S1S0 = 01$, and $C_{in} = 1$? a) Addition b) Add with carry c) Subtraction d) AND
Fill in the Blanks:	
10	The _____ operation in the ALSU sets bits in register A to 1 where corresponding bits in B are 1, using $A \leftarrow A \vee B$.
True or False:	
11	The ALSU's cascaded structure connects the carry-out of one stage to the carry-in of the next for multi-bit arithmetic operations. (T/F)
Multiple Choice Questions	
12	Which of the following is not an advantage of ALSU intergration? a) Centralisation b) Effeciency c) Flexibility d) Integration
Fill in the blanks	
13	The _____ logical operation performs a bitwise OR, producing a 1 if either input bit is 1.
14	The _____ operation computes the difference between two numbers, handling borrow bits as needed.
15	The ALSU is implemented as a _____ structure, with each stage processing one bit of the input data.

5. SUMMARY

Logic and shift micro operations are essential components of computer organisation, forming the foundation for data manipulation in digital systems. Logic micro operations involve bitwise logical functions—AND, OR, XOR, and NOT—performed on binary data in registers. These operations, executed in parallel, treat bits as independent binary variables, enabling tasks like masking, bit setting, clearing, toggling, and data comparison. For instance, AND can clear specific bits, while XOR supports error detection and encryption. Integral to the Arithmetic Logic Unit (ALU), these operations facilitate decision-making and data processing, with applications in cryptography, digital signal processing, and status flag management, ensuring robust control logic and data integrity.

Shift micro operations reposition bits within registers through logical, arithmetic, or circular shifts. Logical shifts insert zeros into vacated positions, suitable for unsigned number manipulation, such as multiplication or division by powers of two. Arithmetic shifts preserve the sign bit for signed numbers, enabling efficient scaling. Circular shifts rotate bits cyclically, which is valuable in cryptographic algorithms and data serialisation. These operations enhance computational efficiency by minimising the need for complex arithmetic circuits.

The Arithmetic Logic Shift Unit (ALU) combines arithmetic, logical, and shift operations into a single, efficient hardware unit within the CPU. Its combinational circuit design, featuring a cascaded structure and 4x1 multiplexer, supports 14 operations, including addition, subtraction, and bit shifts, completed in a single clock cycle. This integration reduces hardware complexity, enhances processing speed, and supports diverse applications like arithmetic calculations, bitwise manipulation, and control operations. The ALU's versatility makes it a critical component of modern CPU architecture, enabling rapid and flexible data processing for computational and control tasks.

6. GLOSSARY

Arithmetic Logic Shift Unit (ALU)	- CPU unit combining arithmetic, logical, and shift operations for efficient data processing in one clock cycle.
Arithmetic Shift	- Shifts signed numbers, preserving the sign bit; left shifts multiply, right shifts divide by powers of two.
Bit Manipulation	- Altering specific bits in a register using logic operations for tasks like masking or toggling.
Circular Shift	- Rotates bits in a register, reintroducing shifted-out bits at the opposite end, used in cryptography.
Logical Shift	- Shifts bits left or right, filling vacated positions with zeros, used for unsigned number manipulation.
Logic Micro-Operations	- Bitwise AND, OR, XOR, NOT operations on registers for data manipulation, masking, and encryption.
Masking	- Using AND to isolate specific bits in a register by clearing others.
Register	- High-speed CPU storage for binary data used in micro-operations.
Selective-Clear	- Clears bits in a register where another register's bits are 1 ($A \leftarrow A \wedge \neg B$).
Selective-Complement	- Toggles bits in a register where another register's bits are 1 ($A \leftarrow A \oplus B$).
Selective-Set	- Sets bits in a register to 1 where another register's bits are 1 ($A \leftarrow A \vee B$).
Sign Extension	- Replicates the sign bit in arithmetic right shifts to maintain a signed number's sign.
Truth Table	- Defines outputs of logical operations for all input combinations.

7. TERMINAL QUESTIONS

1. What are logic micro-operations, and how do they differ from arithmetic micro-operations in terms of data manipulation?
2. Explain the bitwise AND operation with an example, including its truth table and a practical application in bit manipulation.
3. How does the XOR micro-operation function, and what is its significance in error detection or encryption algorithms?
4. Describe the NOT micro-operation with an example, and explain its role in complementing register contents.
5. What is the purpose of the selective-clear operation, and how is it implemented using logic micro-operations? Provide an example.
6. How does the mask operation isolate specific bits in a register? Demonstrate with a step-by-step example.
7. Differentiate between logical shift and arithmetic shift, highlighting their specific uses in unsigned and signed number manipulation.
8. Explain the circular left shift operation with an example, and discuss its application in cryptographic algorithms.
9. How does an arithmetic right shift preserve the sign of a signed binary number? Provide an example using an 8-bit signed number.
10. What are the advantages of shift micro-operations in optimising arithmetic computations, such as multiplication or division?
11. Describe the structure of the Arithmetic Logic Shift Unit (ALSU) and explain how its cascaded design supports multi-bit operations.
12. How does the 4x1 multiplexer in the ALSU select between arithmetic, logical, and shift operations? Refer to the functional table.
13. Explain the role of carry-in and carry-out signals in the ALSU's arithmetic operations, such as addition and subtraction.
14. What are the benefits of integrating arithmetic, logical, and shift operations into a single ALSU in terms of hardware efficiency?
15. Provide an example of how the ALSU can be used to perform a logical shift right operation, including the selection variables required.

8. ANSWERS

Self-Assessment Questions

1. True
2. c) AND
3. selective-complement
4. False
5. b) 1110
6. zero
7. True
8. Circular Shift
9. b) Add with carry
10. selective-set
11. True
12. d) Integration
13. OR
14. Subtraction
15. cascaded

Terminal Questions Answers

Answer 1: Logic micro-operations perform bitwise logical functions (AND, OR, XOR, NOT) on register contents, treating bits as independent binary variables. Unlike arithmetic micro-operations, which interpret data as numerical values for operations like addition, logic micro-operations focus on bit-level manipulation for tasks like masking or toggling. For more details, refer to Section 2.

Answer 2: The bitwise AND operation outputs 1 only when both input bits are 1. Example: $A = 1101$, $B = 1011$, $A \wedge B = 1001$. Truth table: $1 \wedge 1 = 1$, $1 \wedge 0 = 0$, $0 \wedge 1 = 0$, $0 \wedge 0 = 0$. It's used for masking to clear specific bits. For more details, refer to Section 2.1.

Answer 3: The XOR micro-operation outputs 1 if input bits differ. Example: $A = 1010$, $B = 1100$, $A \oplus B = 0110$. It's significant in error detection (e.g., parity checks) and encryption (e.g., bit toggling in algorithms). For more details, refer to Section 2.1.

Answer 4: The NOT micro-operation inverts each bit of a register (0 to 1, 1 to 0). Example: $A = 1010$, $\neg A = 0101$. It's used to complement register contents, such as for flag inversion. For more details, refer to Section 2.1.

Answer 5: Selective-clear sets bits in register A to 0 where corresponding bits in B are 1, using $A \leftarrow A \wedge \neg B$. Example: $A = 1010$, $B = 1100$, $\neg B = 0011$, $A \wedge \neg B = 0010$. It clears specific bits for data manipulation. For more details, refer to Section 2.3.

Answer 6: Masking uses AND to isolate bits where the mask (B) is 1. Example: $A = 1010$, $B = 1100$, $A \wedge B = 1000$. Only bits corresponding to 1s in B are retained, useful for extracting subfields. For more details, refer to Section 2.3.

Answer 7: Logical shifts move bits left/right, filling vacated positions with zeros, used for unsigned numbers (e.g., multiplication/division). Arithmetic shifts preserve the sign bit for signed numbers, used in signed arithmetic. For more details, refer to Section 3.

Answer 8: Circular left shift rotates bits left, reintroducing the MSB into the LSB. Example: 11010011 becomes 10100111. It's used in cryptography for cyclic bit patterns. For more details, refer to Section 3.3.

Answer 9: Arithmetic right shift moves bits right, replicating the sign bit. Example: 11111111 (-1 in 8-bit two's complement) shifts to 11111111 (-1), preserving the sign via sign extension. For more details, refer to Section 3.2.

Answer 10: Shift micro-operations enable rapid multiplication (left shift) and division (right shift) by powers of two, reducing hardware complexity and enhancing computational speed. For more details, refer to Section 3.4.

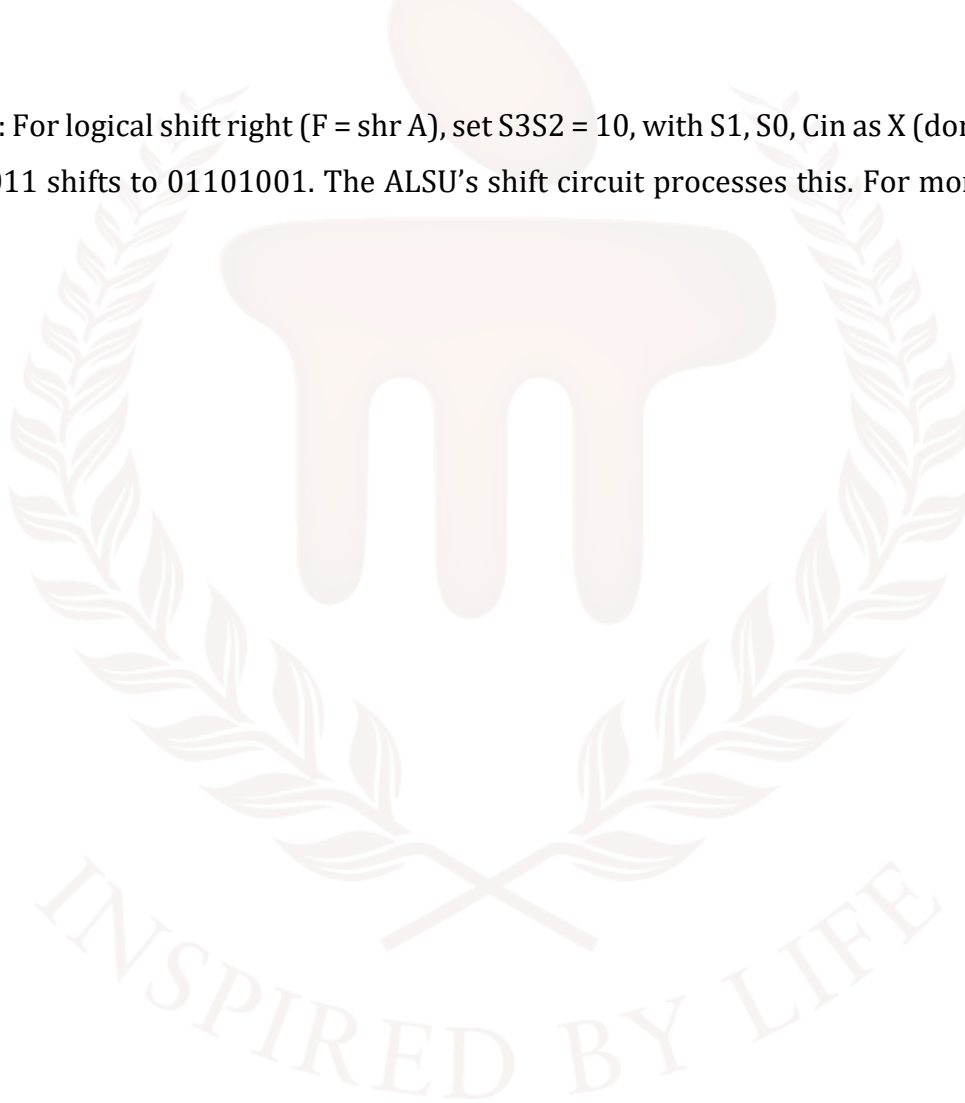
Answer 11: The ALSU is a cascaded structure with n stages for n-bit words, each stage handling one bit with arithmetic, logic, and shift circuits, connected via carry signals for multi-bit operations. For more details, refer to Section 4.2.

Answer 12: The 4x1 multiplexer selects the output of arithmetic, logical, or shift circuits based on control variables S3, S2, S1, S0. The functional table (e.g., S3S2 = 01 for logical operations) defines the operation. For more details, refer to Section 4.3.

Answer 13: Carry-in (Cin) and carry-out signals in the ALSU enable multi-bit arithmetic (e.g., addition: $F = A + B + \text{Cin}$). Carry-out from one stage feeds carry-in to the next, ensuring accurate results. For more details, refer to Section 4.2.

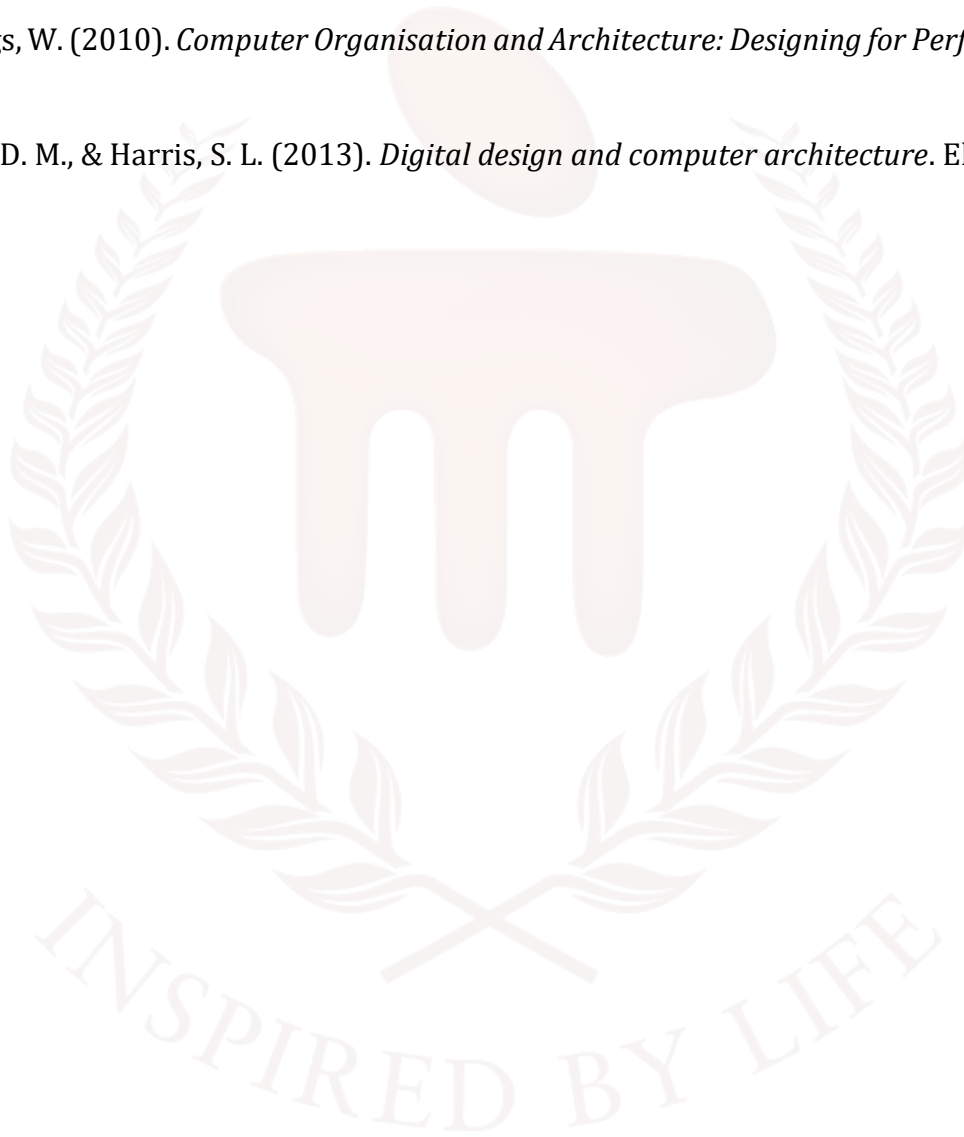
Answer 14: Integrating operations in the ALSU reduces hardware redundancy, simplifies CPU design, and enables single-cycle execution, improving efficiency and flexibility. For more details, refer to Section 4.4.

Answer 15: For logical shift right ($F = \text{shr } A$), set $S3S2 = 10$, with $S1, S0, \text{Cin}$ as X (don't care). Example: $A = 11010011$ shifts to 01101001 . The ALSU's shift circuit processes this. For more details, refer to Section 4.3.



9. REFERENCES

- Mano, M. M. (1982). *Computer System Architecture*. Prentice Hall.
- Stallings, W. (2010). *Computer Organisation and Architecture: Designing for Performance*. Prentice Hall.
- Stallings, W. (2010). *Computer Organisation and Architecture: Designing for Performance*. Prentice Hall.
- Harris, D. M., & Harris, S. L. (2013). *Digital design and computer architecture*. Elsevier.



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 5

Basic Computer Organisation and Design

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
	1.1 Objectives	-	-	
2	Instruction Codes	1 , 2 , 3 , 4 , 5 , 6	1	6 - 11
	2.1 Structure of an Instruction Code	-	-	
	2.2 Types of Instruction Codes	-	-	
	2.3 Classification by Number of Address Fields	-	-	
	2.4 Instruction Formats and Types	-	-	
3	Operation Code	7 , 8 , 9	2	12 -16
	3.1 Opcodes Work: The CPU's Perspective	-	-	
	3.2 Common Opcodes and Their Functions	-	-	
	3.3 The Importance of Opcodes	-	-	
	3.4 Opcodes Enable Complex Operations	-	-	
4	Timing and Control	-	-	17 -27
	4.1 Functions of the Timing and Control Unit	-	-	
	4.2 Major Components of the TCU	-	-	
	4.3 Role of Timing Signals	-	-	
	4.4 Types of Control Organisation	-	-	
	4.5 Timing and Control in Basic Computer Organisation	-	-	
	4.6 Types of Control Units	-	-	
5	Summary	10 , 11 , 12	3	28 -29
6	Glossary	-	-	30 -32
7	Terminal Questions	-	-	33
8	Answers	-	-	34 -36
9	References	-	-	37

1. INTRODUCTION

In the previous unit, you have learned about the fundamentals of computer organisation and data processing. Building on that, this unit explores Logic and Shift Micro Operations, which are essential for manipulating data at the bit level within a computer system. Logic micro operations involve bitwise operations such as AND, OR, XOR, and NOT, which are used to perform logical decision-making and control tasks. Shift micro operations, on the other hand, involve moving bits within a register to the left or right, and include logical shifts (which insert zeros), arithmetic shifts (which preserve the sign bit), and circular shifts (which rotate bits around). These operations are crucial for tasks like multiplication, division, and data alignment. All these functionalities are integrated into the Arithmetic Logic Shift Unit (ALU), a key component of the CPU that combines arithmetic, logic, and shift capabilities to execute complex instructions efficiently.

In this unit, you will learn how a computer's internal structure and control mechanisms work together to execute instructions and manage data efficiently. You'll discover how instruction codes—the language of computers—are structured, with each instruction containing an operation code (opcode) that tells the computer what action to perform, and an address part that points to where the necessary data is located in memory. Understanding this structure will help you see how computers interpret and execute various commands, from simple arithmetic to complex data manipulation.

You will also explore the concept of the instruction cycle, which is the step-by-step process a computer follows to fetch, decode, execute, and store instructions. This cycle is orchestrated by the control unit, which acts like a conductor, ensuring that every micro-operation happens in the correct sequence and at the right time. By examining timing and control, you'll gain insight into how computers maintain precise coordination between their different components, preventing errors and maximising efficiency.

Additionally, this unit will introduce you to different addressing modes, such as direct and indirect addressing, which determine how the computer locates the data it needs for each operation. You'll learn why these modes are important for flexibility and efficiency in programming and hardware design. By the end of this unit, you'll have a clear understanding of the fundamental principles that underpin computer organisation and design, including how the central processing unit (CPU) manages instructions, how memory and registers interact, and how timing and control signals synchronise all activities within the system.

This foundational knowledge is crucial for anyone interested in computer architecture, programming, or hardware engineering, as it reveals the inner workings that make modern computing possible

1.1. Objectives

After studying this unit, you should be able to:

- Define basic computer organisation and its main components.
- Discuss the instruction codes, operation codes, and addressing modes.
- Summarise the role of the control unit in timing and control.
- Identify different instruction formats and their execution.



2. INSTRUCTION CODES

Instruction codes in computer architecture are binary sequences that instruct the computer to perform specific operations, forming the core of how a computer executes tasks. Each instruction code is typically divided into two main components: the operation code (opcode), which defines the operation, such as addition, subtraction, shift, or complement, and the address or operand part, which specifies the registers or memory locations involved in the operation. These codes are stored in memory alongside data and are fetched by the control unit, which decodes them and initiates the required sequence of micro-operations to execute the instruction.

The instruction format varies depending on the architecture but often includes bits for the opcode, address, and sometimes an addressing mode bit to distinguish between direct and indirect addressing. For example, in a system with 4096 memory words, 12 bits are used for the address, and the remaining bits specify the operation. The instructions are executed through an instruction cycle comprising fetch, decode, and execute phases, ensuring that the correct sequence of operations is performed. This system allows for a wide range of instructions, including arithmetic, logical, data transfer, and control operations, making the instruction code fundamental to the stored program concept and the versatility of modern computers.

2.1 Structure of an Instruction Code

Instruction codes are bits that instruct the computer to execute a specific operation. An instruction comprises groups called fields. These fields include:

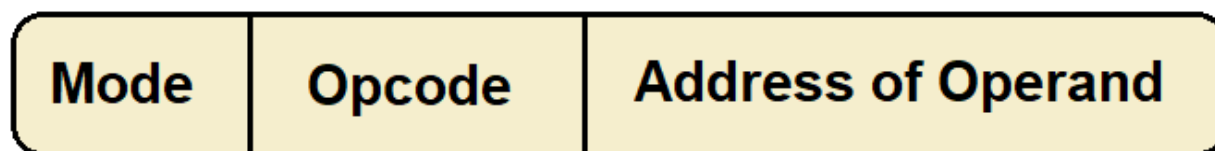


Figure 5-1 Structure of an Instruction Code

- The Operation code (Opcode) field determines the process that needs to be performed.
- The Address field contains the operand's location, i.e., register or memory location.
- The Mode field specifies how the operand is located.

So in Figure 5-1, Instruction code, commonly referred to as the instruction set, constitutes a collection of binary codes that delineate the operations executable by a computer processor. The structure of an instruction code varies according to the processor's architecture but generally comprises the following components:

- **Opcode (Operation Code):** This segment specifies the particular operation to be performed by the processor, encompassing arithmetic functions such as addition, subtraction, multiplication, and division, among others.
- **Operand(s):** This component identifies the data upon which the operation is to be executed. Depending on the architectural design, operands may represent registers containing values, memory addresses referencing data storage locations, or immediate constant values embedded within the instruction itself.
- **Addressing Mode:** This element defines the interpretation method for the operand(s). It may indicate direct addressing, where the operand specifies a memory location; indirect addressing, where the operand refers to a memory address stored in a register; or immediate addressing, where the operand is a constant value included in the instruction.
- **Prefixes or Modifiers:** Certain instruction sets incorporate additional prefixes or modifiers that alter the instruction's behavior. These may impose conditional execution criteria or specify iterative execution until a designated condition is satisfied.

Instruction codes are stored in memory alongside program data. During program execution, the control unit fetches and decodes these codes to orchestrate the requisite sequence of micro-operations.

The precise format and length of each instruction component are contingent upon the underlying architecture; however, their fundamental purpose is to facilitate the systematic interpretation and execution of diverse operations essential to the processor's functionality.

2.2 Types of Instruction Codes

Instruction codes are essential in defining the operations a computer processor can execute. These codes are categorised based on their format, the nature of the operations they perform, and the way they reference operands. Understanding the different types of instruction codes is crucial for grasping how processors interpret and execute instructions.

Memory-Reference Instructions

Memory-reference instructions are designed to operate on data stored in the computer's memory. These instructions typically include an opcode, a memory address field, and an addressing mode bit that specifies whether the address is direct or indirect. Direct addressing means the instruction contains the actual memory address of the operand, while indirect addressing means the address field points to a memory location that holds the effective address. Memory-reference instructions are used for fundamental operations such as arithmetic calculations, logical operations, and data transfer between memory and registers. These instructions often form the bulk of any instruction set, as they enable manipulation of data stored in memory.

Register-Reference Instructions

Register-reference instructions operate directly on the processor's internal registers rather than accessing memory. These instructions typically use a reserved opcode and a set of bits to specify the particular register operation to be performed. Examples include clearing a register, complementing its contents, incrementing or decrementing values, and shifting bits. Because these instructions work on registers, they execute faster than memory-reference instructions and are essential for controlling processor state and performing quick operations on data held within the CPU.

Input-Output (I/O) Instructions

Input-output instructions manage communication between the processor and peripheral devices such as keyboards, printers, disk drives, and network interfaces. These instructions typically have a unique opcode to distinguish them from other instruction types and include fields specifying the particular I/O device and operation. I/O instructions enable the processor to send data to or receive data from external devices, facilitating interaction with the external environment. This category is critical for real-world computing tasks where data input and output are required.

2.3 Classification by Number of Address Fields

Instruction codes can also be classified based on the number of address fields they contain:

- **Three-Address Instructions:** These instructions include three address fields, typically two source operands and one destination operand. This format allows for complex operations where two operands are combined, and the result is stored in a third location. It provides flexibility but requires longer instruction sizes.

opcode	Destination address	Source address	Source address	mode
--------	---------------------	----------------	----------------	------

Figure 5-2 Three-address instruction

- **Two-Address Instructions:** These instructions have two address fields, usually specifying a source and a destination. Often, one of the operands doubles as the destination where the result is stored, which simplifies instruction length but may require additional instructions to preserve the original data.

opcode	Destination address	Source address	mode
--------	---------------------	----------------	------

Figure 5-3 Two-address instruction

- **One-Address Instructions:** These instructions contain a single address field and often rely on an implicit accumulator register. The operand is fetched from the specified address, and the operation is performed with the accumulator. This format reduces instruction size but can limit flexibility.

opcode	operand/address of operand	mode
--------	----------------------------	------

Figure 5-4 One-address instruction

- **Zero-Address Instructions:** Used primarily in stack-based architectures, zero-address instructions do not explicitly specify operands. Instead, operands are implicitly located on the top of the stack. Operations pop operands from the stack, perform the computation, and push the result back onto the stack. This approach simplifies instruction design but requires a stack-oriented processor.

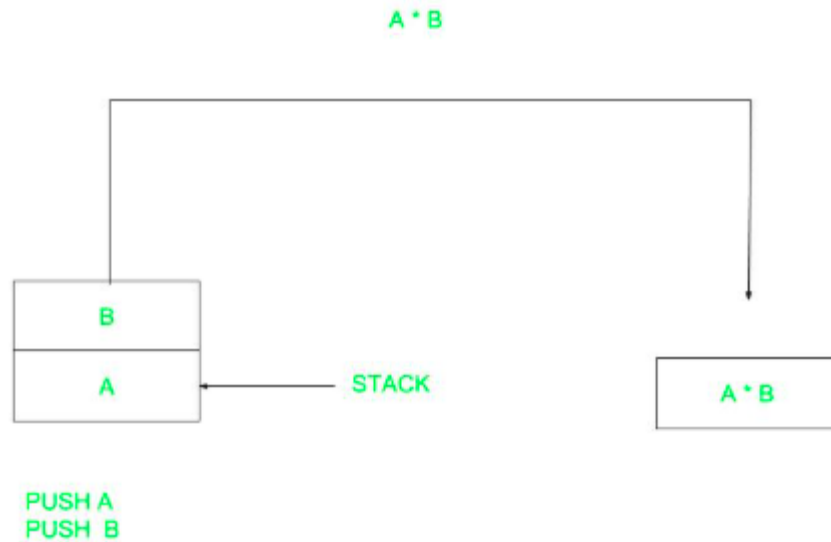


Figure 5-5 Zero-address instruction

2.4 Instruction Formats and Types

Instruction codes can be organised in different formats, depending on the computer's architecture.

Fixed-Length vs. Variable-Length Instructions

- **Fixed-Length:** Every instruction is the same size (e.g., 16 or 32 bits). Easier for hardware to process, common in RISC architectures.
- **Variable-Length:** Instructions can vary in size. Allows for more complex instructions but can be harder to decode, typically in CISC architectures.

Types of Instructions

- **Memory-Reference:** Involve reading from or writing to memory.
- **Register-Reference:** Operate directly on CPU registers.
- **Input/Output:** Handle data transfer between the CPU and I/O devices.

Table 1: Example Table

Type	Example Operation	Use Case
Memory-Reference	LOAD, STORE	Data transfer
Register-Reference	CLEAR, COMPLEMENT	Logical operations
Input/Output	IN, OUT	Communicating with peripherals

SELF-ASSESSMENT QUESTIONS - 1

True or False:	
1	An instruction code is typically divided into an operation code and a data part. (T/F)
Multiple Choice Questions	
2	Which component of an instruction code specifies the action to be performed? a) Address field b) Mode field c) Operation code (Opcode) d) Operand
Fill in the Blanks:	
3	An instruction code is a _____ sequence that instructs the computer to perform specific operations

3. OPERATION CODE

An opcode is a unique binary or hexadecimal value embedded within a machine instruction. It acts as a command, instructing the CPU to carry out a specific operation—such as addition, subtraction, data movement, logical comparison, or program control (like jumping to another instruction).

Analogy:

Think of a machine instruction as a sentence in a language. The opcode is the verb, specifying the action, while the operands are the objects or subjects involved in the action.

Visual Representation

Here's a simple ASCII diagram of an instruction format with an opcode

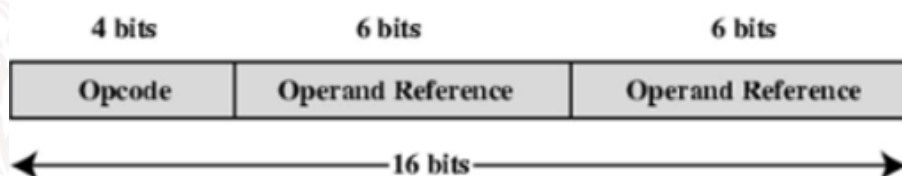


Figure 5-6 ASCII Diagram

For example, in a 16-bit instruction, the first 4 bits might be the opcode, and the remaining 12 bits specify the operands.

Real-World Example

Suppose you have the following instruction in assembly:

ADD R1, R2

- The assembler converts this to a binary instruction.
- The opcode for **ADD** might be **0001**.
- The operands specify registers R1 and R2.
- The CPU decodes **0001** as “add the contents of R1 and R2”.

Structure of an Instruction

A machine instruction typically consists of:

- Opcode: Specifies the operation (e.g., add, move, jump).

- **Operands:** Specify the data, registers, or memory addresses involved.
- **Addressing Mode (optional):** Indicates how to interpret the operands (direct, indirect, immediate, etc.).

Table 2: Example Format

Field	Example Value	Description
Opcode	0001	Add operation
Operand1	0100	Register or memory address
Operand2	0110	Register or memory address

In a 16-bit instruction, the first 4 bits might be the opcode, and the remaining 12 bits are divided among operands and addressing modes.

3.1 Opcodes Work: The CPU's Perspective

Fetch: The CPU fetches the instruction from memory into the instruction register.

Decode: The control unit examines the opcode to determine which operation to perform. This step is crucial because the CPU must distinguish between hundreds of possible actions.

Execute: The CPU carries out the operation specified by the opcode, using the operands provided.

Store/Write-back: If the operation produces a result, it is stored in a register or memory.

Example in Action

Let's say the CPU encounters the binary instruction:

0001 0100 0110

- **0001** (opcode): ADD
- **0100** (operand 1): Register 4
- **0110** (operand 2): Register 6

The CPU decodes this as “Add the contents of Register 4 and Register 6.”

3.2 Common Opcodes and Their Functions

Table 3: Typical opcodes found in many instruction sets

Opcode (Binary)	Assembly Mnemonic	Operation	Description
0000	NOP	No Operation	Do nothing, often used for timing
0001	ADD	Addition	Add two operands
0010	SUB	Subtraction	Subtract second operand from first
0011	MOV	Move	Copy data from source to destination
0100	JMP	Jump	Change program flow to another instruction
0101	AND	Logical AND	Bitwise AND operation
0110	OR	Logical OR	Bitwise OR operation
0111	CMP	Compare	Compare two operands (sets flags)
1000	LOAD	Load	Load data from memory to register
1001	STORE	Store	Store data from register to memory

Each CPU architecture (such as x86, ARM, MIPS, RISC-V) defines its own set of opcodes, collectively called its instruction set architecture (ISA).

3.3 The Importance of Opcodes

Efficiency

The design of opcodes directly affects how quickly and efficiently a CPU can process instructions. Well-structured opcodes allow for faster decoding and execution, which is vital for performance-critical applications.

Compatibility

Software is often compiled for a specific instruction set. The opcode set determines whether a program can run on a particular CPU. This is why software compiled for ARM CPUs won't run on x86 CPUs and vice versa.

Security

Certain opcodes are privileged and can only be executed in kernel or supervisor mode. This prevents user programs from performing dangerous operations, such as directly accessing hardware or altering system memory.

Extensibility

Modern CPUs sometimes add new opcodes to support emerging technologies (like vector processing, cryptography, or AI acceleration) while maintaining backwards compatibility

3.4 Opcodes Enable Complex Operations

Although each opcode typically represents a simple operation, combining them enables the creation of complex programs. For example, a loop in a program might use:

- An opcode for addition or subtraction (to update a counter)
- A compare opcode (to check if the loop should continue)
- A jump opcode (to repeat the loop)

By sequencing opcodes, programmers and compilers can direct the CPU to perform everything from basic calculations to advanced graphics rendering or artificial intelligence.

Challenges in Opcode Design

- **Balancing Simplicity and Power:** Too many opcodes make the CPU design complex; too few can make programming inefficient.
- **Backward Compatibility:** New CPUs must often support old opcodes so that existing software continues to work.
- **Security:** Preventing misuse of powerful opcodes that could compromise system integrity.

SELF-ASSESSMENT QUESTIONS - 2

True or False:	
4	The opcode is the part of a machine instruction that specifies the data involved in the operation. (T/F)
Multiple Choice Questions	
5	Which of the following is an analogy for the opcode in a machine instruction? a) The subject b) The object c) The verb d) The adjective
Fill in the Blanks:	
6	The CPU fetches an instruction from memory into the ____.

4. TIMING AND CONTROL

Timing and control are essential components within a computer system that determine when specific operations should occur. Timing ensures that all elements of the system are synchronised, allowing them to operate together at precise intervals. Control, on the other hand, directs the flow of data and governs how instructions are executed by various parts of the system. Together, these processes work in conjunction with the CPU to guarantee that all operations are performed accurately and at the correct times.

The Timing and Control Unit (TCU) is responsible for generating the timing signals that coordinate the different operations and producing control signals that regulate the movement of data between system components.

Block diagram

Figure 5-7 illustrates the central role of the Control Unit within a computer's CPU, highlighting how it manages and coordinates the flow of information and instructions. The Control Unit receives crucial input from three primary sources: the Instruction Register, Flags, and the Clock.

The Instruction Register supplies the current instruction that the CPU needs to execute, providing the Control Unit with the necessary details to determine the next steps.

Flags, which are special status bits, inform the Control Unit about the outcomes of previous operations, such as whether a calculation resulted in zero, a carry, or an overflow. The Clock delivers regular timing signals, ensuring that all operations are precisely synchronised and executed at the correct intervals.

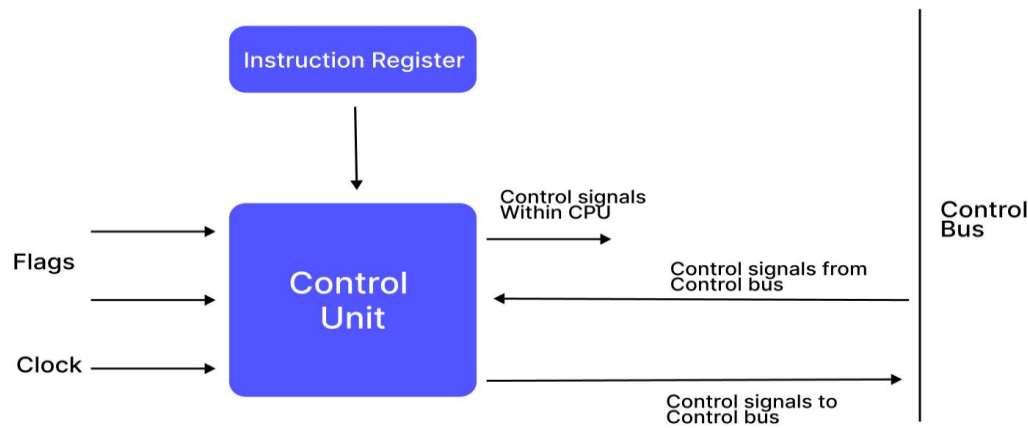


Figure 5-7 Block diagram

Once the Control Unit gathers these inputs, it processes the information and generates a series of control signals. These signals are sent internally within the CPU to direct the actions of various components, such as the Arithmetic Logic Unit (ALU), registers, and other functional units.

For example, the Control Unit might instruct the ALU to perform a specific calculation or tell a register to store or retrieve data. This internal communication is essential for the orderly and efficient execution of instructions, as it ensures that each part of the CPU knows exactly what to do and when to do it, based on the current instruction and system status.

In addition to managing internal CPU operations, the Control Unit also communicates with external components through the Control Bus. It sends control signals out to the Control Bus to coordinate activities such as reading from or writing to memory, or interacting with input/output devices.

The Control Unit can also receive signals from the Control Bus, allowing it to respond to requests or changes in the system, such as interrupts or external commands. This two-way communication enables the CPU to work seamlessly with the rest of the computer system, ensuring that data flows smoothly and operations are carried out accurately and efficiently. Overall, the Control Unit acts as the conductor of the computer's operations, orchestrating every action with precise timing and control.

4.1 Functions of the Timing and Control Unit

The Timing and Control Unit (TCU) is a fundamental and indispensable component within a computer system's architecture. It acts as the central coordinator for the processor's operations, ensuring that all internal components and subsystems work together seamlessly and in perfect temporal harmony.

The TCU's primary responsibility is to orchestrate the precise timing and control signals that regulate the sequential execution of instructions, thereby enabling the processor to function efficiently and reliably. Without the TCU, the processor's various elements—such as the arithmetic logic unit (ALU), registers, memory, and input/output devices—would operate in a disorganised and conflicting manner, leading to errors, data corruption, and system instability.

The TCU organises and manages discrete instances of sequenced execution processes across the entire system, ensuring that each operation occurs at the correct moment in time. This synchronisation is vital for maintaining the integrity of data flow and the correctness of computational results.

The major components of the TCU include timing signals and control signals, each playing a critical role in the processor's operation. In the following sections, we will explore these components in greater depth, highlighting their specific functions and importance.

4.2 Major Components of the TCU

The TCU comprises two essential types of signals:

- **Timing Signals:** These signals provide the temporal framework for the processor's operations. They dictate when specific actions should begin and end, ensuring that all parts of the system operate in lockstep.
- **Control Signals:** These signals direct the processor's functional units to perform specific tasks, such as executing arithmetic operations, moving data between registers and memory, or interacting with input/output devices.

Together, timing and control signals enable the TCU to maintain order and precision throughout the instruction execution cycle.

4.3 Role of Timing Signals

Timing signals are the heartbeat of the computer system. They ensure that all operations occur in a synchronised and orderly fashion, preventing conflicts such as data collisions or simultaneous access to shared resources. These signals are typically generated by a master clock, which produces regular pulses at fixed intervals. Each pulse acts as a trigger for various components to perform their designated operations in a coordinated sequence.

Timing signals are crucial in synchronising the three main phases of the instruction cycle: Fetch, Decode, and Execute. Their role in each phase is described in detail below:

Instruction Fetch

During the instruction fetch phase, a timing signal activates the process of retrieving the next instruction from memory. This signal ensures that the program counter (PC) correctly points to the memory address of the instruction to be fetched. The instruction is then transferred from memory into the instruction register (IR), preparing it for decoding. The timing signal guarantees that this transfer happens at the precise moment, preventing any overlap or loss of data.

Instruction Decode

Once the instruction is loaded into the IR, the next timing signal initiates the decoding process. This phase involves interpreting the instruction's opcode and operands to determine the specific operations required. The control unit uses this timing signal to generate appropriate control signals that direct the ALU, memory, or other components to prepare for execution. Proper timing during decoding ensures that the instruction is correctly understood and that subsequent operations are accurately planned.

Execute

In the execute phase, timing signals regulate when the ALU performs arithmetic or logical operations, or when data is transferred to or from memory. These signals determine the exact moment the operation begins and ends, as well as when the results are stored back into registers or memory locations. This precise timing is essential to maintain data integrity and to ensure that subsequent instructions operate on the correct data.

4.4 Types of Control Organisation

The TCU's control signals can be generated through different organisational methods, each with its own advantages:

- **Hardwired Control:** This approach uses fixed digital circuits such as gates, flip-flops, and decoders to produce control signals. It offers fast execution speeds due to direct hardware implementation but lacks flexibility for modification or updates.
- **Microprogrammed Control:** In this method, control signals are generated by a microprogram stored in a special control memory. This approach provides greater flexibility and ease of modification, as control sequences can be changed by updating the microprogram rather than redesigning hardware.

4.5 Timing and Control in Basic Computer Organisation

Control and timing are absolutely vital in computer organisation, especially during the Fetch-Decode-Execute cycle—the fundamental sequence of steps a computer follows to process and execute program instructions. This cycle ensures that instructions are processed one at a time in a well-coordinated, step-by-step manner. The precise orchestration of timing and control signals is what enables the processor's various components to operate in harmony, preventing errors and ensuring correct program execution.

The Instruction Cycle: Phases and Their Coordination

The instruction cycle, often referred to as the Fetch-Decode-Execute cycle, is the backbone of a computer's operation. It consists of several distinct phases, each of which relies heavily on carefully managed timing and control signals for proper execution:

Fetch Phase

- **Purpose:** Retrieve the next instruction to be executed from main memory.
- **Process:**
 - The control unit issues specific timing signals to initiate the memory read operation.
 - The address of the next instruction is held in the Program Counter (PC).
 - The instruction is fetched from memory and loaded into the Instruction Register (IR).
- **Importance of Timing:**
 - Accurate timing ensures that the memory and data buses are available and not in conflict with other operations.
 - Prevents data corruption and ensures the correct instruction is loaded.

Decode Phase

- **Purpose:** Interpret the fetched instruction to determine the required operation.

- Process:
 - The instruction in the IR is analysed by the control unit.
 - Control signals are generated to configure the Arithmetic Logic Unit (ALU), registers, and other components for the upcoming operation.
- Importance of Control:
 - Control signals must be precisely sequenced to set up the correct pathways and modes for the involved hardware.
 - Ensures that the system interprets the instruction correctly, distinguishing between operations like addition, subtraction, memory access, etc.

Operand Fetch (if required)

- Purpose: Retrieve any additional data (operands) needed to execute the instruction.
- Process:
 - If the instruction references data in memory, the effective address (EA) is calculated.
 - The control unit issues memory read cycles to fetch the operand into a CPU register.
 - This can be represented as: $\text{Register} \leftarrow \text{Memory}[\text{EA}]$.
- Importance of Timing and Control:
 - Ensures operands are fetched at the correct moment, avoiding bus conflicts and ensuring data integrity.
 - Control signals must coordinate the address calculation and data transfer.

Execute Phase

- Purpose: Perform the actual operation specified by the instruction.
- Process:
 - The ALU or other relevant subsystem carries out the operation (e.g., arithmetic calculation, logical comparison, data transfer).
 - Control signals direct the flow of data between the ALU, registers, memory, and I/O devices as required.
- Importance of Control:
 - Precise control signals ensure the correct execution of the operation and the proper routing of results.

- Timing signals guarantee that each step occurs in the correct sequence, preventing errors or data loss.

Synchronisation and Sequencing

Throughout the entire instruction cycle, synchronisation and proper sequencing are achieved through a combination of timing and control signals:

- Timing Signals: Regular pulses (often from a system clock) that dictate when each phase of the cycle should begin and end.
- Control Signals: Direct the specific actions of each hardware component, ensuring that data moves correctly and operations are performed as intended.

4.6 Types of Control Units

There are two main types of control units in a computer's CPU

1. Hardwired Control Unit
2. Microprogrammable control unit

Hardwired Control Unit

Hardwired control units generate control signals using rigid logic circuits. These circuits are designed for specific operations, using fast-controlled hardwired units. The speed of control signals generated by hardwired units is incredible because they depend on physical circuits, and these circuits have the capability of working extremely fast.

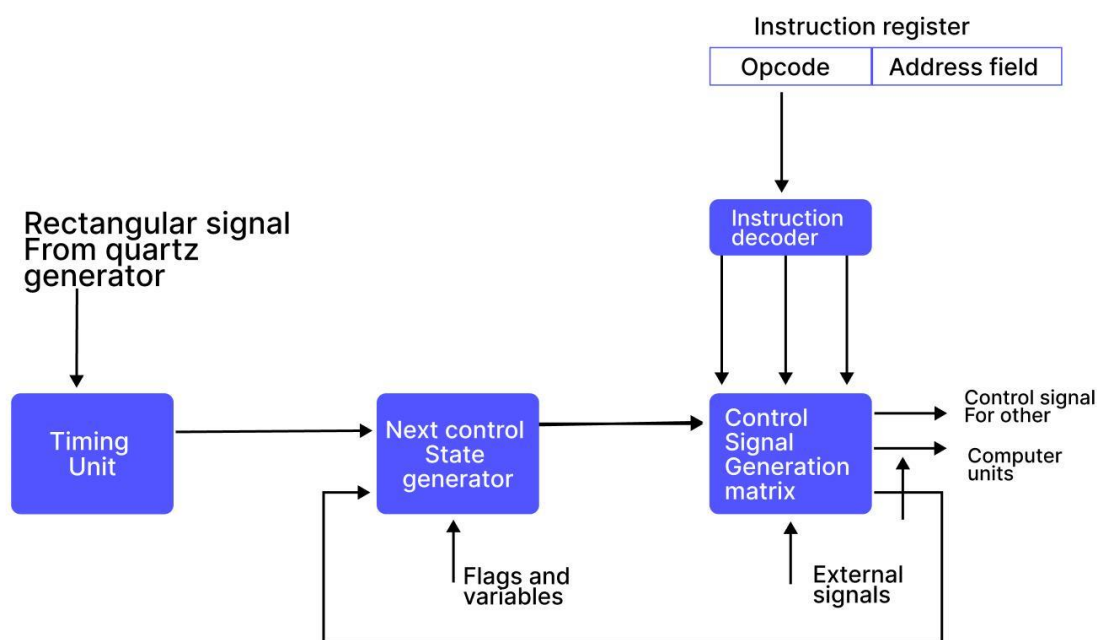


Figure 5-8 Hardwired Control Unit

Figure 5-8 illustrates the internal operation of the control unit in a computer system, focusing on how timing and control signals are generated and managed. It begins with a rectangular signal from a quartz generator, which acts as the system clock. This signal is fed into the Timing Unit, which produces regular timing pulses that synchronise all subsequent operations within the processor. These timing pulses are essential for maintaining the orderly execution of instructions and preventing conflicts between different components.

Next, the Timing Unit sends signals to the Next Control State Generator, which determines the next step in the control sequence based on current timing, status flags, and variables from the CPU. Simultaneously, the Instruction Register holds the current instruction, split into an opcode and an address field. The Instruction Decoder interprets this instruction and sends information to the Control Signal Generation Matrix. The matrix acts as the decision-making hub, receiving inputs from the decoder, the Next Control State Generator, and any external signals such as interrupts.

Finally, the Control Signal Generation Matrix produces the necessary control signals that direct the actions of various computer units, such as the ALU, memory, and I/O devices. It also sends feedback to the Next Control State Generator, updating it with the latest status for the next cycle. This

continuous loop ensures that every instruction is executed correctly and efficiently, with all parts of the system working in perfect coordination.

Microprogrammed Control Unit

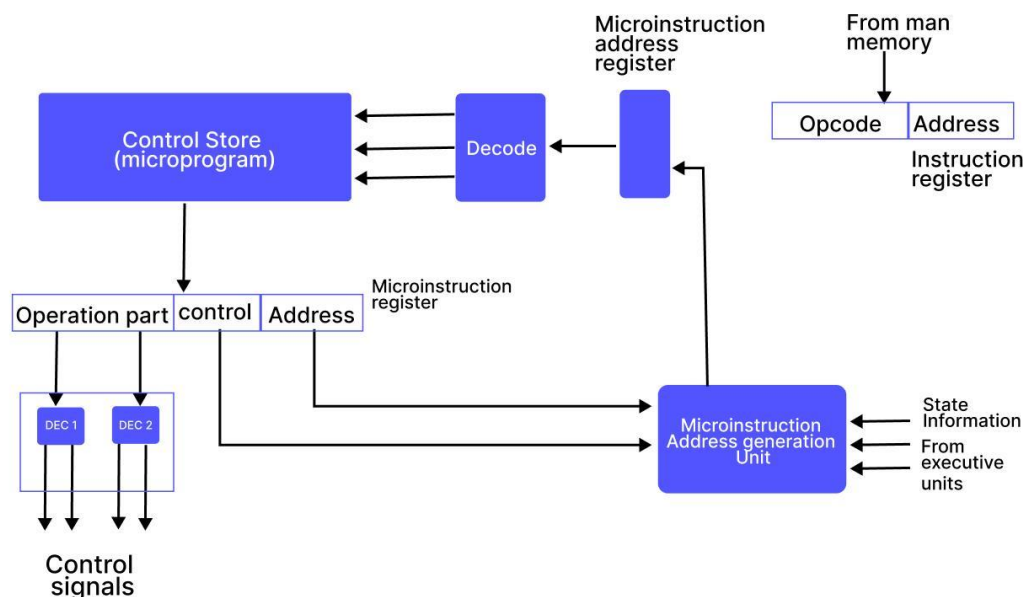


Figure 5-9 Microprogrammed Control Unit

The provided Figure 5-9 illustrates a Microprogrammed Control Unit, a crucial component within a computer's CPU, responsible for generating the precise control signals that orchestrate all internal operations. At its core, the system translates high-level machine instructions, stored in the Instruction Register, into a sequence of low-level microinstructions. The opcode of a fetched instruction is first decoded to identify the starting address of a specific microprogram routine within the Control Store. This Control Store houses the microprogram, a collection of elemental commands or microinstructions, each specifying minute actions like enabling data paths or triggering ALU operations.

Once a microinstruction is fetched and loaded into the Microinstruction Register, its "Operation part" is further decoded to produce the actual electrical control signals disseminated throughout the CPU, dictating the behaviour of registers, the ALU, and other functional units. Concurrently, the "Address part" of the microinstruction, alongside state information and feedback from executive units, feeds

into the Microinstruction Address Generation Unit. This unit dynamically determines the address of the next microinstruction, allowing for sequential execution, conditional branching based on CPU state, and efficient flow of the control process until the current machine instruction is fully executed, thereby enabling complex computational tasks.

SELF-ASSESSMENT QUESTIONS - 3

True or False:	
7	Timing signals dictate the flow of data, while control signals provide the temporal framework for operations. (T/F)
Multiple Choice Questions	
8	Which component is responsible for generating timing and control signals? a) Arithmetic Logic Unit (ALU) b) Program Counter (PC) c) Timing and Control Unit (TCU) d) Instruction Register (IR)
Fill in the blanks	
9	Timing ensures that all elements of the system are synchronised, allowing them to operate together at precise ____ .
Multiple Choice Questions	
10	Which of the following best describes a Hardwired Control Unit? A. It uses microinstructions stored in memory to generate control signals B. It decodes high-level instructions into sequences of microinstructions C. It generates control signals through fixed logic circuits and timing mechanisms D. D. It is slower than a microprogrammed control unit due to complex logic
11	What is the primary function of the Control Signal Generation Matrix in a Hardwired Control Unit? A. Acts as the decision-making hub to produce control signals B. Produces timing pulses to synchronise operations C. Stores all the microprograms used for control D. Generates clock signals from the quartz crystal
12	In a Microprogrammed Control Unit, what does the Control Store contain? A. Status flags and timing units

	B. A set of microprograms made up of microinstructions C. Decoded instructions and ALU operations D. The main system clock and quartz generator
13	How does a Microprogrammed Control Unit determine the next microinstruction to execute? A. By polling the ALU and memory units B. By using a fixed sequence logic generator C. Through the Microinstruction Address Generation Unit using opcode and status D. From the instruction decoder's output directly
14	Which of the following is a key advantage of a Hardwired Control Unit over a Microprogrammed one? A. Easier to modify and update B. Capable of conditional branching during execution C. More flexible in supporting new instructions D. Generally faster due to direct hardware implementation



5. SUMMARY

This unit discusses basic computer organisation and design principles, focusing on how computers execute instructions and manage data. It introduces instruction codes, the binary language computers understand, which are composed of an operation code (opcode) specifying the action and an address part indicating data location. Understanding this structure is crucial for comprehending how computers perform diverse tasks.

The core of computer operation is the instruction cycle, a sequential process of fetching, decoding, executing, and storing instructions. The control unit precisely manages this cycle, acting as a central orchestrator, ensuring timely and accurate micro-operations. Different addressing modes, such as direct and indirect, are explored, highlighting their importance for programming flexibility and hardware efficiency. This foundational knowledge provides insight into CPU instruction management, memory and register interactions, and the synchronisation of system activities through timing and control signals.

Instruction codes, binary sequences, are the essence of computer task execution. They are typically divided into the opcode, which defines the operation (e.g., addition, shift), and the address or operand part, which specifies data locations. These codes are stored in memory, fetched by the control unit, and decoded to initiate micro-operations. Instruction formats vary but commonly include opcode, address, and sometimes an addressing mode bit. The instruction cycle ensures correct operation sequencing with its fetch, decode, and execute phases.

The structure of an instruction code includes fields for the opcode (the operation), the address (operand location), and the mode (how the operand is located). Instruction codes are categorised by their function: memory-reference instructions operate on data in memory (e.g., arithmetic, logical operations); register-reference instructions work directly on CPU registers for faster execution (e.g., clearing, complementing registers); and input-output (I/O) instructions manage communication with peripheral devices. Instructions are also classified by the number of address fields they contain: three-address, two-address, one-address, and zero-address (stack-based) instructions, each offering different levels of complexity and efficiency. Instruction formats can be fixed-length (simpler for hardware, common in RISC) or variable-length (more complex instructions, typical in CISC).

An opcode is a unique binary value within a machine instruction, acting as a command to the CPU for a specific operation like addition, data movement, or program control. It's the "verb" in the instruction "sentence." The CPU fetches an instruction, decodes its opcode to identify the operation, executes the operation using operands, and stores the result. Common opcodes include NOP, ADD, SUB, MOV, JMP, AND, OR, CMP, LOAD, and STORE, each defined by the CPU's instruction set architecture (ISA). Opcodes are critical for CPU efficiency, software compatibility, system security (privileged opcodes), and extensibility to new technologies.

Timing and control are crucial for synchronising computer operations. The Timing and Control Unit (TCU) generates timing signals to coordinate operations and control signals to direct data flow and instruction execution. The Control Unit receives input from the Instruction Register, status flags, and the clock to generate internal control signals for components like the ALU and registers, and external control signals via the Control Bus for memory and I/O devices. Timing signals provide the temporal framework, ensuring operations begin and end precisely. Control signals direct functional units to perform specific tasks.

During the Fetch-Decode-Execute cycle, timing signals coordinate fetching instructions, while control signals direct the decoding and execution phases, including operand fetching and result storage. This synchronisation prevents conflicts and ensures accurate program execution. Control units are primarily of two types: hardwired control units, which use fixed logic circuits for high speed but lack flexibility, and microprogrammed control units, which use a microprogram stored in control memory for greater flexibility and ease of modification.

6. GLOSSARY

Addressing Mode	- The method by which a computer instruction locates the data (operand) it needs for an operation. Examples include direct, indirect, and immediate addressing.
ALU (Arithmetic Logic Unit)	- A digital circuit within the CPU that performs arithmetic (addition, subtraction, etc.) and logical (AND, OR, NOT) operations.
Control Bus	- A special type of memory used in microprogrammed control units to store microprograms, which are sequences of microinstructions.
Control Store	- A special type of memory used in microprogrammed control units to store microprograms, which are sequences of microinstructions.
Control Unit (CU)	- The component of the CPU that directs and coordinates the activities of the entire computer system. It interprets instructions and generates control signals to execute them.
CPU (Central Processing Unit)	- The "brain" of the computer, responsible for executing instructions, performing calculations, and managing the flow of data.
Decode Phase	- The phase of the instruction cycle where the control unit interprets the fetched instruction to determine the operation to be performed and the operands involved.
Direct Addressing	- An addressing mode where the address field in an instruction contains the actual memory address of the operand.
Execute Phase	- The phase of the instruction cycle where the CPU performs the operation specified by the instruction, using the identified operands.
Fetch Phase	- The initial phase of the instruction cycle where the next instruction to be executed is retrieved from main memory.
Flags	- Special status bits within the CPU that indicate the outcome of previous operations (e.g., zero result, carry, overflow).

Hardwired Control Unit	- A type of control unit that uses fixed digital circuits (gates, flip-flops, decoders) to generate control signals, offering high speed but low flexibility.
Indirect Addressing	- An addressing mode where the address field in an instruction points to a memory location that contains the effective (actual) address of the operand.
Input-Output (I/O) Instructions	- Instructions that manage communication between the processor and peripheral devices (e.g., keyboards, printers).
Instruction Code	- A binary sequence that instructs the computer to perform a specific operation. It forms the core of how a computer executes tasks.
Instruction Cycle	- The fundamental, step-by-step process a computer follows to fetch, decode, execute, and store instructions. Also known as the Fetch-Decode-Execute cycle.
Instruction Decoder	- A component within the control unit that interprets the opcode of an instruction to determine the specific operation.
Instruction Format	- The layout or structure of an instruction code, defining the arrangement of its fields (e.g., opcode, address, addressing mode).
Instruction Register (IR)	- A CPU register that holds the instruction currently being executed.
Instruction Set Architecture (ISA)	- The complete set of instructions that a particular CPU can understand and execute, including its defined opcodes.
Memory-Reference Instructions	- Instructions designed to operate on data stored in the computer's main memory.
Microinstruction	- A low-level elemental command within a microprogram that specifies minute actions like enabling data paths or triggering ALU operations.
Microinstruction Address Generation Unit	- A component in a microprogrammed control unit that determines the address of the next microinstruction to be fetched.

Microinstruction Register	- A register that holds the currently fetched microinstruction in a microprogrammed control unit.
Microprogram	- A collection of elemental commands (microinstructions) stored in a control store, used to generate control signals in a microprogrammed control unit.
Microprogrammed Control Unit	- A type of control unit where control signals are generated by a microprogram stored in a special control memory, offering flexibility.
Opcode (Operation Code)	- The field within an instruction code that specifies the particular operation to be performed by the processor (e.g., ADD, SUB, MOV).
Operand	- The data or memory location upon which an operation is to be executed, specified by the address part of an instruction.
Program Counter (PC)	- A CPU register that holds the memory address of the next instruction to be fetched.
Register-Reference Instructions	- Instructions that operate directly on the processor's internal registers rather than accessing main memory.
Timing and Control Unit (TCU)	- A conceptual or physical unit within the CPU responsible for generating timing signals and control signals to coordinate all operations.
Timing Signals	- Regular pulses, often generated by a master clock, that provide the temporal framework for the processor's operations, ensuring synchronisation.

7. TERMINAL QUESTIONS

1. What are the two main parts of an instruction code, and what does each do?
2. Name and differentiate between the three primary types of instruction codes (Memory-Reference, Register-Reference, I/O).
3. Explain addressing modes. How do direct and indirect addressing differ?
4. Briefly describe instruction codes based on the number of address fields (Three-Address, Two-Address, One-Address, Zero-Address).
5. What's the difference between fixed-length and variable-length instruction formats in computer architecture?
6. Define an opcode. How does the CPU use it during the instruction cycle?
7. Illustrate a simple 16-bit instruction format with an opcode and operands, and explain how the CPU interprets it.
8. List five common opcodes and their functions.
9. Why are opcodes important for a computer's efficiency, compatibility, and security?
10. How do simple opcodes combine to create complex programs?
11. What are the roles of timing and control in a computer system?
12. What does the Timing and Control Unit (TCU) do? What are its three main inputs?
13. How do timing signals synchronise the Fetch, Decode, and Execute phases of the instruction cycle?
14. Compare Hardwired Control and Microprogrammed Control. What are their main pros and cons?
15. Explain how timing and control signals orchestrate the Fetch-Decode-Execute cycle

8. ANSWERS

Self-Assessment Questions

1. False
2. Opcode
3. Binary
4. False
5. C) The verb
6. Instruction register
7. False
8. C) Timing and Control Unit (TCU)
9. intervals
10. C. It generates control signals through fixed logic circuits and timing mechanisms
11. A. Acts as the decision-making hub to produce control signals
12. B. A set of microprograms made up of microinstructions
13. C. Through the Microinstruction Address Generation Unit using opcode and status
14. D. Generally faster due to direct hardware implementation

Terminal Questions Answers

Answer 1: An instruction code has an opcode (what to do) and an address/operand part (where to do it). For 4096 memory words, 12 bits are needed for the address, as $2^{12} = 4096$.

For more details, refer to Section 2

Answer 2: Memory-Reference instructions access memory (e.g., LOAD data). Register-Reference instructions operate on CPU registers (e.g., CLEAR a register). Input-Output (I/O) instructions manage device communication (e.g., OUT to printer).

For more details, refer to Section 2.2

Answer 3: Addressing modes define how an instruction finds data. Direct addressing uses the exact memory address, while indirect addressing points to a location holding the address. They offer flexibility in accessing data for efficient programming.

For more details, refer to Section 2.1 & 2.2

Answer 4: Instructions are classified by address fields: Three-Address (flexible but long), Two-Address (source/destination), One-Address (uses accumulator, small), and Zero-Address (stack-based, implicit operands). Each balances complexity and size.

For more details, refer to Section 2

Answer 5: Fixed-length instructions (RISC) are easier for hardware to process due to consistent size. Variable-length instructions (CISC) allow more complex operations but are harder to decode due to varying sizes.

For more details, refer to Section 2.4

Answer 6: An opcode is a command in a machine instruction (the "verb"). The CPU fetches and decodes the opcode to know the action, and then executes that operation using operands.

For more details, refer to Section 3

Answer 7: In a 16-bit instruction, a 4-bit opcode (e.g., 0001 for ADD) would precede 12 bits for operands (e.g., 0100 0110 for Registers 4 and 6), signalling the CPU to add their contents.

For more details, refer to Section 3

Answer 8: Common opcodes include ADD (addition), JMP (jump), LOAD (memory to register), MOV (copy data), and CMP (compare). Each specifies a distinct CPU operation.

For more details, refer to Section 3.2

Answer 9: Opcodes are crucial for CPU efficiency (fast decoding), compatibility (software runs on specific CPUs), and security (privileged operations restricted). They fundamentally define a CPU's capabilities.

For more details, refer to Section 3.3

Answer 10: Simple opcodes combine sequentially for complex tasks. For example, a loop uses ADD/SUB for counting, CMP for condition check, and JMP to repeat. Sequencing these builds intricate program logic.

For more details, refer to Section 3.4

Answer 11: Timing dictates when operations occur, ensuring synchronisation. Control directs how data flows and instructions execute. Together, they precisely coordinate all computer components, preventing errors and ensuring efficient operation.

For more details, refer to Section 4.

Answer 12: The Timing and Control Unit (TCU) generates signals to orchestrate operations. It uses inputs from the Instruction Register (current instruction), Flags (operation results), and the Clock (synchronisation pulses) to produce precise control signals.

For more details, refer to Section 4.

Answer 13: Timing signals synchronise the instruction cycle: they initiate instruction fetch, trigger the decoding process, and regulate the exact start and end of execution, ensuring orderly progression and data integrity in each phase.

For more details, refer to Section 4.3

Answer 14: Hardwired Control offers speed but lacks flexibility (hardware changes needed). Microprogrammed Control is more flexible (software changes to microprogram) but generally slower. Each has trade-offs in performance versus adaptability.

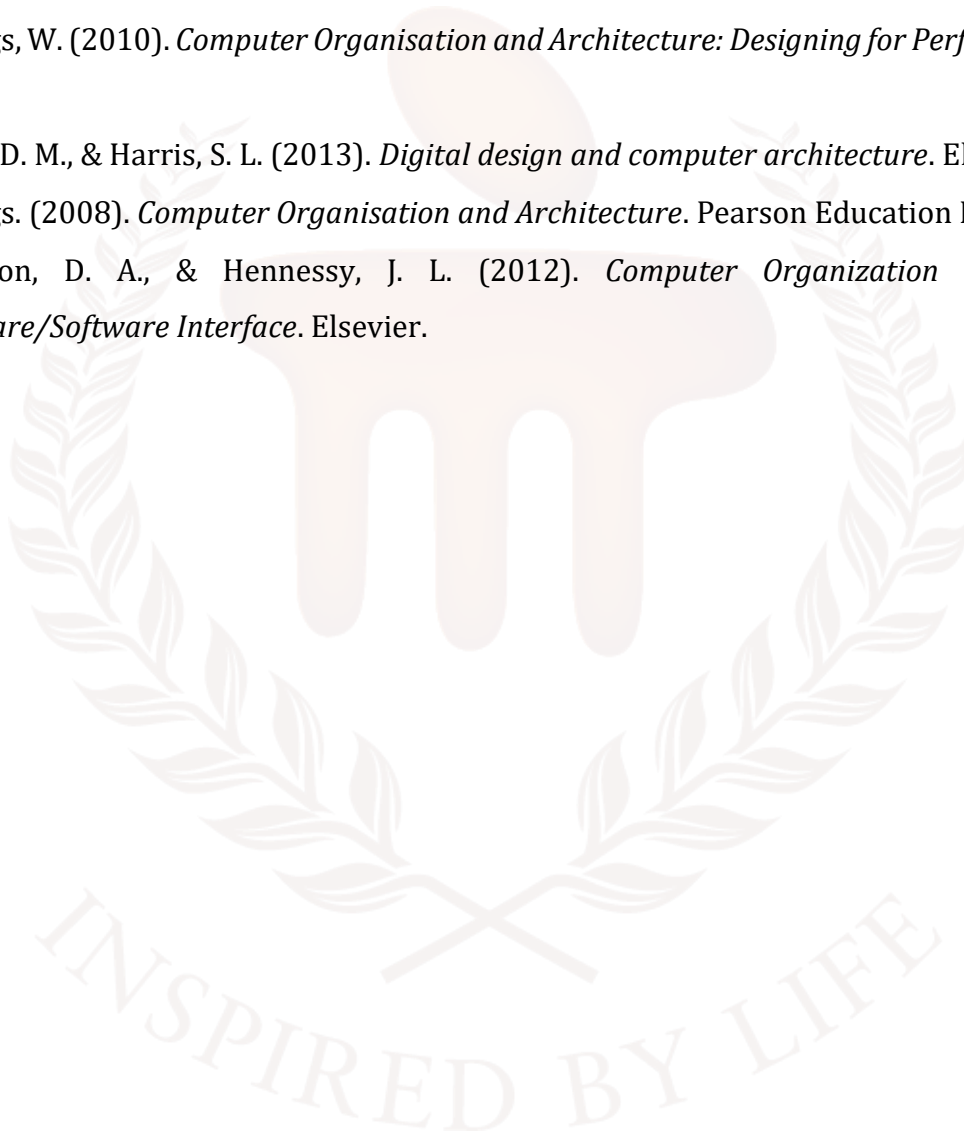
For more details, refer to Section 4.4

Answer 15: In the Fetch-Decode-Execute cycle, timing signals dictate sequence (e.g., when to fetch). Control signals direct specific actions (e.g., configure ALU, transfer data). They collaborate to ensure synchronised, error-free instruction processing step-by-step.

For more details, refer to Section 4.5

9. REFERENCES

- Mano, M. M. (1982). *Computer System Architecture*. Prentice Hall.
- Stallings, W. (2010). *Computer Organisation and Architecture: Designing for Performance*. Prentice Hall.
- Stallings, W. (2010). *Computer Organisation and Architecture: Designing for Performance*. Prentice Hall.
- Harris, D. M., & Harris, S. L. (2013). *Digital design and computer architecture*. Elsevier.
- Stallings. (2008). *Computer Organisation and Architecture*. Pearson Education India.
- Patterson, D. A., & Hennessy, J. L. (2012). *Computer Organization and design: The Hardware/Software Interface*. Elsevier.

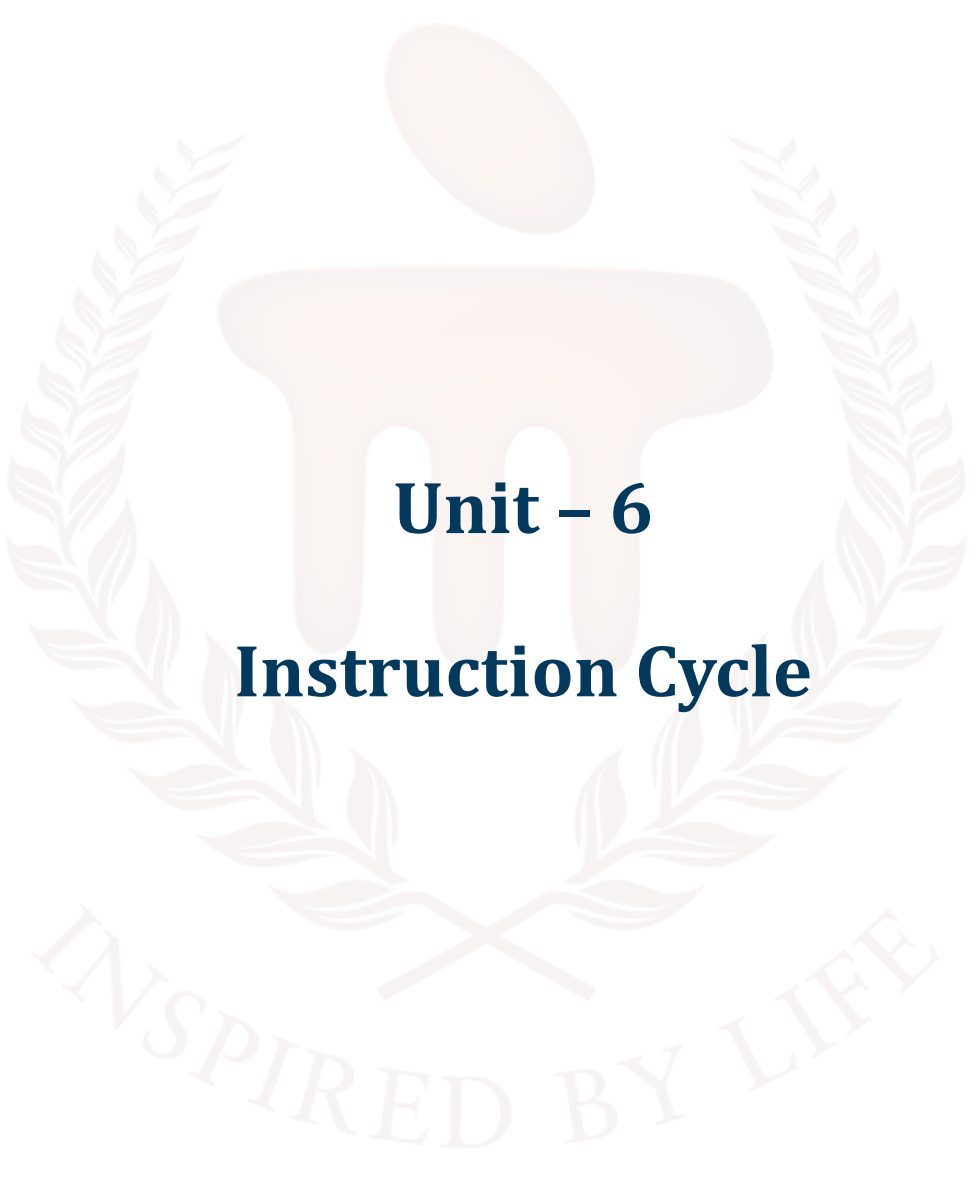


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105
COMPUTER ORGANIZATION AND
ARCHITECTURE



Unit – 6

Instruction Cycle

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 6
1.1	Objectives	-	-	
2	Instruction Cycle as a Finite-State Machine	-	-	7 - 9
2.1	FSM Definition	i	-	
2.2	Pipeline Performance Model	-	-	
2.3	Hazard Detection and Forwarding	-	-	
2.3.1	Structural Hazards	-	-	
2.3.2	Data Hazards (RAW, WAR, WAW)	-	-	
2.3.3	Control Hazards (Branch Hazards)	-	-	
2.3.4	Branch Target Buffers (BTB)	-	-	
2.4	Instruction Cycle State Diagram	-	-	
3	Memory-Reference Instructions	-	-	10 - 19
3.1	Definition and Classification	-	-	
3.2	Addressing Modes	1, 2, 3, 4, 5, 6	-	
3.2.1	Immediate Addressing	-	-	
3.2.2	Direct (Absolute) Addressing	-	-	
3.2.3	Indirect Addressing	-	-	
3.2.4	Register Direct & Register Indirect	-	-	
3.2.5	Indexed (Base-plus-Offset) Addressing	-	-	
3.3	Execution Sequence for Direct Mode	-	-	
3.4	Execution Sequence Micro-Operations	ii	-	
3.5	Memory Hierarchy & Caching	-	-	
3.6	Virtual Memory & Address Translation	7	1	
4	Input/Output Instructions	-	-	20 - 28
4.1	Programmed I/O Instructions	-	-	

	4.2	Interrupt-Driven I/O Instructions	-	-	
	4.2.1	Enabling and Disabling Interrupts	-	-	
	4.2.2	Interrupt Acknowledge and Vectoring	-	-	
	4.2.3	Performance and Latency	-	-	
	4.3	Direct Memory Access (DMA) Setup Instructions	-	-	
	4.3.1	DMA Controller Registers	-	-	
	4.3.2	DMA Setup Sequence	-	-	
	4.3.3	Throughput and CPU Utilisation	-	-	
	4.4	I/O Execution Sequence and Timing	iii	-	
	4.5	Bus Protocols & Arbitration	-	2	
5	Interrupts		-	-	
	5.1	Concept and Classification	-	-	
	5.1.1	Maskable vs. Non-Maskable Interrupts	-	-	
	5.1.2	Hardware vs. Software Interrupts (Traps)	-	-	
	5.2	Interrupt Service Cycle	-	-	29 - 33
	5.3	Interrupt Handling Mechanism	iv	-	
	5.4	Interrupt Vector Table	v	-	
	5.5	Nested and Prioritised Interrupts	-	-	
	5.6	Context Switching on Interrupt	-	3	
6	Summary		-	-	34
7	Glossary		-	-	35
8	Terminal Questions		-	-	36
9	Answers		-	-	37 - 40
10	References		-	-	41

1. INTRODUCTION

In the previous unit, you learned about how computers execute instructions through a structured process known as basic computer organisation and design. The main part of this process are instruction codes, which are binary representations of operations that the computer must perform. Each instruction typically consists of an operation code (opcode) that specifies the type of operation—such as addition, subtraction, or data transfer—and an operand that indicates the data or memory location involved. The control unit of the CPU interprets these instruction codes and generates the necessary signals to carry out the operations. This is coordinated through a system of timing and control, where clock pulses regulate the sequence of actions in the instruction cycle: fetching the instruction, decoding it, executing the operation, and storing the result. Together, these elements ensure that the computer performs tasks accurately and efficiently, following a well-defined sequence governed by both hardware and control logic.

In contemporary computing architectures, the instruction cycle constitutes a formal state-transition system wherein the processor sequentially moves through defined microarchitectural states—Fetch, Decode, Execute, Memory, and Write-back—to effect the execution of each instruction. At its core, this cycle forms a deterministic finite-state machine (FSM) governed by a microprogrammed or hardwired control unit. Beyond this fundamental loop, memory-reference instructions, I/O operations, and interrupts introduce non-trivial interactions with hierarchical memory subsystems and asynchronous events.

In this unit, you will learn about a comprehensive theoretical and practical framework, incorporating formal models, performance analyses, microarchitectural insights, mathematical derivations, and statistical models, all reinforced by illustrative diagrams and infographics.

After studying this unit, you should be able to:

-

2. INSTRUCTION CYCLE AS A FINITE-STATE MACHINE

2.1. FSM Definition

Define the instruction cycle FSM as $((S, I, O, \delta))$ where:

- $S = \{\text{Fetch, Decode, Execute, Memory, WriteBack}\}$
- $I = \{\text{clock, instruction_complete}\}$
- $O = \{\text{assert_control_signals}\}$
- $\delta: S \times I \rightarrow S$
- $\delta: S \times I \rightarrow O$

Table 1: Explaining the different states

State	Input	Next State	Output
Fetch	Clock	Decode	$MAR \leftarrow PC$; $MDR \leftarrow M[MAR]$; $IR \leftarrow MDR$; $PC \leftarrow PC + 1$
Decode	Clock	Execute	Decode opcode; prepare operands
Execute	Clock	Memory	ALU/EAB calculation
Memory	Clock	WriteBack	Load: $MDR \leftarrow M[MAR]$; Store: $M[MAR] \leftarrow MDR$
WriteBack	Clock	Fetch	Write-back to register

2.2. Pipeline Performance Model

- **Clock Period:** $T_{clk}(t_i)$
- **Ideal CPI** = 1; with stalls: $(CPI = 1 + \text{stalls})$.
- **Amdahl's Speedup:** $(S = \frac{1}{\text{fraction of sequential work}})$.

2.3. Hazard Detection and Forwarding

In pipelined CPUs, overlapping instruction phases yields structural, data, and control hazards. Detecting and mitigating these hazards is critical to maintaining throughput near the ideal CPI of 1.

2.3.1. Structural Hazards

A structural hazard occurs when two pipeline stages require the same hardware resource in the same cycle. Common examples:

- **Single ALU** used for arithmetic and address calculations concurrently.
- **Single memory port** contended by instruction fetch (IF) and data access (MEM) stages.

Resolution Techniques

Resolution techniques in computer architecture are strategies used to manage resource conflicts and optimise instruction execution. Resource duplication involves providing separate hardware units, such as distinct Arithmetic Logic Units (ALUs) and address-generation units, to allow parallel execution of operations that would otherwise compete for the same resource.

Pipeline scheduling addresses conflicts by temporarily stalling an instruction until the required resource becomes available, ensuring correct execution order while maintaining pipeline integrity.

Interleaving, particularly in the form of Harvard architecture, uses separate instruction and data caches to eliminate memory port conflicts, allowing simultaneous access to both instruction and data streams and thereby improving throughput and reducing latency. These techniques collectively enhance processor performance by minimising delays and maximising resource utilisation.

2.3.2. Data Hazards (RAW, WAR, WAW)

Three types of data hazards:

- **RAW (Read-After-Write):** True dependency—an instruction reads a register before a previous instruction writes it.
- **WAR (Write-After-Read):** Anti-dependency—a later instruction writes a register before an earlier instruction reads it.
- **WAW (Write-After-Write):** Output dependency—two instructions write to the same destination.

RAW Hazards are most common. The pipeline control unit implements a Hazard Detection Unit (HDU) that compares destination registers of prior instructions in Execute/Memory stages against source registers of the current instruction:

```

ID[Instruction Decode]
EX[Execute]
MEM[Memory]
SUBGRAPH HazardDetectionUnit
  A((Dest_fwd))
  B((Src1))
  C((Src2))
end
  
```

```

EX --> A
ID --> B
ID --> C
A -->|if equal to B or C| Stall
  
```

Forwarding Paths provide the ALU result directly to dependent instructions without waiting for write-back. The forwarding network adds multiplexers at the input of the ALU stage, selecting either the register file or the forwarded ALU_out.

2.3.3. Control Hazards (Branch Hazards)

Branches and jumps alter the PC, invalidating prefetched instructions. Control hazards cause pipeline flushes unless mitigated.

Mitigation Strategies:

- Static Prediction: Always predict taken or not taken.
- Delayed Branch: CPU defines a delay slot—a fixed number of instructions always executed after the branch.
- Dynamic Prediction: Two-bit saturating counters track branch history, offering higher accuracy.

Two-Bit Saturating Counter Model

Each branch has a 2-bit counter: Strongly Taken, Weakly Taken, Weakly Not Taken, Strongly Not Taken. Counters increment or decrement based on actual outcome, mapping to predicted direction.

2.3.4. Branch Target Buffers (BTB)

A BTB caches branch target addresses and prediction bits, allowing the IF stage to fetch from the predicted PC rather than waiting for the DE stage to resolve the branch.

2.4. Instruction Cycle State Diagram

```

[*] --> Fetch
Fetch --> Decode
Decode --> Execute
Execute --> Memory
Memory --> WriteBack
WriteBack --> Fetch
  
```

3. MEMORY-REFERENCE INSTRUCTIONS

3.1. Definition and Classification

Memory-reference instructions form the critical bridge between a CPU's internal registers and its external memory subsystem. They can be rigorously characterised as two fundamental operations within the von Neumann architecture's instruction set:

1. Load (Read) Instructions

- Operation: Transfer data from a specified memory location into a CPU register.
- Syntax Example:
LOAD R1, Addr
; Pseudocode: $R1 \leftarrow M[Addr]$
LOAD R1, 0x2000
- Microarchitectural Impact: Involves computing the Effective Address (EA), issuing a memory read, and writing back into the register file. Total latency typically = fetch (1 cycle) + decode (1) + EA calculation (1) + memory access (≈ 6 with cache) + write-back (1) = ~ 10 cycles (assuming 90% cache hit rate).
- Use Cases: Loading constants, stack operations, accessing array elements, pointer dereferencing.

2. Store (Write) Instructions

- Operation: Transfer data from a CPU register back into a specified memory location.
- Syntax Example:
STORE R1, Addr
; Pseudocode: $M[Addr] \leftarrow R1$
STORE R1, 0x3004
- Microarchitectural Impact: EA computation + register-read + memory write. Total latency ~ 9 cycles (fetch + decode + EA + write + minimal write-back).
- Use Cases: Saving computed results to arrays, updating data structures, writing back spilled registers.

Performance Observation: Because each memory-reference instruction requires both an instruction fetch and a separate data memory access, its CPI (cycles per instruction) is significantly

higher than that of pure ALU operations. Designers mitigate this via multi-level caches, pipelined memory interfaces, and prefetching.

3.2. Addressing Modes

An addressing mode defines the computation of the Effective Address (EA) for memory-reference instructions. Selecting the appropriate mode balances hardware complexity, instruction encoding size, and runtime flexibility.

3.2.1. Immediate Addressing

- Description: Operand value is encoded directly in the instruction's immediate field. No second memory access for data.
- Syntax: `LOAD R1, #5`
- Hardware Cost: Instruction width must accommodate the immediate; EA computation trivial (zero delay).
- Latency: ~4 cycles total (fetch + decode + extend + write-back).
- Example Usage: Initialising counters, small offsets for address arithmetic.

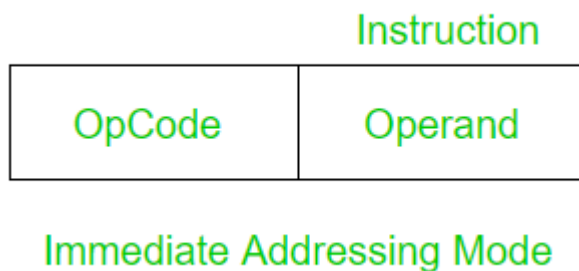
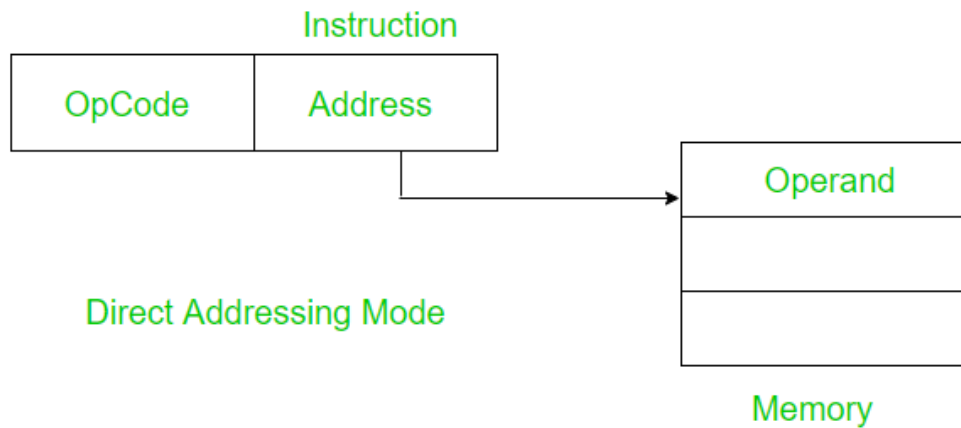


Figure 1: Immediate addressing mode

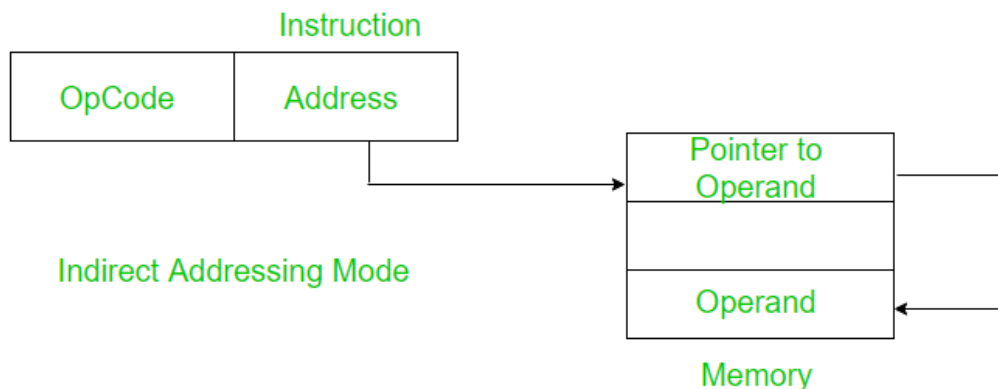
3.2.2. Direct (Absolute) Addressing

- Description: Instruction contains the literal memory address.
- Syntax: `LOAD R1, 0x1000`
- Hardware Cost: Simple MAR load from IR; no extra ALU cycles.
- Latency: ~10 cycles (as in Section 3.1).
- Limitations: Immediate field width limits addressable range; larger addresses require multiple instructions.

**Figure 2: Direct addressing mode**

3.2.3. Indirect Addressing

- Description: Instruction's address field points to a memory cell whose content is the EA. Allows pointers to be stored in memory.
- Syntax: LOAD R1, (0x1000)
- Micro-ops:
 1. M3: $MAR \leftarrow IR[ptr_addr] \text{ (0x1000)}$
 2. M4a: $MDR \leftarrow M[MAR] \rightarrow \text{pointer value}$
 3. M4b: $MAR \leftarrow MDR; MDR \leftarrow M[MAR]$
 4. M5: $R1 \leftarrow MDR$
- Latency: ~16 cycles (two memory accesses in Memory phase).
- Use Case: Implementing arrays of pointers, dynamic data structures.

**Figure 3: Indirect addressing mode**

3.2.4. Register Direct & Register Indirect

- Register Direct:
 - Syntax: LOAD R1, R2 — copy without memory.
 - Micro-ops: M3: MDR $\leftarrow$ R2; M5: R1 $\leftarrow$ MDR; no memory cycles.
 - Latency: ~ 5 cycles (fetch + decode + reg file).
- Register Indirect:
 - Syntax: LOAD R1, (R2)
 - Micro-ops: M3: MAR $\leftarrow$ R2; M4: MDR $\leftarrow$ M[MAR]; M5: R1 $\leftarrow$ MDR
 - Latency: ~ 10 cycles.
- Advantages: Efficient pointer dereference, no immediate field.
- Considerations: Increases register-file read pressure.

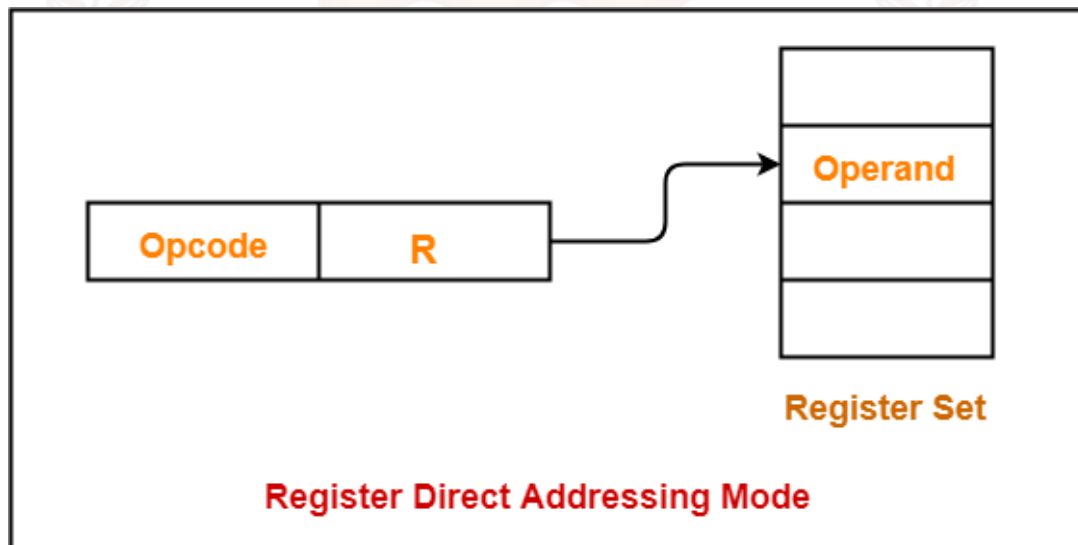


Figure 4: Register direct mode

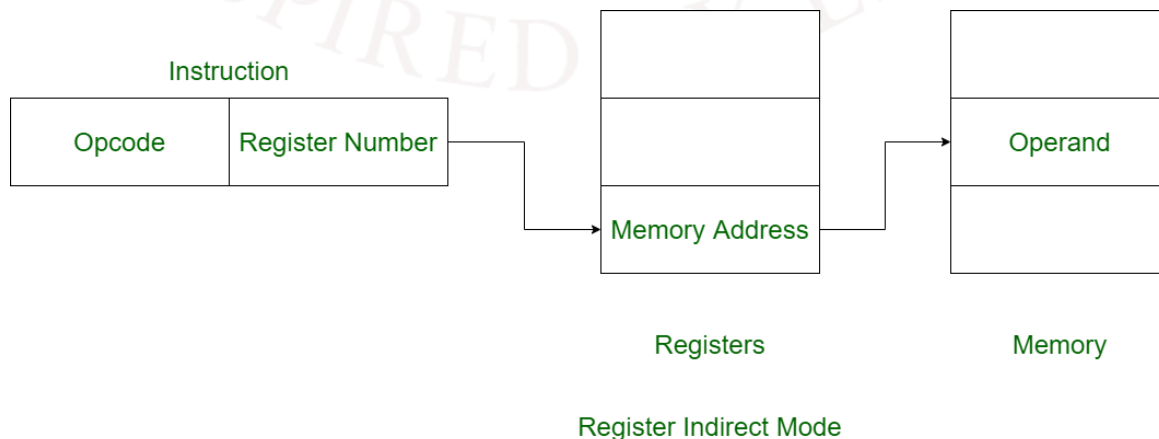


Figure 5: Register indirect mode

3.2.5. Indexed (Base-plus-Offset) Addressing

- Description: $EA = \text{Base register value} + \text{signed offset from instruction}$.
- Syntax: `LOAD R1, 0x1000(R2)`
- Micro-ops:
 1. M3: $A \leftarrow R2; B \leftarrow \text{sign_extend}(\text{IR}[\text{offset}])$
 2. M3: $\text{ALU_out} \leftarrow A + B; \text{MAR} \leftarrow \text{ALU_out}$
 3. M4: $\text{MDR} \leftarrow M[\text{MAR}]$
 4. M5: $R1 \leftarrow \text{MDR}$
- Latency: ~ 11 cycles (one extra ALU cycle vs. direct).
- Use Cases: Array indexing, structure field access, loop traversal.

Trade-offs Summary:

- Immediate & Register Direct: Fastest, minimal hardware; limited flexibility.
- Direct & Register Indirect: Moderate complexity; fixed or register-based EA.
- Indirect & Indexed: Highest flexibility at cost of added cycles and hardware for ALU and extra memory accesses.

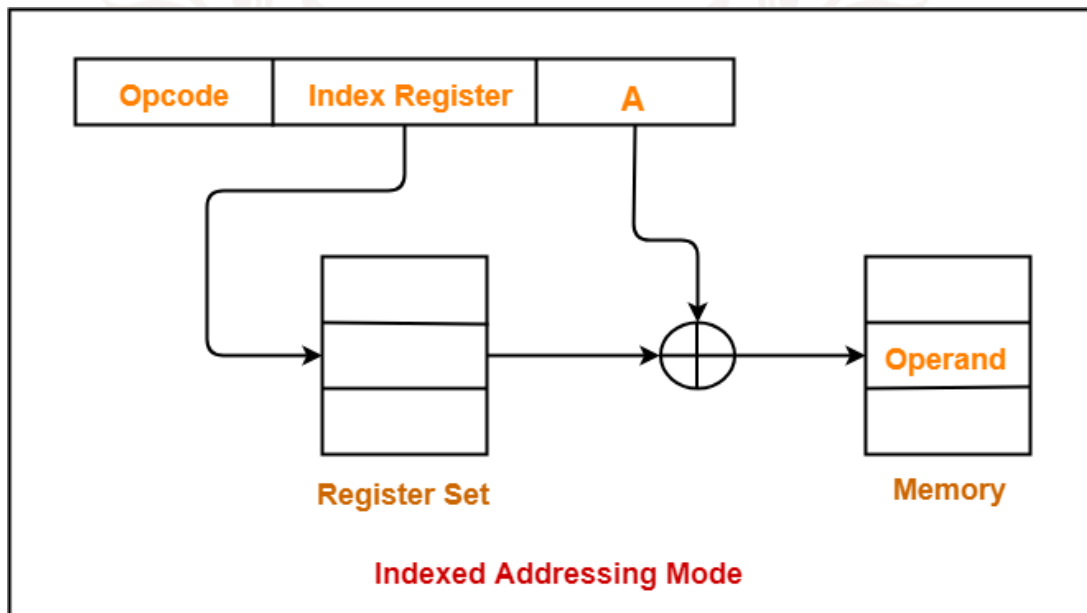


Figure 6: Indexed addressing mode

3.3. Execution Sequence for Direct Mode

Below is the canonical micro-operation breakdown for a LOAD instruction using direct addressing:

1. M1 (Fetch):
 - $MAR \leftarrow PC$
 - $MDR \leftarrow M[MAR]$
 - $IR \leftarrow MDR$
 - $PC \leftarrow PC + 1$
2. M2 (Decode):
 - Control Unit decodes opcode and extracts address field.
3. M3 (Effective Address):
 - $MAR \leftarrow IR[address]$
4. M4 (Memory Read):
 - $MDR \leftarrow M[MAR]$
5. M5 (Write-Back):
 - $R[dest] \leftarrow MDR$

For a STORE, steps 3–5 become:

3. M3: $MAR \leftarrow IR[address];$
4. M4: $MDR \leftarrow R[src];$
5. M5: $M[MAR] \leftarrow MDR.$

This sequence highlights why memory-reference instructions require additional cycles compared to register-only operations and underscores the importance of caches and pipelining in hiding latency.

3.4. Execution Sequence Micro-Operations

Table 2: Consolidated micro-operation sequence for LOAD and STORE in direct addressing mode

Cycle	LOAD Direct	STORE Direct
M1	$MAR \leftarrow PC; MDR \leftarrow M[MAR]; IR \leftarrow MDR; PC \leftarrow PC + 1$	$MAR \leftarrow PC; MDR \leftarrow M[MAR]; IR \leftarrow MDR; PC \leftarrow PC + 1$
M2	Decode IR	Decode IR
M3	$MAR \leftarrow IR[address]$	$MAR \leftarrow IR[address]$
M4	$MDR \leftarrow M[MAR]$	$MDR \leftarrow R[src]$
M5	$R[dest] \leftarrow MDR$	$M[MAR] \leftarrow MDR$

- For a LOAD, step M4 reads from memory; M5 writes into the register file.

- For a STORE, step M4 reads the register; M5 writes into memory.

3.5. Memory Hierarchy & Caching

Modern processors mitigate the high latency of main memory accesses through a multi-level cache hierarchy. At a minimum, most CPUs include an L1 data cache (L1d), optionally an L1 instruction cache (L1i), an L2 unified cache, and sometimes an L3 cache shared across cores.

Cache Levels and Access Times

Cache levels in a computer system are organised hierarchically to balance speed and storage capacity.

The L1 cache is the smallest and fastest, located closest to the processor core, typically ranging from 32 to 64 KB with an access time of about 1 to 4 cycles. It stores frequently used instructions and data for rapid retrieval.

The L2 cache is larger, usually between 256 KB and 1 MB, but slower, with access times around 10 to 20 cycles. It acts as a secondary buffer when data is not found in L1.

The L3 cache is even larger, ranging from 2 MB to 32 MB, and is shared across multiple cores in multi-core processors. Its access time is slower, around 30 to 50 cycles, but it helps reduce memory traffic to the main memory.

Finally, main memory (typically DRAM) has the largest capacity but the slowest access time, around 100 to 200 cycles, and is used when data is not found in any of the cache levels. This layered approach ensures efficient data access while managing cost and complexity.

Average Memory Access Time (AMAT)

Let (h_1, h_2, h_3) denote hit rates at L1, L2, L3. Then:

$$T_{\text{avg}} = h_1 t_{\text{L1}} + (1-h_1) [h_2 t_{\text{L2}} + (1-h_2) [h_3 t_{\text{L3}} + (1-h_3) t_{\text{MM}}]]$$

Example:

$(h_1=0.95, h_2=0.90, h_3=0.85), (t_{\text{L1}}=2), (t_{\text{L2}}=12), (t_{\text{L3}}=35), (t_{\text{MM}}=150):$

$$T_{\text{avg}} = 0.95 \times 2 + 0.05 (0.90 \times 12 + 0.10 (0.85 \times 35 + 0.15 \times 150))$$

Detailed computation yields $(T_{\text{avg}} \approx 3.4)$ cycles.

Cache Write Policies and Coherency

- Write-Through vs. Write-Back: Write-through updates all cache levels and memory, simpler

but high bandwidth; write-back marks dirty lines, writes to lower levels on eviction.

- Cache Coherency (Multiprocessor): Protocols like MESI ensure consistency: Modified, Exclusive, Shared, Invalid states.

Worked Impact Example

If L1 hit rate drops from 95 % to 90 %, recompute (T_{avg}) and observe the impact on overall CPI for memory-reference instructions. A simple sensitivity analysis shows a disproportionate slowdown, underscoring the importance of high L1 hit rates.

3.6. Virtual Memory & Address Translation

Virtual memory provides each process with a private linear address space, translated to physical addresses via the Memory Management Unit (MMU). Memory-reference instructions trigger TLB lookups and page table walks on misses.

Translation Lookaside Buffer (TLB)

- TLB Hit: Fast lookup (~1 cycle); physical frame number returned.
- TLB Miss: Hardware/software page table walk: multiple memory accesses (2–6 cycles per level) until the PTE (page table entry) is found.

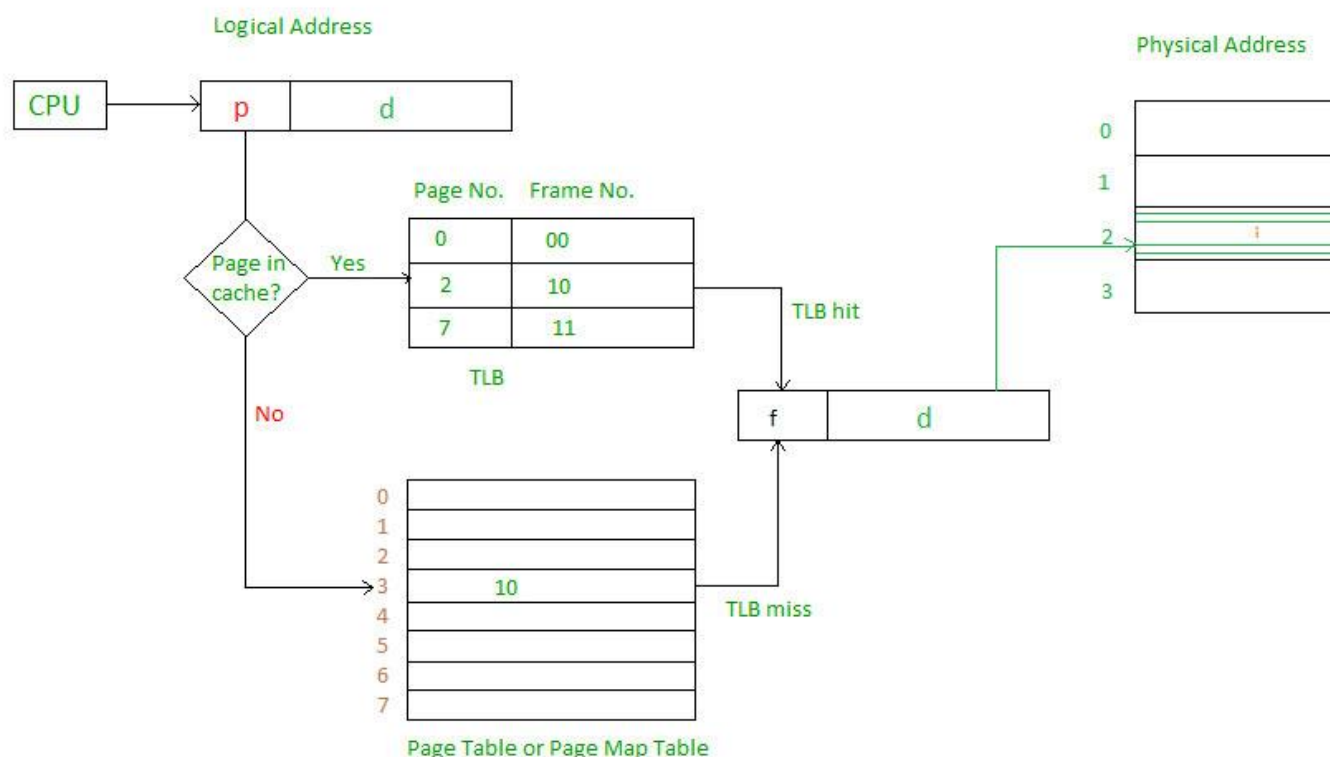


Figure 7: TLB diagram

Effective Memory Access Time with TLB

Let $(lpha) = \text{TLB hit rate}$, $(t_{\text{TLB}})=1 \text{ cycle}$, $(t_{\text{miss}})=50 \text{ cycles}$ (page-table access + load PTE + update TLB). Then:

$$T_{\text{mem}} = lpha \cdot (t_{\text{TLB}} + t_{\text{cache_avg}}) + (1-lpha) \cdot (t_{\text{TLB}} + t_{\text{miss}} + t_{\text{cache_avg}}).$$

High $(lpha)$ values ($>99 \%$) are critical for performance.

Page Fault Handling

- Page Fault: If page not resident, CPU traps to OS, which loads page from disk ($\sim$ microseconds–milliseconds).
- Sequence Diagram: TLB miss $\rightarrow$ page table walk $\rightarrow$ PTE invalid $\rightarrow$ page fault ISR $\rightarrow$ disk I/O $\rightarrow$ resume instruction.

CPU->>MMU: VA

MMU-->>CPU: TLB Miss

CPU->>Memory: Walk page tables

alt PTE valid

Memory-->>MMU: PFN

MMU-->>CPU: Update TLB; Physical Addr

else PTE invalid

MMU-->>CPU: Page Fault

CPU->>OS: Trap to page-fault handler

OS->>Disk: Read page

Disk-->>OS: Page data

OS->>Memory: Install PTE, update TLB

CPU->>CPU: Retry instruction

End

SELF-ASSESSMENT QUESTIONS - 1

True or False:	
1	In immediate addressing, the operand value is fetched from memory during the Memory phase.
Multiple Choice	
2	Which addressing mode requires two memory accesses in the Memory phase? A. Direct B. Indexed C. Indirect D. Register Direct
Short Answer	
3	Explain one scenario in which you would prefer indexed addressing over direct addressing.
Calculation	
4	Given a cache-hit latency of 1 cycle, memory latency of 50 cycles, and a hit rate of 0.9, compute the approximate cycle count for a LOAD R1,0x2000(R2) instruction in direct mode assuming fetch+decode+EA each cost 1 cycle and write-back costs 1 cycle.
Fill in the Blank	
5	Register-indirect addressing uses the contents of register ___ as the Effective Address.

4. INPUT/OUTPUT INSTRUCTIONS

In order to transfer data between the CPU and peripheral devices (keyboards, disks, network cards), the processor provides specialised I/O instructions. There are three primary paradigms:

- Programmed I/O (Polling)
- Interrupt-Driven I/O
- Direct Memory Access (DMA)

This section dives deep into programmed I/O, detailing its operation, instruction semantics, micro-operations, performance impact, and real-world considerations.

4.1. Programmed I/O Instructions

In programmed I/O, the CPU explicitly issues each I/O read or write instruction and then actively waits—polling a status register—until the peripheral is ready. While straightforward to implement, polling monopolises CPU cycles, making this approach suitable only for low-throughput or simple embedded systems.

4.1.1. IN Instruction

The IN instruction reads a byte or word from a specified I/O port into a CPU register. Its execution proceeds as follows:

; Pseudo-assembly for IN R1, PortAddr

IN R1, PortAddr ; $R1 \leftarrow IO[PortAddr]$

Micro-operations:

1. Fetch (M1–M2):
 - $MAR \leftarrow PC$; $MDR \leftarrow M[MAR]$; $IR \leftarrow MDR$; $PC \leftarrow PC + 1$
2. Decode (M3):
 - Decode opcode = IN; extract PortAddr field.
3. Execute (M4):
 - $MAR \leftarrow PortAddr$; $MDR \leftarrow IO[MAR]$; read from device register
4. Write-Back (M5):
 - $R1 \leftarrow MDR$; transfer to register file

Timing and CPU Overhead:

- Instruction fetch & decode: ~2 cycles
- I/O read cycle: depends on peripheral (e.g., 50–200 cycles)
- Register write-back: 1 cycle

Overall latency per IN can range from a few dozen to several hundred cycles. Since the CPU must wait until IO[PortAddr] is valid, high I/O latencies stall the pipeline completely.

Polling Loop Example:

POLL_LOOP:

```
IN  R0, STATUS_PORT    ; Read device status
AND R0, R0, #0x01      ; Mask ready bit
BEQ POLL_LOOP          ; If zero, keep polling
IN  R1, DATA_PORT     ; Device ready: read data
```

```
participant CPU
participant DEVICE

CPU->>DEVICE: IN STATUS_PORT
DEVICE-->>CPU: status bits
alt not ready
    CPU->>CPU: branch to POLL_LOOP
else ready
    CPU->>DEVICE: IN DATA_PORT
    DEVICE-->>CPU: data
end
```

4.1.2. OUT Instruction

The OUT instruction writes a byte or word from a register to a specified I/O port. Its micro-operations mirror IN, but data flows from CPU to the device:

; Pseudo-assembly for OUT PortAddr, R2

OUT PortAddr, R2 ; IO[PortAddr] $\leftarrow$ R2

Micro-operations:

1. Fetch (M1–M2):

This phase retrieves the instruction from memory. The Memory Address Register (MAR) is loaded with the value of the Program Counter (PC), which points to the address of the next instruction. The Memory Data Register (MDR) then fetches the instruction from memory at that address (M[MAR]). The instruction is transferred to the Instruction Register (IR), and the PC is incremented to point to the next instruction.

$$\text{MAR} \leftarrow \text{PC}; \text{MDR} \leftarrow \text{M}[\text{MAR}]; \text{IR} \leftarrow \text{MDR}; \text{PC} \leftarrow \text{PC} + 1$$

2. Decode (M3):

The CPU decodes the instruction stored in the IR. It identifies the operation code (opcode), which in this case is OUT, and extracts the operands: the destination PortAddr and the source register R2.

Decode opcode = OUT; extract PortAddr and R2.

3. Execute (M4):

The CPU prepares for the I/O operation. The MAR is loaded with the PortAddr, indicating the I/O device address. The MDR is loaded with the contents of R2, which holds the data to be sent to the device.

$$\text{MAR} \leftarrow \text{PortAddr}; \text{MDR} \leftarrow \text{R2}; \text{prepare data}$$

4. I/O Write (M5):

The actual data transfer occurs. The contents of the MDR are written to the I/O device at the address specified by MAR.

$$\text{IO}[\text{MAR}] \leftarrow \text{MDR}; \text{transfer to device}$$

Timing and CPU Overhead:

- Instruction fetch & decode: ~2 cycles
- I/O write cycle: 50–200 cycles (depending on device)
- No register write-back

Use Cases & Limitations:

- Suitable for simple control/status registers (e.g., toggling LEDs, sending serial bytes).

- Excessive polling or frequent OUT instructions can saturate the bus and starve the CPU of cycles.

4.2. Interrupt-Driven I/O Instructions

Rather than continuously polling device status, interrupt-driven I/O allows the CPU to issue an I/O command and then proceed with other instructions. When the peripheral completes the operation or needs service, it asserts an interrupt request (IRQ) line. The CPU acknowledges the interrupt at the next instruction boundary and vectors to the appropriate Interrupt Service Routine (ISR), minimising wasted CPU cycles.

4.2.1. Enabling and Disabling Interrupts

- EI (Enable Interrupts): Sets the global interrupt-enable flag, allowing maskable interrupts to be recognised.
- DI (Disable Interrupts): Clears the interrupt-enable flag, preventing further maskable interrupts until re-enabled.

Use Case: In critical code sections where state must not change (e.g., updating shared OS data structures), the kernel disables interrupts (DI) to avoid race conditions, then re-enables (EI) once complete.

4.2.2. Interrupt Acknowledge and Vectoring

1. Issue I/O Command (e.g., OUT Port, Rn or a DMA setup).
2. Continue Execution: CPU proceeds with subsequent instructions.
3. Peripheral Signals IRQ: Device asserts its IRQ line upon completion or error.
4. Interrupt Acknowledge Cycle:
 - CPU finishes current instruction.
 - When an interrupt is signaled, the CPU does not immediately stop its current task. Instead, it completes the execution of the current instruction to maintain consistency and avoid partial execution. This ensures that the system state remains stable before handling the interrupt.
 - CPU places an INT_ACK on the bus.

After finishing the current instruction, the CPU sends an interrupt acknowledge signal (INT_ACK) on the control bus. This signal informs the interrupting device or controller that the CPU is ready to handle the interrupt and is requesting more information about it.
 - Device/Interrupt Controller places a vector number on the data bus.

In response to the INT_ACK, the device or interrupt controller places an interrupt vector number on the data bus. This vector is a unique identifier that tells the CPU which interrupt service routine (ISR) to execute. The CPU uses this vector to index into an interrupt vector table, which contains the addresses of all ISRs.

5. Context Save: CPU pushes PC and flags (and optionally GPRs) onto the stack or uses shadow registers.
6. Vector Fetch: CPU computes $\text{vector_addr} = \text{IVT_base} + (\text{vector_number} \times \text{entry_size})$ and loads ISR address into PC.
7. ISR Execution: Peripheral handler in software clears device status, processes data, possibly signals higher-level processes.
8. Return from Interrupt: IRET/RETI instruction pops PC and flags, optionally restores GPRs, and resumes pre-interrupt execution, re-enabling interrupts.

participant CPU

participant Device

participant InterruptController

Note over CPU: EI ; Issue OUT Port, Rn

CPU->>Device: OUT Port, Rn

Note over CPU: Execute other instructions

Device-->>InterruptController: IRQ Signal

InterruptController-->>CPU: INT_ACK + VectorNum

CPU->>CPU: Push PC, Flags

CPU->>Memory: Read IVT[VectorNum]

Memory-->>CPU: ISR_Address

CPU->>CPU: PC ← ISR_Address

Note over CPU: Execute ISR ; IRET

CPU->>CPU: Pop PC, Flags

CPU->>CPU: Resume

4.2.3. Performance and Latency

- CPU Overhead: Context save (10–20 cycles) + vector fetch (2 cycles) + ISR body + IRET (5 cycles).
- Latency Advantages: Eliminates wasted polling loops; CPU time used for useful work until interrupt arrives.
- Considerations: ISR latency must be low for real-time responsiveness; excessive interrupts can lead to “interrupt storm” and high context-switch overhead.

4.3. Direct Memory Access (DMA) Setup Instructions

DMA enables peripheral controllers to transfer data blocks between memory and I/O devices autonomously, bypassing per-word CPU involvement.

4.3.1. DMA Controller Registers

1. Source Address Register (SAR)
2. Destination Address Register (DAR)
3. Byte Count Register (BCR)
4. Control/Status Register (CSR)

4.3.2. DMA Setup Sequence

The CPU programs the DMA controller via memory-mapped I/O or special instructions:

1. LOAD R1, SourceAddr ; Load base memory address into a register.
2. LOAD R2, DestAddr ; Load device/memory destination address.
3. LOAD R3, ByteCount ; Number of bytes (or words) to transfer.
4. Write DMA Registers:

```

OUT DMA_SAR, R1    ; SAR ← R1
OUT DMA_DAR, R2    ; DAR ← R2
OUT DMA_BCR, R3    ; BCR ← R3
OUT DMA_CSR, START_BIT ; CSR ← (Start + direction + IRQ enable)
  
```

5. CPU Continues Execution: DMA controller arbitrates bus mastership, performs transfers.
6. DMA Completion Interrupt: On BCR expiration, controller asserts IRQ; CPU runs DMA-completion ISR to clear CSR and notify software.

```

CPU -- OUT DMA_SAR,R1 --> DMA
CPU -- OUT DMA_DAR,R2 --> DMA
CPU -- OUT DMA_BCR,R3 --> DMA
CPU -- OUT DMA_CSR --> DMA
CPU -->|continues| OtherTasks
DMA -->|bus-grant| Memory
Memory -->|data| DMA
DMA -->|IRQ| CPU
  
```

4.3.3. Throughput and CPU Utilisation

- Setup Overhead: $\approx 5-10$ instructions (10–50 cycles).
- Data Transfer: CPU-free; DMA transfers at device/memory bandwidth.
- Completion Overhead: DMA interrupt (~ 15 cycles) + ISR.
- Comparison: For a 1 MB transfer, programmed I/O may consume millions of CPU cycles; DMA uses only tens of cycles for setup and completion.

4.4. I/O Execution Sequence and Timing

Table 3: Consolidated view of the cycle-by-cycle micro-operations and relative latencies for the three I/O paradigms

Phase	Programmed I/O	Interrupt-Driven I/O	DMA Setup & Completion
Fetch	M1–M2: Fetch IN/OUT	M1–M2: Fetch IN/OUT or OUT	M1–M2: Fetch OUT to DMA_CSR
Decode	M3: Decode & port addr	M3: Decode & port addr	M3: Decode & register select
Execute	M4: MAR $\leftarrow$ Port; MDR $\leftarrow$ IO[MAR]	M4: MAR $\leftarrow$ Port; MDR $\leftarrow$ IO[MAR] (initial)	M4: OUT SAR/DAR/BCR
WriteBack	M5: R $\leftarrow$ MDR or IO[MAR] $\leftarrow$ MDR (store)	M5: R $\leftarrow$ MDR / IO[MAR] $\leftarrow$ MDR	M5: OUT CSR $\rightarrow$ start DMA
Idle/Other	—	CPU continues until IRQ	CPU continues until IRQ
ISR Latency	N/A	Context save + ISR (≈ 30 cycles)	DMA-completion ISR (≈ 20 cycles)

Below that, CPUs must arbitrate access to the system bus when multiple masters contend for memory or I/O devices. High-performance I/O depends on robust bus protocols and arbitration schemes.

4.5. Bus Protocols & Arbitration

Modern systems use standardised bus protocols (e.g., PCIe, AMBA AXI) to interconnect CPUs, memory controllers, and peripherals. Key concepts include:

- **Multi-Master Arbitration:** Multiple agents (CPU, DMA controller, I/O devices) may request bus ownership. An arbiter grants access based on priority schemes (fixed priority, round-robin, time-division).
- **Handshake Signals:** A typical cycle uses REQ and GNT lines or AXI's ARVALID/ARREADY and AWVALID/AWREADY phases to ensure both master and slave are ready before data transfer.
- **Burst Transfers:** Agents can transfer a sequence of data beats in one bus acquisition, reducing arbitration overhead. For example, AXI supports LEN-encoded bursts where the master issues multiple data beats back-to-back.
- **Split Transactions:** Protocols like PCIe allow a device to split a long transfer into request and completion phases, freeing the bus in between for other masters.

Bus Arbitration Example (Round-Robin)

```
CPU_REQ([CPU Request])
DMA_REQ([DMA Request])
DEV_REQ([Device Request])
Arbiter --> GRANT_CPU([CPU Grant])
Arbiter --> GRANT_DMA([DMA Grant])
Arbiter --> GRANT_DEV([Device Grant])
CPU_REQ --> |round-robin| Arbiter
DMA_REQ --> Arbiter
DEV_REQ --> Arbiter
```

Case Study: AXI4 Read Burst

AXI4 read transactions involve:

1. AR Channel: Master asserts ARVALID with ARADDR and ARLEN.
2. Slave acknowledges ARREADY.

3. R Channel: Slave returns RDATA beats, asserting RVALID; master responds with RREADY.
4. Last beat: Slave asserts RLAST, indicating end of burst.

This decoupling allows the bus to interleave other AR/AW requests while data beats are in flight, improving throughput and latency under contention.

SELF-ASSESSMENT QUESTIONS - 2

True or False:	
6	Programmed I/O completely frees the CPU to perform other work while waiting for a peripheral.
Multiple Choice	
7	In interrupt-driven I/O, which micro-operation immediately follows the CPU's INT_ACK cycle? A. Pipeline flush B. Device places vector number on bus C. Context restore D. DMA setup
Short Answer	
8	List the four registers you must program in the DMA controller for a memory-to-peripheral transfer.
Calculation	
9	If programmed I/O for a 1 KB block takes 1 μ s/word and DMA setup+ completion overhead is 10 μ s, how much CPU time is saved by using DMA instead of programmed I/O?

5. INTERRUPTS

Interrupts provide a mechanism for asynchronous events—both hardware and software—to temporarily halt the processor's normal instruction cycle and transfer control to specialised handlers. This enables responsive I/O, precise exception handling, and multitasking in operating systems.

5.1. Concept and Classification

An interrupt is a signal indicating that an event requiring immediate attention has occurred. The CPU suspends its current stream of instructions, saves relevant state, and jumps to an Interrupt Service Routine (ISR). After servicing the event, the CPU restores its state and resumes execution.

5.1.1. Maskable vs. Non-Maskable Interrupts

- **Maskable Interrupts (IRQ):**
 - Definition: Software-configurable via an interrupt-enable flag.
 - Use Cases: General-purpose device interrupts—network cards, disk controllers, timers.
 - Control: CPU executes `DI` to disable, `EI` to re-enable.
- **Non-Maskable Interrupts (NMI):**
 - Definition: Cannot be disabled; reserved for critical conditions.
 - Use Cases: Power-fail signals, memory errors, watchdog timeouts.
 - Guarantee: ISR will always run, ensuring system does not ignore catastrophic faults.

5.1.2. Hardware vs. Software Interrupts (Traps)

Interrupts are mechanisms that allow a CPU to respond to events, either from external hardware or internal software conditions. Hardware interrupts originate from external devices that assert signals on interrupt request (IRQ) lines. For example, a keyboard press might trigger `IRQ1`, or a network interface might signal an event requiring CPU attention. These interrupts are asynchronous and their latency—the time it takes for the CPU to respond—can vary depending on the CPU's current workload and pending operations, leading to jitter.

In contrast, software interrupts, also known as traps, are triggered by specific instructions or processor exceptions. Examples include the `INT` or `TRAP` instructions used in system calls (like `INT 0x80` in x86 architecture), or faults such as division by zero, page faults, or invalid opcodes. These interrupts are deterministic, meaning they occur precisely at the instruction boundary where the

triggering condition arises, allowing for predictable handling and debugging. Together, hardware and software interrupts form a critical part of system responsiveness and error handling.

5.2. Interrupt Service Cycle

When an interrupt request arrives (during the Execute phase), the CPU executes the Interrupt Acknowledge Cycle before entering the ISR:

1. Complete Current Instruction: Preserve atomicity and avoid partial state.
2. *Interrupt Acknowledge (M**) Phase:** CPU issues INT_ACK cycle on the bus; interrupt controller places vector number onto data bus.
3. Context Save: Push PC, flags, and optionally GPRs onto stack or save in shadow registers.
4. Interrupt Masking: Disable further maskable interrupts (DI) to prevent nested corruption unless priority allows.
5. Vector Fetch: Compute $\text{vector_address} = \text{IVT_base} + (\text{vector_number} \times \text{EntrySize})$; fetch ISR entry.
6. Update PC: $\text{PC} \leftarrow \text{ISR entry}$.
7. Execute ISR: Service device, acknowledge in peripheral, perform necessary I/O or state changes.
8. Return from Interrupt: IRET or RETI instruction pops flags and PC, re-enables interrupts, resumes interrupted code.

participant CPU

participant IntCtrl

Note over CPU: IRQ asserted by Device

CPU->>CPU: Finish current instruction

CPU->>IntCtrl: Issue INT_ACK

IntCtrl-->>CPU: Provide vector number

CPU->>Memory: Read IVT entry

Memory-->>CPU: ISR address

CPU->>CPU: Push context; $\text{PC} \leftarrow \text{ISR address}$; DI

Note over CPU: Execute ISR

CPU->>CPU: IRET $\rightarrow$ Pop context; EI; resume

5.3. Interrupt Handling Mechanism

Table 4: A step-by-step breakdown

Step	Action	Micro-operations
1	Device raises IRQ line.	IRQ line sampled at instruction boundary.
2	CPU finishes current instruction.	Normal execution for last instruction.
3	Push PC and flags onto stack.	$SP \leftarrow SP - 4$; $M[SP] \leftarrow PC$; $SP \leftarrow SP - 4$; $M[SP] \leftarrow FLAGS$
4	Mask further maskable interrupts.	$FLAGS.IF \leftarrow 0$
5	Fetch ISR address from Interrupt Vector Table (IVT).	$MAR \leftarrow IVT_base + (vec \times 4)$; $MDR \leftarrow M[MAR]$; $PC \leftarrow MDR$
6	Execute ISR. Clear device status and perform service.	Device register access; OS calls.
7	IRET: Pop PC and flags; re-enable interrupts.	$FLAGS \leftarrow M[SP]$; $SP \leftarrow SP + 4$; $PC \leftarrow M[SP]$; $SP \leftarrow SP + 4$

This mechanism ensures the CPU can respond swiftly to asynchronous events without losing the context of its current work.

5.4. Interrupt Vector Table

The Interrupt Vector Table (IVT) is a contiguous memory region where each entry corresponds to an ISR's address. In a 32-bit system with 4-byte entries:

$IVT_Base = 0x00000000$

$Entry[i] = M[IVT_Base + (i \times 4)]$; 32-bit ISR address

Table 5: Interrupt Vector Table

Vector #	Offset (bytes)	ISR	Typical Use
0	0x00	Reset Handler	Power-on reset
1	0x04	NMI Handler	Non-maskable events
2	0x08	Hard Fault	Fatal exceptions
9	0x24	Keyboard IRQ (IRQ1)	Keyboard input
...	...	...	...

Efficient IVT lookup reduces ISR dispatch latency. Advanced interrupt controllers (APIC, GIC) support configurable IVT bases and dynamic remapping.

5.5. Nested and Prioritised Interrupts

When multiple interrupt sources contend for the CPU's attention, the interrupt controller must decide which one to service first based on priority. Nested interrupts allow a higher-priority interrupt to preempt a currently executing lower-priority Interrupt Service Routine (ISR), improving responsiveness for critical events.

- **Priority Levels:** Each interrupt source is assigned a unique priority number. A lower numerical value often indicates higher urgency.
- **Nested Servicing:** If a higher-priority interrupt arrives during a lower-priority ISR, the controller issues a new INT_ACK, the CPU saves the ISR's context again, and jumps to the higher-priority ISR.
- **Return Sequence:** After servicing the high-priority ISR, the CPU restores the context of the preempted ISR and completes it before resuming normal execution.

5.6. Context Switching on Interrupt

A context switch during an interrupt is a carefully orchestrated process that ensures the CPU can temporarily pause its current task, handle an urgent event, and then resume execution seamlessly.

It begins with pipeline drain or flush, where any in-progress instructions are discarded to prevent partial execution and ensure a clean transition. Next, the CPU performs a context save, storing the current state of execution—this includes general-purpose registers, the Program Counter (PC), the Processor Status Word (which holds condition flags), and any special-purpose registers like those used for floating-point operations. These saved values are typically pushed onto the stack or stored in dedicated shadow registers, depending on the architecture.

Once the context is saved, the CPU proceeds to Interrupt Service Routine (ISR) entry. It loads the PC with the address of the ISR from the Interrupt Vector Table, and may also mask further interrupts to prevent nested or recursive handling unless explicitly allowed. After the ISR completes its task, the CPU performs a context restore using an instruction like IRET (Interrupt Return). This step involves popping the saved registers and flags from the stack, restoring the PC, and refilling the pipeline so that normal execution can resume exactly where it left off.

5.7. Examples

- Timer Interrupt in an RTOS

A high-precision timer (highest priority) pre-empts a disk-transfer ISR (lower priority) to maintain real-time scheduling.

- UART vs. DMA Controller

A UART's receive interrupt (medium priority) can be nested within a lower-priority keyboard interrupt to avoid data loss.

- Software Trap vs. Hardware IRQ

A synchronous trap (e.g., division-by-zero) is treated with the highest priority, pre-empting any asynchronous hardware ISRs.

SELF-ASSESSMENT QUESTIONS - 3

True or False:	
11	Non-maskable interrupts (NMIs) can be disabled by the DI instruction.
Multiple Choice	
12	Which of these is not typically saved during an interrupt context-save? A. General-purpose registers B. Program Counter C. Processor flags D. Cache contents
Short Answer	
13	Define a nested interrupt and explain how the CPU handles its context.
Fill in the Blank	
14	The Interrupt Vector Table base address in a 32-bit system is usually at 0x_____.
Calculation	
15	If saving all registers takes 30 cycles and flushing the pipeline takes 5 cycles, what is the total entry-latency overhead when an interrupt arrives?

6. SUMMARY

In this unit, we have systematically dissected the core microarchitectural processes that govern modern CPU operation. Beginning with the instruction cycle, we modeled the stages—Fetch, Decode, Execute, Memory, and Write-Back—as a finite-state machine (FSM), clarifying how control signals guide transitions between states. We also introduced a pipeline performance model, analysing cycles per instruction (CPI) under ideal and hazard-prone conditions, and applied Amdahl's Law to illustrate the practical limits of speedup.

Next, we explored hazard detection and forwarding mechanisms, distinguishing between structural, data (RAW, WAR, WAW), and control hazards. Hardware solutions such as resource duplication, interleaved caches, forwarding networks, branch prediction strategies, and branch-target buffers were discussed in detail. The unit then delved into memory-reference instructions, defining Load and Store operations and breaking down their micro-operations.

We examined five addressing modes—Immediate, Direct, Indirect, Register-Direct/Indirect, and Indexed—comparing their hardware implications and execution sequences across MIPS and x86 instruction sets. Further, we studied memory hierarchy and caching, outlining multi-level cache architectures and deriving the Average Memory Access Time formula. Topics such as write policies, MESI cache coherency protocols, and the impact of hit-rate variations on CPI were thoroughly analysed. Virtual memory and address translation were introduced with a focus on TLB operation, page-fault handling, and effective memory-access time. The unit also covered I/O paradigms, contrasting Programmed I/O, Interrupt-Driven I/O, and DMA, while highlighting timing breakdowns, bus arbitration techniques, and real-world applications. Finally, we examined interrupts, classifying types and detailing the interrupt service cycle, vector tables, context-switching, and pipeline-related latency.

Together, these topics form a cohesive foundation for understanding how instructions are processed and how asynchronous events and peripheral interactions are managed—essential knowledge for designing and optimising CPU architectures.

7. GLOSSARY

Amdahl's Law	-	A formula that predicts overall system speedup based on the fraction of execution time that can be parallelised.
Branch Target Buffer (BTB)	-	A cache that stores the target addresses of previously encountered branch instructions to speed up fetch for predicted paths.
Cache Coherency (MESI)	-	A protocol ensuring consistency of data in multi-level or multi-processor cache hierarchies (Modified, Exclusive, Shared, Invalid states).
Context Switch	-	The process of saving and restoring CPU state (registers, PC, flags) when transitioning between tasks or ISRs.
Control Hazard	-	A pipeline stall caused by uncertainty in the program counter due to branches or jumps.
Data Hazard	-	A condition where instructions depend on the results of prior instructions that have not yet completed.
Pipeline Flush	-	Clearing all partially completed instructions in the pipeline in response to control-flow changes.
Programmed I/O	-	An I/O method where the CPU actively polls a device's status register, stalling until the device is ready.
Saturating Counter	-	A small counter used in dynamic branch prediction that increments or decrements within fixed bounds to track history.
TLB (Translation Lookaside Buffer)	-	A small, fast cache that holds recent virtual-to-physical page translations to speed up address translation.
Two-Bit Predictor	-	A dynamic branch prediction scheme using a 2-bit saturating counter per branch, reducing misprediction rates.
Write-Back vs. Write-Through	-	Cache write policies: write-back buffers modified data until eviction; write-through updates both cache and main memory immediately.

8. TERMINAL QUESTIONS

1. Draw and label the five-state FSM for the instruction cycle, including the inputs, outputs, and micro-operations for each state transition.
2. Compare and contrast static (“always-taken”) branch prediction with two-bit saturating-counter prediction in terms of accuracy and hardware cost.
3. Explain the differences between structural, data (RAW, WAR, WAW), and control hazards, and describe one hardware technique to mitigate each.
4. List and characterise the five addressing modes covered in the chapter, and give a realistic use-case for each mode.
5. Detail the execution sequence of micro-operations.
6. Illustrate how the hazard-detection unit and forwarding network collaborate to resolve a RAW dependency without stalling.
7. Compare programmed I/O, interrupt-driven I/O, and DMA for a 10 MB transfer in terms of CPU utilisation, latency, and throughput.
8. Outline the full interrupt-service cycle—from IRQ assertion through INT_ACK, context-save, vector fetch, ISR execution, to IRET—including key micro-operations.
9. Describe how nested and prioritised interrupts are handled, and analyse their impact on worst-case ISR latency in a real-time system.
10. Propose a bus-arbitration scheme for CPU, DMA controller, and GPU sharing an AMBA AXI bus that balances fairness and low-latency service for high-priority masters.

9. ANSWERS

Self-Assessment Questions

1. False (immediate addressing encodes the operand in the instruction itself).
2. C. Indirect.
3. Indexed addressing is preferred when accessing elements of an array at runtime, because the base register holds the start address and the offset encodes the element index.
4. $\text{Fetch} + \text{decode} + \text{EA} + \text{write-back} = 1 + 1 + 1 + 1 = 4$ cycles
5. plus memory access: $0.9 \times 1 + 0.1 \times 50 = 0.9 + 5 = 5.9$. $0.9 \times 1 + 0.1 \times 50 = 0.9 + 5 = 5.9$. Total ≈ 9.9 cycles.
6. The register name (e.g., R2) holds the EA.
7. False (programmed I/O polls and blocks the CPU).
8. B. Device places vector number on bus.
9. SAR (Source Address Register), DAR (Destination Address Register), BCR (Byte Count Register), CSR (Control/Status Register).
10. Programmed I/O: $1 \mu\text{s}/\text{word} \times 1024 = 1024 \mu\text{s}$. DMA overhead = $10 \mu\text{s}$. CPU time saved $\approx 1014 \mu\text{s}$.
11. False (NMIs cannot be disabled).
12. D. Cache contents.
13. A nested interrupt occurs when a higher-priority interrupt arrives during a lower-priority ISR; the CPU pushes the lower-priority context again, services the high-priority ISR, then pops back to resume.
14. 0x00000000.
15. $30 + 5 = 35$ cycles.

Terminal Questions Answers

Answer 1: Five-State FSM Diagram

- States:
 - Fetch
 - Decode
 - Execute
 - Memory

- WriteBack
- Input: clock
- Outputs (micro-ops):

Transition	Output
Fetch → Decode	$MAR \leftarrow PC$; $MDR \leftarrow M[MAR]$; $IR \leftarrow MDR$; $PC \leftarrow PC+1$
Decode → Execute	Decode opcode; prepare operands
Execute → Memory	ALU/EAB calculation
Memory → WriteBack	Load: $MDR \leftarrow M[MAR]$; Store: $M[MAR] \leftarrow MDR$
WriteBack → Fetch	Write back to register

For more details, refer section 2.

Answer 2: Static vs. Two-Bit Saturating-Counter Prediction

Aspect	Static ("always-taken"/"always-not-taken")	Two-Bit Saturating Counter
Hardware cost	None (no extra bits or logic)	2 bits per branch entry + inc/dec logic
Accuracy	Very low (cannot adapt)	Significantly higher (adapts to patterns)
Response to noise	Mis-predicts on first occurrence	Requires two mispredictions to flip, reducing noise impact

For more details, refer section 2.

Answer 3: Hazard Types & Mitigations

- Structural Hazard: simultaneous resource use (e.g., single ALU).
Mitigation: Resource duplication (separate ALU or separate I-cache/D-cache).
- Data Hazard (RAW, WAR, WAW): data dependencies between instructions.
 - RAW (true dependency) → Mitigation: Forwarding network bypasses results to dependent stage.
 - WAR/WAW → Mitigation: Register renaming avoids name conflicts.
- Control Hazard: uncertain PC after branch/jump.
Mitigation: Dynamic two-bit branch predictors + BTB to speculatively fetch correct path.

For more details, refer section 2.

Answer 4: Addressing Modes & Use-Cases

- Immediate: operand encoded in instruction.
Use-case: initialising loop counters.
- Direct: instruction contains literal memory address.
Use-case: accessing global variables.
- Indirect: memory cell holds target address.
Use-case: pointer dereferencing in linked data structures.
- Register Direct: operand is in a register.
Use-case: moving data between registers.
- Register Indirect: register holds effective address.
Use-case: stack operations via stack pointer.
- Indexed: $EA = \text{base register} + \text{offset}$.
Use-case: array indexing.

For more details, refer section 3.

Answer 5:

Cycle	LOAD Direct	STORE Direct
M1	$MAR \leftarrow PC$; $MDR \leftarrow M[MAR]$; $IR \leftarrow MDR$; $PC \leftarrow PC + 1$	$MAR \leftarrow PC$; $MDR \leftarrow M[MAR]$; $IR \leftarrow MDR$; $PC \leftarrow PC + 1$
M2	Decode IR	Decode IR
M3	$MAR \leftarrow IR(\text{address})$	$MAR \leftarrow IR(\text{address})$
M4	$MDR \leftarrow M[MAR]$	$MDR \leftarrow R[\text{src}]$
M5	$R[\text{dest}] \leftarrow MDR$	$M[MAR] \leftarrow MDR$

For more details, refer section 3.

Answer 6: Hazard Detection & Forwarding Collaboration

- Compare EX/MEM destination register vs. ID stage source registers.
- If a match (RAW) and data is available in EX/MEM or MEM/WB stage,
- Forward via multiplexers at ALU inputs instead of stalling.
- If no forwarding path covers the dependency, the control unit inserts a one-cycle stall.

For more details, refer section 2.

Answer 7: I/O Paradigms Comparison for 10 MB

Paradigm	CPU Utilisation	Latency	Throughput
Programmed I/O	100 % (polling)	High per-word stalls	Low (device-bound)
Interrupt-Driven	Moderate (ISR on IRQ)	Reduced (only ISR)	Moderate
DMA	Minimal (setup + ISR)	Low (bulk transfer)	Highest (bus-max)

For more details, refer section 4.

Answer 8: Interrupt-Service Cycle Steps

- Finish current instruction.
- Flush pipeline.
- Issue INT_ACK; peripheral places vector number on bus.
- Push PC, flags, GPRs onto stack.
- Mask further maskable interrupts.
- Compute $\text{vector_addr} = \text{IVT_base} + \text{vector} \times \text{entry_size}$; $\text{MAR} \leftarrow \text{vector_addr}$; $\text{MDR} \leftarrow \text{M}[\text{MAR}]$; $\text{PC} \leftarrow \text{MDR}$.
- Execute ISR body.
- IRET: Pop flags, PC, GPRs; refill pipeline; resume program.

For more details, refer section 4.

Answer 9: Nested & Prioritised Interrupt Handling

- On a higher-priority IRQ during a lower-priority ISR:
 1. Push low-priority ISR context again.
 2. Service high-priority ISR.
 3. IRET: pop to resume low-priority ISR; then IRET back to user code.
- Worst-case latency increases by two context-save/restore sequences plus two pipeline flushes.

For more details, refer section 5.

Answer 10: Proposed AXI Bus Arbitration

- Weighted Round-Robin with Priority Boost:
 - Each master (CPU, DMA, GPU) normally gets equal time-slots.
 - GPU (real-time) may “steal” one slot per cycle if requests pending, up to its weight.
 - Idle slots are redistributed to other masters, ensuring fairness while honoring real-time needs.

For more details, refer section 4.

10. REFERENCES

1. Hennessy, J. L., & Patterson, D. A. (2019). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.
2. Patterson, D. A., & Hennessy, J. L. (2017). Computer Organization and Design: The Hardware/Software Interface (5th ed.). Morgan Kaufmann.
3. Stallings, W. (2018). Computer Organization and Architecture: Designing for Performance (11th ed.). Pearson.
4. Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson.
5. Heuring, V. P., & Jordan, H. F. (2014). Computer Systems Design and Architecture (2nd ed.). Addison-Wesley.
6. Furber, S. (2010). ARM System-on-Chip Architecture (2nd ed.). Addison-Wesley.

BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 7

Control Memory

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Control Word and Microoperations	1	-	6 - 9
2.1	Structure of a Control Word	-	-	
2.2	Microoperations and Control Signals	-	-	
2.3	Encoding Control Signals	-	1	
3	Microinstructions	-	-	10 - 15
3.1	Types of Microinstructions	-	-	
3.2	Format and Fields of a Microinstruction	i	-	
3.3	Execution of Microinstructions	-	2	
4	Microprogrammed Control	-	-	16 - 21
4.1	Basic Concept and Advantages	2	-	
4.2	Microprogram Sequencing	-	-	
4.3	Control Address Register and Control Memory	ii	3	
5	Hardwired Control	3, iii	-	22 - 27
5.1	Design Principles	4, iv	-	
5.2	Comparison with Microprogrammed Control	-	-	
5.3	Use Cases and Applications	-	4	
6	Control Memory	-	-	28 - 34
6.1	Structure of Control Memory	5	-	
6.2	Control Memory Addressing	-	-	
6.3	ROM vs RAM in Control Units	v	5	
7	Summary	-	-	35
8	Glossary	-	-	36
9	Terminal Questions	-	-	37
10	Answers	-	-	38 - 42
11	References	-	-	43

1. INTRODUCTION

In the previous unit, we focused on the Instruction Cycle and gained an understanding of how a processor fetches, decodes, and executes instructions. We specifically examined the behaviour of Memory Reference Instructions, Input/Output Instructions, and the mechanism of Interrupts. These topics helped us understand how the CPU interacts with memory and peripheral devices, and how it manages control flow when external events occur. We also saw how the control unit plays a critical role in orchestrating each phase of the instruction cycle by generating the necessary control signals.

In this unit, we will take a deeper look into the internal mechanisms of the control unit itself. In particular, we explore the concept of Control Memory, which is responsible for storing the micro-level instructions—called microinstructions—that drive the execution of each machine instruction. These microinstructions are bundled into microprograms, and together they define the behaviour of the CPU at the hardware level.

We begin with an introduction to the Control Word, which is a binary representation of control signals that activate specific micro-operations in various parts of the CPU. From there, we move on to understand how microinstructions are structured and sequenced. This leads us into the design of Microprogrammed Control Units, where a control memory stores the microcode required for instruction execution.

In contrast to this, we will also study Hardwired Control Units, where the control logic is implemented using fixed digital circuits rather than stored instructions. While hardwired control offers faster execution, it lacks the flexibility of microprogrammed control and is more difficult to modify or extend.

By the end of this unit, you will have a solid understanding of how a CPU's control unit is designed and implemented, the difference between hardwired and microprogrammed control approaches, and the role of control memory in enabling instruction execution at the micro-operation level. This unit is crucial for bridging the gap between high-level instruction execution and low-level hardware behaviour, and it sets the stage for more advanced topics in computer architecture and design.

1.1. Objectives

After studying this unit, you should be able to:

- Explain the concept of a Control Word and how it represents a set of control signals for micro-operations.
- Discuss the structure and role of Microinstructions.
- Distinguish between Hardwired Control Units and Microprogrammed Control Units.
- Illustrate how Control Memory is used to store and execute microinstructions.
- Identify the components involved in microprogram execution.
- Explain how control signals are generated using both hardwired logic and microprogrammed sequencing techniques.
- Apply the concept of microprogramming to design or interpret control sequences for instruction execution.



2. CONTROL WORD AND MICROOPERATIONS

In a digital computer system, each instruction is executed through a series of microoperations—small, fundamental steps like data transfers between registers, ALU operations, or memory reads. The control unit coordinates these microoperations by issuing control signals, but instead of triggering each signal individually, it uses a control word: a binary pattern where each bit corresponds to a specific control line.

For example, in a system with a 10-bit control word, each bit may represent actions like loading the accumulator (AC), initiating memory read, or performing an ALU addition. A control word like 1010010001 could simultaneously load data from memory into the Data Register (DR), perform addition using the ALU, and store the result into the AC—all in one clock cycle. Each '1' in the control word enables a specific microoperation, allowing for parallel activation of hardware components.

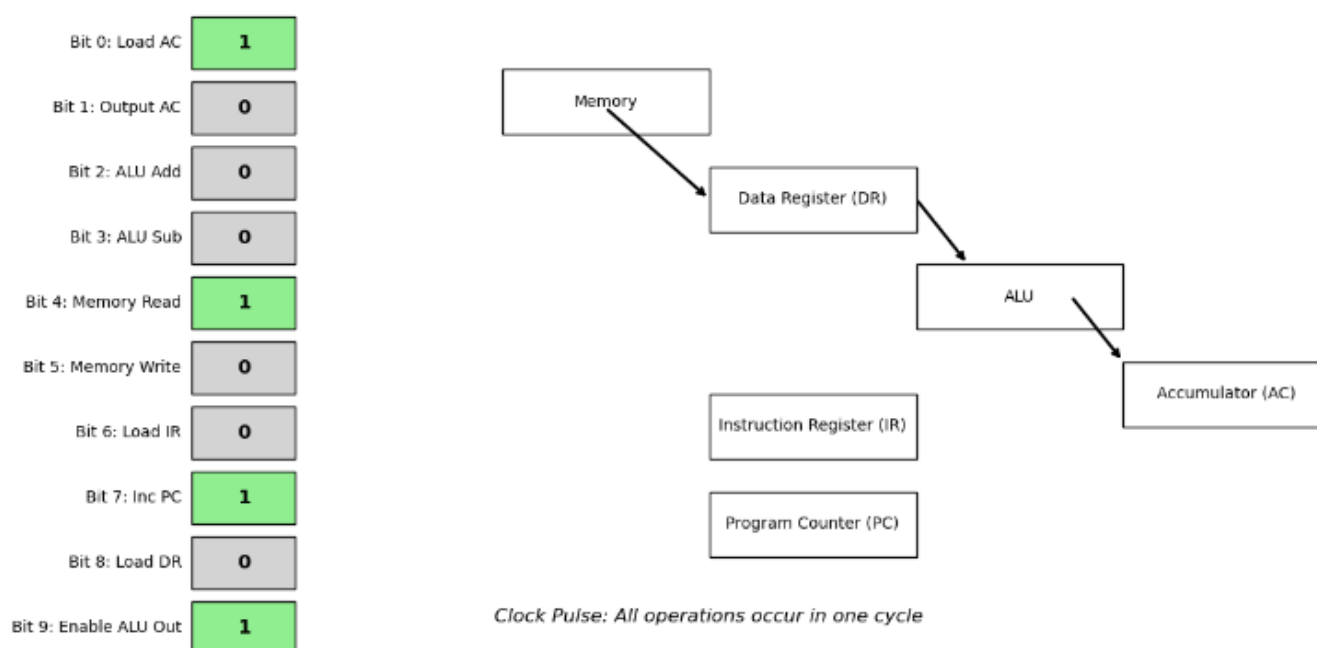


Fig 1 - 10-bit Control Word Activating CPU Microoperations in a Single Clock Cycle

Control words are essential in both hardwired control units, which use logic circuits, and microprogrammed control units, where control words are stored as microinstructions in control memory. These microinstructions sequence complex instruction execution like ADD or STORE. Thus, control words form the internal language that bridges software instructions and hardware execution.

2.1. Structure of a Control Word

The structure of a control word is crucial in CPU design, as it defines how multiple microoperations are encoded and triggered in a single clock cycle. A control word is a fixed-length binary pattern, typically divided into fields, with each field targeting a specific hardware unit—like the ALU, registers, memory, or buses. Each bit or group of bits within these fields activates corresponding control lines or selects operations.

Consider a 16-bit control word in a simple CPU:

- [15–12]: ALU Operation (e.g., ADD, SUB)
- [11–8]: Source Register (e.g., R3)
- [7–4]: Destination Register (e.g., R12)
- [3]: Memory Read Enable
- [2]: Memory Write Enable
- [1]: Increment Program Counter
- [0]: Load Instruction Register

For example, the control word 0101001111001010 performs an ADD (R3 to R12), reads memory, and increments the program counter.

By organising control signals into structured fields, the CPU can execute multiple internal actions in parallel with one instruction cycle. More complex CPUs may add fields for buses, flags, or microinstruction jumps. Thus, the control word acts as a blueprint that ensures synchronised and efficient internal operation across the processor's architecture.

2.2. Microoperations and Control Signals

Microoperations are the basic hardware-level actions a CPU performs to execute instructions. They involve tasks like transferring data between registers, performing ALU operations, and accessing memory. Each microoperation is activated by a control signal, a binary trigger (1 = active, 0 = inactive) generated by the control unit.

Microoperations fall into categories:

- Register Transfer: e.g., $R1 \leftarrow R2$
- Arithmetic: e.g., $R3 \leftarrow R1 + R2$
- Logic: e.g., $R1 \leftarrow R1 \text{ AND } R2$

- Memory: e.g., $DR \leftarrow M[AR]$

To perform $R1 \leftarrow R2 + R3$, control signals must:

- Enable R2 and R3 outputs to the ALU
- Select ALU "Add" operation
- Enable ALU output and load it into R1

Each control signal—like “Enable R2 Output” or “Load R1”—corresponds to a bit in a control word. In a memory read ($DR \leftarrow M[AR]$), signals like “Enable AR”, “Memory Read Enable”, and “Load DR” must be precisely sequenced.

In hardwired units, logic circuits generate these signals. In microprogrammed systems, control words stored in control memory define them. Accurate timing and coordination of these control signals are critical to avoid data corruption and ensure reliable CPU operation.

2.3. Encoding Control Signals

In microprogrammed control units, using one bit per control signal in each microinstruction would result in excessively wide control words. To reduce size and improve efficiency, control signals are typically encoded. Encoding compresses control information by representing multiple signals with shorter fields, which are later decoded to activate the appropriate hardware lines.

Instead of dedicating a bit for each signal, control signals are grouped into categories like ALU operations, register selection, or memory access. Each group is assigned a field in the control word containing a binary code that selects one option from the group. These codes are interpreted by internal decoders in the control unit.

For example, a 12-bit control word might be structured as:

- [11–9]: ALU Operation Code
- [8–6]: Source Register
- [5–3]: Destination Register
- [2]: Memory Read
- [1]: Memory Write
- [0]: Program Counter Increment

This format allows for efficient instruction encoding while covering a wide range of microoperations. Though encoding requires additional decoding logic and can limit simultaneous signal activation, it

significantly reduces control memory size and simplifies microinstruction design. Hybrid encoding, combining encoded fields with direct control bits, is often used for more flexibility.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What does a control word primarily define in a microprogrammed control unit? A) The memory address B) The instruction format C) The set of microoperations to be executed D) The clock cycle duration
2	Which of the following is a typical component of a control word? A) Opcode B) Control signals C) Program counter D) ALU result
3	What is the purpose of encoding control signals? A) To increase the number of control lines B) To reduce hardware complexity C) To slow down execution D) To eliminate microoperations
Fill in the Blank	
4	A _____ is a binary string that specifies a set of microoperations to be performed during one clock cycle.
5	Control signals are used to activate specific _____ within the CPU.
6	Encoding control signals helps reduce the number of _____ required to implement control logic.
True or False	
7	A control word can activate multiple microoperations simultaneously.
8	Microoperations are executed one at a time and cannot be combined.
9	Control signals are only used for memory access and not for ALU operations.
10	Encoding control signals allows for more efficient use of hardware resources.

3. MICROINSTRUCTIONS

In microprogrammed control units, a microinstruction is a single control word stored in control memory that specifies one or multiple microoperations to execute in one clock cycle. Each microoperation (e.g., register transfer, ALU action, memory access) is triggered by binary control signals defined in the control word. A microprogram is a sequence of such microinstructions, stored in control memory (typically ROM or writable control storage) and orchestrated by a microprogram counter or sequencer to implement machine level instructions.

When executing a typical instruction (e.g., ADD R1, R2), the CPU enters the corresponding microprogram, which sequentially performs fetch, decode, operand read, ALU execution, and result store steps via successive microinstructions. Each microinstruction also includes sequencing fields—such as next address, condition bits, or branch logic—to determine control flow.

Microinstructions are formatted in two main styles:

- Horizontal: One bit per control signal—very wide words enabling high parallelism and speed, at the cost of memory size and complexity.
- Vertical: Encoded control fields that require decoding—more compact and simpler, but slower and less parallel.

Advantages include clear sequencing of complex instruction logic, simplified and modifiable control design, and ease of debugging. Unlike hard wired control, updating microprograms often just means rewriting control memory—not redesigning hardware.

3.1. Types of Microinstructions

Microinstructions, depending on their structure and purpose, can be categorised into different types based on how they encode control information and how they manage the sequencing of microoperations. Broadly, microinstructions are classified as either horizontal or vertical, based on the level of control signal encoding. In a horizontal microinstruction format, each individual control signal has a dedicated bit position in the microinstruction. This allows for multiple microoperations to be activated simultaneously, offering high parallelism and efficient execution. For example, one bit may enable the output of a register, another may activate an ALU operation, and another may trigger a memory write. Because all these bits can be set independently, horizontal microinstructions offer precise and direct control over the datapath. However, the trade-off is that they result in long control

words, often with many unused bits if fewer operations are needed in a given cycle, which increases the size of the control memory.

In contrast, vertical microinstructions use encoded fields to represent groups of control signals. For instance, instead of having separate bits for each possible register or ALU function, a few bits may encode the operation to be performed, which is then decoded internally to generate the required control signals. Vertical microinstructions are more compact and save memory space, making them suitable for control units where instruction storage is a concern. However, because fields are mutually exclusive, they generally permit only one microoperation per functional group at a time, thus reducing parallelism and potentially slowing execution in complex operations.

In addition to format-based classification, microinstructions can also be categorised by their function. Control microinstructions contain the control word needed to perform specific microoperations within the datapath, such as transferring data or performing ALU operations. These are the most common types and directly impact the processor's behaviour in a given clock cycle. Another class includes sequencing microinstructions, which are concerned with controlling the flow of execution within the microprogram. These may include fields for branching, testing condition flags, or specifying the next address in control memory. In many designs, a microinstruction may include both control and sequencing information in the same word.

As an illustrative example, consider a microinstruction that performs a memory read operation and increments the program counter. In a horizontal format, this might be represented as a control word like 1001001000100000, where separate bits correspond to enabling the memory address register, activating the memory read line, and enabling the program counter increment. In a vertical format, the same functionality might be expressed as several compact fields: a 3-bit field for selecting the ALU function (e.g., ADD), a 4-bit field for selecting the destination register, and a 2-bit field for the sequencing mode (e.g., go to next, jump, conditional branch).

Ultimately, the type of microinstruction used in a control unit depends on the design priorities of the system—whether the goal is compact control memory usage, maximum execution speed, or design simplicity. Understanding the types of microinstructions is essential for appreciating how microprogrammed control units optimise instruction execution and maintain flexibility in CPU design.

3.2. Format and Fields of a Microinstruction

The format of a microinstruction defines how the various control-related fields are arranged within a fixed-length binary word stored in the control memory of a microprogrammed control unit. Each

microinstruction contains specific fields that determine the control signals to be activated, the conditions under which they are applied, and the sequencing logic for determining the address of the next microinstruction. The design of this format must strike a balance between functional flexibility and memory efficiency. A typical microinstruction format includes fields such as the control field, which encodes the microoperations to be performed; the next address field, which indicates the location of the next microinstruction to be executed; and condition or branch fields, which determine the flow of control based on CPU status flags or external conditions.

For example, in a 32-bit microinstruction, the layout might be structured as follows: bits 31–20 could represent the control field, with each bit or group of bits activating specific hardware control lines such as register enables, ALU operations, or memory access; bits 19–16 could serve as a condition field that determines whether a branch should be taken based on flags like zero, carry, or overflow; and bits 15–0 could represent the next address field, which points to the next microinstruction in control memory. This allows for conditional branching within the microprogram, enabling complex instruction behaviours such as loops or decision-making to be implemented efficiently at the microinstruction level.

In a horizontal microinstruction format, the control field may contain dozens of bits—each corresponding directly to a control signal. This supports parallel activation of multiple microoperations but leads to longer and wider microinstructions. In contrast, a vertical format will divide the control field into compact, encoded subfields, where each field selects one operation from a group, such as one ALU function, one register destination, and one source. This saves memory space but limits the ability to execute multiple microoperations simultaneously. Some designs employ a hybrid format, combining both encoded and direct control fields to balance parallelism with compactness.

To illustrate, consider the following sample microinstruction format used in a simple microprogrammed system.

Table 1: Microinstruction format

Bit Range	Field Name	Purpose
31–24	ALU	Specifies the operation to be performed by the Arithmetic Logic Unit (e.g., ADD, SUB, AND).
23–20	SRC	Indicates the source register or operand for the ALU operation.
19–16	DEST	Indicates the destination register where the result will be stored.

15–8	FLAGS	Contains status flags (e.g., Zero, Carry, Overflow) that influence conditional operations.
7–0	NEXTADDR	Specifies the address of the next microinstruction to execute (used for branching or sequencing).

In this example, the ALU field selects the operation (ADD, SUB, AND, etc.), SRC and DEST specify the source and destination registers, FLAGS indicate conditions for branching or control decisions, and NEXTADDR holds the address of the next microinstruction. The control unit decodes each of these fields and activates the corresponding control signals to perform the desired internal CPU actions during that clock cycle.

A well-designed microinstruction format allows for the efficient sequencing of control signals, supports conditional execution paths, and ensures that the hardware can perform complex tasks with minimal microinstruction count. The choice of format and field layout directly influences the speed, size, and flexibility of the control unit, making it a fundamental aspect of microprogrammed control design.

3.3. Execution of Microinstructions

The execution of microinstructions is the process by which the control unit interprets and activates the control signals encoded in a microinstruction to carry out the corresponding microoperations within the CPU. When an instruction is fetched from main memory and decoded, the control unit uses its opcode to determine the starting address of the corresponding microprogram stored in control memory. The control unit then begins fetching and executing microinstructions sequentially, using a register known as the Control Address Register (CAR) to hold the address of the current microinstruction. Each microinstruction is fetched from control memory and loaded into the Control Data Register (CDR), which temporarily holds the instruction during decoding and execution. The control fields of the microinstruction are decoded to generate the specific control signals that activate hardware components, such as enabling register transfers, selecting ALU functions, performing memory reads or writes, and updating status flags.

The execution of a microinstruction typically happens in a single clock cycle, during which all specified microoperations are carried out simultaneously if the format permits parallelism. For example, in one cycle, the control word might enable the output of register R2, pass it to the ALU, perform an addition with the contents of register R3, and store the result into register R1—all governed by the set bits in the control field. After the microinstruction is executed, the control unit

determines the address of the next microinstruction. This can be done in several ways: by incrementing the CAR to fetch the next sequential instruction, by using an address field specified in the current microinstruction, or by performing a conditional branch based on processor status flags or external conditions.

Branching decisions are guided by a condition field within the microinstruction, which checks flags such as zero, carry, or overflow. If the condition is met, the control unit loads a new address into the CAR, allowing the microprogram to jump to a different execution path. This mechanism supports looping, conditional execution, subroutine calls, and other control flow operations similar to those found in high-level programs, but at the micro-level of CPU operation.

Once the sequence of microinstructions that implements a particular machine instruction has been executed, control is either passed to the next instruction's microprogram or returned to the instruction fetch cycle. In this way, the execution of microinstructions forms the invisible layer beneath the instruction cycle, translating machine-level operations into coordinated hardware-level actions.

Thus, the execution of microinstructions is a cyclical process involving fetching from control memory, decoding the control fields to activate specific control signals, executing the corresponding microoperations, and determining the address of the next microinstruction. This cycle continues until the complete microprogram for a machine instruction is executed. The efficiency and accuracy of this process directly affect the performance and correctness of the entire CPU, making it a foundational component in the design of microprogrammed control units.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
1	Which of the following best describes a microinstruction? A) A high-level programming command B) A binary instruction that specifies microoperations C) A hardware signal D) A compiler directive
2	What are the two main types of microinstructions? A) Arithmetic and Logical B) Horizontal and Vertical

	C) Control and Data D) Simple and Complex
3	In a horizontal microinstruction format, control signals are: A) Encoded B) Grouped C) Directly specified D) Ignored
Fill in the Blank	
4	A _____ microinstruction uses encoded control signals to reduce the number of bits required.
5	The _____ field in a microinstruction specifies the operations to be performed by the ALU.
6	Microinstructions are stored in the _____ memory of the control unit.
True or False	
7	Microinstructions are executed during the fetch phase of the instruction cycle.
8	Vertical microinstructions are more compact than horizontal microinstructions.
9	The format of a microinstruction includes fields for control signals, ALU operations, and next address.
10	Microinstructions are used to define the behaviour of the control unit in a microprogrammed CPU.



4. MICROPROGRAMMED CONTROL

Microprogrammed control designs the CPU's control unit using a software-like method: control signals are stored as microinstructions in control memory, instead of being hardwired via fixed logic. Each microinstruction encodes control signals for a single clock cycle, and a microprogram—a sequence of these—implements a complete machine instruction.

When the CPU fetches and decodes a machine instruction, its opcode determines the start address in control memory. This address sits in the Control Address Register (CAR). During each cycle, a microinstruction is fetched into the Control Data Register (CDR) and activates control signals for microoperations, such as ALU activity, register transfers, or memory access.

Sequencing occurs via updates to the CAR: it may simply increment or branch based on embedded microinstruction bits. This mechanism enables conditionals, loops, and subroutine-like behaviour in microprogram routines. A microsequencer handles this logic.

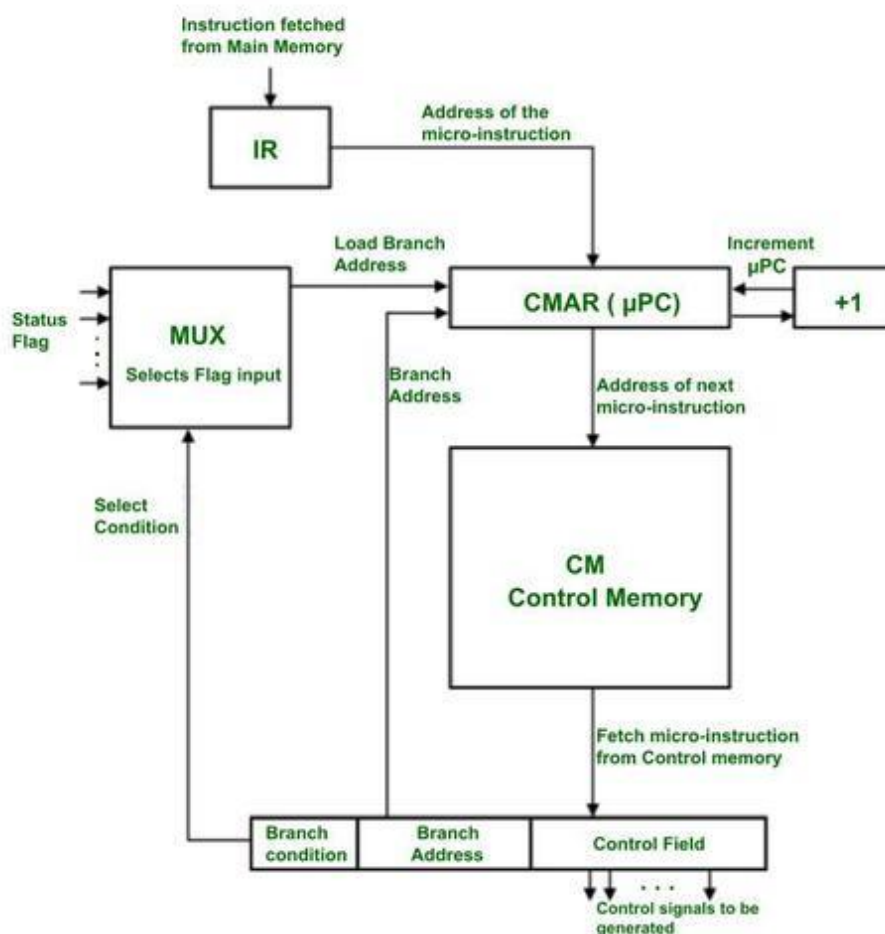


Fig 2- Microprogrammed Control Unit Structure and Flow

The figure illustrates a microprogrammed control unit, which is a type of control unit used in computer architecture to manage the execution of instructions. At the heart of this system is the Instruction Register (IR), which fetches instructions from the main memory. These instructions are then used to determine the address of the corresponding micro-instruction, which is stored in the Control Memory Address Register (CMAR), also known as the microprogram counter (μ PC).

A Multiplexer (MUX) plays a crucial role in decision-making by selecting inputs based on status flags and select conditions. This helps determine whether the control unit should continue with the next sequential micro-instruction or branch to a different one. The CMAR can either increment its value to point to the next instruction or load a new branch address depending on the conditions evaluated by the MUX.

Once the appropriate address is determined, it is sent to the Control Memory (CM), which stores the micro-instructions. The CM then fetches the micro-instruction and generates the necessary control signals to execute the instruction. These control signals are sent to various parts of the CPU to carry out the desired operations.

The microprogrammed control approach offers significant advantages—flexibility, ease of modification, and simpler debugging—as changes in the ISA or control logic require only updates to control memory, not hardware redesign. Its main drawback is slightly slower operation compared to hardwired control, due to microinstruction fetches.

However, for complex instruction sets and evolving architectures, microprogrammed control remains a scalable, maintainable solution, harnessing firmware-like structure to manage CPU operations efficiently.

4.1. Basic Concept and Advantages

Microprogrammed control replaces complex hardwired logic with a programmable, flexible system. Control signals for CPU operations are stored as microinstructions in control memory. Executing a machine-level instruction triggers its corresponding microprogram, a sequence of microinstructions fetched and executed step-by-step. Each microinstruction issues control signals for micro-operations like register transfers, ALU tasks, memory access, or program counter updates.

Key Advantages:

- **Flexibility & Upgradability**

Control logic resides in memory and can be updated simply by altering microcode. This adaptability supports firmware updates, new instruction sets, and multi-ISA emulation—without hardware redesign.

- **Simplified Design & Debugging**

Because microinstructions are software-like, sequential and modular, they're easier to write, test, and maintain. Operations can be reused across instructions, improving modularity and reducing errors.

- **Structured Control Flow**

Microprograms incorporate next address fields and conditional branches, enabling loops, subroutines, and conditional flows within control code.

Although microprogrammed control runs slightly slower than hardwired control due to instruction fetch overhead, this trade-off is often acceptable—especially in CISC processors, emulators, or systems requiring frequent ISA updates.

4.2. Microprogram Sequencing

Microprogram sequencing controls the order of fetching and executing microinstructions. Each machine-level instruction maps to a microprogram—a structured sequence of microinstructions. The Control Address Register (CAR) holds the address of the next microinstruction in control memory and is updated after each fetch.

CAR updates occur via:

- Sequential increment, advancing to the next microinstruction;
- Unconditional branching, loading a specified address;
- Conditional branching, selecting between next or branch address based on status flags like zero or carry.

Microinstructions often include a next-address field, which explicitly defines branch targets. More complex formats embed conditional logic and mapping functions to resolve the next CAR value.

The microprogram sequencer combines incrementer, branch logic, mapping logic, a multiplexer, and sometimes a Subroutine Register (SBR) to support subroutine calls and returns.

Conditional branching allows decision-making, supporting loops and dynamic execution based on flag conditions—e.g., branching if zero after an ALU operation .

Mapping logic uses the opcode from the Instruction Register to load the starting microinstruction address into CAR, enabling immediate access to the relevant microprogram .

In summary, microprogram sequencing uses CAR and sequencer logic to enable modular, flexible, and maintainable execution of microprograms via controlled sequencing, branching, and subroutine management.

4.3. Control Address Register and Control Memory

The Control Address Register (CAR) and Control Memory are two fundamental components of a microprogrammed control unit. Together, they manage the fetching and sequencing of microinstructions that direct the CPU's internal operations. The Control Address Register holds the address of the microinstruction to be fetched from control memory in the next clock cycle. It functions similarly to the Program Counter (PC) in general instruction execution, but at the microinstruction level. Once the microinstruction is fetched, it is transferred to the Control Data Register (CDR) or directly decoded to generate the required control signals.

Control Memory is a read-only memory (ROM) or sometimes a writable memory (like EEPROM or RAM in reconfigurable systems) that stores all the microinstructions (or microcode) for the CPU. Each location in control memory corresponds to one microinstruction, which includes fields for control signal activation, branching decisions, and the next microinstruction address.

For example, suppose the control memory contains a microinstruction at address 11001000 (binary) that directs the CPU to read from memory into a data register and then proceed to address 11001001. The CAR initially holds 11001000. On the next clock pulse:

- The control unit uses the value in CAR to fetch the microinstruction from control memory at address 11001000.
- The contents of that memory location are interpreted to produce control signals like "Enable Memory Read" and "Load Data Register".
- The next address field in the microinstruction (e.g., 11001001) is loaded into the CAR.
- On the following clock cycle, the control unit fetches the microinstruction from address 11001001, and the cycle continues.

This process can be summarised as:

CAR = 11001000

→ Fetch control word from CM[11001000]

→ Execute microoperations: Memory Read, Load DR

→ CAR ← 11001001 (from next address field)

The microinstruction itself may look like this in binary

Table 2: Binary representation

Control Signals	Condition	Next Address
1010010010001000	00	11001001

Here:

- 1010010010001000 activates signals such as Memory Read, Load DR, etc.
- 00 indicates unconditional sequencing.
- 11001001 specifies the address of the next microinstruction.

In systems with conditional branching, the next address field may depend on status flags or external inputs. For example, if a condition bit is set (like zero or carry), the control unit may load a different address into CAR. This supports branching, looping, and subroutines at the microprogram level.

Control memory is often implemented using ROM for stability and reliability, especially in commercial CPUs. However, in academic or experimental systems, RAM-based control memory allows designers to modify the microcode, supporting flexibility and ease of experimentation.

In conclusion, the Control Address Register and Control Memory together form the heart of microprogram sequencing. The CAR determines which microinstruction is executed next, while control memory holds the actual logic needed to perform each CPU operation. Their interaction ensures the structured, efficient, and programmable flow of microoperations in a microprogrammed control unit.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
1	What is the main advantage of microprogrammed control over hardwired control? A) Faster execution B) Easier modification and flexibility C) Lower power consumption D) Larger control memory

2	What does the control address register (CAR) hold? A) The current instruction B) The next microinstruction address C) The ALU result D) The opcode
3	Which component stores the microinstructions in a microprogrammed control unit? A) RAM B) Control Memory C) ALU D) Register File
Fill in the Blank	
4	Microprogrammed control uses a sequence of _____ to implement control logic.
5	The _____ memory stores the microinstructions that define the control signals.
6	The process of determining the next microinstruction to execute is called _____.
True or False	
7	Microprogrammed control is more flexible than hardwired control.
8	The control address register is updated after each microinstruction is executed.
9	Microprogram sequencing is only used in RISC processors.
10	Control memory is volatile and loses its contents when power is off.

5. HARDWIRED CONTROL

Hardwired control units generate control signals via fixed combinational and sequential logic (gates, decoders, flip-flops), rather than reading microinstructions from memory. Inputs—such as the instruction opcode, timing pulses or cycle counter, and flag/status bits—are processed by dedicated FSM-style circuitry to produce the precise control signals required at each step of execution.

For example, in an ADD R1, R2 operation, the control logic might include conditions like:

If (Opcode=ADD) && (T=2) → Enable R1 output → Load into ALU input A

If (Opcode=ADD) && (T=3) → Enable R2 output → Load into ALU input B

If (Opcode=ADD) && (T=4) → ALU perform ADD → Store result in R1

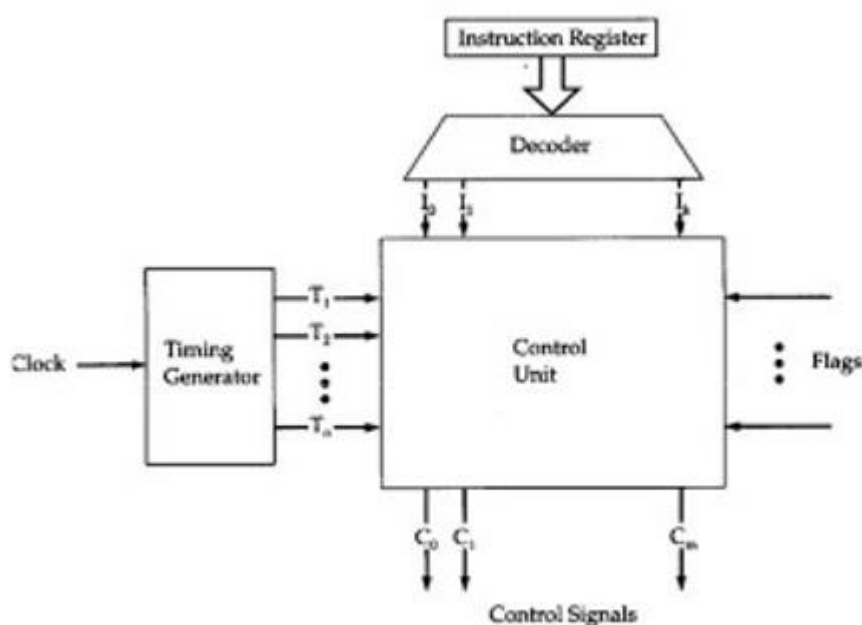


Fig 3 – Hardwired Control Unit Block Diagram

The image depicts a Hardwired Control Unit used in computer architecture to manage instruction execution. Here's a detailed explanation of each component shown in the figure:

Table 3: Description of each component

Component	Description
Instruction Register	Holds the instruction currently being executed. It serves as the input to the control logic.

Decoder	Decodes the instruction from the Instruction Register and translates it into control signals that guide the operation of the CPU.
Control Unit	The core of the system. It receives inputs from the Decoder, Timing Generator, and Flags, and generates control signals (C0, C1, ..., Cn) to orchestrate the execution of instructions.
Timing Generator	Produces timing signals (T0, T1, ..., Tn) that help synchronize the operations of the control unit and other components.
Flags	Status indicators that reflect the outcome of operations (e.g., zero result, carry, overflow). These are used by the Control Unit to make decisions during instruction execution.
Control Signals (C0, C1, ..., Cn)	Output signals from the Control Unit that control various parts of the CPU, such as registers, ALU operations, and data paths.

Pros:

- Speed: Control signals are generated directly by hardware, eliminating microinstruction fetch delays—ideal for high-performance or real-time systems.
- Compact design: Often smaller and faster than storing microcode.

Cons:

- Inflexibility: Modifying the instruction set or control logic requires redesigning hardware—making updates costly and time-consuming.
- Complexity: Hardwired control grows unwieldy for complex or evolving ISAs, complicating testing and maintenance.

Typically used in RISC architectures with regular, simple instruction sets, hardwired control balances raw speed against limited adaptability, making it suitable for stable environments demanding maximum performance.

5.1. Design Principles

Hardwired control units rely on digital logic (combinational and sequential circuits) to generate precise control signals based on the instruction opcode, timing steps, and status flags. Key design elements include:

1. Opcode decoding: The instruction decoder reads the opcode from the IR and passes it to control logic and timing circuits.

2. Timing control: A step counter or FSM divides execution into phases (T1, T2, T3, ...), synchronising control signals with clock cycles.
3. Modular blocks: Separate modules manage decoding, sequencing, and signal generation, improving clarity and testability.
4. Logic optimisation: Boolean simplification (often via Karnaugh maps) minimises gate count and latency.
5. Precise synchronisation: Control signals are perfectly timed to clock-generated states for reliable execution.

Control signals are grouped—register control, ALU functions, memory I/O—and driven by logic expressions combining opcode, timing, and flags. The result is a fast, deterministic control mechanism, ideal for simple, stable ISAs. However, its rigidity hinders updates—altering behaviour requires physical redesign.

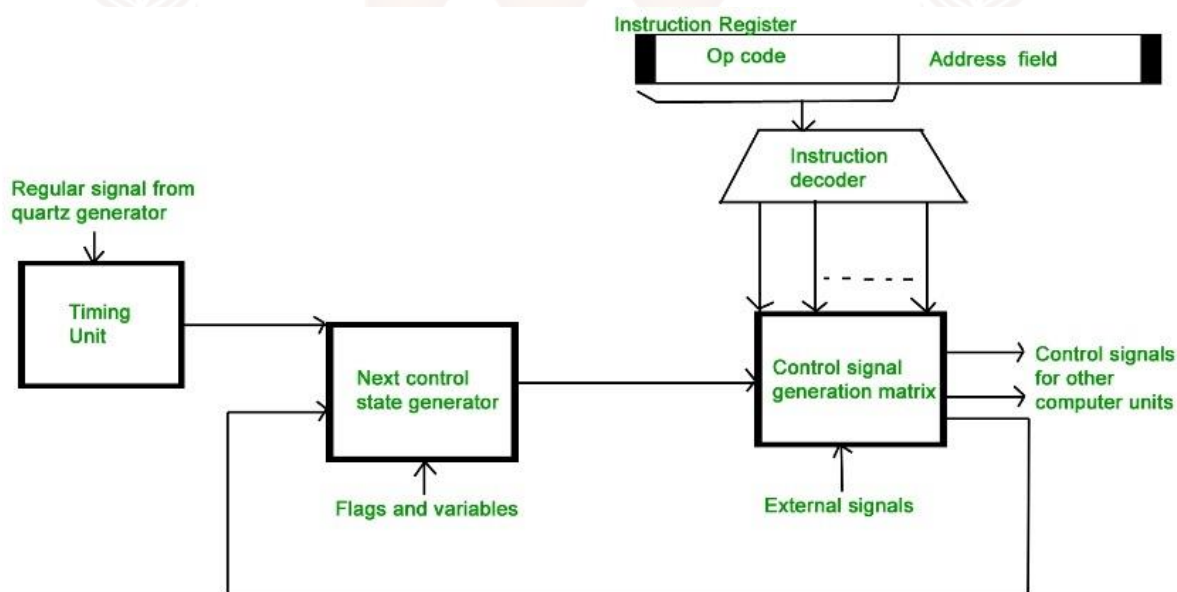


Fig 4 - Design Principles of Hardwired Control

The figure illustrates a hardwired control unit used in computer architecture, showing how various components interact to execute instructions efficiently. Here's a detailed explanation of each component:

Table 4: Explanation of each component

Component	Function
Instruction Register	Holds the current instruction being executed. It includes fields like the Op code and Address field, which are essential for decoding and execution.

Instruction Decoder	Decodes the instruction from the Instruction Register and translates it into signals that guide the control logic. These decoded signals are sent to the Control Signal Generation Matrix.
Timing Unit	Receives regular pulses from a quartz generator and produces timing signals (T0, T1, T2, ...) that synchronize the operations of the control unit and other components.
Next Control State Generator	Determines the next control state based on current flags and variables. It sends signals to both the Timing Unit and the Control Signal Generation Matrix to guide the next steps in instruction execution.
Control Signal Generation Matrix	The core logic block that generates control signals. It receives inputs from the Instruction Decoder, Next Control State Generator, and External Signals, and outputs control signals to various parts of the CPU.
Flags and Variables	Provide status information (e.g., zero result, carry, overflow) that helps the Next Control State Generator decide the next control state.
External Signals	Additional inputs from outside the control unit that influence the generation of control signals, allowing the system to respond to external events or conditions.

5.2. Comparison with Microprogrammed Control

The two primary approaches to control unit design in a CPU are hardwired control and microprogrammed control, and each has distinct characteristics that make it suitable for different use cases. Hardwired control uses fixed digital logic circuits to generate control signals directly in response to instruction opcodes, timing states, and status flags. It is known for its speed and efficiency, as control signals are produced instantaneously through logic gates without requiring memory access. In contrast, microprogrammed control stores control information in a special memory, known as control memory, and executes instructions by fetching and interpreting microinstructions sequentially. This approach introduces a small delay due to the memory access cycle but provides greater flexibility and modularity.

One of the most significant advantages of microprogrammed control is its ease of modification. Because the control logic is stored in memory, changes to the instruction set or execution behaviour can be made by simply updating the microcode—no hardware redesign is needed. This makes it ideal for systems that require frequent updates, complex instruction sets, or instruction emulation.

Hardwired control, on the other hand, is less adaptable; modifying its logic often requires significant changes to the physical circuitry, which can be time-consuming and costly.

In terms of performance, hardwired control is faster because it eliminates the need to fetch microinstructions, making it suitable for RISC (Reduced Instruction Set Computer) architectures where the number of instructions is limited and each instruction executes in a small number of fixed steps. Microprogrammed control, while slightly slower, excels in CISC (Complex Instruction Set Computer) architectures where instructions are more elaborate and require longer or variable-length execution sequences.

From a design perspective, hardwired control units are more complex to implement and debug, as they require careful planning of logic pathways and precise coordination of timing signals. Microprogrammed control units, in contrast, offer software-like structure, making them easier to write, organise, test, and maintain. Designers can use labels, branching, and subroutines in the microprogram, just like in conventional programming, which promotes modularity and clarity.

In summary, hardwired control offers speed and compactness, making it suitable for high-performance and simple instruction set environments, while microprogrammed control provides flexibility, scalability, and ease of maintenance, especially beneficial for complex or evolving CPU architectures. The choice between the two depends on factors like system complexity, performance requirements, development cost, and expected need for future updates.

5.3. Use Cases and Applications

Choosing between hardwired and microprogrammed control depends on design goals and ISA complexity. Hardwired control excels in speed and efficiency, as control signals are generated via fixed logic pathways. It's ideal for real-time and high-performance domains—especially in RISC processors with simple, fixed-length instructions. Practical examples include ARM Cortex M microcontrollers, many DSPs, and performance focused CPUs in embedded or mobile systems.

In contrast, microprogrammed control thrives in flexibility and maintainability. CISC architectures like Intel x86, legacy mainframes, emulators, and industrial automation systems benefit from this approach. Complex, variable-length instructions are easier to implement using microprograms, and firmware updates—such as Intel's microcode patches—can fix bugs post-fabrication. It also supports educational CPUs and research platforms, where its software-like microcode eases testing and ISA experimentation.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
1	What is the key characteristic of hardwired control? A) Uses microinstructions B) Uses fixed logic circuits C) Requires control memory D) Is slower than microprogrammed control
2	Which of the following is an advantage of hardwired control? A) Easier to modify B) More flexible C) Faster execution D) Requires more memory
3	Hardwired control is best suited for: A) Complex instruction sets B) Systems requiring frequent updates C) Simple and fast processors D) Embedded systems with large control memory
Fill in the Blank	
4	Hardwired control uses _____ circuits to generate control signals.
5	Unlike microprogrammed control, hardwired control does not use _____ memory.
6	Hardwired control is typically implemented using _____ logic components.
True or False	
7	Hardwired control is more flexible than microprogrammed control.
8	Hardwired control is generally faster than microprogrammed control.
9	Hardwired control is ideal for systems that require frequent changes in control logic.
10	Use cases for hardwired control include simple CPUs and embedded systems.

6. CONTROL MEMORY

Control memory is a dedicated, high-speed store in microprogrammed control units that houses microinstructions—binary control words that guide CPU internal operations every clock cycle. It differs from general-purpose memory by acting as the core repository for control logic. Each microinstruction typically includes:

- A control-word field that asserts control signals (e.g., ALU functions, register transfers, memory access).
- Sequencing fields such as next-address logic, conditional bits, or branch indicators.

When an opcode is decoded, the control unit retrieves the starting address of its microprogram in control memory. The microinstructions are then fetched sequentially—or selectively via branching—to orchestrate precisely timed micro-operations.

Control memory can be implemented using:

- ROM/ROS for fixed microcode in commercial CPUs (fast, cost-effective),
- EPROM, EEPROM, or RAM (writable control store) for systems needing runtime updates, experimentation, or patching.

Typical architectures include a 10-bit address bus (supporting 1,024 microinstructions) with word lengths of 32–64 bits for control and sequencing fields. Some designs incorporate a microinstruction decoder to compress encoded formats into explicit control signals.

The main advantage of control memory lies in its programmable flexibility: behavior can be changed by rewriting microcode, enabling updates, backward compatibility, or CPU emulation without hardware redesign. Though fetching microinstructions introduces minor delays compared to hardwired logic, the overall benefits in modularity, adaptability, and maintainability far outweigh the speed trade-off.

6.1. Structure of Control Memory

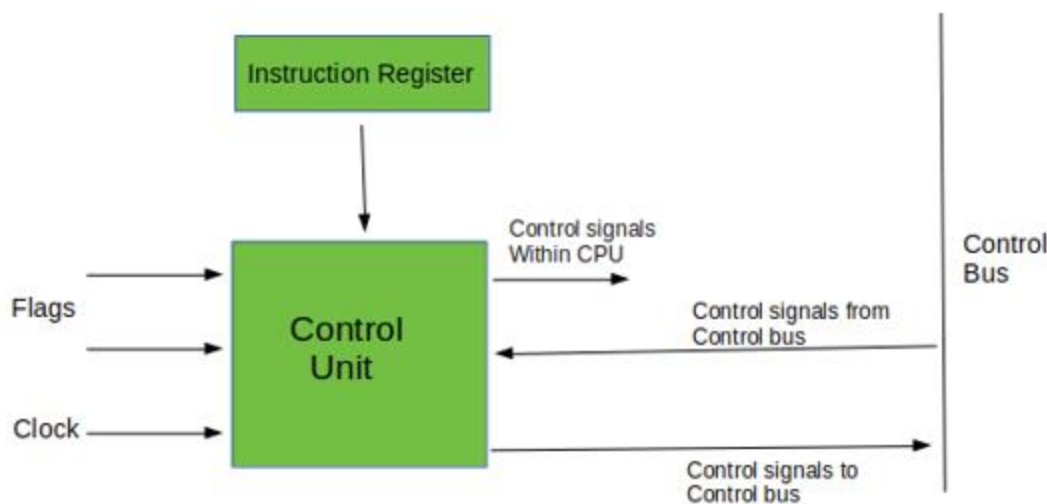


Fig 5 – Diagram of control unit

Control memory is a dedicated, high-speed ROM-like storage that holds microinstructions—fixed-width binary words used to direct CPU control flow. Each entry in control memory typically contains:

- Control field: Activates specific control signals to perform micro-operations (e.g., enabling ALU, register, or memory operations).
- Condition field: Holds flags or bits that determine whether branching should occur.
- Next-address field: Specifies the location of the following microinstruction, enabling both sequential steps and conditional/unconditional jumps.

For example, a 32-bit microinstruction might allocate bits 31–20 for control signals, 19–16 for condition coding, and 15–0 for sequencing logic, combining direct control of the datapath with address control for microprogram flow. Once a machine opcode is decoded, its initial microprogram address is loaded into the Control Address Register (CAR), which then fetches and loads each microinstruction into the Microinstruction Register (MIR). CAR is updated on each cycle—incremented for next microinstruction or overridden by the next-address field if branching is required.

Physically, control memory may use ROM, PROM, or writable storage (like EEPROM or RAM) for patchable designs or educational use. Designs may further optimise memory width using horizontal formats (one bit per control signal for direct control) or more compact vertical formats (code fields decoded into control signals).

This structure delivers efficient, structured sequencing: control words are fetched and interpreted in a single cycle, enabling precise control logic execution. While slightly slower than hardwired control

due to memory access, control memory's programmable nature allows easy updates to control logic without hardware redesign—ideal for flexible, extensible CPU architectures.

6.2. Control Memory Addressing

Control memory addressing refers to the process by which the control unit selects the correct location in control memory to fetch the next microinstruction. Each microinstruction is stored at a unique address, and the correct sequencing of microinstructions depends on accurate and efficient address generation. The address of the next microinstruction is held in a register called the Control Address Register (CAR). During each clock cycle, the CAR provides the address to control memory, allowing the corresponding microinstruction to be fetched and executed.

There are several ways the address in the CAR can be updated to ensure correct sequencing. The most basic method is sequential addressing, in which the CAR is incremented by one after each microinstruction, similar to how the Program Counter operates during normal instruction execution. This method is suitable for instructions that require a fixed, linear sequence of microoperations.

More advanced microprogramming systems support branching and conditional jumps by including a next-address field within each microinstruction. This field may contain either the actual address of the next microinstruction (direct addressing) or a symbolic label that maps to an address. The control unit uses this field to update the CAR directly, allowing for non-sequential execution, such as loops, conditional branches, and subroutine calls.

In systems that support conditional branching, the microinstruction also contains a condition field that determines whether the next address should be selected from the microinstruction or simply incremented. This condition could be based on internal flags like zero, carry, overflow, or specific external control signals. For example, if the zero flag is set after an ALU operation, the next address may jump to an error-handling microroutine; otherwise, execution continues sequentially.

In some architectures, control memory addressing begins by mapping the opcode of a machine instruction to the starting address of its corresponding microprogram. This mapping is often achieved through a decoder or lookup table that translates instruction opcodes into microprogram entry points in control memory. For instance, if the machine instruction ADD has an opcode of 0100, the mapping logic might associate it with control memory address 00101000. This address is loaded into the CAR to begin fetching the first microinstruction of the ADD microprogram.

A simplified example of address sequencing might look like this:

Instruction: SUB

Opcode: 0101

Mapped Microprogram Start: 00110000

→ $CAR \leftarrow 00110000$

→ Fetch and execute microinstruction from 00110000

→ Next address from microinstruction: 00110001

→ $CAR \leftarrow 00110001$

→ Repeat...

This mechanism allows for structured and flexible control flow, enabling microprogrammed CPUs to implement even complex instruction sets and behaviors with precise and predictable control memory access.

In summary, control memory addressing ensures that microinstructions are fetched in the correct order by maintaining and updating the Control Address Register. It supports linear sequencing, conditional branching, opcode-based entry points, and subroutine linking—all essential features for implementing reliable and adaptable microprogrammed control logic.

6.3. ROM vs RAM in Control Units

In a microprogrammed control unit, the choice between using ROM (Read-Only Memory) or RAM (Random Access Memory) for control memory has a significant impact on the design's flexibility, speed, cost, and upgradability. Both types of memory serve the same functional purpose—to store microinstructions—but they differ in behaviour and use cases.

ROM is the most commonly used memory type in commercial control units. It is non-volatile, meaning its contents remain intact even when the power is turned off. ROM is typically used to store permanent microprograms that define the control sequences for executing all supported machine-level instructions. Because the microcode is embedded during manufacturing or programming (in the case of PROM or EPROM), ROM-based control units are highly stable and reliable. ROM also tends to be faster and cheaper than RAM for fixed logic applications, making it ideal for CPUs where the instruction set is finalised and unlikely to change.

For example, in a typical microprogrammed processor, the control memory might store microinstructions for instructions like ADD, SUB, LOAD, and JUMP, each occupying a fixed location in ROM. These instructions are executed repeatedly with high performance and minimal latency.

On the other hand, RAM-based control memory is volatile and allows the contents to be modified at runtime or during system initialisation. This capability is extremely useful in development, research, or prototyping environments, where designers need the flexibility to test new instruction sets, patch bugs, or reconfigure control behaviour without physically modifying hardware. RAM also enables dynamic reconfiguration, where the control logic of the CPU can be adapted for different modes of operation or tasks.

For instance, an educational CPU may use RAM for control memory so you can upload new microcode to implement custom instructions or change the behaviour of existing ones. Similarly, in firmware-upgradable CPUs, the microcode may be stored in EEPROM or flash memory (a form of rewritable ROM) to allow manufacturers to distribute updates that improve performance or add features after deployment.

However, RAM-based control units are generally slower and more expensive than ROM-based systems when used for permanent control storage. They also require additional hardware for initialisation, such as a loader that transfers microcode into RAM on startup.

In summary, ROM is ideal for stable, high-performance control units in production environments where the microprogram does not change, while RAM provides flexibility and reusability for systems that need to be updated, reprogrammed, or customised. The choice between ROM and RAM depends on the trade-offs between speed, cost, flexibility, and use case requirements.

Table 5: Comparison table for ROM vs RAM in Control Memory, clearly outlining key differences relevant to microprogrammed control units:

Aspect	ROM (Read-Only Memory)	RAM (Random Access Memory)
Volatility	Non-volatile (retains data without power)	Volatile (data lost when power is off)
Modifiability	Fixed once programmed; not easily modified	Easily programmable and reprogrammable
Usage Scenario	Used in final, commercial CPUs with fixed microcode	Used in development, prototyping, or reconfigurable CPUs

Flexibility	Low – not suitable for changing instruction sets	High – supports updates, patches, and experimentation
Speed	Generally faster access for fixed logic	Slightly slower due to read/write circuitry
Cost	Cheaper for mass-produced, permanent logic	More expensive due to reprogrammable nature
Typical Applications	Commercial microprocessors, stable instruction sets	Educational CPUs, firmware-upgradable systems, research CPUs
Initialisation Required	No – contents permanently embedded	Yes – microprogram must be loaded at startup
Reusability	One-time use (or limited write cycles for EPROM/Flash)	Can be reused and updated repeatedly
Example Usage	Intel x86 instruction microcode in embedded ROM	VLIW processors or academic control units with user-defined ISA

SELF-ASSESSMENT QUESTIONS - 5

Multiple Choice Questions	
1	What is the primary function of control memory in a control unit? A) Store user data B) Store microinstructions C) Perform arithmetic operations D) Manage cache memory
2	Which of the following is used to access a specific location in control memory? A) Program Counter B) Control Address Register C) Instruction Register D) Data Bus
3	Which type of memory is typically used for permanent storage of microinstructions? A) RAM B) ROM C) Cache D) Register

Fill in the Blank	
4	The _____ memory holds the microinstructions that define control signals for the CPU.
5	The _____ register contains the address of the next microinstruction to be fetched.
6	RAM in control units allows for _____ microinstruction updates, while ROM does not.
True or False	
7	Control memory is used to store microinstructions in a microprogrammed control unit.
8	ROM is volatile and loses its contents when power is off.
9	RAM-based control memory allows for dynamic changes to control logic.
10	Control memory addressing is managed by the Control Address Register.



7. SUMMARY

- In this unit, we learn how computers handle and change binary data using logic and shift micro-operations. Logic micro-operations work on each bit in a register using basic logical functions like AND, OR, XOR, and NOT. These operations help in tasks such as turning bits on or off, checking conditions, and securing data through encryption. For example, AND can be used to clear specific bits, while XOR is useful in error detection.
- These operations are very important in the CPU's Arithmetic Logic Unit (ALU), which performs calculations and decisions. We also learn about advanced bit manipulation techniques like selective-set, selective-clear, masking, and inserting new bits into registers. Shift micro-operations move bits left or right and are used for fast multiplication, division, and organising data.
- There are three types: logical shifts (which fill empty spots with zeros), arithmetic shifts (which keep the sign bit for signed numbers), and circular shifts (which rotate bits around without losing any). These shifts are useful in encryption and data transmission.
- The unit also introduces the Arithmetic Logic Shift Unit (ALSU), a special part of the CPU that combines arithmetic, logic, and shift operations in one place. It uses a smart design to perform many tasks quickly and efficiently in just one clock cycle. The ALSU helps the CPU work faster and handle different types of data operations. Overall, this unit helps us understand how computers process data at the bit level, making them powerful and efficient.

8. GLOSSARY

Arithmetic Logic Shift Unit (ALU)	- A part of the CPU that can perform arithmetic, logic, and shift operations all in one place, helping the computer process data quickly.
Arithmetic Shift	- A way to move bits left or right in signed numbers. Left shifts multiply the number by 2, and right shifts divide it by 2 while keeping the sign.
Bit Manipulation	- Changing specific bits in a register using logic operations like AND, OR, XOR, and NOT to control or modify data.
Circular Shift	- Also called rotation. It moves bits around in a loop—bits that go out on one side come back in on the other side. Useful in encryption.
Logical Shift	- Moves bits left or right and fills empty spots with zeros. It's used for unsigned numbers like regular counting.
Logic Micro-Operations	- Basic bit-level operations like AND, OR, XOR, and NOT help the computer make decisions and manage data.
Masking	- A technique using AND to keep only the bits you want and clear the rest. It's like putting a filter on data.
Register	- A small, fast memory space inside the CPU where binary data is stored and used for operations.
Selective-Clear	- Clears (sets to 0) bits in a register where another register has 1s. Done using $A \leftarrow A \wedge \neg B$.
Selective-Complement	- Flips (toggles) bits in a register where another register has 1s. Done using $A \leftarrow A \oplus B$.
Selective-Set	- Sets bits to 1 in a register where another register has 1s. Done using $A \leftarrow A \vee B$.
Sign Extension	- Keeps the sign of a number during an arithmetic right shift by copying the sign bit into the empty space.
Truth Table	- A chart that shows all possible input combinations and their corresponding outputs for a logic operation.

9. TERMINAL QUESTIONS

1. What is a control word in a CPU, and how does its structure allow multiple microoperations to be executed in a single clock cycle?
2. Explain how microoperations and control signals are related in a control unit. Provide an example of how control signals orchestrate a register transfer operation.
3. What is control signal encoding, and how does it optimise the size and efficiency of control words in microprogrammed control units?
4. Differentiate between horizontal and vertical microinstruction formats. What are the trade-offs in terms of performance and memory usage?
5. Describe the key fields in a typical microinstruction format and explain how these fields contribute to control signal generation and sequencing.
6. What role does the Control Address Register (CAR) and Control Data Register (CDR) play in the execution of microinstructions?
7. Explain the basic concept of microprogrammed control. What are the main advantages and limitations compared to hardwired control?
8. How does microprogram sequencing work? Describe the role of the sequencer and the mechanisms used to determine the next microinstruction.
9. In what ways can branching be implemented in microprogrammed control units? How are conditional and unconditional branches handled?
10. Compare the design principles of hardwired control and microprogrammed control. In what situations is hardwired control more suitable?
11. Describe the structure and function of the hardwired control unit. How are timing signals and opcodes used to generate control outputs?
12. List real-world use cases for both hardwired and microprogrammed control units. Why are RISC and CISC architectures associated with these approaches respectively?
13. What is control memory and how does its structure support microinstruction storage and sequencing in a CPU?
14. How does control memory addressing enable sequencing of microinstructions? Provide examples of sequential and conditional address updates.
15. Compare ROM and RAM as control memory options in control units. What are their advantages and limitations in commercial and educational CPU designs?

10. ANSWERS

Self-Assessment Questions

Self-Assessment Questions - 1

Multiple Choice Questions (MCQs)

1. C) The set of microoperations to be executed
2. B) Control signals
3. B) To reduce hardware complexity

Fill in the Blanks

4. control word
5. components
6. control lines

True or False

7. True
8. False
9. False
10. True

Self-Assessment Questions - 2

Multiple Choice Questions (MCQs)

1. B) A binary instruction that specifies microoperations
2. B) Horizontal and Vertical
3. C) Directly specified

Fill in the Blanks

4. vertical
5. operation
6. control

True or False

7. False

- 8. True
- 9. True
- 10. True

Self-Assessment Questions - 3

Multiple Choice Questions (MCQs)

- 1. B) Easier modification and flexibility
- 2. B) The next microinstruction address
- 3. B) Control Memory

Fill in the Blanks

- 4. microinstructions
- 5. control
- 6. microprogram sequencing

True or False

- 7. True
- 8. True
- 9. False
- 10. False

Self-Assessment Questions - 4

Multiple Choice Questions (MCQs)

- 1. B) Uses fixed logic circuits
- 2. C) Faster execution
- 3. C) Simple and fast processors

Fill in the Blanks

- 4. combinational
- 5. control
- 6. sequential

True or False

- 7. False
- 8. True

- 9. False
- 10. True

Self-Assessment Questions - 5

Multiple Choice Questions (MCQs)

- 1. B) Store microinstructions
- 2. B) Control Address Register
- 3. B) ROM

Fill in the Blanks

- 4. control
- 5. Control Address
- 6. dynamic

True or False

- 7. True
- 8. False
- 9. True
- 10. True

Terminal Questions Answers

Answer 1: A control word is a binary string in which each bit represents a specific control signal. These bits are grouped into fields targeting hardware units like the ALU, memory, and registers. Because each bit can activate a specific microoperation, multiple microoperations can occur in parallel during a single clock cycle. Refer to Section 2.1 for more details.

Answer 2: Microoperations are basic CPU actions (e.g., register transfer, ALU operation), each activated by control signals. For a transfer like $R1 \leftarrow R2 + R3$, control signals must: enable R2 and R3 outputs, select ALU add operation, and enable R1 to load the result. Refer to Section 2.2 for more details.

Answer 3: Encoding reduces the number of bits in a control word by grouping signals into fields and assigning codes to operations. Decoders later translate these codes into control lines. This saves memory space and simplifies instruction formats. Refer to Section 2.3 for more details.

Answer 4: Horizontal format uses one bit per control signal—large word size but supports parallelism. Vertical format uses encoded fields—compact but allows only one operation per group. Horizontal is faster; vertical saves memory. Refer to Section 3.1 for more details.

Answer 5: Fields include: control field (activates microoperations), condition field (checks flags), and next address field (guides sequencing). This structure supports logic flow, conditional branching, and efficient control signal execution. Refer to Section 3.2 for more details.

Answer 6: CAR holds the address of the next microinstruction. The CDR holds the fetched microinstruction. Together, they enable the fetch-decode-execute cycle of microinstructions. Refer to Sections 3.3 and 4.3 for more details.

Answer 7: Microprogrammed control stores control signals as microinstructions in control memory. It's flexible, easier to modify/debug, and ideal for complex ISAs. However, it's slower than hardwired control due to fetch delays. Refer to Section 4.1 for more details.

Answer 8: Microprogram sequencing uses the Control Address Register (CAR) to fetch microinstructions. The sequencer updates the CAR via incrementing, branching, or condition-checking using status flags. Refer to Section 4.2 for more details.

Answer 9: Branching uses next-address fields and condition codes. Unconditional branches load a fixed address into CAR; conditional branches do so only if a flag (like zero or carry) is set. Refer to Sections 4.2 and 4.3 for more details.

Answer 10: Hardwired control uses fixed logic (decoders, FSMs) and is faster but inflexible. It's best for simple, stable ISAs like RISC. Microprogrammed control is more flexible but slower. Refer to Sections 5.1 and 5.2 for more details.

Answer 11: Hardwired control uses logic gates, decoders, and timing counters. Opcodes and timing steps trigger control signals directly without needing memory fetches. Refer to Section 5.1 for more details.

Answer 12: Hardwired: RISC processors, DSPs – where speed matters.

Microprogrammed: CISC CPUs, x86, emulators – where flexibility and updatability matter. Refer to Section 5.3 for more details.

Answer 13: Control memory stores microinstructions with control, condition, and next-address fields. These enable sequential or branched execution of control logic. Refer to Section 6.1 for more details.

Answer 14: The CAR points to the next instruction. It can increment (sequential), or load a new address (conditional). E.g., if the zero flag is set, jump to the error routine. Refer to Section 6.2 for more details.

Answer 15: ROM is fast, non-volatile, and used in commercial CPUs. RAM is flexible, reprogrammable, and ideal for development or educational CPUs. RAM needs initialisation at startup. Refer to Section 6.3 for more details.



11. REFERENCES

BOOKS

- Computer Architecture: A Quantitative Approach, John L. Hennessy, David A. Patterson, 6th Edition, 2017
- Computer Organisation and Architecture: Designing for Performance, William Stallings, 11th Edition, 2018
- Computer System Architecture, M. Morris Mano, 3rd Edition, 2006
- Computer Organisation and Design: The Hardware/Software Interface, David A. Patterson, John L. Hennessy, 5th Edition, 2013
- Digital Design and Computer Architecture, Sarah Harris, David Harris, 3rd Edition, 2021

E-REFERENCES

- Logical and Shift Micro-operations Computer Organisation and Architecture, <https://care4you.in/logical-and-shift-micro-operations/>
- Logic and Shift Micro Operations, <https://www.scribd.com/document/394026693/U-I-Logic-and-Shift-micro-operations>

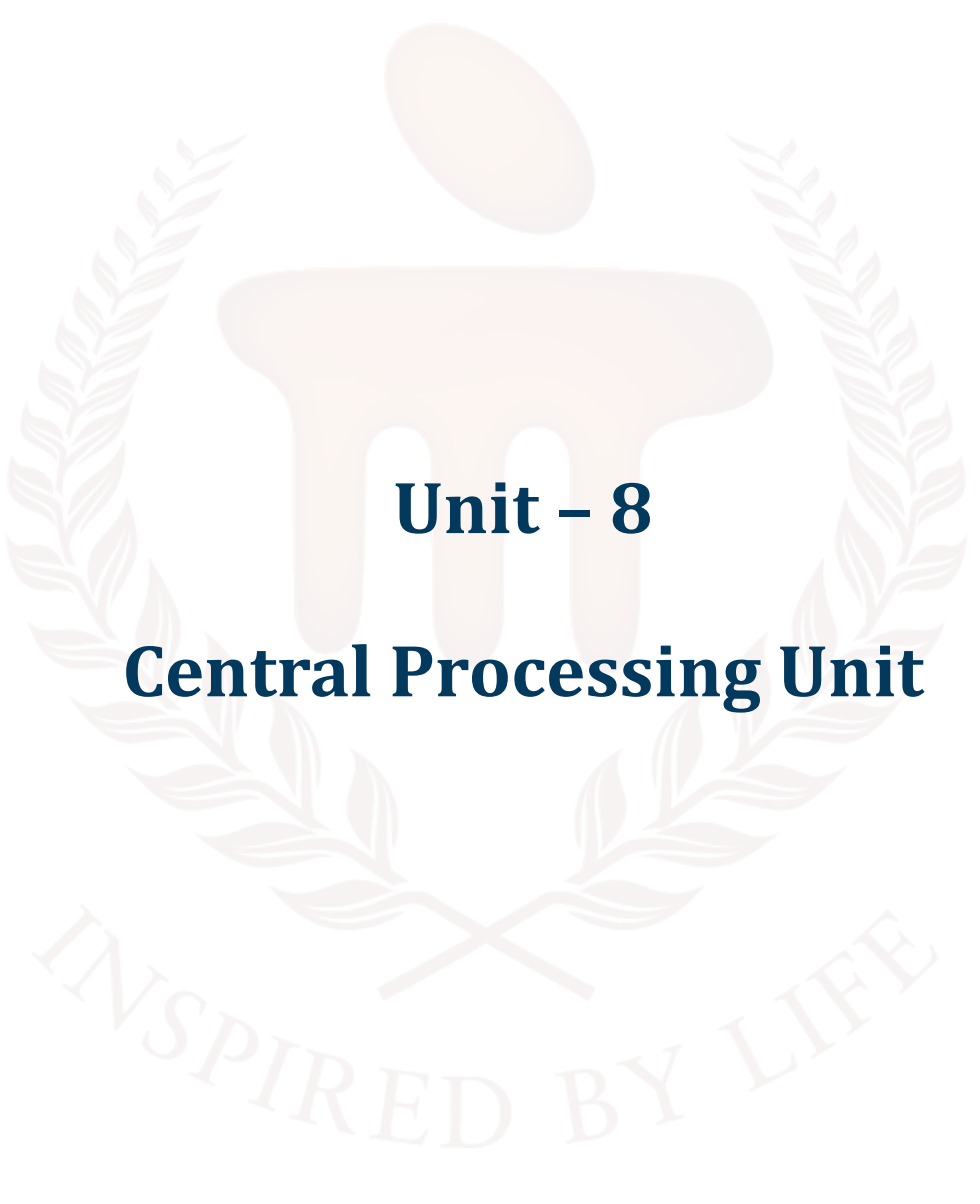
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 8

Central Processing Unit

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	General Register Organisation	-	-	6 - 14
2.1	Register Set and Data Flow	-	-	
2.2	Register Transfer Operations	-	-	
2.3	Control Logic and Timing	-	1	
3	Stack Organisation	-	-	15 - 18
3.1	Stack-Based CPU Architecture	-	-	
3.2	Stack Operations and Implementation	-	-	
3.3	Stack Pointer and Status Flags	-	2	
4	Instruction Formats	-	-	19 - 23
4.1	Types of Instruction Formats	-	-	
4.2	Field Definitions and Layout	-	-	
4.3	Format Selection and CPU Design Impact	-	3	
5	Addressing Modes	-	-	24 - 29
5.1	Overview of Addressing Techniques	-	-	
5.2	Immediate, Direct, Indirect Addressing	-	-	
5.3	Indexed, Register, and Relative Addressing	-	-	
5.4	Addressing Mode Applications and Examples	-	4	
6	Summary	-	-	30
7	Glossary	-	-	31 - 32
8	Terminal Questions	-	-	33
9	Answers	-	-	34 - 38
10	References	-	-	39

1. INTRODUCTION

In the previous unit, we explored the inner workings of the control unit within a CPU, focusing on how control signals are generated and managed to execute instructions. We examined the concept of microinstructions and microprogramming, the structure and role of control memory, and the differences between hardwired and microprogrammed control units. This gave us a foundational understanding of how CPUs orchestrate internal operations at the micro-level, using control words and sequencing logic to manage data flow and execution.

Building on that foundation, this unit shifts our focus to the organisation of the Central Processing Unit (CPU) itself. In this unit, we will delve into the structural components that make up the CPU and how they interact to perform computation. We begin with the General Register Organisation, where we study how multiple registers are interconnected and used for efficient data manipulation and instruction execution. Next, we explore the Stack Organisation, a powerful method for managing temporary data, function calls, and expression evaluation using Last-In-First-Out (LIFO) principles.

We then move on to Instruction Formats, where we analyse how machine instructions are structured, including the layout of operation codes, operands, and addressing information. Finally, we study Addressing Modes, which define how operands are accessed and manipulated during instruction execution. These modes—such as immediate, direct, indirect, and indexed—play a crucial role in determining the flexibility and efficiency of a CPU's instruction set.

By the end of this unit, you will have a comprehensive understanding of how the CPU is organised internally, how instructions are formatted and executed, and how different addressing techniques influence performance and design. This unit bridges the gap between control logic and execution architecture, preparing you for deeper insights into processor design and optimisation.

1.1. Objectives

After studying this unit, you should be able to:

- Explain the structure and function of the General Register Organisation within a CPU.
- Discuss the role of stack organisation in CPU design.
- Differentiate between various instruction formats.
- Identify and compare different addressing modes
- Illustrate how data flows between registers, memory, and the ALU in a register-based CPU architecture.
- Explain how CPU organisation affects instruction cycle efficiency, memory access patterns, and overall system performance.



2. GENERAL REGISTER ORGANISATION

The General Register Organisation refers to the internal arrangement and functioning of the set of registers in a computer's central processing unit (CPU). These registers are small, fast storage units that play a vital role in the execution of instructions by temporarily holding data, instructions, and addresses.

Purpose of General Registers

General-purpose registers (GPRs) are used to:

- Store operands for arithmetic and logic operations
- Hold intermediate results during computation
- Facilitate data transfer between memory and the CPU
- Support addressing and control functions in instruction execution

These registers allow for high-speed data access and reduce reliance on slower main memory, significantly improving processing efficiency.

Components of General Register Organisation

A typical general register organisation includes the following components:

1. Register Set:

A collection of general-purpose registers (e.g., R0, R1, R2, ..., Rn), where each register can hold a word of data. These registers are used by the CPU during instruction execution.

2. Arithmetic Logic Unit (ALU):

The ALU performs operations such as addition, subtraction, logical AND, OR, NOT, etc., using data from the registers.

3. Control Unit:

Generates control signals to select specific registers, initiate ALU operations, and manage data flow.

4. Multiplexers (MUX):

Used to select which registers feed data to the ALU inputs, based on control signals.

5. Buses:

Internal pathways that carry data between registers, the ALU, and memory. Typical designs include:

- Input Bus A
- Input Bus B
- Output Bus

Register Selection and Operation

To execute an operation like ADD R1, R2, R3 ($R1 = R2 + R3$), the following steps occur:

1. Selection: Control unit selects R2 and R3 as inputs using multiplexers.
2. Execution: ALU performs the addition.
3. Storage: Result is stored in R1, enabled by the control unit.

The operation is completed in a single or a few clock cycles, depending on the architecture.

Advantages of General Register Organisation

- Speed: Registers are much faster than main memory.
- Flexibility: General-purpose use enables efficient programming and compilation.
- Reduced Memory Access: Intermediate results are stored in registers, lowering memory traffic.
- Parallelism Support: Multiple registers enable simultaneous operations, improving performance.

2.1. Register Set and Data Flow

The Register Set in a CPU consists of a group of high-speed storage elements used to hold data temporarily during instruction execution. These registers play a critical role in enabling fast access to operands and results, significantly enhancing the performance of a processor.

Register Set

A register set typically includes multiple general-purpose registers (GPRs), and may also contain special-purpose registers depending on the architecture. General-purpose registers can be used interchangeably for storing data, addresses, or intermediate results.

- Number of Registers: Varies by processor (e.g., 8 in early Intel CPUs, 32 in MIPS, 16 in ARM).
- Register Width: Depends on architecture—typically 32-bit or 64-bit.
- Naming Conventions: Registers may be named numerically (R0, R1, R2...) or symbolically (EAX, EBX in x86; x0–x31 in RISC-V).

Registers are directly connected to the processor's internal data paths and ALU, ensuring minimal delay in accessing or modifying data.

Data Flow in Register Operations

Data flow refers to the movement of data between the registers, the Arithmetic Logic Unit (ALU), and memory or I/O. In a general register organisation, the data flow is managed by the control unit, which activates specific components to perform operations such as loading, storing, or computing.

Key stages of data flow include:

1. Reading Operands:
 - The control unit selects two source registers via multiplexers.
 - Their contents are routed to the ALU input buses (usually labeled A and B).
2. ALU Operation:
 - The ALU performs the desired arithmetic or logic operation on the inputs.
 - The result is passed to an output bus.
3. Writing Results:
 - The result from the ALU is written back into a destination register.
 - This register is enabled via a write control signal.
4. Interaction with Memory (when needed):
 - Load: Data from memory is read into a register.
 - Store: Data from a register is written into memory.

Example: Data Flow for an ADD Instruction

Instruction:

ADD R1, R2, R3 ; $R1 \leftarrow R2 + R3$

Data Flow Steps:

- R2 and R3 are selected and sent to the ALU via input buses A and B.
- The ALU computes the sum of R2 and R3.
- The result is sent via the output bus to R1.
- R1 is enabled to store the new value.

Control Signals in Data Flow

To coordinate these steps, the control unit generates specific control signals such as:

- Register Select Lines: Choose which registers to read/write.
- Load/Store Signals: Enable data transfer.
- ALU Control Lines: Specify the operation (add, subtract, etc.).
- Clock Pulses: Synchronise data movement across components.

Importance of Efficient Data Flow

Efficient data flow ensures:

- Minimal delay in executing instructions.
- Effective utilisation of registers and the ALU.
- High throughput, especially in pipelined or superscalar processors.

2.2. Register Transfer Operations

Register Transfer Operations are fundamental actions that move data between registers and other components within the CPU. These operations form the basis of instruction execution, enabling the CPU to process data, perform arithmetic/logical functions, and control the flow of a program.

Definition

A register transfer operation involves copying data from one register to another, or between a register and another hardware unit, such as memory or the Arithmetic Logic Unit (ALU). These operations are performed under the control of the CPU's control unit, which generates appropriate signals to facilitate the transfer.

Notation

Register transfers are typically represented using a symbolic notation:

$R1 \leftarrow R2$

This means that the contents of register R2 are transferred to register R1.

Types of Register Transfer Operations

1. Data Transfer Between Registers

- Example: $R1 \leftarrow R2$

The contents of register R2 are copied to register R1.

2. Data Transfer Between Register and ALU

- Registers provide operands to the ALU.

- The result from the ALU is stored back in a register.
 - Example: $R3 \leftarrow R1 + R2$
3. Data Transfer Between Register and Memory
- Load Operation:
 $R1 \leftarrow M[1000]$
 - Data from memory address 1000 is loaded into R1.
 - Store Operation:
 $M[1000] \leftarrow R1$
Contents of R1 are stored into memory address 1000.
4. Data Transfer with Immediate Values
- Example: $R1 \leftarrow \#25$
The constant value 25 is loaded directly into R1.

Control Signals for Register Transfers

To perform register transfer operations, the control unit issues specific signals:

- Load Enable (LE): Enables a register to accept data.
- Output Enable (OE): Enables a register to place data onto a bus.
- Select Lines: Determine which register is involved in the transfer.
- Clock Pulse: Synchronises the data transfer with the system clock.

Example: Executing a Register Transfer

Assume the instruction is:

$R3 \leftarrow R1 + R2$

Steps involved:

1. R1 and R2 are selected using control signals.
2. Their values are routed to the ALU input buses.
3. The ALU performs the addition.
4. The result is placed on the output bus.
5. R3's load signal is enabled to capture the result.

Importance of Register Transfer Operations

- Foundation of Instruction Execution: Most CPU instructions involve register transfers either explicitly or implicitly.

- **High-Speed Execution:** Transfers occur within the CPU, enabling much faster data movement compared to memory access.
- **Simplifies Control Logic:** Standardising register transfers simplifies the design of control units in hardware.

2.3. Control Logic and Timing

The Control Logic and Timing subsystem of a CPU is responsible for orchestrating the execution of instructions by generating the appropriate control signals at the right time. It ensures that register transfers, ALU operations, and memory accesses occur in the correct sequence and are properly synchronised with the system clock.

Control Logic

The control logic is the part of the CPU that interprets instructions and generates signals to coordinate various operations such as register transfers, ALU computations, and data movement.

There are two primary methods for implementing control logic:

1. **Hardwired Control**
 - Uses fixed combinational logic circuits (e.g., gates, decoders, flip-flops).
 - Fast and efficient but difficult to modify or upgrade.
 - Best suited for RISC architectures with a simple instruction set.
2. **Microprogrammed Control**
 - Uses a sequence of microinstructions stored in control memory.
 - More flexible and easier to design and maintain.
 - Suitable for CISC architectures where instructions are more complex.

Functions of Control Logic:

- Select source and destination registers.
- Enable ALU operations.
- Manage read/write operations to memory.
- Control input/output device interactions.
- Synchronise data flow through buses.

Timing

Timing in a CPU refers to how operations are synchronised with the system clock to ensure orderly and predictable execution of instructions.

Each register transfer or operation is executed within a defined clock cycle. The system clock produces a sequence of pulses, and control signals are activated in synchronisation with these pulses.

Key Timing Concepts:

1. Clock Cycle:
 - A single oscillation of the clock signal.
 - Represents the smallest unit of time in synchronous systems.
2. Instruction Cycle:
 - The sequence of steps required to fetch, decode, and execute an instruction.
 - Usually consists of multiple clock cycles.
3. Control Timing Signals:
 - Generated by the control unit to activate specific components at the right moment (e.g., load a register, start an ALU operation).
 - Often derived from a timing generator or counter that steps through predefined states.

Example: Timing in a Register Transfer Operation

Consider the instruction:

$R3 \leftarrow R1 + R2$

The control logic generates the following sequence of signals, aligned with clock pulses:

Clock Cycle	Operation	Control Signals Activated
T1	Select R1 and R2	Enable R1 and R2 outputs via MUX
T2	Perform ALU addition	Activate ALU with control signal for ADD
T3	Load result into R3	Enable R3 for loading (Load Enable)

Each operation must be completed within its designated clock cycle to ensure correctness and stability of data.

Importance of Control Logic and Timing

- Ensures Coordination: All components work together in harmony.
- Improves Efficiency: Operations are executed without unnecessary delays.

- Prevents Conflicts: Avoids data corruption by managing access to buses and registers.
- Enables Pipelining: Precise timing allows overlapping execution stages in modern CPUs.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the primary function of general-purpose registers in a CPU? A) Store permanent data B) Perform I/O operations C) Temporarily hold operands and results during instruction execution D) Control memory access
2	In a general register organisation, which component performs arithmetic and logical operations? A) Control Unit B) Stack Pointer C) Arithmetic Logic Unit (ALU) D) Instruction Register
3	Which of the following best describes a register transfer operation? A) Moving data from memory to disk B) Transferring data between two registers C) Executing a conditional branch D) Fetching an instruction from memory
Fill in the Blank	
4	The _____ is responsible for generating control signals that coordinate register selection, ALU operations, and data movement.
5	In a register transfer operation, the _____ signal enables a register to accept data from the output bus.
6	The movement of data between registers and the ALU is managed through internal pathways called _____.
True or False	
7	The control logic in a CPU ensures that operations are synchronised with the system clock.
8	Register transfer operations can only occur between registers and memory.

9	In a general register organisation, multiple registers allow for parallel execution of operations.
10	The ALU in a CPU can only perform addition and subtraction operations.



3. STACK ORGANISATION

A stack is a memory structure used in CPUs that works on the Last-In, First-Out (LIFO) principle—meaning the last value pushed onto the stack is the first to be popped off. It's mainly used for handling function calls, return addresses, local variables, and temporary data during program execution.

The stack is accessed using a Stack Pointer (SP), which points to the top of the stack. In many systems (like x86), the stack grows downward—from high memory addresses to low ones. When a PUSH operation is executed (e.g., PUSH R1), the SP is decremented, and the data from R1 is stored at the new address. A POP operation (e.g., POP R2) does the reverse—data is read from the current SP location into R2, and then SP is incremented.

For example, calling a function like `sum()` causes the CPU to push the return address onto the stack. Inside the function, local variables or parameters may also be pushed. When the function finishes, the return address is popped, allowing the program to continue correctly.

Stacks also support recursion. Calling `factorial(3)` results in pushing calls for `factorial(3)`, `factorial(2)`, and `factorial(1)`, each with its own data. The stack then unwinds as each call returns.

In stack-based CPUs, operations like ADD automatically pop the top two values, compute the result, and push it back—no need for specifying registers.

Overall, stack organisation helps manage temporary data and control flow efficiently, especially in nested or recursive programming.

3.1. Stack-Based CPU Architecture

In a stack-based CPU architecture, all operations are performed using a stack rather than named registers. Instructions implicitly operate on the top elements of the stack, making the instruction set simpler and more compact.

Unlike register-based CPUs where operands are explicitly mentioned (e.g., `ADD R1, R2, R3`), stack-based CPUs use instructions like PUSH, POP, and ADD without specifying registers. The top values on the stack are used as operands.

For example, to calculate the result of $5 + 3$ in a stack-based CPU:

PUSH 5 ; Push first operand

PUSH 3 ; Push second operand

ADD ; Pop 5 and 3, add them, push result (8)

After the ADD, the stack contains only the result: 8.

These architectures are commonly used in virtual machines such as the Java Virtual Machine (JVM) and Forth language systems. They are also useful for evaluating arithmetic expressions, especially those written in postfix (Reverse Polish) notation, where operations are written after operands, like:

$(5 + 3) * 2 \rightarrow 5\ 3\ +\ 2\ *$

Stack-based CPUs evaluate this efficiently by pushing operands and applying operations in order.

Because operands are implied, instructions are shorter, which reduces memory usage. However, more stack operations (like multiple PUSHes and POPs) may be needed compared to register-based architectures.

In summary, stack-based CPU architectures simplify instruction formats by relying on an internal stack for all operations. While they may not be as common in general-purpose processors, they are widely used in specialised systems like interpreters, calculators, and virtual machines.

3.2. Stack Operations and Implementation

A stack works on the Last-In, First-Out (LIFO) principle, using two main operations: PUSH (to add data) and POP (to remove data). The Stack Pointer (SP) keeps track of the top of the stack, which usually grows downward in memory (from higher to lower addresses).

For a PUSH operation, the SP is decremented and the data is stored at the new address. For example:

SP = 0x1000

PUSH 10 $\rightarrow$ SP = 0x0FFC, M[0x0FFC] = 10

For a POP operation, the value is read from the current SP location, and the SP is incremented:

POP $\rightarrow$ data = M[0x0FFC], SP = 0x1000

Stacks are heavily used in function calls. When a function is called, the return address and parameters are pushed onto the stack. When the function returns, they are popped to restore the previous state. This makes stacks ideal for nested and recursive functions.

For example, calling factorial(3) will push calls for factorial(3), factorial(2), and factorial(1). As the function returns, each result is popped off and used.

In stack-based CPUs, instructions like ADD automatically pop the top two values, add them, and push the result—no need to name registers.

3.3. Stack Pointer and Status Flags

The Stack Pointer (SP) is a special-purpose register that always points to the top of the stack. It plays a key role in PUSH and POP operations. When data is pushed onto the stack, the SP is updated (usually decremented in a descending stack) and the data is stored at the new top. When data is popped, it is read from the SP location, and the SP is updated accordingly.

For example, if SP = 0x1000, executing PUSH 5 in a descending stack would change SP to 0x0FFC and store 5 at that address. A following POP would retrieve 5 and return SP to 0x1000.

In addition to SP, CPUs use status flags to indicate the result of operations. These flags are bits set or cleared in a special status register after arithmetic or logical operations. Common status flags include:

- Zero (Z): Set if the result of an operation is zero.
- Carry (C): Set if there is a carry out of the most significant bit.
- Sign (S): Set if the result is negative (in signed operations).
- Overflow (O or V): Set if a signed overflow occurs.

While the SP is not directly involved in setting status flags, the data pushed or popped might affect subsequent operations that do set them. For example, if a POP instruction loads a value into a register and an arithmetic operation is performed on it, that operation may change flags like Zero or Carry.

In summary, the Stack Pointer controls access to the stack, ensuring correct data flow during subroutine calls and returns, while status flags help monitor and react to the outcomes of CPU operations.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
1	What principle does a stack follow in CPU architecture? A) First-In, First-Out (FIFO) B) Last-In, First-Out (LIFO) C) Random Access D) Direct Mapping

2	Which register is used to track the top of the stack? A) Instruction Register B) Program Counter C) Stack Pointer D) Status Register
3	What happens during a PUSH operation in a descending stack? A) Stack Pointer is incremented B) Stack Pointer remains unchanged C) Stack Pointer is decremented D) Data is removed from the stack
Fill in the Blank	
4	A _____ operation removes the top element from the stack and updates the stack pointer.
5	In stack-based CPUs, arithmetic operations like ADD use _____ operands from the top of the stack.
6	The _____ flag is set when the result of an operation is zero.
True or False	
7	In a stack-based CPU, operands must be explicitly specified in each instruction.
8	The stack pointer increases during a PUSH operation in a descending stack.
9	Stack-based architectures are well-suited for evaluating expressions written in postfix notation.
10	Status flags like Zero and Carry are never affected by stack operations.

4. INSTRUCTION FORMATS

An instruction format defines the structure or layout of bits in a machine-level instruction. It tells the CPU how to interpret the instruction—including what operation to perform, which operands to use, and where to store the result. The format must balance simplicity, flexibility, and efficiency.

A typical instruction includes fields such as:

- **Opcode:** Specifies the operation (e.g., ADD, SUB, LOAD).
- **Operand(s):** Specifies the data or registers involved.
- **Address/Immediate:** For memory addresses or constant values.

Depending on the CPU architecture, instructions may have fixed length (e.g., 32 bits in MIPS) or variable length (e.g., x86). Common instruction formats include:

- **Zero-address (Stack-based):** Operands are implicitly on the stack.
Example: ADD (adds top two stack values)
- **One-address:** Uses an accumulator and one memory/register operand.
Example: ADD A $\rightarrow$ ACC = ACC + A
- **Two-address:** One operand is both source and destination.
Example: ADD R1, R2 $\rightarrow$ R1 = R1 + R2
- **Three-address:** Separate destination and two source operands.
Example: ADD R1, R2, R3 $\rightarrow$ R1 = R2 + R3

Instruction format affects how the CPU decodes and executes

4.1. Types of Instruction Formats

Instruction formats refer to the different ways in which machine instructions are structured. The format determines how many operands are used, how they are specified, and how much space is allocated for each field in the instruction. The main types of instruction formats are based on the number of addresses used in the instruction.

1. Zero-Address Format:

Used in stack-based architectures. Operands are implicitly taken from the stack, and results are pushed back.

Example:

PUSH 5

PUSH 3

ADD ; result = 5 + 3 (pushed back to stack)

2. One-Address Format:

Uses a single explicit operand, typically with an implicit accumulator register.

Example:

LOAD A ; $ACC \leftarrow M[A]$

ADD B ; $ACC \leftarrow ACC + M[B]$

STORE C ; $M[C] \leftarrow ACC$

3. Two-Address Format:

Specifies two operands—one acts as both a source and a destination.

Example:

ADD R1, R2 ; $R1 \leftarrow R1 + R2$

4. Three-Address Format:

Includes separate fields for two source operands and one destination. Offers flexibility but requires more bits.

Example:

ADD R1, R2, R3 ; $R1 \leftarrow R2 + R3$

Each format has trade-offs: zero- and one-address formats simplify hardware and reduce instruction size, while two- and three-address formats offer faster execution and fewer instructions in complex programs.

4.2. Field Definitions and Layout

In machine language, each instruction is made up of fields that specify what the instruction does and how it should be executed. The way these fields are arranged is known as the instruction layout or format. Understanding these fields is key to understanding how CPUs decode and execute instructions.

A typical instruction consists of the following fields:

- **Opcode (Operation Code):** Specifies the operation to perform (e.g., ADD, SUB, LOAD).
- **Address/Operand Fields:** Specify the location of operands—can be a register, a memory address, or an immediate value.
- **Mode Field (optional):** Indicates the addressing mode used (e.g., direct, indirect, immediate).

- Register Fields: Specify which registers are involved in the operation.
- Immediate Field (optional): Contains a constant value to be used directly in the operation.
- Displacement/Offset Field (optional): Used in indexed or base-relative addressing to compute memory addresses.

Example: MIPS instruction format (R-type)

[opcode (6 bits) | rs (5 bits) | rt (5 bits) | rd (5 bits) | shamt (5 bits) | funct (6 bits)]

Here:

- opcode specifies the instruction type.
- rs and rt are source registers.
- rd is the destination register.
- shamt is the shift amount (used in shift instructions).
- funct gives the specific ALU operation.

Example: Instruction

ADD R1, R2, R3

In binary, the fields would encode:

- opcode for ADD,
- rs = R2, rt = R3, rd = R1.

The layout ensures the CPU knows how to interpret and execute the instruction correctly.

4.3. Format Selection and CPU Design Impact

The selection of instruction format plays a critical role in CPU design, directly affecting performance, hardware complexity, and code efficiency. The instruction format defines how many bits are used for the opcode, registers, addresses, and other fields, and how instructions are structured in memory.

A short, fixed-length format (e.g., 32-bit instructions in RISC architectures like MIPS) simplifies instruction decoding, allows fast pipelining, and leads to uniform execution cycles. However, it limits the number of operands or addressing modes that can be specified in one instruction.

On the other hand, variable-length formats (common in CISC architectures like x86) offer more flexibility, supporting more complex instructions and addressing modes. This can reduce the total

number of instructions in a program, but makes the instruction decoding logic more complex and can slow down pipeline performance.

Format selection also affects memory usage and compiler design. For instance, a 3-address format can perform more work per instruction but uses more bits, increasing program size. A 1-address format (e.g., accumulator-based) reduces instruction size but may require more instructions to perform the same task.

In summary, the choice of instruction format involves a trade-off: fixed, simple formats support faster and more predictable CPU behavior, while complex, variable formats support richer instructions but require more sophisticated hardware. This decision shapes everything from pipeline depth and ALU complexity to instruction fetch speed and power consumption.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
1	Which instruction format uses implicit operands from the stack? A) One-address format B) Zero-address format C) Two-address format D) Three-address format
2	In a typical instruction layout, what does the opcode field represent? A) The memory address of the operand B) The register to store the result C) The operation to be performed D) The instruction length
3	Which format allows specifying two source operands and one destination operand? A) One-address format B) Two-address format C) Three-address format D) Zero-address format
Fill in the Blank	
4	The _____ field in an instruction specifies the operation to be performed by the CPU.
5	In a MIPS R-type instruction format, the _____ field determines the specific ALU operation.

6	Instruction formats affect CPU design by influencing decoding complexity, memory usage, and _____ performance.
True or False	
7	In a one-address format, the accumulator is used as an implicit operand.
8	Fixed-length instruction formats simplify decoding and support faster pipelining.
9	Variable-length instruction formats are easier to decode and execute than fixed-length formats.
10	The choice of instruction format has no impact on compiler design or instruction fetch speed.



5. ADDRESSING MODES

Addressing modes define how the operand of an instruction is specified and accessed during execution. They provide flexibility in programming by allowing data to be accessed in various ways—directly from memory, through registers, using offsets, or even as immediate values.

The choice of addressing mode affects how the CPU interprets the instruction, accesses operands, and performs operations. It also impacts the efficiency and power of the instruction set.

Each addressing mode has its advantages: immediate and register modes are fast, while indirect and indexed modes offer flexibility for handling arrays, pointers, and dynamic memory.

5.1. Overview of Addressing Techniques

Addressing techniques refer to the various methods a CPU uses to locate the operands (data) required by an instruction. These techniques determine how and where the processor retrieves data—whether from memory, registers, or within the instruction itself.

Each addressing technique has its own advantages in terms of speed, flexibility, and instruction size. Choosing the right addressing mode can optimise program performance and reduce memory access time.

Here's a quick overview of the most commonly used addressing techniques:

- **Immediate Addressing:**

The operand is part of the instruction itself. Fast and simple.

Example: `MOV R1, #5` → Load the constant value 5 into R1.

- **Register Addressing:**

Operand is in a register. Very fast since no memory access is needed.

Example: `ADD R1, R2` → Add contents of R2 to R1.

- **Direct Addressing:**

The instruction contains the actual memory address of the operand.

Example: `LOAD R1, 3000` → Load value from memory address 3000 into R1.

- **Indirect Addressing:**

Instruction points to a memory location that holds the address of the operand.

Example: `LOAD R1, (3000)` → If $M[3000] = 5000$, load data from address 5000.

- **Register Indirect Addressing:**

A register holds the memory address of the operand.

Example: `MOV R1, (R2)` → Load value from address in R2 into R1.

- **Indexed Addressing:**

Combines a base address (from a register) and an offset to calculate the operand's address.

Example: `LOAD R1, 10(R2)` → Access memory at $R2 + 10$.

- **Relative Addressing:**

Address is determined relative to the current Program Counter (PC), commonly used in branch instructions.

Example: `JMP +4` → Jump to instruction 4 steps ahead.

In summary, addressing techniques are essential for providing access to data in various locations. They allow instructions to be short and flexible while still supporting a wide range of data access patterns such as constants, variables, arrays, and function calls.

5.2. Immediate, Direct, Indirect Addressing

In computer architecture, Immediate, Direct, and Indirect addressing are three fundamental techniques used to specify the location of operands during instruction execution. Each method offers a different way for the CPU to access data, balancing between speed, flexibility, and complexity.

Immediate Addressing is the simplest form, where the operand is part of the instruction itself. No memory access is needed because the value is embedded directly in the instruction. This makes it very fast.

Example: `MOV R1, #10`

Here, the constant value 10 is moved directly into register R1.

Direct Addressing provides the actual memory address of the operand within the instruction. The CPU fetches the operand by reading from that specific memory location.

Example: `LOAD R1, 3000`

This means load into R1 the value found at memory address 3000.

Indirect Addressing adds a level of indirection: the instruction contains an address that points to another memory location, which holds the actual operand. This is useful for accessing dynamic data like arrays or pointers.

Example: LOAD R1, (3000)

Here, memory location 3000 holds the address 5000, and the value at address 5000 is loaded into R1.

In summary, Immediate addressing is fastest but limited to constant values, Direct addressing is simple but fixed, and Indirect addressing is powerful for dynamic memory access at the cost of one extra memory fetch.

5.3. Indexed, Register, and Relative Addressing

These three addressing modes offer flexible ways for the CPU to access data stored in memory or registers, often used for arrays, loops, and control flow.

Indexed Addressing is commonly used to access elements of arrays. It combines a base address (usually in a register) with a fixed offset or index to calculate the effective address of the operand.

Example: LOAD R1, 20(R2)

Here, the CPU adds 20 to the value in R2 to get the memory address, then loads the data from that location into R1. If R2 = 1000, the CPU loads from address 1020.

Register Addressing accesses operands that are stored directly in CPU registers. This is very fast since no memory access is required.

Example: ADD R1, R2

Adds the value in R2 to R1. Both operands are in registers, so execution is efficient.

Relative Addressing is mainly used in control transfer instructions like jumps or branches. The target address is calculated by adding a signed offset to the current Program Counter (PC).

Example: JMP +6

This means jump to the instruction located 6 positions ahead of the current instruction, relative to the PC.

In summary, indexed addressing is ideal for structured data like arrays, register addressing is the fastest for temporary data, and relative addressing is crucial for control flow and branching in programs.

5.4. Addressing Mode Applications and Examples

Different addressing modes are used based on the needs of a program—such as accessing constants,

handling arrays, performing pointer operations, or managing control flow. Understanding where each mode fits helps in writing efficient and optimised code.

Immediate Addressing is useful when working with fixed constant values.

Example: MOV R1, #10

This loads the constant value 10 directly into register R1—no memory access required. It's ideal for initialising variables or counters.

Direct Addressing is typically used to access global or static variables stored at known memory locations.

Example: LOAD R2, 3000

Loads the value from memory address 3000 into R2.

Indirect Addressing is suited for pointer-based access and dynamic memory.

Example: LOAD R1, (5000)

If $\text{memory}[5000] = 8000$, the value at address 8000 is loaded into R1. This is useful for pointer dereferencing.

Register Addressing is efficient for temporary or frequently used values.

Example: ADD R1, R2

Performs addition using only register values—no memory access—making it fast for arithmetic operations.

Indexed Addressing is ideal for array access and loop operations.

Example: LOAD R1, 4(R3)

Accesses the element at $R3 + 4$, allowing easy iteration through an array.

Relative Addressing is used in conditional or unconditional branching.

Example: JMP +6

Jumps to a location 6 instructions ahead—helpful in loops, if-else statements, and function calls.

In summary, each addressing mode has specific use cases:

- Immediate: constants
- Direct: known memory locations

- Indirect: pointers
- Register: fast operations
- Indexed: arrays
- Relative: branching

Choosing the right addressing mode improves code performance, memory efficiency, and instruction clarity.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
1	Which addressing mode includes the operand directly within the instruction? A) Direct Addressing B) Indirect Addressing C) Immediate Addressing D) Indexed Addressing
2	In register indirect addressing, the operand is accessed by: A) Using a constant value B) Using a register that holds the memory address C) Using a fixed memory address D) Using the program counter
3	Which addressing mode is most suitable for accessing array elements? A) Immediate Addressing B) Indexed Addressing C) Relative Addressing D) Register Addressing
Fill in the Blank	
4	In _____ addressing, the instruction contains the actual memory address of the operand.
5	_____ addressing mode is commonly used in branch and jump instructions, where the target address is relative to the current program counter.
6	_____ addressing mode is ideal for pointer-based operations, where the instruction refers to a memory location that contains the address of the operand.
True or False	
7	Immediate addressing requires an extra memory access to fetch the operand.

8	Register addressing is faster than memory-based addressing because it avoids memory access.
9	Indexed addressing combines a base register and an offset to calculate the effective address.
10	All addressing modes are equally efficient for all types of operations.



6. SUMMARY

In this unit, we explored the internal organisation of the Central Processing Unit (CPU), focusing on how its structural components work together to execute instructions efficiently. The unit began with an in-depth look at the General Register Organisation, where we learned how multiple general-purpose registers (GPRs) are used to store operands, intermediate results, and final outputs. We examined the data flow between registers, the ALU, and memory, and understood how register transfer operations are coordinated using control signals and timing mechanisms.

Next, we studied the Stack Organisation, a powerful method for managing temporary data and function calls using a Last-In, First-Out (LIFO) structure. We learned how stack-based CPUs operate using implicit operands, and how the stack pointer (SP) and status flags play a crucial role in managing stack operations like PUSH and POP. The stack's role in recursion, subroutine handling, and expression evaluation was also highlighted.

The unit then covered Instruction Formats, explaining how machine instructions are structured into fields such as opcode, operand, and addressing mode. We explored various formats—zero-address, one-address, two-address, and three-address—and analysed how format selection impacts CPU design, instruction decoding, and execution efficiency.

Finally, we examined Addressing Modes, which define how operands are accessed during instruction execution. We studied immediate, direct, indirect, register, indexed, and relative addressing, and discussed their applications in accessing constants, arrays, pointers, and control flow instructions. Each mode was illustrated with examples to show its practical use in programming and system design.

By the end of this unit, we understood how CPU organisation affects instruction execution, memory access, and overall system performance. The concepts covered serve as a foundation for advanced topics in computer architecture, including pipelining, parallelism, and processor optimisation.

7. GLOSSARY

General Register Organisation	- A CPU design that uses multiple general-purpose registers to store operands and intermediate results, enabling faster and more flexible instruction execution.
Register Transfer Operation	- The movement of data from one register to another or between a register and memory is controlled by specific control signals.
Control Logic	- The part of the CPU that generates signals to coordinate data movement, ALU operations, and instruction execution based on the current instruction and clock cycle.
Stack	- A Last-In, First-Out (LIFO) data structure is used in CPUs to manage temporary data, function calls, and return addresses.
Stack Pointer (SP)	- A special-purpose register that holds the address of the top of the stack and is updated during PUSH and POP operations.
PUSH Operation	- Adds a data item to the top of the stack and updates the stack pointer accordingly.
POP Operation	- Removes the top data item from the stack and adjusts the stack pointer to point to the new top.
Instruction Format	- The layout of bits in a machine instruction typically includes fields like opcode, operand(s), and addressing mode.
Opcode	- A field in an instruction that specifies the operation to be performed, such as ADD, SUB, or LOAD.
Addressing Mode	- A method used to determine how the operand of an instruction is accessed, such as immediate, direct, indirect, or indexed.
Immediate Addressing	- An addressing mode where the operand is specified directly within the instruction.
Direct Addressing	- An addressing mode where the instruction contains the actual memory address of the operand.

Indirect Addressing	-	An addressing mode where the instruction points to a memory location that contains the address of the operand.
Indexed Addressing	-	An addressing mode that calculates the effective address by adding a constant offset to the contents of a register.
Relative Addressing	-	An addressing mode where the operand's address is determined by adding an offset to the current program counter (PC), often used in branch instructions.
Register Addressing	-	An addressing mode where operands are located in CPU registers, allowing for fast access and execution.

8. TERMINAL QUESTIONS

1. What is the purpose of general-purpose registers in a CPU, and how do they improve instruction execution efficiency?
2. Explain the data flow in a general register organisation. How do buses and multiplexers facilitate register operations?
3. Describe the steps involved in executing a register transfer operation such as $R3 \leftarrow R1 + R2$.
4. What is the role of control logic in coordinating register transfers and ALU operations?
5. Compare hardwired control and microprogrammed control in terms of flexibility, speed, and design complexity.
6. Define stack organisation and explain how it supports subroutine calls and recursion in CPU architecture.
7. What is the function of the stack pointer (SP), and how is it updated during PUSH and POP operations?
8. Illustrate how a stack-based CPU evaluates the expression $(A + B) * (C - D)$ using postfix notation.
9. What are the advantages and limitations of stack-based CPU architectures compared to register-based designs?
10. Describe the structure of a typical machine instruction. What are the key fields included in an instruction format?
11. Differentiate between zero-address, one-address, two-address, and three-address instruction formats with examples.
12. How does the choice of instruction format affect CPU design, instruction decoding, and execution performance?
13. Define addressing mode. Why are multiple addressing modes important in CPU instruction sets?
14. Explain the difference between immediate, direct, and indirect addressing modes. Provide examples of each.
15. How does indexed addressing work, and why is it useful for accessing array elements in memory?

9. ANSWERS

Self-Assessment Questions

Self-Assessment Questions - 1

Multiple Choice Questions (MCQs)

1. C) Temporarily hold operands and results during instruction execution
2. C) Arithmetic Logic Unit (ALU)
3. B) Transferring data between two registers

Fill in the Blanks

4. Control Unit
5. Load Enable (LE)
6. Buses

True or False

7. True
8. False
9. True
10. False

Self-Assessment Questions - 2

Multiple Choice Questions (MCQs)

1. B) Last-In, First-Out (LIFO)
2. C) Stack Pointer
3. C) Stack Pointer is decremented

Fill in the Blanks

4. POP
5. implicit
6. Zero

True or False

7. False

- 8. False
- 9. True
- 10. False

Self-Assessment Questions - 3

Multiple Choice Questions (MCQs)

- 1. B) Zero-address format
- 2. C) The operation to be performed
- 3. C) Three-address format

Fill in the Blanks

- 4. Opcode
- 5. Funct
- 6. Execution

True or False

- 7. True
- 8. True
- 9. False
- 10. False

Self-Assessment Questions - 4

Multiple Choice Questions (MCQs)

- 1. C) Immediate Addressing
- 2. B) Using a register that holds the memory address
- 3. B) Indexed Addressing

Fill in the Blanks

- 4. Direct
- 5. Relative
- 6. Indirect

True or False

- 7. False
- 8. True

9. True

10. False

Terminal Questions Answers

Answer 1: General-purpose registers temporarily store operands, intermediate results, and outputs during instruction execution. They reduce memory access, improve speed, and support flexible data manipulation.

(Refer to section 2.1 - Register Set and Data Flow)

Answer 2: In general register organisation, data flows from source registers to the ALU via input buses (A and B), and the result flows back to a destination register via an output bus. Multiplexers and control signals manage register selection and data routing.

(Refer to section 2.2 - Register Transfer Operations)

Answer 3: For $R3 \leftarrow R1 + R2$:

R1 and R2 are selected as inputs.

Their values are sent to the ALU.

The ALU performs addition.

The result is stored in R3.

(Refer to section 2.2 - Register Transfer Operations)

Answer 4: Control logic generates signals to select registers, initiate ALU operations, and manage timing. It ensures that each step of instruction execution occurs in the correct sequence and synchronises with the system clock.

(Refer to section 2.3 - Control Logic and Timing)

Answer 5: Hardwired control uses fixed logic circuits for fast execution, but is difficult to modify. Microprogrammed control uses microinstructions stored in memory, offering flexibility and easier updates, though slightly slower.

(Refer to section 2.3 - Control Logic and Timing)

Answer 6: A stack is a LIFO structure used to manage temporary data, function calls, and return addresses. It simplifies subroutine handling and supports recursion by storing return points and local variables.

(Refer to section 3.1 - Stack-Based CPU Architecture)

Answer 7: The Stack Pointer (SP) holds the address of the top of the stack. It is decremented during a PUSH (to add data) and incremented during a POP (to remove data), adjusting the stack's top position.

(Refer to section 3.3 - Stack Pointer and Status Flags)

Answer 8: Postfix evaluation of $(A + B) * (C - D)$:

PUSH A, PUSH B, ADD $\rightarrow$ result1

PUSH C, PUSH D, SUB $\rightarrow$ result2

MUL $\rightarrow$ result1 * result2 is pushed

(Refer to section 3.1 - Stack-Based CPU Architecture)

Answer 9: Advantages: Simplified instruction set, efficient for recursion, compact code.

Limitations: Limited random access, more PUSH/POP operations, harder to optimise.

(Refer to section 3.1 - Stack-Based CPU Architecture)

Answer 10: A machine instruction typically includes:

Opcode: operation to perform

Operand fields: registers or memory addresses

Addressing mode: how to access operands

(Refer to section 4.1 - Types of Instruction Formats)

Answer 11: Zero-address: ADD (operands on stack)

One-address: ADD A (uses accumulator)

Two-address: ADD R1, R2

Three-address: ADD R1, R2, R3

(Refer to section 4.1 - Types of Instruction Formats)

Answer 12: Instruction format affects decoding complexity, execution speed, and memory usage. Fixed formats simplify hardware; variable formats offer flexibility but increase decoding time.

(Refer to section 4.3 - Format Selection and CPU Design Impact)

Answer 13: Addressing modes define how operands are accessed. They allow instructions to work with constants, memory, registers, or computed addresses, enhancing flexibility and efficiency.

(Refer to section 5.1 - Overview of Addressing Techniques)

Answer 14: Immediate: Operand is in the instruction (e.g., MOV R1, #10)

Direct: Address is in the instruction (e.g., LOAD R1, 5000)

Indirect: Address points to another address (e.g., LOAD R1, (5000))

(Refer to section 5.2 - Immediate, Direct, Indirect Addressing)

Answer 15: Indexed addressing adds a constant offset to a base register to compute the effective address. It's ideal for accessing array elements (e.g., LOAD R1, 4(R2) accesses R2 + 4).

(Refer to section 5.3 - Indexed, Register, and Relative Addressing)

10. REFERENCES

BOOKS

1. Computer Architecture: A Quantitative Approach, John L. Hennessy, David A. Patterson, 6th Edition, 2017
2. Computer Organisation and Architecture: Designing for Performance, William Stallings, 11th Edition, 2018
3. Computer System Architecture, M. Morris Mano, 3rd Edition, 2006
4. Computer Organisation and Design: The Hardware/Software Interface, David A. Patterson, John L. Hennessy, 5th Edition, 2013
5. Digital Design and Computer Architecture, Sarah Harris, David Harris, 3rd Edition, 2021

E-REFERENCES

1. Logical and Shift Micro-operations Computer Organisation and Architecture, <https://care4you.in/logical-and-shift-micro-operations/>
2. Logic and Shift Micro Operations, <https://www.scribd.com/document/394026693/U-I-Logic-and-Shift-micro-operations>

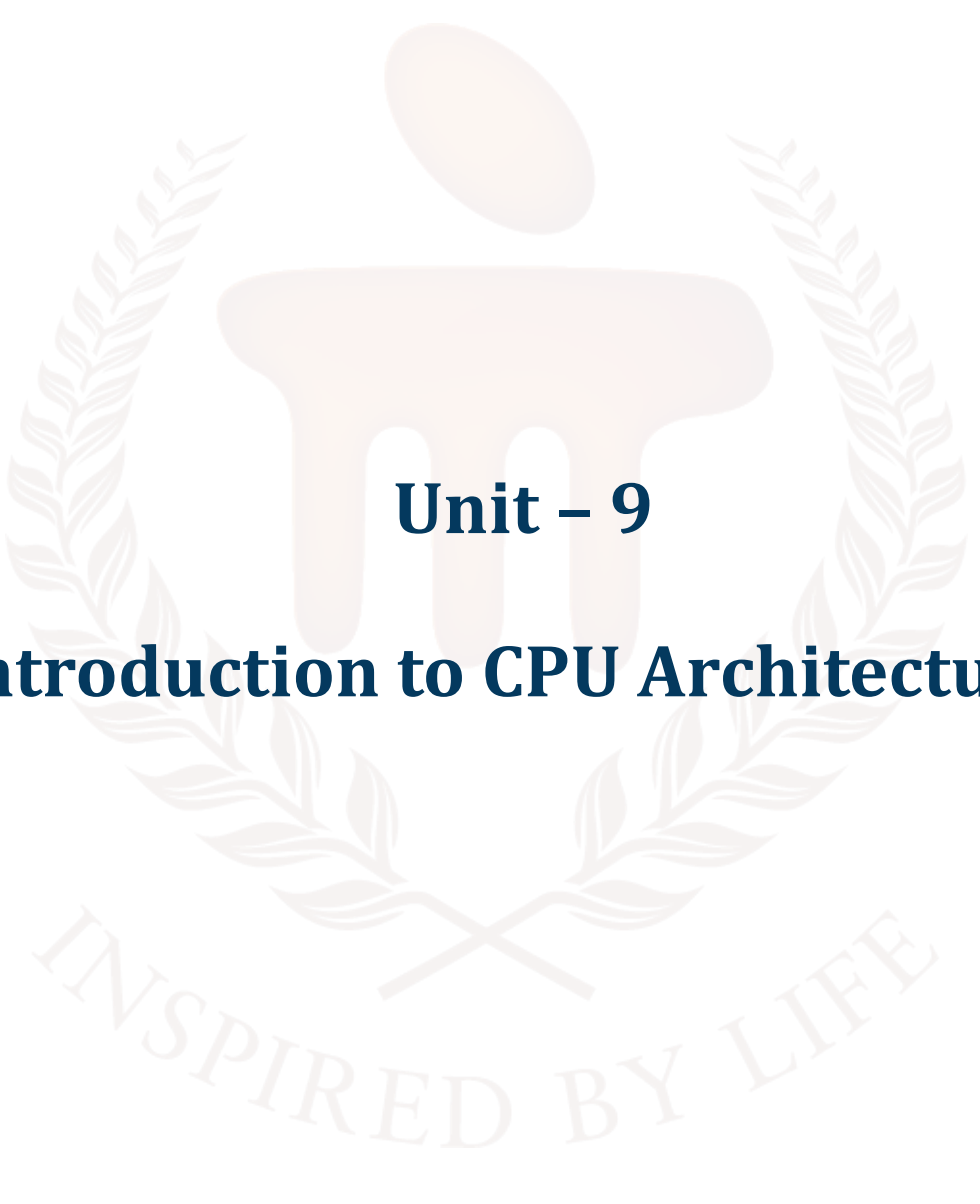
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 9

Introduction to CPU Architectures

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Superscalar Architecture	-	-	6 - 10
2.1	Concept and Design Principles	-	-	
2.2	Instruction-Level Parallelism	-	-	
2.3	Pipeline and Execution Units	-	1	
3	VLIW Architecture	-	-	11 - 16
3.1	VLIW Fundamentals	-	-	
3.2	Compiler Role in VLIW	-	-	
3.3	Advantages and Limitations	-	2	
4	SIMD Architecture	-	-	17 - 21
4.1	SIMD Basics	-	-	
4.2	Data Parallelism	-	-	
4.3	SIMD in Modern Processors	-	3	
5	Comparative Analysis	-	-	22 - 26
5.1	Superscalar vs VLIW	-	-	
5.2	VLIW vs SIMD	-	-	
5.3	Use Cases and Performance	-	4	
6	Summary	-	-	27
7	Glossary	-	-	28 - 29
8	Terminal Questions	-	-	30
9	Answers	-	-	31 - 34
10	References	-	-	35

1. INTRODUCTION

In the previous unit, we explored the internal organisation of the Central Processing Unit (CPU), focusing on how its structural components—such as general-purpose registers, stacks, instruction formats, and addressing modes—work together to execute instructions efficiently. We examined the data flow within the CPU, the role of control logic and timing, and how different instruction formats and addressing techniques influence performance and design. This foundational knowledge gave us a clear understanding of how instructions are processed, how data is managed within a CPU, and how architectural decisions impact computational efficiency and flexibility.

Building upon that foundation, this unit shifts our focus to the architectural paradigms that define how modern CPUs achieve high performance in increasingly complex computing environments. As software demands grow—ranging from real-time data processing to artificial intelligence and multimedia applications—CPU architectures must evolve to deliver greater speed, efficiency, and parallelism.

In this unit, we will delve into three prominent architectural models: Superscalar, VLIW (Very Long Instruction Word), and SIMD (Single Instruction, Multiple Data). Each of these architectures introduces unique strategies for enhancing instruction throughput and exploiting parallelism at different levels. Superscalar architectures improve performance by dynamically issuing multiple instructions per clock cycle, leveraging sophisticated hardware mechanisms like out-of-order execution and branch prediction. VLIW architectures, in contrast, shift the complexity to the compiler, which statically schedules multiple operations into a single long instruction word, simplifying hardware design while maintaining parallel execution. SIMD architectures focus on data-level parallelism, enabling the simultaneous execution of the same operation across multiple data elements—an approach particularly effective in scientific computing, image processing, and machine learning.

Throughout this unit, we will analyse the design principles, operational mechanisms, and real-world applications of these architectures. You will gain insights into how instruction scheduling, execution pipelines, and parallel data processing are implemented in modern processors. Additionally, we will compare these architectures in terms of hardware complexity, compiler dependency, scalability, and suitability for various computational tasks.

By the end of this unit, you will be equipped to critically evaluate the trade-offs involved in each architectural approach, understand their respective strengths and limitations, and appreciate their significance in the design of high-performance computing systems.

1.1. Objectives

After studying this unit, you should be able to:

- Explain the fundamental principles behind Superscalar, VLIW, and SIMD architectures
- Discuss how Superscalar processors achieve instruction-level parallelism.
- Analyse the role of the compiler in VLIW architectures.
- Illustrate how SIMD architectures perform data-level parallel operations.
- Compare and contrast Superscalar, VLIW, and SIMD architectures.



2. SUPERSCALAR ARCHITECTURE

Superscalar architecture is a type of CPU design that allows a processor to execute multiple instructions simultaneously during a single clock cycle. Unlike scalar processors, which fetch and execute one instruction at a time, superscalar processors fetch, decode, and issue two or more instructions in parallel, improving overall performance and instruction throughput.

This architecture uses multiple execution units such as ALUs, FPU, and load/store units, which operate in parallel. The instruction pipeline is widened so that more than one instruction can pass through each stage simultaneously. For this to work efficiently, the CPU must include advanced instruction scheduling, dependency checking, and register renaming to prevent data hazards and ensure correct execution order.

For example, if a superscalar CPU has two ALUs, it can execute two independent arithmetic instructions like `ADD R1, R2, R3` and `SUB R4, R5, R6` at the same time, provided there are no data dependencies.

Modern CPUs (e.g., Intel Core, AMD Ryzen) are typically superscalar, often capable of issuing 2 to 6 instructions per cycle. However, the actual performance gain depends on how well the instruction stream can be parallelised—this is influenced by instruction-level parallelism (ILP).

In summary, superscalar architecture boosts CPU performance by executing multiple instructions per cycle, using parallel execution units and intelligent control logic to manage dependencies and resource conflicts.

2.1. Concept and Design Principles

The core concept of superscalar architecture is to enhance processor performance by allowing a CPU to issue and execute multiple instructions simultaneously during a single clock cycle. Instead of relying solely on increasing clock frequency to improve speed, superscalar processors aim to boost throughput by executing more instructions per cycle. This approach leverages instruction-level parallelism (ILP)—the ability to perform several independent operations at the same time.

To accomplish this, the processor is equipped with multiple parallel execution units, such as Arithmetic Logic Units (ALUs) for integer operations, Floating Point Units (FPUs) for real-number calculations, and Load/Store units for memory access. These units operate concurrently, allowing the

CPU to handle different types of instructions in parallel without waiting for one to complete before starting another.

A superscalar processor is designed to fetch, decode, issue, and execute more than one instruction per cycle. To do this effectively, the architecture must incorporate several advanced features and design principles:

- **Instruction Fetching and Decoding in Parallel:** The processor can fetch multiple instructions from memory and decode them in the same cycle, preparing several instructions for potential execution.
- **Multiple Execution Units:** Superscalar CPUs have duplicated functional units (e.g., multiple ALUs or FPU) so that several instructions can be processed simultaneously, each in a different pipeline.
- **Out-of-Order Execution:** Instructions are not bound to execute in their original program order. Instead, they are executed as soon as their operands are ready, which helps prevent idle cycles and maximises hardware utilisation.
- **Dependency Checking:** To maintain correctness, the CPU performs dynamic analysis to detect data hazards, such as:
 - RAW (Read After Write): True dependency
 - WAR (Write After Read): Anti-dependency
 - WAW (Write After Write): Output dependency
- Efficient detection and resolution of these hazards are crucial for safe parallel execution.
- **Register Renaming:** This technique eliminates false dependencies by assigning unique physical registers to logical ones, allowing independent instructions to execute without unnecessary delays caused by name conflicts.
- **Dynamic Instruction Scheduling:** The CPU includes complex hardware logic to schedule and issue instructions dynamically, based on operand availability and resource readiness. This is essential to maximise parallel execution and avoid stalls.

Together, these features enable superscalar processors to make intelligent decisions about which instructions to run in parallel and when, all while preserving the logical behaviour of the program.

In summary, superscalar processors significantly increase instruction throughput by executing multiple independent instructions per clock cycle. Achieving this requires a combination of parallel hardware units and sophisticated control mechanisms to manage instruction dependencies, maintain

execution order, and ensure program correctness. Superscalar design is a key advancement in modern high-performance processors, balancing hardware complexity with increased processing speed.

2.2. Instruction-Level Parallelism (ILP)

Instruction-Level Parallelism (ILP) refers to the ability of a processor to execute multiple independent instructions at the same time. Superscalar processors exploit ILP to improve performance by issuing several instructions in parallel during a single clock cycle.

To take advantage of ILP, the processor looks for instructions that do not depend on each other—meaning one instruction's output is not needed by the next. If such independence exists, those instructions can be executed simultaneously using different execution units.

Example:

ADD R1, R2, R3 ; $R1 = R2 + R3$

MUL R4, R5, R6 ; $R4 = R5 \times R6$

Since these two instructions work on different registers and have no data dependency, they can be executed in parallel.

Superscalar CPUs use techniques like out-of-order execution, register renaming, and dynamic scheduling to detect and exploit ILP at runtime. The higher the ILP, the more instructions can be executed per cycle, increasing throughput.

However, ILP is limited by:

- Data dependencies (one instruction needs the result of another)
- Control dependencies (e.g., branches)
- Resource conflicts (limited execution units)

In summary, ILP is a key factor in superscalar performance, enabling the processor to maximise hardware usage by executing independent instructions in parallel.

2.3. Pipeline and Execution Units

In a superscalar architecture, the CPU uses multiple pipelines and execution units to process several instructions in parallel. Each pipeline is a sequence of stages—typically fetch, decode, execute,

memory access, and write-back—through which an instruction flows. Unlike scalar CPUs (which have only one pipeline), superscalar CPUs have two or more pipelines operating in parallel.

To support this, the processor includes multiple execution units, such as:

- ALUs (Arithmetic Logic Units) for integer operations
- FPUs (Floating Point Units) for real-number calculations
- Load/Store Units for memory operations
- Branch Units for control flow instructions

Each instruction is sent to a suitable execution unit depending on its type. For example, if two instructions—one addition and one multiplication—are independent, the CPU can send them to an ALU and an FPU respectively in the same cycle.

ADD R1, R2, R3 → ALU pipeline

MUL F1, F2, F3 → FPU pipeline

Both can execute simultaneously if no dependencies exist.

Superscalar processors also use dynamic scheduling, instruction dispatch, and hazard detection logic to manage the flow of instructions across multiple pipelines while ensuring correct execution.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the main advantage of Superscalar architecture? A) Reduced clock speed B) Execution of one instruction per cycle C) Parallel execution of multiple instructions D) Use of a single execution unit
2	Which technique is used in Superscalar processors to avoid false dependencies? A) Branch prediction B) Register renaming C) Instruction bundling D) Static scheduling
3	Instruction-Level Parallelism (ILP) is limited by: A) Clock speed

	B) Compiler efficiency C) Data and control dependencies D) Instruction format
Fill in the Blank	
4	Superscalar processors use _____ execution to allow instructions to be processed as soon as their operands are ready.
5	The ability to execute multiple independent instructions simultaneously is known as _____.
6	Superscalar CPUs include multiple _____ units to support parallel instruction execution.
True or False	
7	Superscalar architecture executes only one instruction per clock cycle.
8	Register renaming helps eliminate false data dependencies in Superscalar processors.
9	Superscalar processors rely entirely on the compiler for instruction scheduling.
10	Multiple pipelines in Superscalar CPUs allow different instructions to be processed simultaneously.

3. VLIW ARCHITECTURE (VERY LONG INSTRUCTION WORD)

VLIW (Very Long Instruction Word) architecture is a CPU design approach where the compiler—not the hardware—takes responsibility for identifying instruction-level parallelism (ILP) and packing multiple operations into a single, long instruction word. Each instruction word in a VLIW system typically contains several independent operations that can be executed simultaneously by different functional units.

Unlike superscalar processors, which rely on complex hardware to dynamically schedule and dispatch multiple instructions at runtime, VLIW processors execute multiple operations in parallel using a single instruction that specifies them explicitly. This greatly simplifies hardware design by removing dynamic scheduling, dependency checking, and branch prediction logic, shifting the burden to the compiler.

Structure Example:

A VLIW instruction might be 128 bits wide and include 4 operations, such as:

```
[ ADD R1, R2, R3 | MUL R4, R5, R6 | LOAD R7, 100(R8) | NOP ]
```

Here, all four operations are executed in parallel during the same clock cycle, assuming no dependencies.

Key Features:

- Fixed-width instructions: Each instruction contains multiple operations, typically one for each functional unit.
- Parallel execution: All operations in a VLIW instruction execute simultaneously.
- Simplified hardware: No dynamic scheduling; the control logic is minimal compared to superscalar CPUs.
- Compiler dependence: The compiler must detect parallelism and schedule instructions carefully to avoid hazards.

Applications:

VLIW architectures are commonly used in digital signal processors (DSPs), media processors, and embedded systems, where workloads are predictable and can be optimised at compile-time.

3.1. VLIW Fundamentals

VLIW (Very Long Instruction Word) architecture is based on the idea of executing multiple operations in parallel by encoding them into a single, wide instruction word. Each VLIW instruction typically contains several operations (e.g., integer arithmetic, floating-point, load/store), and all are intended to be executed simultaneously during one clock cycle.

Unlike superscalar processors, which rely on hardware to schedule and dispatch instructions dynamically, VLIW systems shift this responsibility to the compiler. The compiler analyses the program, identifies independent operations, and packs them into one VLIW instruction. This eliminates the need for complex hardware features like instruction scheduling, dependency checking, and speculative execution.

Example: A 128-bit VLIW instruction might include four 32-bit operations like:

[ADD R1, R2, R3 | MUL R4, R5, R6 | LOAD R7, 0(R8) | NOP]

All operations are independent and can be executed in parallel by separate functional units.

Key characteristics of VLIW include:

- Fixed-length, wide instructions with multiple parallel operations
- Simplified hardware, as scheduling is handled by the compiler
- Efficient use of instruction-level parallelism (ILP)

VLIW is best suited for applications where parallelism is known at compile time, such as digital signal processing, multimedia, and embedded systems.

3.2. Compiler Role in VLIW

In VLIW architecture, the compiler plays a central role in ensuring efficient parallel execution. Unlike superscalar processors, where the hardware decides which instructions to execute in parallel at runtime, VLIW relies entirely on the compiler to detect instruction-level parallelism (ILP) and schedule multiple independent operations into a single, long instruction word.

The compiler performs several critical tasks:

- Dependency analysis: It checks for data, control, and resource dependencies between instructions to avoid conflicts.

- **Instruction scheduling:** It arranges instructions so that operations in a VLIW word can run simultaneously without hazards.
- **Filling instruction slots:** The compiler packs as many operations as possible into each instruction word. If not enough independent operations are available, it inserts NOPs (no-operations) to fill unused slots.
- **Register allocation and renaming:** It manages register usage to maximise parallelism and reduce false dependencies.

Example:

Given these operations:

ADD R1, R2, R3

MUL R4, R5, R6

LOAD R7, 0(R8)

If they are independent, the compiler will combine them into one VLIW instruction:

[ADD | MUL | LOAD | NOP]

In summary, the compiler in VLIW architecture is responsible for finding parallelism, scheduling instructions, and ensuring correctness, making VLIW hardware simpler but requiring more sophisticated compilation techniques.

3.3. Advantages and Limitations

VLIW (Very Long Instruction Word) architecture presents a unique balance of strengths and trade-offs, primarily due to its design philosophy that shifts complexity from hardware to the compiler. By depending on compile-time scheduling, VLIW simplifies processor hardware but introduces challenges in software generation and flexibility.

Advantages:

- **Simplified Hardware Design:**

One of the most notable benefits of VLIW is its reduced hardware complexity. It eliminates the need for intricate mechanisms such as out-of-order execution, dynamic instruction scheduling, register renaming, and hardware-based hazard detection. As a result, the processor requires fewer transistors, leading to a smaller silicon footprint, lower power consumption, and often

higher clock speeds due to simplified control logic. This makes VLIW especially attractive for cost-sensitive and power-constrained environments.

- **High Performance Potential:**

By allowing the execution of multiple operations in a single clock cycle—assuming those operations are independent—VLIW architecture can achieve very high instruction throughput. This parallelism, when effectively exploited by the compiler, allows VLIW processors to perform competitively with or even outperform superscalar processors in some workloads, particularly where the instruction-level parallelism (ILP) is high and predictable.

- **Deterministic and Predictable Execution:**

Since instruction scheduling is done at compile time, the behaviour of VLIW programs is more deterministic than in dynamically scheduled processors. This makes VLIW a good choice for real-time systems, where predictability is critical for meeting timing constraints and ensuring reliable operation under strict deadlines.

- **Optimised for Specialised Applications:**

VLIW excels in domains where the nature of computation is regular and parallelisable, such as embedded systems, digital signal processing (DSP), audio/video encoding, and image processing. In these domains, the compiler can effectively identify parallel operations, and the workloads do not change frequently, reducing the need for dynamic execution flexibility.

Limitations:

- **Heavy Compiler Dependence:**

A major drawback of VLIW is its reliance on the compiler to detect parallelism and produce optimal instruction bundles. This places significant burden on compiler technology. If the compiler fails to extract sufficient ILP or schedule instructions efficiently, the resulting performance can be poor—even if the hardware has the capability for high parallelism.

- **Increased Code Size:**

Because instruction words must be fixed in width (e.g., 4 or 6 operations per instruction), unused slots in an instruction word are filled with NOPs (No Operation) when there are not enough independent operations to execute in parallel. These padding instructions increase the overall code size, which can impact memory usage and instruction cache efficiency.

- **Lack of Runtime Flexibility:**

Unlike superscalar processors, VLIW processors do not adjust scheduling decisions dynamically. This means they cannot adapt to variable memory latencies, branch mispredictions, or other

runtime changes in workload behaviour. As a result, performance may suffer in applications with irregular control flow or unpredictable data access patterns.

- **Binary Compatibility Challenges:**

Programs compiled for one VLIW hardware configuration (e.g., a processor with 4 execution slots) may not run efficiently—or at all—on a processor with a different number of slots (e.g., 6). This tight coupling between compiled code and hardware limits binary portability, making software reuse and system upgrades more difficult.

In summary, VLIW architecture offers impressive benefits in terms of hardware simplicity, power efficiency, and parallel performance—especially in well-defined, compute-intensive applications. However, these benefits come at the cost of compiler complexity, inflexibility at runtime, and compatibility limitations. VLIW is best suited for specialised systems where workloads are known in advance and can be aggressively optimised at compile time.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
1	In VLIW architecture, who is responsible for scheduling instructions for parallel execution? A) Hardware B) Operating System C) Compiler D) Control Unit
2	What is a major advantage of VLIW over Superscalar architecture? A) Higher clock speed B) Simpler hardware design C) Dynamic instruction scheduling D) Better branch prediction
3	What happens when there are not enough independent operations to fill a VLIW instruction word? A) The processor stalls B) The compiler inserts NOPs C) The instruction is skipped D) The hardware fills the slots automatically
Fill in the Blank	

4	VLIW stands for _____.
5	In VLIW systems, multiple operations are packed into a single _____ instruction.
6	VLIW architecture is commonly used in _____ systems and digital signal processors.
True or False	
7	VLIW processors rely on hardware to detect instruction-level parallelism at runtime.
8	The execution of operations in a VLIW instruction occurs simultaneously.
9	VLIW architecture is highly adaptable to unpredictable runtime conditions.
10	Code compiled for one VLIW configuration may not be portable to another configuration.



4. SIMD ARCHITECTURE (SINGLE INSTRUCTION, MULTIPLE DATA)

SIMD (Single Instruction, Multiple Data) is a computer architecture that allows a single instruction to operate on multiple data elements simultaneously. It is a form of data-level parallelism, ideal for applications that perform the same operation repeatedly on large data sets—such as graphics, image processing, matrix operations, and scientific computing.

In SIMD, a single control unit broadcasts the same instruction to multiple processing elements (or lanes), each operating on different data. This is different from traditional scalar processors, which execute one instruction on one piece of data at a time.

Example:

To add two arrays of numbers element by element:

```
// Scalar (one element at a time)
```

```
for (i = 0; i < 4; i++)
```

```
    C[i] = A[i] + B[i]
```

```
// SIMD (one instruction handles all)
```

```
C = A + B
```

Here, SIMD executes one ADD instruction that simultaneously adds all corresponding elements of arrays A and B.

Features of SIMD:

- Parallel data processing: Same operation on multiple data elements.
- Efficient for vector and matrix operations.
- Widely used in multimedia, 3D graphics, AI, and signal processing.

Examples of SIMD Architectures:

- Intel MMX, SSE, AVX: SIMD extensions in x86 CPUs.
- ARM NEON: SIMD engine in ARM processors.
- GPUs: Often follow a SIMD-like execution model for massive parallelism.

Advantages:

- High performance for parallel workloads.
- Reduced instruction fetch and decode overhead.
- Energy efficient compared to multiple scalar processors.

Limitations:

- Not suitable for tasks with irregular control flow or data dependencies.
- Requires aligned and parallelisable data, making programming more complex.

In summary, SIMD architecture is powerful for applications involving repetitive, parallel computations across large data sets, offering improved speed and efficiency with fewer instructions.

4.1. SIMD Basics

SIMD (Single Instruction, Multiple Data) is a parallel computing model where a single instruction is applied simultaneously to multiple data elements. It is ideal for applications with data-level parallelism, such as image processing, signal processing, and scientific computing.

In SIMD systems, the control unit issues one instruction, which is executed in parallel by multiple processing elements (also called lanes or cores), each operating on different pieces of data. This model boosts performance by reducing the number of instructions needed to process large data sets.

Example:

Adding two arrays element-wise:

```
// Scalar
```

```
C[0] = A[0] + B[0]
```

```
C[1] = A[1] + B[1]
```

```
...
```

```
// SIMD
```

```
C = A + B // all elements processed in parallel
```

SIMD is implemented in hardware using vector processors, GPU cores, or CPU extensions like Intel SSE/AVX and ARM NEON. These units process multiple data elements (e.g., 4, 8, or 16 values) in parallel using a single instruction.

In summary, SIMD improves performance and efficiency in programs that perform the same operation repeatedly on large blocks of data, by processing multiple data items per instruction cycle.

4.2. Data Parallelism

Data parallelism is a form of parallel computing where the same operation is applied simultaneously to multiple data elements. It is the foundation of SIMD (Single Instruction, Multiple Data) architecture, which enables efficient processing of large data sets.

In data-parallel tasks, the data is divided into chunks, and each chunk is processed in parallel by separate processing units, all executing the same instruction. This model is ideal for operations on arrays, vectors, or matrices, where each element can be treated independently.

Example:

If you want to multiply every element of an array by 2:

```
// Scalar (sequential)
```

```
for (i = 0; i < 4; i++)
```

```
    A[i] = A[i] * 2
```

```
// SIMD (parallel)
```

```
A = A * 2 // all elements updated at once
```

Data parallelism is widely used in graphics processing, machine learning, signal processing, and scientific simulations—anywhere the same computation needs to be applied to large volumes of data.

In summary, data parallelism enables high performance and efficiency by allowing the same instruction to process multiple data points in parallel, reducing execution time and maximising hardware utilisation.

4.3. SIMD in Modern Processors

Modern processors integrate SIMD (Single Instruction, Multiple Data) capabilities directly into their architecture to enhance performance for data-parallel tasks. These SIMD extensions allow a single instruction to operate on multiple data elements simultaneously, significantly accelerating workloads like multimedia processing, image filtering, machine learning, and scientific computing.

In CPUs, SIMD is implemented through specialised instruction sets:

- Intel: MMX, SSE, AVX, AVX-512

- ARM: NEON
- IBM Power: AltiVec

These instruction sets operate on vector registers, which can hold multiple data items (e.g., 4, 8, or 16 integers or floating-point numbers). A single instruction performs operations like addition, multiplication, or comparison on all elements in the vector at once.

Example using AVX (256-bit register):

```
ADDPS XMM1, XMM2
```

This adds eight 32-bit floating-point values from two registers in parallel.

GPUs, which are designed for massive parallelism, also use SIMD-like execution across thousands of cores. While technically classified as SMT (Single Instruction, Multiple Threads), the concept is similar: multiple data streams processed using the same instruction.

In summary, modern processors use SIMD to boost performance and reduce power consumption in data-heavy applications by processing multiple data elements per instruction. This makes SIMD a key feature in both general-purpose CPUs and specialised accelerators.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
1	What does SIMD stand for in computer architecture? A) Single Instruction, Multiple Devices B) Simple Instruction, Multiple Data C) Single Instruction, Multiple Data D) Simultaneous Instruction, Multiple Dispatch
2	Which type of parallelism does SIMD architecture primarily exploit? A) Instruction-Level Parallelism B) Thread-Level Parallelism C) Data-Level Parallelism D) Task-Level Parallelism
3	Which of the following is a common use case for SIMD architecture? A) Operating system scheduling B) Image processing

	C) Database indexing D) Network routing
Fill in the Blank	
4	SIMD architecture applies the same _____ to multiple data elements simultaneously.
5	SIMD is implemented in modern CPUs using specialised _____ sets like SSE, AVX, and NEON.
6	In SIMD, multiple processing elements operate in _____ under a single control unit.
True or False	
7	SIMD architecture is ideal for tasks with irregular control flow and branching.
8	SIMD improves performance by reducing the number of instructions fetched and decoded.
9	SIMD operations are typically performed using vector registers that hold multiple data elements.
10	SIMD is not supported in modern processors.

5. COMPARATIVE ANALYSIS: SUPERSCALAR VS VLIW VS SIMD

Aspect	Superscalar	VLIW	SIMD
Parallelism Type	Instruction-Level Parallelism (ILP)	Instruction-Level Parallelism (ILP)	Data-Level Parallelism (DLP)
Instruction Scheduling	Done by hardware at runtime	Done by compiler at compile time	Same instruction applied to multiple data
Hardware Complexity	High (dynamic scheduling, hazard detection)	Low (simplified control logic)	Moderate (requires vector units)
Compiler Role	Less dependent	Very high — must detect and schedule ILP	Minimal — just vectorised data
Execution Units	Multiple (ALUs, FPUs, etc.)	Multiple, controlled via long instruction word	Multiple SIMD lanes (vector units)
Best Use Cases	General-purpose CPUs	Embedded systems, DSPs, real-time apps	Graphics, multimedia, AI, scientific computing
Scalability	Limited by hardware complexity	Limited by compiler ability and code density	Scales well with wide data sets
Code Portability	Good across similar hardware	Poor — tightly coupled to specific width	Good if SIMD extensions are supported
Performance Potential	High (dynamic and adaptive)	High if code is well-compiled	Very high for data-parallel workloads

Conclusion:

- Superscalar focuses on exploiting ILP dynamically using complex hardware.
- VLIW shifts that responsibility to the compiler, simplifying hardware but increasing software complexity.
- SIMD excels at applying the same operation to large sets of data simultaneously, making it ideal for parallel data processing.

5.1. Superscalar vs VLIW

Superscalar and VLIW (Very Long Instruction Word) architectures both aim to exploit instruction-level parallelism (ILP) by executing multiple instructions per clock cycle. However, they differ significantly in who handles the parallelism and how execution is controlled.

In a Superscalar architecture, the hardware dynamically analyses the instruction stream at runtime to identify independent instructions and schedule them for parallel execution. This requires complex hardware units like dependency checkers, dynamic schedulers, and branch predictors, making the processor more flexible but also more power-hungry and expensive to design.

In contrast, VLIW architecture shifts the responsibility for scheduling to the compiler. The compiler statically determines which instructions can be executed in parallel and bundles them into a wide instruction word. The hardware then executes all instructions in the bundle simultaneously, without runtime analysis. This simplifies the hardware design but puts more pressure on the compiler to find parallelism.

Example:

Both architectures can execute:

ADD R1, R2, R3

MUL R4, R5, R6

- In Superscalar, hardware detects independence and schedules them together.
- In VLIW, the compiler places them in the same instruction word ahead of time.

Summary:

- Superscalar: Hardware-driven, flexible, complex control logic.
- VLIW: Compiler-driven, simpler hardware, relies on compile-time optimisation.

5.2. VLIW vs SIMD

VLIW (Very Long Instruction Word) and SIMD (Single Instruction, Multiple Data) architectures both achieve parallelism but operate on different levels and are designed for different types of workloads.

VLIW exploits Instruction-Level Parallelism (ILP) by executing multiple independent operations in a single instruction. Each operation in the VLIW word is intended for a different functional unit, such

as an ALU or FPU. The compiler is responsible for scheduling these instructions and ensuring they are independent.

SIMD, on the other hand, focuses on Data-Level Parallelism (DLP). It applies one instruction to multiple data elements at the same time using vector or parallel data paths. All processing units execute the same operation in lockstep but on different data inputs.

Example:

- VLIW:
[ADD R1, R2, R3 | MUL R4, R5, R6 | LOAD R7, 0(R8)]
→ Executes different operations in parallel.
- SIMD:
ADDPS XMM1, XMM2
→ Adds corresponding elements of two vectors (e.g., arrays) in parallel.

Summary:

- VLIW is suited for general-purpose programs with multiple independent instructions.
- SIMD is ideal for repetitive operations on large data sets, like image or signal processing.
- VLIW parallelism is across instructions, while SIMD parallelism is across data.

5.3. Use Cases and Performance

Superscalar, VLIW, and SIMD architectures are each optimised for different kinds of workloads, based on how they handle parallelism and execution.

- Superscalar processors are well-suited for general-purpose computing like desktop CPUs and mobile processors. They dynamically extract instruction-level parallelism (ILP) at runtime, making them effective for running complex, unpredictable programs like operating systems, compilers, or web browsers. Performance is high but depends on hardware complexity and the ability to detect independent instructions on the fly.
- VLIW is ideal for embedded systems, digital signal processing (DSP), and real-time applications, where workloads are predictable and can be optimised at compile time. Performance is high when the compiler can efficiently fill all instruction slots, but it suffers when instruction-level parallelism is low or data dependencies exist.

- SIMD is best used for data-parallel workloads such as image processing, machine learning, 3D graphics, and scientific simulations. SIMD delivers excellent performance by applying the same operation to many data elements simultaneously. It scales well with vector size and offers high throughput, but it's not suited for control-heavy or highly branched code.

In summary:

- Superscalar: High performance, versatile, best for complex general-purpose tasks.
- VLIW: High performance in predictable tasks, efficient but compiler-dependent.
- SIMD: Best for high-speed processing of large, uniform data sets.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
1	Which architecture relies on the compiler for instruction scheduling? A) Superscalar B) SIMD C) VLIW D) RISC
2	SIMD architecture is best suited for which type of workload? A) Control-heavy tasks B) Irregular branching C) Data-parallel operations D) Real-time scheduling
3	Superscalar architecture performs instruction scheduling at: A) Compile time B) Runtime using hardware C) Runtime using software D) During boot-up
Fill in the Blank	
4	Superscalar architecture exploits _____ to execute multiple instructions per cycle
5	VLIW architecture simplifies hardware by shifting scheduling responsibilities to the _____.
6	SIMD architecture applies the same instruction to multiple _____ elements simultaneously.
True or False	

7	Superscalar processors are more flexible than VLIW processors in handling unpredictable instruction patterns.
8	VLIW architecture is highly adaptable to runtime changes in instruction flow.
9	SIMD is ideal for executing different instructions on different data elements.
10	Superscalar, VLIW, and SIMD architectures all aim to improve CPU performance through parallelism.



6. SUMMARY

Superscalar architecture is designed to exploit instruction-level parallelism (ILP) by allowing multiple instructions to be issued and executed simultaneously in a single clock cycle. The key design principle is dynamic scheduling, where hardware logic determines which instructions can be executed in parallel based on data dependencies and resource availability. Execution units in a superscalar processor are often supported by deep pipelines and sophisticated hazard detection and forwarding mechanisms, enabling efficient parallel execution and increased throughput.

VLIW (Very Long Instruction Word) architecture also aims to exploit ILP but takes a compiler-driven approach. In VLIW, multiple operations are grouped into a long instruction word and scheduled at compile time, eliminating the need for complex dynamic scheduling hardware. This leads to simpler processor designs and potentially higher performance per watt. However, VLIW relies heavily on the compiler's ability to statically analyse dependencies and optimise instruction grouping, which can limit flexibility and performance in unpredictable or branching workloads.

SIMD (Single Instruction, Multiple Data) architecture focuses on data-level parallelism, executing the same operation across multiple data elements simultaneously. This is particularly effective in applications such as graphics processing, scientific simulations, and multimedia tasks. SIMD processors use wide vector registers and specialised instructions to apply operations across entire arrays of data, significantly accelerating workloads with high regularity and minimal branching. Modern processors, including CPUs and GPUs, incorporate SIMD capabilities through extensions like AVX and NEON.

When comparing these architectures, superscalar and VLIW both target ILP but differ in their handling of instruction scheduling—hardware-driven in superscalar versus compiler-driven in VLIW. VLIW and SIMD differ in their parallelism granularity, with VLIW operating at the instruction level and SIMD at the data level. Superscalar is well-suited for general-purpose computing with unpredictable control flow, while VLIW excels in embedded systems with predictable code paths. SIMD is ideal for highly parallel, data-intensive workloads. Ultimately, the choice between these architectures depends on the application's characteristics and the desired balance between performance, complexity, and power efficiency.

7. GLOSSARY

Superscalar Architecture	- A CPU design that allows multiple instructions to be issued and executed in parallel during a single clock cycle, using multiple execution units and dynamic scheduling.
Instruction-Level Parallelism (ILP)	- The ability of a processor to execute multiple independent instructions simultaneously, improving throughput and performance.
Out-of-Order Execution	- A technique used in superscalar processors where instructions are executed as soon as their operands are ready, rather than strictly following program order.
VLIW Architecture	- A CPU design where the compiler schedules multiple operations into a single, wide instruction word, enabling parallel execution with simplified hardware.
Static Instruction Scheduling	- The process performed by a compiler in VLIW systems to arrange instructions for parallel execution, avoiding runtime scheduling complexity.
NOP (No Operation)	- A placeholder instruction used in VLIW to fill unused slots in an instruction word when not enough independent operations are available.
SIMD Architecture	- A parallel computing model where a single instruction operates on multiple data elements simultaneously, ideal for data-parallel tasks like image processing and scientific computing.
Data-Level Parallelism (DLP)	- A form of parallelism where the same operation is applied to multiple data elements at once, commonly used in SIMD architectures.
Vector Processing	- A technique in SIMD systems where operations are performed on entire vectors (arrays of data) using specialised hardware like vector registers.
Execution Units	- Functional components within a CPU (e.g., ALUs, FPU, Load/Store units) that perform specific types of operations during instruction execution.
Dynamic Scheduling	- A hardware-based method used in superscalar processors to determine which instructions can be executed in parallel at runtime.

Compiler Dependency	- A characteristic of VLIW architecture where performance relies heavily on the compiler's ability to detect and schedule parallel instructions.
SIMD Extensions	- Specialised instruction sets (e.g., Intel SSE/AVX, ARM NEON) that enable SIMD operations in general-purpose CPUs.
Parallel Pipelines	- Multiple instruction pipelines in superscalar processors that allow simultaneous execution of different instructions across various execution units.
Hazard Detection	- A mechanism in superscalar processors that identifies potential conflicts between instructions, such as data dependencies, and prevents incorrect execution.
Branch Prediction	- A technique used in superscalar CPUs to guess the outcome of conditional branches in advance, allowing the pipeline to continue executing instructions without waiting.
Vector Register	- A register that holds multiple data elements (e.g., integers or floats) for SIMD operations, enabling parallel processing of data.
Code Portability	- The ability of software to run efficiently across different hardware platforms; often a challenge in VLIW due to architecture-specific instruction widths.
Execution Throughput	- A measure of how many instructions a processor can complete in a given time; higher in architectures that support parallelism like Superscalar, VLIW, and SIMD.
Compiler Optimisation	- Techniques used by compilers to improve performance, especially important in VLIW systems where the compiler schedules parallel instructions.
Vectorisation	- The process of converting scalar operations into vector operations to take advantage of SIMD capabilities.
Instruction Slot	- A position within a VLIW instruction word that holds an operation; unused slots are typically filled with NOPs.
Parallel Execution	- The simultaneous execution of multiple instructions or operations, central to Superscalar, VLIW, and SIMD architectures.

8. TERMINAL QUESTIONS

1. What is the primary goal of Superscalar architecture in CPU design?
2. How does out-of-order execution improve performance in Superscalar processors?
3. Explain the role of register renaming in avoiding false dependencies.
4. What are the key differences between Superscalar and VLIW architectures in terms of instruction scheduling?
5. Why is the compiler critical in VLIW architecture?
6. Describe how a VLIW instruction is structured and executed.
7. What are the advantages and limitations of VLIW architecture in embedded systems?
8. How does SIMD architecture achieve data-level parallelism?
9. Give an example of a real-world application where SIMD architecture is highly effective.
10. What are the challenges of using SIMD for irregular data patterns?
11. Compare Superscalar, VLIW, and SIMD architectures in terms of hardware complexity and scalability.
12. What is the difference between instruction-level parallelism (ILP) and data-level parallelism (DLP)?
13. How do modern processors integrate SIMD capabilities to enhance performance?
14. In what scenarios would VLIW architecture outperform Superscalar?
15. Why is SIMD considered energy-efficient for data-parallel workloads?

9. ANSWERS

Self-Assessment Questions

Self-Assessment Questions - 1

1. C) Parallel execution of multiple instructions
2. B) Register renaming
3. C) Data and control dependencies
4. out-of-order
5. Instruction-Level Parallelism (ILP)
6. execution
7. False
8. True
9. False
10. True

Self-Assessment Questions - 2

1. C) Compiler
2. B) Simpler hardware design
3. B) The compiler inserts NOPs
4. Very Long Instruction Word
5. wide
6. embedded
7. False
8. True
9. False
10. True

Self-Assessment Questions - 3

1. C) Single Instruction, Multiple Data
2. C) Data-Level Parallelism
3. B) Image processing
4. instruction
5. instruction

6. parallel
7. False
8. True
9. True
10. False

Self-Assessment Questions - 4

1. C) VLIW
2. C) Data-parallel operations
3. B) Runtime using hardware
4. compiler
5. data
6. True
7. False
8. False
9. True

Terminal Questions Answers

Answer 1: To increase the overall performance of the CPU by enabling the execution of multiple instructions in parallel during a single clock cycle. Superscalar architecture achieves this by using multiple execution units and advanced control logic to identify and dispatch independent instructions simultaneously, thereby improving instruction throughput without increasing clock speed.

(Refer to section 2.1 - Concept and Design Principles)

Answer 2: Out-of-order execution allows the processor to execute instructions as soon as their operands are available, rather than waiting for previous instructions to complete. This reduces idle time in execution units and helps maintain a steady flow of instruction processing, especially in cases where instruction dependencies would otherwise cause delays.

(Refer to section 2.1 - Concept and Design Principles)

Answer 3: Register renaming helps eliminate false dependencies between instructions that use the same register names. By assigning unique identifiers to destination registers, the CPU ensures that instructions can be executed in parallel without unintended interference, thus improving instruction-level parallelism and reducing pipeline stalls.

(Refer to section 2.1 - Concept and Design Principles)

Answer 4: Superscalar architecture uses hardware to dynamically analyse and schedule instructions at runtime, while VLIW relies on the compiler to statically schedule instructions during compilation. This means Superscalar processors are more flexible and adaptive to runtime conditions, whereas VLIW processors depend heavily on compile-time optimisation.

(Refer to section 5.1 - Superscalar vs VLIW)

Answer 5: In VLIW architecture, the compiler is responsible for identifying independent operations and packing them into a single instruction word. It must perform dependency analysis, instruction scheduling, and register allocation to ensure that all operations in a VLIW instruction can be executed in parallel without conflicts.

(Refer to section 3.2 - Compiler Role in VLIW)

Answer 6: A VLIW instruction is a wide, fixed-length word that contains multiple operations intended for different functional units. These operations are scheduled by the compiler and executed simultaneously by the processor, assuming there are no data dependencies or resource conflicts among them.

(Refer to section 3.2 - Compiler Role in VLIW)

Answer 7: VLIW architecture offers simplified hardware and predictable execution timing, making it ideal for embedded systems. However, its performance is limited by the compiler's ability to find parallelism, and code size may increase due to unused instruction slots filled with NOPs, which can affect memory efficiency.

(Refer to section 3.3 - Advantages and Limitations)

Answer 8: SIMD architecture achieves data-level parallelism by applying a single instruction to multiple data elements at once. This is done using vector registers or parallel processing lanes, allowing operations like addition, multiplication, or comparison to be performed simultaneously across large datasets.

(Refer to section 4.2 - Data Parallelism)

Answer 9: A common real-world application of SIMD is image processing, where the same operation—such as adjusting brightness or applying a filter—is applied to every pixel in an image. SIMD enables these operations to be performed in parallel, significantly speeding up processing time.

(Refer to section 5.3 - Use Cases and Performance)

Answer 10: SIMD struggles with irregular data patterns because it requires uniform operations across all data elements. If different elements need different instructions or if control flow varies, SIMD lanes may be underutilised or require masking, which reduces efficiency and complicates programming.

(Refer to section 4 - SIMD Architecture)

Answer 11: Superscalar processors have high hardware complexity due to dynamic scheduling and hazard detection, but they offer flexible scalability. VLIW has simpler hardware but limited scalability due to compiler constraints. SIMD has moderate hardware complexity and scales well with wide data sets, making it ideal for parallel workloads.

(Refer to section 5 - Comparative Analysis: Superscalar vs VLIW vs SIMD)

Answer 12: Instruction-level parallelism (ILP) involves executing multiple independent instructions simultaneously, while data-level parallelism (DLP) involves executing the same instruction on multiple data elements. ILP is exploited by Superscalar and VLIW architectures, whereas DLP is the foundation of SIMD.

(Refer to section 2.2 - Instruction-Level Parallelism (ILP))

Answer 13: Modern processors integrate SIMD capabilities through specialised instruction sets like Intel SSE/AVX and ARM NEON. These extensions allow CPUs to perform vector operations on multiple data elements in parallel, enhancing performance in multimedia, scientific, and AI applications.

(Refer to section 4.3 - SIMD in Modern Processors)

Answer 14: VLIW architecture can outperform Superscalar in scenarios where workloads are predictable and can be optimised at compile time. Examples include digital signal processing and embedded systems, where deterministic execution and simplified hardware are advantageous.

(Refer to section 3.3 - Advantages and Limitations)

Answer 15: SIMD is energy-efficient because it reduces the number of instructions fetched and decoded, and performs operations on multiple data elements with a single instruction. This minimises control overhead and maximises hardware utilisation, making it ideal for high-throughput, low-power applications.

(Refer to section 4.3 - SIMD in Modern Processors)

10. REFERENCES

BOOKS

1. Computer Architecture: A Quantitative Approach, John L. Hennessy, David A. Patterson, 6th Edition, 2017
2. Computer Organisation and Architecture: Designing for Performance, William Stallings, 11th Edition, 2018
3. Computer System Architecture, M. Morris Mano, 3rd Edition, 2006
4. Computer Organisation and Design: The Hardware/Software Interface, David A. Patterson, John L. Hennessy, 5th Edition, 2013
5. Digital Design and Computer Architecture, Sarah Harris, David Harris, 3rd Edition, 2021

E-REFERENCES

1. Logical and Shift Micro-operations Computer Organisation and Architecture, <https://care4you.in/logical-and-shift-micro-operations/>
2. Logic and Shift Micro Operations, <https://www.scribd.com/document/394026693/U-I-Logic-and-Shift-micro-operations>

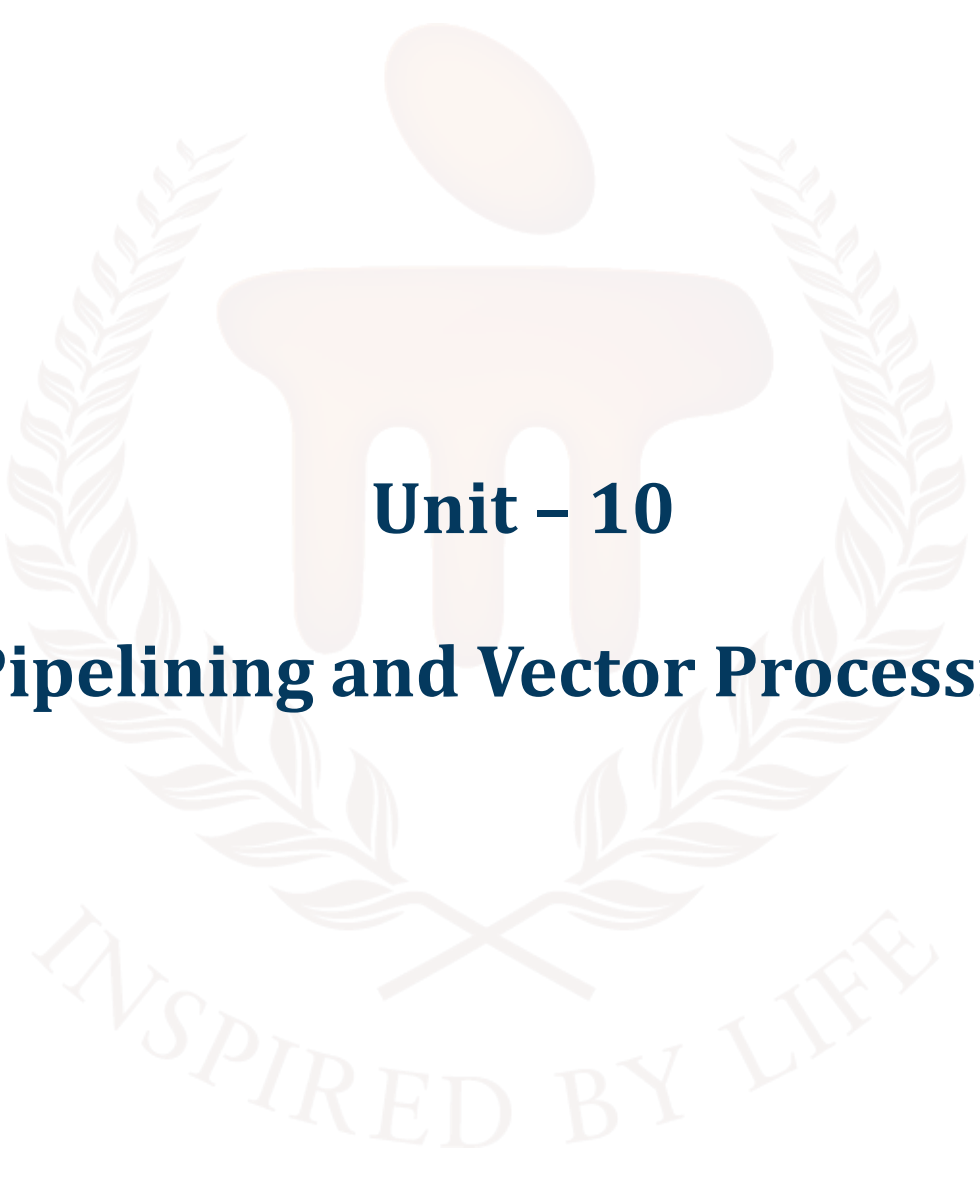
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 10

Pipelining and Vector Processing

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Parallel Processing	-	-	6 - 13
2.1	Concept and Design Principles	-	-	
2.2	Types of Parallelism	-	-	
2.3	Benefits and Challenges	-	1	
3	Pipelining	-	-	14 - 20
3.1	Pipeline Fundamentals	-	-	
3.2	Pipeline Stages	-	-	
3.3	Pipeline Hazards	-	2	
4	Arithmetic Pipeline	-	-	21 - 26
4.1	Design and Implementation	-	-	
4.2	Examples and Use Cases	-	3	
5	Instruction Pipeline	-	-	27 - 31
5.1	Instruction Fetch and Decode	-	-	
5.2	Execution and Write-back	-	4	
6	Vector Processing	-	-	32 - 38
6.1	Vector Architecture	-	-	
6.2	Vector Instructions	-	-	
6.3	Applications of Vector Processing	-	5	
7	Array Processors	-	-	39 - 41
7.1	SIMD vs MIMD	-	-	
7.2	Use Cases and Performance	-	6	
8	Summary	-	-	42
9	Glossary	-	-	43 - 44

10	Terminal Questions	-	-	45
11	Answers	-	-	46 - 49
12	References	-	-	50



1. INTRODUCTION

In the previous unit, we explored the architectural paradigms that enable modern CPUs to achieve high performance through instruction-level and data-level parallelism. We examined Superscalar, VLIW (Very Long Instruction Word), and SIMD (Single Instruction, Multiple Data) architectures—each offering unique strategies to enhance instruction throughput and computational efficiency. You learned how Superscalar processors dynamically schedule instructions using complex hardware, how VLIW shifts scheduling responsibilities to the compiler for simplified hardware design, and how SIMD accelerates data-parallel tasks by applying the same instruction across multiple data elements simultaneously.

Building upon that foundation, this unit delves deeper into the mechanisms that make such parallelism possible at the microarchitectural level. We will focus on Pipelining and Vector Processing, two critical techniques that significantly boost processing speed and throughput in modern computing systems.

We begin by understanding the concept of Parallel Processing, which forms the basis for executing multiple operations concurrently. From there, we explore Pipelining, a technique that breaks down instruction execution into discrete stages, allowing multiple instructions to be processed simultaneously in different stages of the pipeline. You will learn about Arithmetic Pipelines and Instruction Pipelines, their design principles, and how they handle hazards and dependencies.

Next, we turn to Vector Processing, which extends the idea of SIMD by enabling operations on entire vectors of data using specialised hardware. We will also examine Array Processors, which are designed to handle large-scale parallel computations efficiently.

By the end of this unit, you will have a comprehensive understanding of how pipelining and vector processing contribute to high-performance computing. You will be able to analyse their advantages, limitations, and real-world applications, and appreciate their role in shaping the architecture of modern processors.

1.1. Objectives

After studying this unit, you should be able to:

- Explain the concept of parallel processing.
- Discuss the principles and stages of pipelining.
- Identify and analyse pipeline hazards and techniques for their resolution.
- Analyse the architecture and functioning of vector processors and array processors.
- Illustrate how vector processing enhances performance in data-intensive applications.
- Evaluate the role of SIMD and MIMD models in array processor design.



2. PARALLEL PROCESSING

In computer organisation and architecture, parallel processing refers to the simultaneous execution of multiple instructions or operations by a processor or multiple processors. This architectural approach is fundamental to increasing system performance, especially as limitations in clock speed improvements (due to thermal and power constraints) have made traditional frequency scaling less effective.

Parallel processing aims to improve instruction throughput and reduce execution latency by exploiting the natural parallelism found in many computing tasks. Instead of processing one instruction at a time (as in scalar or sequential systems), parallel processors are capable of executing multiple instructions or multiple data operations concurrently.

Parallel processing is implemented through a combination of architectural strategies such as:

- Multiple functional units (e.g., multiple ALUs, FPU),
- Pipelined execution units,
- Multi-core and multi-processor designs, and
- Vector and SIMD execution units.

This technique is essential in high-performance systems such as scientific computing platforms, multimedia processors, and modern general-purpose CPUs and GPUs.

At the architectural level, parallelism is categorised into several types:

- Instruction-Level Parallelism (ILP): Executing independent instructions simultaneously.
- Data-Level Parallelism (DLP): Applying the same operation to multiple data elements in parallel.
- Task-Level Parallelism (TLP): Executing separate program threads or tasks concurrently.

The success of parallel processing in a system depends heavily on its ability to manage shared resources, maintain memory consistency, balance workloads across processing elements, and scale effectively with increased hardware resources.

2.1. Concept and Design Principles

Parallel processing, in the context of computer organisation and architecture, refers to the coordinated execution of multiple instructions simultaneously to improve processing speed and

efficiency. Unlike sequential execution in traditional processors, parallel systems leverage multiple functional units, cores, or processors to perform computations concurrently.

Concept of Parallel Processing

Modern computing systems are expected to handle increasingly complex and data-intensive tasks. Relying solely on increasing clock speed (frequency scaling) has become inefficient due to power and heat constraints. To overcome this, parallel processing exploits the inherent parallelism in data and instructions to increase throughput without necessarily increasing clock frequency.

At the architectural level, parallelism can be applied through:

- Multiple Execution Units (ALUs, FPUs, Load/Store units)
- Multiple Instruction Pipelines
- Multiple Cores or Processors
- Vector Units or SIMD Lanes

Core Design Principles

1. Instruction-Level Parallelism (ILP)

ILP involves identifying independent instructions that can be executed simultaneously. Superscalar and VLIW processors exploit ILP using multiple pipelines or wide instruction words.

Example:

If `ADD R1, R2, R3` and `SUB R4, R5, R6` are independent, a superscalar processor may issue both in the same clock cycle using separate ALUs.

2. Data-Level Parallelism (DLP)

Data-level parallelism refers to applying the same operation on multiple data elements simultaneously. SIMD (Single Instruction, Multiple Data) architectures like AVX or NEON utilise this principle.

Example:

Adding two vectors of 8 integers each using one instruction.

3. Task-Level Parallelism (TLP)

TLP involves executing independent processes or threads on separate processing units. This is commonly implemented in multi-core or multi-processor systems.

4. Pipelining

Instruction pipelines divide instruction execution into stages (Fetch, Decode, Execute, etc.). In parallel processors, multiple pipelines may operate in parallel, increasing instruction throughput.

5. Resource Replication

To support simultaneous execution, parallel processors replicate functional units such as multiple ALUs, memory ports, or register files.

Architectural Considerations

- **Synchronisation Mechanisms:** Hardware supports synchronisation (e.g., atomic operations, memory barriers) to maintain consistency when multiple units access shared memory.
- **Cache Coherence:** In multi-core systems, cache coherence protocols (like MESI) ensure all cores see consistent memory views.
- **Interconnection Networks:** In distributed or multi-core architectures, buses, crossbars, or meshes are used to connect cores efficiently.
- **Scalability:** Architectures must scale with added cores or units without introducing bottlenecks in communication or memory access.

2.2. Types of Parallelism

Parallelism is categorised based on the level at which operations are executed concurrently. The major types include bit-level, instruction-level, data-level, task-level, and pipeline parallelism.

Bit-Level Parallelism

This occurs when operations on wider word sizes allow more bits to be processed in a single instruction. For example, a 32-bit adder can perform four 8-bit additions at once, reducing the total number of operations needed.

Instruction-Level Parallelism (ILP)

Independent instructions that do not depend on one another can be executed simultaneously. ILP is exploited using multiple functional units, pipelining, and dynamic scheduling.

ADD R1, R2, R3 ; Executed in ALU1

SUB R4, R5, R6 ; Executed in ALU2

If there's no data dependency, both can execute in the same clock cycle.

Data-Level Parallelism (DLP)

Multiple data elements are processed using the same operation. DLP is common in vector and SIMD architectures, where a single instruction operates on an entire data set.

```
int A[4] = {1,2,3,4}, B[4] = {10,20,30,40}, C[4];
```

```
for (int i = 0; i < 4; i++)
```

```
    C[i] = A[i] + B[i];
```

A SIMD instruction can perform these additions in parallel using a vector register.

Task-Level Parallelism (TLP)

Different threads or tasks are executed concurrently. Each task may perform a different operation and run on a separate core.

```
// Thread 1 handles file I/O
```

```
// Thread 2 performs calculations
```

This form is utilised in multi-threaded and multi-core systems.

Pipeline Parallelism

An instruction is divided into stages—such as fetch, decode, execute—allowing multiple instructions to be processed simultaneously in different stages.

Example stages: Fetch → Decode → Execute → Memory Access → Writeback

While one instruction is being decoded, another can be fetched, and another can be executed, enabling high instruction throughput.

2.3. Benefits and Challenges

Parallel processing provides significant improvements in system performance, especially in workloads that involve large-scale computation. However, it also introduces architectural and programming complexities that must be carefully managed.

Benefits

1. Increased Throughput

Multiple instructions or data operations can be processed simultaneously, significantly increasing the number of computations completed per unit time.

2. Reduced Execution Time

By distributing work across multiple processing units, the overall time required to complete a task is reduced, particularly for computation-heavy applications.

3. Efficient Resource Utilisation

Multi-core and multiprocessor systems can handle more workloads without idle hardware, improving the overall utilisation of functional units like ALUs, FPUs, and memory controllers.

4. Scalability

Parallel systems can scale with additional cores or processors, making them suitable for both small embedded systems and large data centers.

5. Support for High-Performance Applications

Tasks such as 3D rendering, scientific simulations, AI model training, and real-time signal processing benefit greatly from parallel execution.

Challenges

1. Data Dependency

Instructions that depend on the results of others cannot be executed in parallel, reducing the available parallelism and requiring complex dependency analysis.

2. Synchronisation Overhead

Coordinating access to shared resources (such as memory) introduces delays due to the need for locking, barriers, or other synchronisation mechanisms.

3. Load Imbalance

Unequal distribution of work across cores leads to some processors being idle while others are overloaded, reducing efficiency.

4. Communication Overhead

In distributed systems or multi-core processors with separate caches, frequent communication or data transfer between units increases latency.

5. Debugging and Testing Complexity

Non-deterministic behaviour caused by concurrent execution makes parallel programs harder to test and debug compared to sequential ones.

6. Hardware Complexity

Supporting parallelism requires additional hardware structures like multiple execution units, advanced control logic, and interconnection networks, increasing design complexity and power consumption.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the main goal of parallel processing in computer architecture? A) Reduce memory usage B) Increase clock speed C) Improve instruction throughput D) Simplify hardware design
2	Which of the following is an example of data-level parallelism (DLP)? A) Executing multiple threads on different cores B) Executing independent instructions in parallel C) Applying the same operation to multiple data elements D) Fetching instructions from memory
3	Which type of parallelism is most commonly used in multi-core processors? A) Bit-level parallelism B) Instruction-level parallelism C) Task-level parallelism D) Pipeline parallelism
Fill in the Blank	
4	The ability to execute multiple independent instructions simultaneously is known as _____.
5	_____ parallelism involves applying the same operation to multiple data elements at once.
6	One of the major challenges in parallel processing is managing _____ between processing units.
True or False	
7	Parallel processing can reduce execution time by distributing tasks across multiple processing units.
8	Bit-level parallelism is the most scalable form of parallelism in modern processors.

9	Load imbalance occurs when all processing units are equally utilised.
10	Cache coherence is important in multi-core systems to ensure consistent memory views.



3. PIPELINING

Pipelining is a technique used in processor design to improve instruction throughput by overlapping the execution of multiple instructions. It divides the instruction execution cycle into separate stages, allowing multiple instructions to be in different stages of execution simultaneously.

A pipeline works like an assembly line, where each stage performs a specific operation and passes the result to the next stage. As one instruction moves to the next stage, a new instruction enters the first stage, keeping all stages active and improving efficiency.

Basic Pipeline Stages

A typical RISC instruction pipeline consists of the following five stages:

1. **Instruction Fetch (IF):** The processor fetches the instruction from memory.
2. **Instruction Decode (ID):** The instruction is decoded, and operands are identified.
3. **Execute (EX):** The required operation is performed using the ALU.
4. **Memory Access (MEM):** If needed, data is read from or written to memory.
5. **Write Back (WB):** The result is written back to the register file.

Each stage completes in one clock cycle. Once the pipeline is full, one instruction completes every cycle.

Pipeline Execution Example

Assume a 5-stage pipeline and three instructions:

1. ADD R1, R2, R3
2. SUB R4, R5, R6
3. AND R7, R8, R9

Cycle	ADD	SUB	AND
1	IF		
2	ID	IF	
3	EX	ID	IF
4	MEM	EX	ID
5	WB	MEM	EX
6		WB	MEM
7			WB

Each instruction moves through the stages in a staggered manner, completing one per cycle after the pipeline is filled.

Throughput and Latency

- Throughput increases, as one instruction is completed per cycle.
- Latency, or the time to execute one instruction end-to-end, remains the same, but total program execution time is reduced.

3.1. Pipeline Fundamentals

Pipelining increases instruction throughput by dividing the instruction execution process into a sequence of discrete stages, with each stage handled by dedicated hardware. This structure allows multiple instructions to be in different stages of execution at the same time, rather than waiting for one instruction to complete before starting the next.

Principle of Overlap

Each instruction passes through the same stages in the same order, and while one instruction is in a given stage, another instruction can enter the stage behind it. This overlapping of instruction execution maximises hardware utilisation and reduces idle time in functional units.

Ideal Pipeline Conditions

An ideal pipeline assumes:

- All instructions take the same number of stages.
- Each stage takes exactly one clock cycle.
- There are no data dependencies or resource conflicts.
- Instructions are continuously fed into the pipeline without stalls.

Under these ideal conditions:

- The latency for one instruction remains unchanged.
- The throughput becomes one instruction per cycle after filling the pipeline.
- The speedup approaches the number of pipeline stages.

Speedup (ideal) = Number of pipeline stages

Pipeline Performance Terms

- **Latency:** Time taken for a single instruction to pass through the pipeline.
- **Throughput:** Number of instructions completed per unit time.
- **Cycle Time:** Determined by the slowest stage; all stages must complete within one cycle.

Instruction Pipeline vs Data Pipeline

- **Instruction Pipeline:** Used in CPUs for overlapping instruction fetch, decode, execute, etc.
- **Data Pipeline:** Common in signal processors, where each stage performs part of a data transformation (e.g., FFT).

Hardware Requirements

- Separate hardware registers (pipeline registers) between each stage to hold intermediate results.
- Control logic to manage instruction flow and handle stalls or hazards.
- Bypass paths and hazard detection units in advanced pipelines.

3.2. Pipeline Stages

Instruction execution in a pipelined processor is divided into a sequence of stages, each performing a distinct function. These stages are connected through pipeline registers and operate in parallel, allowing multiple instructions to be processed concurrently.

A classic RISC pipeline typically consists of five stages:

1. Instruction Fetch (IF)

The instruction is fetched from memory using the address in the Program Counter (PC). The PC is then incremented to point to the next instruction.

Key operations:

- Read instruction from memory
- Update PC

2. Instruction Decode (ID)

The fetched instruction is decoded to determine the operation type and identify the source and destination registers. The required operands are read from the register file.

Key operations:

- Decode opcode and addressing mode
- Access register operands
- Sign-extend immediate values

3. Execute (EX)

The actual operation is performed in this stage. For arithmetic instructions, the ALU performs the computation. For branch instructions, the branch target address is calculated and condition evaluated.

Key operations:

- ALU operations
- Branch address calculation
- Comparison for branching

4. Memory Access (MEM)

This stage is used for instructions that require access to memory, such as load and store operations. For arithmetic or logic instructions, this stage is simply passed through.

Key operations:

- Read from or write to memory
- Use ALU result as memory address

5. Write Back (WB)

The result of computation or memory read is written back into the destination register. This is the final stage of instruction execution.

Key operations:

- Write result to register file
- Complete instruction execution

These stages form a pipeline where each stage processes a different instruction every clock cycle. Pipeline registers are placed between each stage to hold the intermediate data and control signals, enabling smooth handover of data between stages.

3.3. Pipeline Hazards

Pipeline hazards are conditions that prevent the next instruction in the pipeline from executing during its designated clock cycle. They reduce the efficiency of the pipeline and can lead to delays, known as pipeline stalls or bubbles.

Hazards are classified into three main types: data hazards, control hazards, and structural hazards.

1. Data Hazards

A data hazard occurs when instructions that exhibit data dependencies are scheduled too closely in the pipeline.

Types of Data Hazards:

- **Read After Write (RAW)** – True dependency

Instruction needs a value that a previous instruction has not yet written.

ADD R1, R2, R3 ; $R1 = R2 + R3$

SUB R4, R1, R5 ; depends on R1

- **Write After Read (WAR)** – Anti-dependency

A later instruction writes to a register before a previous one finishes reading it.

- **Write After Write (WAW)** – Output dependency

Two instructions write to the same register; the order of writes must be preserved.

Solutions:

- Data forwarding (bypassing)
- Register renaming
- Inserting pipeline stalls

2. Control Hazards

These arise from branch instructions that alter the program flow, causing the pipeline to fetch the wrong instructions.

Example:

BEQ R1, R2, LABEL

ADD R3, R4, R5 ; fetched before knowing branch outcome

Solutions:

- Branch prediction (static or dynamic)
- Branch delay slots
- Flushing incorrect instructions after misprediction

3. Structural Hazards

Structural hazards occur when two or more instructions require the same hardware resource at the same time and the hardware cannot support both requests.

Example:

Both instruction fetch and data access require memory, but there's only one memory unit.

Solutions:

- Duplicating hardware resources (e.g., separate instruction and data memory)
- Stalling one of the instructions

These hazards must be detected and handled by control logic within the processor to ensure correct execution while minimising performance loss.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
1	What is the main advantage of pipelining in processor design? A) Reduced clock speed B) Increased instruction latency C) Improved instruction throughput D) Simplified control logic
2	Which of the following is a typical stage in a RISC instruction pipeline? A) Instruction Scheduling B) Register Renaming C) Instruction Fetch D) Branch Prediction
3	What type of hazard occurs when two instructions require the same hardware resource simultaneously? A) Data hazard B) Control hazard C) Structural hazard D) Logical hazard

Fill in the Blank	
4	The _____ stage in a pipeline is responsible for performing arithmetic and logical operations.
5	_____ hazards occur due to dependencies between instructions that are too close together in the pipeline.
6	The time taken for a single instruction to pass through all pipeline stages is called _____.
True or False	
7	In a pipelined processor, multiple instructions can be in different stages of execution at the same time.
8	Pipeline hazards always result in incorrect program execution.
9	Branch prediction is a technique used to reduce control hazards in pipelines.
10	Once a pipeline is filled, one instruction completes every two clock cycles.

4. ARITHMETIC PIPELINE

An arithmetic pipeline is a specialised type of instruction pipeline designed to perform arithmetic operations, particularly those involving floating-point or complex multi-step calculations. The operation is broken down into sequential stages, with each stage executing a part of the computation. This type of pipeline is useful in applications that require repetitive and high-speed arithmetic, such as digital signal processing, graphics, and scientific computing.

Basic Structure

Arithmetic pipelines are typically structured to handle operations like addition, multiplication, and division in multiple stages:

- **Stage 1:** Operand Fetch
- **Stage 2:** Partial Computation (e.g., exponent alignment in floating-point add)
- **Stage 3:** Main Operation (e.g., multiplication of mantissas)
- **Stage 4:** Normalisation and Rounding
- **Stage 5:** Result Writeback

Each stage completes in one clock cycle, and multiple arithmetic operations can be in different stages at once.

Example: Floating-Point Addition Pipeline

Floating-point addition involves several steps:

1. **Align Exponents** – Shift the smaller operand's mantissa to match the larger exponent.
2. **Add Mantissas** – Perform the actual addition or subtraction.
3. **Normalise Result** – Adjust the result to fit the standard floating-point format.
4. **Round Result** – Apply rounding mode.
5. **Write Result** – Store back in the register or memory.

Each step is assigned to a stage, and as one operand is being rounded, another can be added in a previous stage.

Pipeline Diagram (Conceptual)

Clock Cycle	Stage 1 (Align)	Stage 2 (Add/Sub)	Stage 3 (Normalise)	Stage 4 (Round)	Stage 5 (Writeback)
-------------	--------------------	----------------------	------------------------	--------------------	------------------------

1	Instr 1				
2	Instr 2	Instr 1			
3	Instr 3	Instr 2	Instr 1		
...	...	...	...	...	...

Features and Use Cases

- Used extensively in floating-point units (FPUs).
- Enables high-speed arithmetic in matrix multiplication, DSP algorithms, and real-time computations.
- Often combined with data forwarding to handle dependencies efficiently.

4.1. Design and Implementation

The design and implementation of an arithmetic pipeline involve dividing a complex arithmetic operation into a series of smaller, sequential stages, each performing a part of the computation. The goal is to ensure that each stage takes approximately the same time, allowing smooth data flow and maximising throughput.

Stage Partitioning

An effective pipeline design starts by identifying independent steps in the arithmetic operation. These steps are then mapped to individual pipeline stages. For floating-point operations, the stages may include:

- Operand Fetch
- Exponent Comparison and Alignment
- Mantissa Operation
- Normalisation
- Rounding
- Result Storage

Each stage is separated by pipeline registers that hold intermediate results and control signals.

Hardware Components

Key elements involved in the implementation:

- **Arithmetic Logic Units (ALUs):** Perform integer or fixed-point operations.
- **Floating-Point Units (FPUs):** Specialised for floating-point operations.

- **Shifters and Comparators:** Used in alignment and normalisation.
- **Pipeline Registers:** Store data between stages.
- **Control Unit:** Coordinates stage transitions, handles hazards, and manages data dependencies.

Example: 4-Stage Floating-Point Adder Pipeline

Stage	Operation
Stage 1	Fetch operands and compare exponents
Stage 2	Align mantissas by shifting
Stage 3	Perform addition/subtraction of mantissas
Stage 4	Normalise and round result

Timing and Performance Considerations

- Each stage should ideally complete in one clock cycle.
- The overall cycle time is determined by the slowest stage.
- Balancing stages is critical to avoid bottlenecks.
- Use of data forwarding or stall control may be needed to manage dependencies.

Implementation Example

// Pseudo-code for pipelined floating-point addition

Stage1: Fetch operands A and B

Stage2: Align exponents by shifting mantissa

Stage3: Add mantissas

Stage4: Normalise result and write back

Instructions flow through stages like an assembly line. After the pipeline is filled, one result is produced every clock cycle.

This structure allows efficient hardware utilisation and high-speed arithmetic, especially in repetitive or parallelisable numerical tasks.

4.2. Examples and Use Cases

Arithmetic pipelines are widely used in systems where high-speed numerical computation is essential. These pipelines improve performance by executing parts of arithmetic operations in parallel stages, making them suitable for repetitive and latency-sensitive tasks.

Example 1: Floating-Point Addition Pipeline

In floating-point addition, multiple steps like exponent comparison, alignment, mantissa operation, normalisation, and rounding can be executed in a pipelined manner.

Operation Breakdown:

- Align exponents of two numbers
- Add or subtract mantissas
- Normalise the result
- Round and store

Each of these steps is assigned to a pipeline stage. Once the pipeline is filled, the adder can produce one result per clock cycle.

Application: Real-time physics calculations in game engines and simulations.

Example 2: Multiplication Pipeline in DSP

Digital Signal Processors (DSPs) use pipelined multipliers to process large arrays of data in real time.

```
// Multiply two vectors
for (int i = 0; i < N; i++)
    C[i] = A[i] * B[i];
```

With pipelined multipliers, multiple multiplication operations can be performed concurrently, greatly improving performance in filters, transforms, and convolution.

Application: Audio/video processing, radar signal analysis.

Example 3: Graphics Processing

Graphics Processing Units (GPUs) implement deep arithmetic pipelines to perform vector and matrix operations required for rendering, lighting, and transformation.

Use Case: 3D transformations using matrix multiplication:

```
result = projection_matrix × model_matrix × vertex_vector;
```

Pipelined ALUs in GPUs process many such operations in parallel across multiple data points.

Example 4: Scientific Simulations

Simulations involving physical models (e.g., weather, fluid dynamics) require fast computation of large floating-point datasets. Arithmetic pipelines accelerate operations like dot products, matrix

solves, and vector updates.

Use Case:

- Climate models
- Finite element analysis
- Molecular dynamics

Example 5: Machine Learning

Matrix operations in neural network training and inference—such as weighted sum and activation function computations—are accelerated using pipelined arithmetic units.

Use Case:

Multiply-and-accumulate (MAC) operations in deep learning accelerators.

Arithmetic pipelines are thus essential in any architecture designed for high-speed numeric computation, ranging from embedded systems to supercomputers.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
1	What is the primary purpose of an arithmetic pipeline? A) To reduce memory usage B) To perform high-speed arithmetic operations in stages C) To simplify instruction decoding D) To manage control hazards
2	Which of the following is typically a stage in a floating-point addition pipeline? A) Instruction Fetch B) Register Renaming C) Exponent Alignment D) Branch Prediction
3	Arithmetic pipelines are most commonly used in which type of applications? A) File management systems B) Real-time signal processing and scientific computing C) Operating system scheduling D) Database indexing

Fill in the Blank	
4	The _____ stage in an arithmetic pipeline adjusts the result to fit the standard floating-point format.
5	_____ and rounding are typically the final steps in a floating-point arithmetic pipeline.
6	Arithmetic pipelines improve performance by executing parts of a computation in _____ stages.
True or False	
7	Arithmetic pipelines are only used for integer operations.
8	Each stage in an arithmetic pipeline ideally completes in one clock cycle.
9	Floating-point operations like addition and multiplication can be broken into multiple pipeline stages.
10	Arithmetic pipelines are not suitable for repetitive numerical tasks.



5. INSTRUCTION PIPELINE

An instruction pipeline is a hardware implementation technique that allows overlapping execution of multiple instructions in a processor. It divides the instruction cycle into a series of discrete stages, each handled by a different part of the processor. As one instruction moves from one stage to the next, a new instruction enters the first stage, enabling parallelism and increasing instruction throughput.

This approach improves the overall execution rate by allowing several instructions to be processed simultaneously at different stages of completion.

Stages in a Typical Instruction Pipeline

A standard RISC processor uses a five-stage pipeline:

1. **Instruction Fetch (IF):** Retrieves the next instruction from memory using the program counter.
2. **Instruction Decode (ID):** Decodes the instruction and reads source operands from registers.
3. **Execute (EX):** Performs the required operation, such as an arithmetic calculation or branch address computation.
4. **Memory Access (MEM):** Reads from or writes to memory, if required.
5. **Write Back (WB):** Writes the result of the computation back to the destination register.

Each stage is separated by pipeline registers that hold intermediate results and control information.

Example Execution Flow

For the following instructions:

ADD R1, R2, R3

SUB R4, R5, R6

AND R7, R8, R9

The pipeline execution (with no hazards) proceeds as follows:

Cycle	Instruction 1	Instruction 2	Instruction 3
1	IF		
2	ID	IF	
3	EX	ID	IF
4	MEM	EX	ID
5	WB	MEM	EX

6		WB	MEM
7			WB

After the pipeline is filled, one instruction completes per clock cycle.

Key Characteristics

- **Increased Throughput:** The processor completes one instruction per cycle after the pipeline is full.
- **Same Latency:** The total time to execute a single instruction remains unchanged.
- **Instruction Overlap:** Multiple instructions are active at different stages at any given time.

Instruction pipelining forms the foundation of high-performance uniprocessor architectures and is widely used in modern microprocessors.

5.1. Instruction Fetch and Decode

The Instruction Fetch (IF) and Instruction Decode (ID) stages are the first two phases of an instruction pipeline. These stages prepare the instruction and its operands for execution by retrieving it from memory and interpreting its format.

Instruction Fetch (IF)

In this stage, the processor fetches the instruction to be executed from the memory using the Program Counter (PC), which holds the address of the next instruction.

Key operations:

- Read the instruction from memory at the address given by PC.
- Increment the PC to point to the next instruction ($PC = PC + 4$ for 32-bit instructions).
- Store the fetched instruction in the Instruction Register (IR) for use in the next stage.

The IF stage typically interfaces with the instruction cache to speed up access.

Instruction Decode (ID)

After fetching, the instruction must be decoded to determine what operation to perform and which operands are involved.

Key operations:

- Decode the opcode to identify the instruction type (e.g., arithmetic, branch, memory).
- Identify source and destination registers.

- Retrieve operand values from the register file.
- Sign-extend immediate values if required.
- Generate control signals needed by later stages.

This stage uses the instruction format (R-type, I-type, etc.) to parse fields like opcode, register addresses, and immediate values.

Example

For the instruction:

ADD R1, R2, R3

- IF stage reads this instruction from memory at address in PC.
- ID stage:
 - Decodes opcode for ADD.
 - Identifies R2 and R3 as source registers.
 - Fetches their values.
 - Prepares to write the result to R1.

These initial stages are critical in setting up the execution of instructions. Errors or delays here propagate downstream, affecting overall performance.

5.2. Execution and Write-back

The Execution (EX) and Write-back (WB) stages are the final functional phases in the instruction pipeline. After fetching and decoding, these stages perform the actual computation and store the result.

Execution (EX)

In the execution stage, the processor performs the operation specified by the instruction. This could involve arithmetic computation, logical operations, or address calculation for memory access.

Key operations:

- Perform arithmetic or logic operations using the Arithmetic Logic Unit (ALU).
- Calculate branch target addresses (for control instructions).
- Compute effective memory addresses (for load/store).
- Evaluate branch conditions.

This stage often includes multiplexers to select operands and forwarding logic to reduce data hazards.

Example (Arithmetic Instruction)

SUB R4, R2, R3

- ALU computes $R4 = R2 - R3$ using values obtained in the decode stage.
- The result is passed to the write-back stage via a pipeline register.

Write-back (WB)

In the write-back stage, the result of the execution (or memory access) is written into the destination register specified in the instruction.

Key operations:

- Update the register file with the result from ALU or memory.
- Ensure correct register is updated based on instruction type.

Write-back ensures that the computation result becomes available for future instructions.

Example

For:

ADD R1, R2, R3

- After EX: ALU computes $R2 + R3$.
- In WB: Result is written to register R1.

These two stages complete the instruction's lifecycle within the pipeline, returning the result to the register file and preparing the processor to execute the next instruction seamlessly.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
1	What is the role of the Instruction Fetch (IF) stage in a pipeline? A) Perform arithmetic operations B) Decode the instruction C) Fetch the instruction from memory D) Write the result to a register
2	Which stage is responsible for performing the actual computation in the instruction pipeline? A) Instruction Fetch

	B) Instruction Decode C) Execute D) Write-back
3	What does the Write-back (WB) stage do in a pipeline? A) Fetch the next instruction B) Store the result into the destination register C) Decode the instruction D) Access memory
Fill in the Blank	
4	The _____ stage decodes the instruction and identifies the source and destination registers.
5	The result of an arithmetic operation is written back to the register file in the _____ stage.
6	The _____ holds the address of the next instruction to be fetched.
True or False	
7	The Execute (EX) stage performs operations like addition, subtraction, and branch evaluation.
8	The Instruction Fetch stage is responsible for writing results to registers.
9	Pipeline registers are used to store intermediate results between stages.
10	Once the pipeline is filled, one instruction completes every two clock cycles.

6. VECTOR PROCESSING

Vector processing is a method of executing operations on entire vectors (arrays of data) instead of scalar values (individual elements). It is a form of data-level parallelism (DLP) where a single instruction operates on multiple data elements simultaneously. This approach is especially effective in applications involving large datasets and repetitive computations.

Vector Registers and Vector Instructions

In a vector processor, operands are stored in vector registers, which can hold multiple data elements. A vector instruction specifies an operation to be applied element-wise across all elements of the operand vectors.

Example: Vector Addition

```
// Scalar
for (int i = 0; i < 4; i++)
    C[i] = A[i] + B[i];

// Vector instruction (conceptual)
C = A + B;
```

Instead of executing four additions sequentially, a vector processor performs them in parallel using a single instruction.

Vector Processor Architecture

Key components:

- **Vector Registers:** Hold multiple data elements.
- **Vector ALU:** Performs arithmetic/logical operations on entire vectors.
- **Pipelined Functional Units:** Allow overlapping processing of elements.
- **Control Unit:** Issues and manages vector instructions.

Each vector instruction typically requires multiple cycles but completes multiple operations, resulting in high throughput.

Advantages

- High performance on array-based computations.
- Fewer instructions needed compared to scalar execution.

- Efficient use of memory bandwidth through block data access.
- Ideal for scientific computing, graphics, and machine learning.

Typical Applications

- Matrix multiplication
- Image filtering and processing
- Physics simulations
- Signal and audio processing

Vector processing remains a powerful technique in both general-purpose CPUs (via SIMD extensions like AVX or NEON) and specialised vector processors used in high-performance computing systems.

6.1. Vector Architecture

Vector architecture is designed to efficiently handle operations on one-dimensional arrays (vectors) of data. Unlike scalar architectures that process one data element per instruction, vector architectures execute the same operation on multiple data elements simultaneously, offering significant speedup for data-parallel workloads.

Key Components

1. Vector Registers

These are wide registers capable of holding an entire vector of data elements. A single vector register may store 64, 128, or more elements, depending on the implementation.

2. Vector Functional Units

Dedicated hardware units perform arithmetic or logical operations on vector elements. Operations may be pipelined to allow continuous processing of data elements.

3. Vector Load/Store Units

These manage memory access, allowing blocks of data to be moved between memory and vector registers in a single instruction. They often support stride-based access for processing matrix rows/columns efficiently.

4. Control Unit

Interprets vector instructions and manages control logic, such as instruction sequencing and element count tracking.

Types of Vector Instructions

- Vector-Vector Operations: Operate on two source vectors.
VADD V1, V2, V3 ; $V1 = V2 + V3$
- Vector-Scalar Operations: Operate between a vector and a scalar value.
VMUL V1, V2, #4 ; Multiply each element of V2 by 4
- Vector-Load/Store: Transfer blocks of data between memory and vector registers.
VLD V1, [R1] ; Load vector V1 from memory address in R1

Instruction Execution

Vector instructions are decomposed into element-wise micro-operations that are executed in a pipelined fashion. Once the pipeline is filled, a new vector element is processed each cycle, leading to high throughput.

Features

- Reduces the number of instructions required.
- Exploits data-level parallelism with minimal control overhead.
- Simplifies loop operations by processing all iterations in one instruction.

Vector architectures are foundational in high-performance computing systems and continue to influence modern SIMD-based designs in CPUs and GPUs.

6.2. Vector Instructions

Vector instructions operate on entire vectors rather than individual scalar data elements. A single vector instruction performs the same operation on each element of the source vectors, exploiting data-level parallelism for higher throughput and reduced instruction overhead.

General Format

A typical vector instruction has the format:

OPCODE DEST, SRC1, SRC2

Where:

- OPCODE specifies the operation (e.g., ADD, MUL)
- DEST is the destination vector register
- SRC1 and SRC2 are source vector registers (or scalar values)

Types of Vector Instructions

1. Vector-Vector Instructions

Operate element-wise on two source vectors.

VADD V1, V2, V3 ; $V1[i] = V2[i] + V3[i]$

VMUL V4, V5, V6 ; $V4[i] = V5[i] \times V6[i]$

Each pair of elements from V2 and V3 is added, and the result is stored in V1.

2. Vector-Scalar Instructions

Operate on a vector and a scalar value, applying the scalar to all elements of the vector.

VADD V1, V2, #10 ; $V1[i] = V2[i] + 10$

This reduces memory access and is efficient for broadcasting constants across data.

3. Vector Load and Store

Used to transfer blocks of data between memory and vector registers.

VLD V1, [R1] ; Load vector from memory address in R1 to V1

VST V2, [R2] ; Store contents of V2 to memory address in R2

These instructions may support stride-based access to handle non-contiguous memory layouts.

4. Vector Control Instructions

Manage the behaviour of vector processing.

- VSETVL: Sets the active vector length (used in RISC-V Vector Extension)
- VMASK: Enables or disables elements for conditional operations

Pipelined Execution

Each element operation flows through functional units like an arithmetic pipeline. While the first result is delayed by pipeline depth, subsequent results are produced each cycle (once the pipeline is full).

Example: Vector Addition

// $C[i] = A[i] + B[i]$, where $i = 0$ to 3

VLD V1, A_ADDR ; Load vector A into V1

VLD V2, B_ADDR ; Load vector B into V2

VADD V3, V1, V2 ; $V3[i] = V1[i] + V2[i]$

VST V3, C_ADDR ; Store result to C

This entire loop is replaced by four compact instructions, improving performance and reducing control complexity.

6.3. Applications of Vector Processing

Vector processing is widely applied in domains where the same operation must be performed on large sets of data. Its ability to exploit data-level parallelism makes it ideal for workloads involving arrays, matrices, signals, and images. This reduces execution time and improves computational efficiency.

1. Scientific Computing

Many scientific problems involve numerical simulations and modeling based on matrices and vectors. Vector processors efficiently execute repetitive operations like matrix multiplication, dot products, and numerical integration.

Examples:

- Solving systems of linear equations
- Finite element analysis
- Climate and weather simulation models

2. Signal Processing

Digital Signal Processing (DSP) tasks like filtering, transformation, and modulation rely on repetitive arithmetic operations across streams of input data. Vector units enable real-time performance in these applications.

Examples:

- Fast Fourier Transform (FFT)
- FIR/IIR filters
- Echo cancellation and audio enhancement

3. Graphics and Image Processing

Graphics applications involve transformations, lighting, rasterisation, and pixel manipulation, which require high-throughput vector operations on coordinate data, pixel arrays, and color channels.

Examples:

- 3D transformations using matrix-vector multiplication
- Edge detection and blurring in images
- Real-time shading and rendering pipelines in GPUs

4. Machine Learning and AI

Training and inference in machine learning models require matrix operations and activation functions over large datasets. Vector instructions are used to accelerate neural network computations.

Examples:

- Multiply-and-accumulate (MAC) operations in deep learning
- Batch normalisation and activation layers
- Tensor operations in frameworks like TensorFlow and PyTorch

5. Cryptography

Encryption algorithms involve arithmetic operations on blocks of data, which can be parallelised using vector instructions to enhance performance.

Examples:

- AES encryption using vectorised S-box lookups
- Hash functions using SIMD

6. Multimedia Applications

Vector processing improves the efficiency of audio and video encoding, decoding, compression, and playback by handling multiple samples or pixels per instruction.

Examples:

- MP3/AAC audio decoding
- JPEG/MP4 video compression
- Real-time streaming and transcoding

These applications demonstrate the importance of vector processing in achieving high performance across a wide range of modern computing systems—from mobile devices and desktops to supercomputers and AI accelerators.

SELF-ASSESSMENT QUESTIONS - 5

Multiple Choice Questions	
1	What is the main advantage of vector processing? A) Reduced instruction size B) Increased control complexity C) High throughput for data-parallel tasks D) Improved branching performance
2	Which component holds multiple data elements for vector operations? A) Scalar Register B) Instruction Cache C) Vector Register D) Control Unit
3	Which type of instruction applies the same operation to all elements of a vector? A) Scalar Instruction B) Vector-Scalar Instruction C) Vector-Vector Instruction D) Branch Instruction
Fill in the Blank	
4	Vector processing is a form of _____ parallelism, where a single instruction operates on multiple data elements.
5	The _____ unit in a vector processor performs arithmetic and logical operations on entire vectors.
6	Vector instructions reduce _____ overhead by replacing multiple scalar instructions with a single vector instruction.
True or False	
7	Vector instructions are executed sequentially on each data element.
8	Vector processing is ideal for applications like image filtering and matrix multiplication.
9	Vector load/store instructions can support stride-based memory access.
10	Vector processing is not supported in modern CPUs.

7. ARRAY PROCESSORS

An array processor is a parallel processing system composed of multiple processing elements (PEs) arranged in a regular structure, typically controlled by a single control unit. All PEs execute the same instruction simultaneously but operate on different data, following the SIMD (Single Instruction, Multiple Data) model. This architecture is highly efficient for workloads that involve applying the same operation to large datasets—such as vectors, matrices, or image pixels.

Each processing element in the array has its own local registers and possibly local memory, but all elements share a common instruction stream from a central control unit. The PEs are connected through an interconnection network that may use a mesh, ring, or crossbar topology, depending on design requirements.

Array processors are best suited for structured, regular computations with minimal branching. Their predictable control flow and uniform instruction execution make them ideal for applications in scientific computing, image processing, and signal filtering. Modern GPUs can be seen as evolved array processors, consisting of thousands of cores that process data in parallel with minimal control overhead.

7.1. SIMD vs MIMD

SIMD and MIMD represent two foundational models for parallel processing, each suited for different types of workloads.

In a SIMD (Single Instruction, Multiple Data) system, one control unit broadcasts a single instruction to all processing elements, which then execute that instruction on their respective data. This model is highly efficient for uniform, data-parallel tasks such as vector operations, image transformations, or matrix multiplication. SIMD systems are relatively simple to design and control, but they require that all processing units stay in sync, which can lead to inefficiencies if the data or operations are not uniform.

By contrast, MIMD (Multiple Instruction, Multiple Data) systems consist of independent processors, each capable of executing its own instruction stream on its own dataset. This model supports asynchronous execution, making it more flexible for general-purpose and task-parallel computing. MIMD architectures are found in multi-core CPUs and distributed systems, where each core or node may handle a different portion of a larger task, or even a different task altogether.

While SIMD offers better performance for highly regular, parallel tasks with identical operations, MIMD is better suited for irregular or diverse workloads where instructions and data vary across processors.

7.2. Use Cases and Performance

Array processors are designed to accelerate computations involving repetitive and uniform operations across large datasets. Their ability to apply a single instruction to many data elements simultaneously makes them ideal for structured, compute-intensive tasks.

One key application area is scientific computing, where array processors can efficiently perform matrix operations, solve differential equations, and run simulations involving physical models. These tasks often involve large arrays of numbers that undergo identical computations, such as additions or multiplications.

In image and signal processing, array processors are used for filtering, transformations, and compression. Each pixel or audio sample can be processed independently using the same algorithm, making SIMD execution both fast and power-efficient.

Graphics rendering in GPUs relies heavily on array-like execution units to perform shading, texture mapping, and geometric transformations. Likewise, machine learning models benefit from vector and matrix operations, especially in training and inference stages involving large tensors.

In terms of performance, array processors achieve high throughput when all processing elements are kept busy with uniform tasks. However, if the data is sparse, irregular, or requires conditional branching, performance may degrade due to idle units or control inefficiencies. Thus, workload suitability is a key factor in achieving optimal performance with array processors.

SELF-ASSESSMENT QUESTIONS - 6

Multiple Choice Questions	
1	What is the key characteristic of SIMD architecture? A) Each processor executes a different instruction B) All processors share a single data stream C) A single instruction is executed on multiple data elements D) Instructions are executed sequentially
2	Which architecture is more suitable for irregular workloads with diverse operations?

	A) SIMD B) MIMD C) VLIW D) Superscalar
3	Array processors are most effective when: A) Tasks require frequent branching B) Data is sparse and unpredictable C) Operations are uniform across large datasets D) Instructions vary across processors
Fill in the Blank	
4	In _____ architecture, each processor executes its own instruction stream on its own data.
5	Array processors are composed of multiple _____ elements arranged in a regular structure.
6	_____ processors are ideal for structured computations like matrix operations and image filtering.
True or False	
7	SIMD architecture is ideal for tasks with irregular control flow.
8	MIMD systems support asynchronous execution across processors.
9	Array processors are commonly used in graphics rendering and machine learning.
10	All processing elements in an array processor execute different instructions simultaneously.

8. SUMMARY

- In this unit, we explored the foundational techniques that enable high-performance computing through parallelism at the microarchitectural level. We began with an overview of Parallel Processing, examining its types—Instruction-Level, Data-Level, Task-Level, and Pipeline Parallelism—and understanding how they contribute to increased throughput and reduced execution time.
- We then studied Pipelining, a method of overlapping instruction execution across multiple stages, and analysed both Arithmetic Pipelines and Instruction Pipelines. These structures allow multiple instructions or operations to be processed simultaneously, improving efficiency in tasks like floating-point arithmetic and instruction execution.
- Next, we delved into Vector Processing, which applies operations to entire arrays of data using specialised vector instructions and registers. We learned how vector architectures and instructions reduce control overhead and accelerate computations in domains such as scientific simulations, graphics, and machine learning.
- Finally, we examined Array Processors, focusing on the SIMD vs MIMD models, and their use in structured, data-parallel workloads. These processors are essential in applications requiring uniform operations across large datasets, such as image processing and neural network inference.
- By the end of this unit, you should understand how pipelining and vector processing enhance computational speed and efficiency, and how these techniques are implemented in modern processors to support advanced applications across various domains.

9. GLOSSARY

Parallel Processing	- The simultaneous execution of multiple instructions or operations to improve system performance and reduce execution time.
Pipelining	- A technique that divides instruction execution into stages, allowing multiple instructions to be processed concurrently in different stages.
Pipeline Stages	- The sequential phases of instruction execution—typically Fetch, Decode, Execute, Memory Access, and Write Back.
Pipeline Hazards	- Conditions that disrupt the smooth execution of instructions in a pipeline, including data, control, and structural hazards.
Data Hazard	- A pipeline issue caused by dependencies between instructions, such as Read After Write (RAW), Write After Read (WAR), and Write After Write (WAW).
Control Hazard	- A disruption caused by branch instructions that change the flow of execution, potentially leading to incorrect instruction fetching.
Structural Hazard	- A conflict that arises when hardware resources are insufficient to support concurrent instruction execution.
Arithmetic Pipeline	- A specialised pipeline designed for high-speed arithmetic operations, often used in floating-point units and DSPs.
Instruction Pipeline	- A pipeline used in CPUs to overlap the execution of multiple instructions, improving throughput.
Vector Processing	- A method of executing operations on entire arrays (vectors) of data simultaneously, exploiting data-level parallelism.
Vector Architecture	- A processor design that includes vector registers and functional units to perform operations on multiple data elements in parallel.
Vector Instruction	- A single instruction that performs the same operation on each element of a vector, reducing control overhead.

Vector Register	-	A wide register capable of holding multiple data elements for vector operations.
SIMD (Single Instruction, Multiple Data)	-	A parallel computing model where one instruction is applied to multiple data elements simultaneously.
MIMD (Multiple Instruction, Multiple Data)	-	A parallel computing model where multiple processors execute different instructions on different data independently.
Array Processor	-	A parallel system with multiple processing elements executing the same instruction on different data, typically following the SIMD model.
Data-Level Parallelism (DLP)	-	Parallelism achieved by applying the same operation to multiple data elements simultaneously.
Instruction-Level Parallelism (ILP)	-	Parallelism achieved by executing multiple independent instructions at the same time.
Task-Level Parallelism (TLP)	-	Parallelism achieved by executing different tasks or threads concurrently on separate cores or processors.

10. TERMINAL QUESTIONS

1. What is the primary purpose of pipelining in processor design?
2. Explain the difference between instruction-level parallelism (ILP) and data-level parallelism (DLP).
3. Describe the five basic stages of a RISC instruction pipeline.
4. What are pipeline hazards, and how do they affect instruction execution?
5. How does data forwarding help resolve data hazards in a pipeline?
6. What is the role of vector registers in vector processing?
7. Compare SIMD and MIMD architectures in terms of instruction control and data handling.
8. What are the advantages of using arithmetic pipelines in floating-point operations?
9. How does vector processing improve performance in scientific computing applications?
10. What is stride-based memory access, and why is it useful in vector load/store operations?
11. Explain the concept of task-level parallelism and give an example of its application.
12. What are the challenges associated with parallel processing in multi-core systems?
13. How do control hazards arise in instruction pipelines, and what techniques are used to mitigate them?
14. Describe the structure and function of an array processor.
15. In what types of applications is vector processing most beneficial, and why?

11. ANSWERS

Self-Assessment Questions

Self-Assessment Questions – 1

1. C) Improve instruction throughput
2. C) Applying the same operation to multiple data elements
3. C) Task-level parallelism
4. Instruction-Level Parallelism (ILP)
5. Data-Level
6. synchronisation
7. True
8. False
9. False
10. True

Self-Assessment Questions - 2

1. C) Improved instruction throughput
2. C) Instruction Fetch
3. C) Structural hazard
4. Execute (EX)
5. Data
6. Latency
7. True
8. False
9. True
10. False

Self-Assessment Questions - 3

1. B) To perform high-speed arithmetic operations in stages
2. C) Exponent Alignment
3. B) Real-time signal processing and scientific computing
4. Normalisation
5. Normalisation

6. sequential
7. False
8. True
9. True
10. False

Self-Assessment Questions - 4

1. C) Fetch the instruction from memory
2. C) Execute
3. B) Store the result into the destination register
4. Instruction Decode (ID)
5. Write-back (WB)
6. Program Counter (PC)
7. True
8. False
9. True
10. False

Self-Assessment Questions - 5

1. C) High throughput for data-parallel tasks
2. C) Vector Register
3. C) Vector-Vector Instruction
4. data-level
5. Vector ALU
6. control
7. False
8. True
9. True
10. False

Self-Assessment Questions - 6

1. C) A single instruction is executed on multiple data elements
2. B) MIMD
3. C) Operations are uniform across large datasets

4. MIMD
5. processing
6. Array
7. False
8. True
9. True
10. False

Terminal Questions Answers

Answer 1: To increase the overall performance of the CPU by enabling the execution of multiple instructions in parallel during a single clock cycle.

(refer section 2.1 - Concept and Design Principles)

Answer 2: It allows the processor to execute instructions as soon as their operands are available, reducing idle time and improving instruction throughput.

(refer section 2.1 - Concept and Design Principles)

Answer 3: It eliminates false dependencies by assigning unique identifiers to destination registers, allowing parallel execution without conflicts.

(refer section 2.1 - Concept and Design Principles)

Answer 4: Superscalar uses dynamic hardware-based scheduling at runtime, while VLIW relies on compile-time scheduling by the compiler.

(refer section 5.1 -- Instruction Fetch and Decode)

Answer 5: The compiler detects parallelism, schedules instructions, and ensures correctness, making it essential for VLIW performance.

(refer section 3.2 - Pipeline Stages)

Answer 6: A VLIW instruction is a wide, fixed-length word containing multiple operations that are executed simultaneously by different functional units.

(refer section 3.2 - Pipeline Stages)

Answer 7: VLIW simplifies hardware and offers predictable execution, but depends on compiler efficiency and may increase code size.

(refer section 3.3 - Pipeline Hazards)

Answer 8: SIMD applies a single instruction to multiple data elements simultaneously using vector registers or lanes.

(refer section 6 - Vector Processing)

Answer 9: Image processing, such as applying filters to every pixel, benefits greatly from SIMD's parallel execution.

(refer section 6.3 - Applications of Vector Processing)

Answer 10: SIMD struggles with irregular control flow or data dependencies, leading to underutilised lanes and reduced efficiency.

(refer section 6.3 - Applications of Vector Processing)

Answer 11: Superscalar has high hardware complexity, VLIW has simpler hardware but compiler limitations, and SIMD scales well with wide data sets.

(refer section 5 - Instruction Pipeline)

Answer 12: ILP executes multiple independent instructions simultaneously; DLP executes the same instruction on multiple data elements.

(refer section 2.2 - Types of Parallelism)

Answer 13: SIMD is integrated via instruction sets like SSE, AVX, and NEON, enabling parallel operations on multiple data elements.

(refer section 6.3 - Applications of Vector Processing)

Answer 14: VLIW outperforms Superscalar in predictable, compile-time optimised workloads like DSP and embedded systems.

(refer section 3.3 - Pipeline Hazards)

Answer 15: SIMD reduces instruction fetch/decode overhead and performs operations on multiple data elements with a single instruction.

(refer section 6.3 - Applications of Vector Processing)

12. REFERENCES

BOOKS

1. Computer Architecture: A Quantitative Approach, John L. Hennessy, David A. Patterson, 6th Edition, 2017
2. Computer Organisation and Architecture: Designing for Performance, William Stallings, 11th Edition, 2018
3. Computer System Architecture, M. Morris Mano, 3rd Edition, 2006
4. Computer Organisation and Design: The Hardware/Software Interface, David A. Patterson, John L. Hennessy, 5th Edition, 2013
5. Digital Design and Computer Architecture, Sarah Harris, David Harris, 3rd Edition, 2021

E-REFERENCES

1. Logical and Shift Micro-operations Computer Organisation and Architecture, <https://care4you.in/logical-and-shift-micro-operations/>
2. Logic and Shift Micro Operations, <https://www.scribd.com/document/394026693/U-I-Logic-and-Shift-micro-operations>

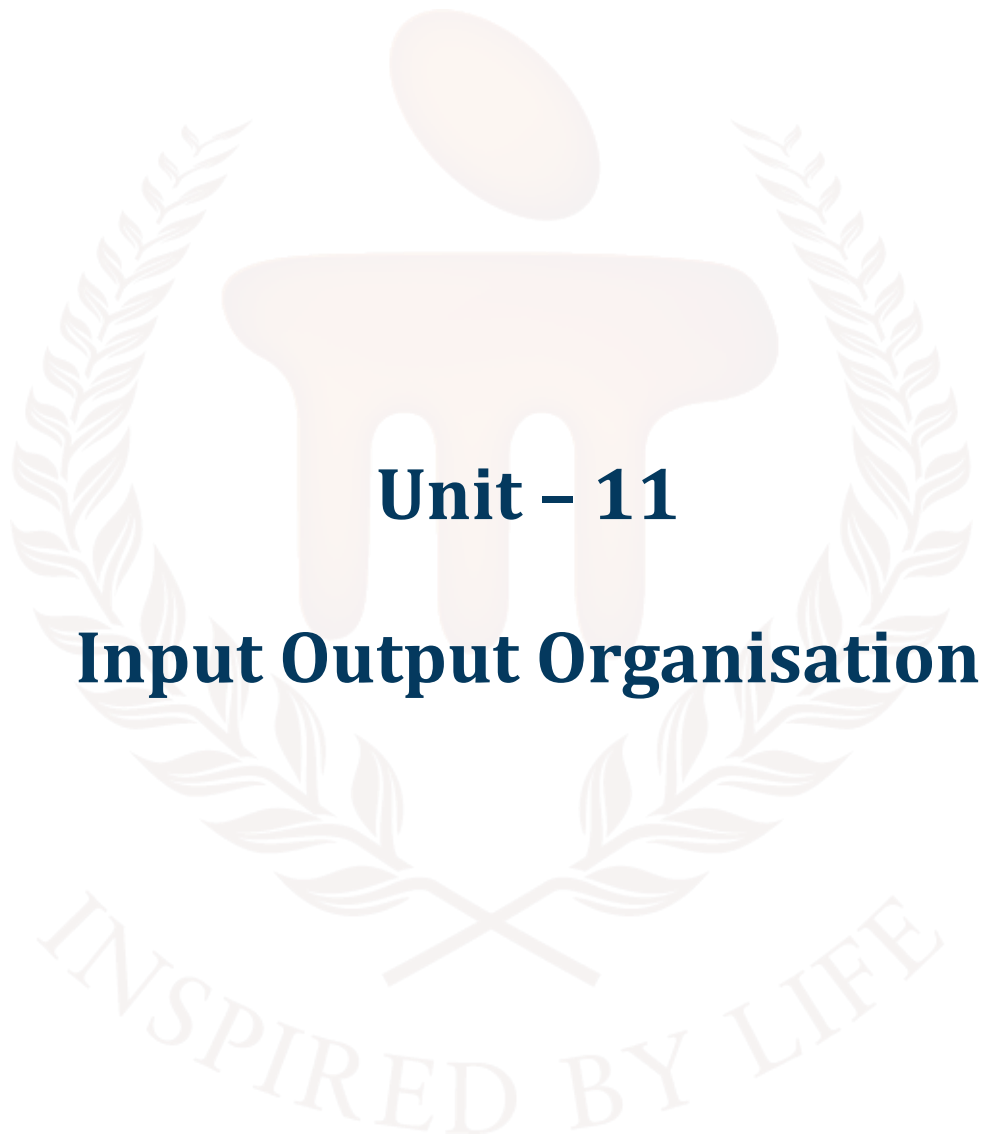
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 11

Input Output Organisation

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	I/O Interface	-	-	6 - 13
2.1	Purpose and Types of Interfaces	-	-	
2.2	I/O Mapped vs Memory Mapped I/O	1 i	-	
2.3	Interface Characteristics	-	1	
3	Asynchronous Data Transfer	-	-	14 - 20
3.1	Handshaking Method	-	-	
3.2	Strobe Control Method	2	-	
3.3	Asynchronous vs Synchronous Transfer	ii	2	
4	Modes of Transfer	-	-	21 - 27
4.1	Programmed I/O	-	-	
4.2	Interrupt-Driven I/O	3	-	
4.3	Direct Memory Access (DMA)	4	3	
5	Priority Interrupt	-	-	28 - 36
5.1	Interrupt Cycle	5, iii	-	
5.2	Priority Schemes	-	-	
5.3	Daisy-Chaining and Polling	iv	-	
5.4	Software vs Hardware Interrupts	v	4	
6	Summary	-	-	37
7	Glossary	-	-	38 - 39
8	Terminal Questions	-	-	40
9	Answers	-	-	41 - 44
10	References	-	-	45

1. INTRODUCTION

In the previous unit, we explored the internal mechanisms of a computer that contribute to high-performance execution, focusing on pipelining and vector processing. We examined how instruction pipelines improve throughput by overlapping instruction execution stages, and how arithmetic pipelines enable efficient processing of complex computations. Additionally, we delved into vector architectures and their applications in domains requiring data-level parallelism, such as graphics, scientific simulations, and machine learning. The unit also introduced array processors and compared SIMD and MIMD models, highlighting their roles in parallel computation.

Building on this understanding of internal execution efficiency, this unit shifts focus to a critical aspect of system performance—Input/Output (I/O) Organisation. Unlike CPU and memory operations that occur internally, I/O operations enable a computer system to interact with external devices, making them essential for real-world functionality.

In this unit, we will begin by examining the I/O interface, understanding how processors communicate with peripheral devices. We will explore asynchronous data transfer mechanisms, including strobe control and handshaking, which help coordinate data exchange between devices operating at different speeds. Next, we will study various modes of data transfer, such as programmed I/O, interrupt-driven I/O, and Direct Memory Access (DMA)—each with its own trade-offs in terms of performance and complexity.

A key topic in this unit is priority interrupt handling, which ensures that urgent I/O tasks are serviced efficiently. You will learn about interrupt cycles, priority schemes, and common techniques like daisy-chaining and polling used to manage multiple I/O requests.

By the end of this unit, you will have a foundational understanding of how I/O operations are organised and managed in computer systems. This knowledge is essential for designing systems that efficiently integrate and coordinate processing units with external devices such as keyboards, displays, sensors, and network interfaces.

1.1. Objectives

After studying this unit, you should be able to:

- Explain the role and purpose of I/O interfaces in computer systems.
- Differentiate between memory-mapped I/O and I/O-mapped (isolated) I/O techniques.
- Discuss asynchronous data transfer methods, including strobe control and handshaking.
- Compare different modes of data transfer, such as programmed I/O, interrupt-driven I/O, and DMA.
- Illustrate the functioning and sequence of operations in a typical interrupt cycle.
- Analyse various priority interrupt handling techniques, including daisy-chaining and polling.
- Distinguish between hardware and software interrupts and understand their respective use cases.



2. I/O INTERFACE

The I/O interface is a crucial component that connects the processor to external devices, enabling data exchange and control signals to flow between them. It manages communication, handles speed mismatches, and ensures proper coordination between the fast CPU and slower peripheral devices.

2.1. Purpose and Types of Interfaces

In a computer system, the Input/Output (I/O) interface acts as a bridge between the central processing unit (CPU) and the external peripheral devices. These devices—such as keyboards, mice, printers, displays, storage drives, and sensors—operate at different speeds and in different formats compared to the CPU. Hence, a standardised interface is required to ensure smooth, accurate, and reliable communication between them.

Purpose of an I/O Interface

The primary purposes of an I/O interface are:

- **Communication Mediation:** It enables data exchange between the CPU and external devices by translating internal CPU signals to a format the peripheral can understand, and vice versa.
- **Speed Matching:** External devices are typically much slower than the CPU. The interface ensures that communication happens without data loss by synchronising the speed differences.
- **Data Formatting:** Peripheral devices often operate with different data formats. The interface converts data between device-specific and CPU-compatible formats.
- **Device Control:** The I/O interface allows the CPU to send control commands (such as "start", "stop", or "read") to the device and to receive status information (such as "ready", "busy", or "error").
- **Interrupt Handling:** It provides mechanisms for devices to notify the CPU asynchronously via interrupt signals, improving efficiency by avoiding constant polling.
- **Address Decoding:** The interface helps identify which specific device the CPU is communicating with, often through address lines or port numbers.

Types of I/O Interfaces

I/O interfaces are broadly categorised based on how the CPU communicates with the peripheral devices:

1. Memory-Mapped I/O

In this technique, I/O devices are treated as if they are part of the memory. Each device is assigned a unique address in the memory space. The CPU uses standard memory access instructions (like LOAD/STORE) to communicate with the devices.

Features:

- Shared address space between memory and I/O.
- Allows use of all addressing and data manipulation instructions.
- Larger address space requirement.

Example: Writing data to a printer may involve storing a value at a specific memory address assigned to the printer.

2. I/O-Mapped I/O (Isolated I/O)

Here, I/O devices are assigned a separate address space. Special instructions such as IN and OUT are used to perform I/O operations.

Features:

- Separate address space for memory and I/O.
- Requires special control signals (e.g., I/O Read, I/O Write).
- Conserves memory address space.

Example: An OUT instruction may send a byte from the CPU to an output port like a display controller.

3. Serial and Parallel Interfaces

- **Serial Interface:** Sends and receives one bit at a time over a single communication line. It is cost-effective and used in long-distance communication (e.g., USB, RS-232).
- **Parallel Interface:** Sends multiple bits simultaneously across multiple lines. It is faster but costlier and suitable for short distances (e.g., internal data buses, printer ports).

4. Standard Bus Interfaces

These are industry-standard protocols that define how devices communicate with the system bus. Examples include:

- PCI (Peripheral Component Interconnect)
- USB (Universal Serial Bus)

- SATA (Serial ATA)

These interfaces simplify device integration and allow plug-and-play functionality.

5. Custom or Dedicated Interfaces

Some systems use custom-built interfaces for specific embedded applications (e.g., microcontroller peripherals like ADCs or temperature sensors). These are optimised for size, power, and cost.

The I/O interface plays a crucial role in ensuring seamless communication between the CPU and a diverse range of peripherals. By offering various types of interfacing techniques—from memory-mapped I/O to standardised serial/parallel protocols—the interface abstracts device-specific details from the CPU and provides a flexible, manageable framework for external communication.

2.2. I/O Mapped vs Memory Mapped I/O

To enable communication between the CPU and peripheral devices, two major interfacing techniques are used: Memory-Mapped I/O and I/O-Mapped (Isolated) I/O. Both approaches define how the processor accesses input/output devices but differ in terms of address space usage, instructions, and control signals.

Memory-Mapped I/O

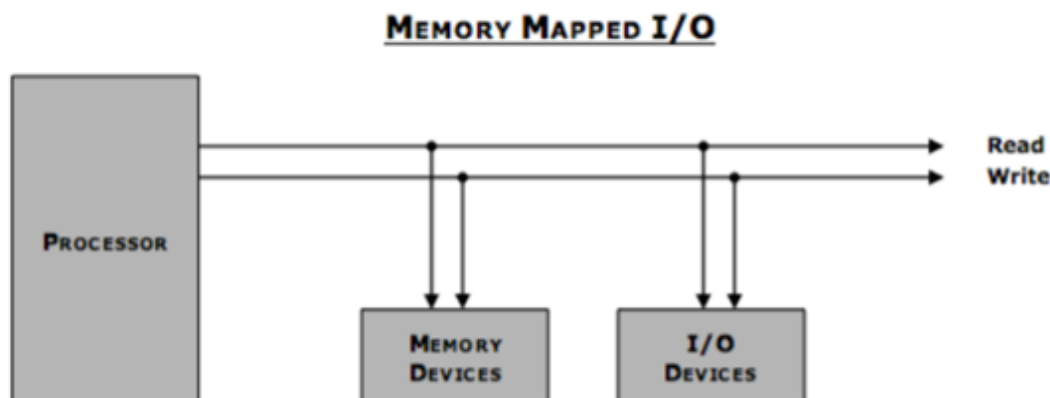


Figure 1: Memory Mapped I/O

In Memory-Mapped I/O, peripheral devices are assigned specific addresses within the same address space as regular memory. This means both memory and I/O devices share the same bus and are accessed using standard data transfer instructions like MOV, LOAD, and STORE.

Features:

- Uses standard memory instructions for I/O access.
- Each I/O device is given a unique memory address.
- Simplifies software design as I/O devices are treated like memory locations.
- Reduces the need for special I/O instructions.
- Consumes part of the memory address space, potentially limiting the memory available for programs.

Example:

To send data to an output device, the CPU may execute:

MOV 0x8000, R1 ; Move the value in R1 to the device at address 0x8000

I/O-Mapped I/O (Isolated I/O)

In I/O-Mapped I/O, also called Isolated I/O, the I/O devices occupy a separate address space from memory. The CPU uses special instructions, typically IN and OUT, to read from or write to I/O ports.

Features:

- Memory and I/O have distinct address spaces.
- Requires dedicated I/O instructions (IN, OUT).
- Conserves memory address space.
- Typically allows for faster decoding due to a smaller I/O address space.
- Only the accumulator or specific registers may be used in some architectures.

Example:

To send a byte to a port, the CPU might use:

OUT 0x20, R1 ; Send the contents of R1 to I/O port 0x20

Table 1: Comparison Table

Feature	Memory-Mapped I/O	I/O-Mapped I/O (Isolated)
Address Space	Shared with memory	Separate from memory
Instructions Used	Standard memory instructions	Special I/O instructions (IN, OUT)
Control Signals	Memory Read/Write	I/O Read/Write

Addressable Devices	Limited by memory space	Limited by I/O address range
Flexibility	Higher (can use any CPU register)	Lower (often restricted to accumulator)
Complexity	More complex decoding	Simpler decoding logic

Both methods serve the same purpose—enabling communication between the CPU and peripherals—but are chosen based on system requirements. Memory-mapped I/O is more flexible and powerful, while I/O-mapped I/O is simpler and more efficient for systems with limited I/O needs.

2.3. Interface Characteristics

An effective I/O interface must bridge the gap between the CPU and a wide variety of peripheral devices that differ in speed, data formats, and control mechanisms. To do this reliably and efficiently, an I/O interface is designed with several key characteristics that govern how data is transferred and how the CPU and devices interact.

1. Device Addressing

Every peripheral connected to the system must have a unique address or port number so the CPU can distinguish between multiple devices. Addressing may be:

- Memory-mapped, where device addresses are part of the memory space.
- I/O-mapped, where devices are assigned separate I/O port addresses.

2. Data Transfer Capability

The interface must support the required data width (e.g., 8-bit, 16-bit, 32-bit) and direction (input, output, or bidirectional). It should handle:

- Byte or word-sized transfers
- Full-duplex or half-duplex communication
- Buffered transfer, allowing data to be temporarily stored to reduce CPU wait time

3. Synchronisation

Since peripheral devices typically operate at slower and often irregular speeds compared to the CPU, synchronisation is essential. The interface must manage:

- Ready/Busy signals to inform the CPU when the device is ready

- Handshaking mechanisms for asynchronous communication
- Clock signals in synchronous systems to coordinate timing

4. Control and Status Signalling

An interface includes control lines to manage the operation of peripherals and status lines to report their condition. These include:

- Control signals: Read, Write, Enable, Interrupt Request
- Status flags: Ready, Busy, Error, Data Available

These allow the CPU to send commands and receive real-time feedback from the device.

5. Interrupt Handling Capability

Modern I/O interfaces often support interrupt-driven communication, where devices can notify the CPU when they need attention. This feature:

- Improves efficiency by avoiding constant polling
- Requires priority handling in multi-device environments
- Is essential for real-time and high-speed systems

6. Bus Compatibility

The interface must be compatible with the system bus, which includes:

- Data bus (for transferring data)
- Address bus (for selecting the target device)
- Control bus (for managing operations)

Proper coordination with the bus ensures correct timing and signal integrity.

7. Error Detection and Handling

Reliable communication requires mechanisms for detecting and correcting errors. Common features include:

- Parity bits or checksums
- Error flags for notifying the CPU of data transmission failures
- Retry mechanisms to resend corrupted data

8. Modularity and Expandability

A good interface allows easy addition or removal of devices without extensive changes to the system.

This is especially important in modern systems that support plug-and-play functionality.

I/O interface characteristics define how efficiently and reliably a peripheral device can communicate with the CPU. By addressing factors like synchronisation, control, interrupt handling, and error management, these characteristics ensure seamless integration of diverse hardware components into a unified computing system.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which of the following is a feature of memory-mapped I/O? a) Uses special I/O instructions b) Shares address space with memory c) Requires separate control signals d) Limited to accumulator register
2	In I/O-mapped I/O, communication with devices is done using: a) LOAD and STORE instructions b) MOV and ADD instructions c) IN and OUT instructions d) PUSH and POP instructions
3	Which interface type is best suited for long-distance communication? a) Parallel Interface b) Serial Interface c) Memory-Mapped Interface d) Custom Interface
4	What is the role of address decoding in an I/O interface? a) Formatting data b) Matching device speed c) Identifying the target device d) Controlling data flow
Fill in the Blank	
5	In memory-mapped I/O, I/O devices are assigned addresses within the _____ space.
6	The _____ interface sends multiple bits simultaneously across multiple lines.
7	_____ signals are used to manage operations like read, write, and enable in I/O interfaces.

8	USB and PCI are examples of _____ bus interfaces.
True or False	
9	I/O-mapped I/O allows the use of all CPU registers for data transfer.
10	Serial interfaces are more cost-effective than parallel interfaces for long-distance communication.
11	An I/O interface helps in synchronising the speed between CPU and peripheral devices.
12	Memory-mapped I/O conserves memory address space.



3. ASYNCHRONOUS DATA TRANSFER

In asynchronous data transfer, data is exchanged between devices that do not share a common clock. Instead, control signals are used to coordinate communication, ensuring that both sender and receiver are ready before data is transmitted. This method is essential for connecting high-speed processors with slower or independently timed peripherals.

3.1. Handshaking Method

In computer systems, devices often operate at different speeds, and not all components share the same clock. To manage communication between such unsynchronised devices, asynchronous data transfer is used. Unlike synchronous systems where a common clock controls timing, asynchronous systems use control signals to coordinate data transfers.

One of the most reliable and widely used asynchronous techniques is the handshaking method.

What is Handshaking?

Handshaking is a protocol in which the source and destination devices exchange control signals before transferring data. This ensures that both ends are ready for communication and prevents data loss or corruption. It provides synchronisation without requiring a shared clock.

Basic Principle of Handshaking

The method involves two key control signals:

- Request (or Data Valid) — sent by the sender to indicate that data is available and ready to be transferred.
- Acknowledge — sent by the receiver to confirm that it is ready to accept the data or that the data has been received successfully.

This two-way signalling creates a handshake similar to two people agreeing to pass an object—one offers, the other accepts.

Types of Handshaking

There are two common types:

1. Source-Initiated Handshaking

- The source (sender) initiates the transfer.

- It places data on the bus and activates the Request signal.
- The destination (receiver) reads the data and responds with an Acknowledge signal.
- Once the source detects the acknowledge, it removes the data and resets the Request signal.

Timing Diagram:

Source: [Data Available] -- Request ↑

Receiver: -- Acknowledge ↑

Source: -- Request ↓

Receiver: -- Acknowledge ↓

2. Destination-Initiated Handshaking

- The destination (receiver) initiates the data transfer.
- It sends a Request signal indicating it is ready to receive data.
- The source places data on the bus and responds with Acknowledge.
- The receiver then reads the data and resets the Request.

This type is used when the receiver controls the pace of data flow, such as in slow or power-constrained devices.

Advantages of Handshaking

- **Clock Independence:** No need for a shared clock between devices.
- **Speed Flexibility:** Devices of different speeds can communicate safely.
- **Data Integrity:** Ensures that data is transferred only when both sender and receiver are ready.
- **Error Reduction:** Helps avoid data corruption due to mismatched timing.

Disadvantages

- **Slower Data Rate:** Compared to synchronous methods, handshaking can introduce delay due to waiting for acknowledgments.
- **Increased Complexity:** Additional control signals and logic are needed.

Applications

Handshaking is widely used in:

- Microprocessor-to-peripheral communication

- Serial communication protocols (e.g., UART, RS-232)
- Bus systems like I²C and USB
- Embedded systems where low-speed peripherals need coordination with high-speed processors

The handshaking method is a robust solution for managing asynchronous data transfers in systems with diverse devices. By using simple request-acknowledge signaling, it ensures safe, reliable, and controlled communication—even without a shared clock.

3.2. Strobe Control Method

The strobe control method is a type of asynchronous data transfer technique used to synchronise communication between the CPU and peripheral devices. It uses a single control signal called a strobe to inform the receiving device when valid data is available on the data bus. Unlike handshaking, which involves two-way communication, the strobe method is unidirectional and simpler, but also less flexible.

Working Principle

In the strobe method, data is transferred in two phases:

1. Data Placement:

The sender (either CPU or I/O device) places data on the data bus.

2. Strobe Signal Activation:

The sender then activates the strobe signal, indicating to the receiver that valid data is present.

After a short, predefined delay, the sender deactivates the strobe signal, and the data is removed from the bus. The receiver must read the data while the strobe signal is active.

Types of Strobe Control

There are two variations, depending on who initiates the transfer:

1. Source-Initiated Strobe

- The source (sender) places the data on the bus.
- Then it activates the strobe signal.
- The destination (receiver) reads the data upon detecting the strobe signal.

This is typically used when the CPU sends data to an output device.

2. Destination-Initiated Strobe

- The destination device activates the strobe signal to request data.
- The source detects the strobe and places data on the bus.

This method is common in systems where the receiver controls the timing, such as input devices reading from a buffer.

Timing Diagram (Source-Initiated)

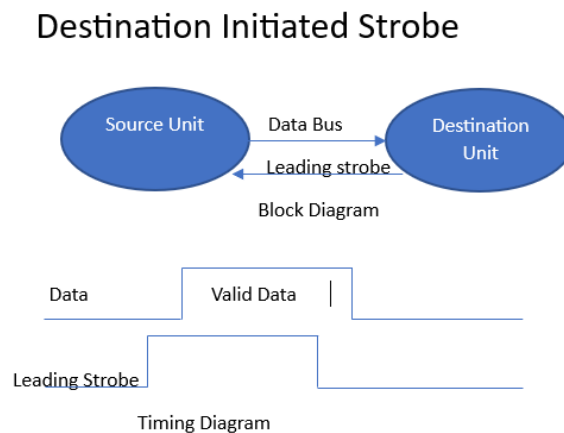


Figure 2: Destination Initiated Strobe

Advantages

- Simple implementation using only one control line.
- Lower hardware cost compared to handshaking.
- Suitable for basic or low-speed I/O operations.

Disadvantages

- No feedback mechanism to confirm whether the data was received.
- Timing must be carefully managed to avoid data loss.
- Not suitable for high-reliability systems or when devices operate at significantly different speeds.

Applications

- Simple microcontroller-based systems
- Basic communication between CPU and memory-mapped I/O devices
- Early computer systems and legacy buses

The strobe control method offers a straightforward way to transfer data asynchronously using a single control line. While it's easy to implement, the lack of acknowledgment from the receiver makes it less reliable than handshaking, particularly in systems where precise timing and data integrity are critical.

3.3. Asynchronous vs Synchronous Transfer

Data transfer in a computer system can be broadly categorised into synchronous and asynchronous methods, based on how the communication between devices is timed. Understanding the difference between these two techniques is essential for designing efficient and reliable I/O systems.

Synchronous Data Transfer

In synchronous transfer, both the sender and receiver operate in coordination with a common clock signal. Data is transmitted at predefined clock intervals, and both devices are synchronised to the same timing reference.

Characteristics:

- Uses a shared clock between devices.
- Data is transferred at regular intervals.
- Faster and more efficient in high-speed systems.
- Requires both sender and receiver to operate at compatible speeds.
- Simpler control logic due to predictable timing.

Example Use Cases:

- Memory-to-CPU communication
- High-speed buses (e.g., SDRAM, PCIe)
- Serial protocols like SPI (Serial Peripheral Interface)

Asynchronous Data Transfer

In asynchronous transfer, there is no shared clock between the sender and receiver. Instead, control signals such as handshaking or strobe signals are used to indicate when data is valid or when a device is ready to send/receive data.

Characteristics:

- Independent clocks or no clocks at all.
- Transfer timing is controlled by control signals (e.g., Request/Acknowledge, Strobe).

- Supports communication between devices with different speeds.
- More flexible but typically slower due to coordination overhead.
- Additional logic is needed to handle timing and synchronisation.

Example Use Cases:

- Communication with external I/O devices
- UART/RS-232 serial communication
- Low-speed embedded systems and sensors

Table 2: Comparison Table

Feature	Synchronous Transfer	Asynchronous Transfer
Clock Requirement	Shared/common clock	No common clock
Speed	Typically faster	Slower due to control signaling
Complexity	Less complex timing	Requires more control logic
Flexibility	Less flexible	Highly flexible
Device Compatibility	Devices must match clock speeds	Devices can operate at different speeds
Examples	Memory buses, SPI	UART, USB, Handshaking I/O

The choice between synchronous and asynchronous transfer depends on system requirements such as speed, device compatibility, and design complexity. While synchronous methods offer higher performance in tightly coupled systems, asynchronous techniques are better suited for environments where devices operate independently or at varying speeds.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
1	Which control signals are used in the handshaking method? a) Start and Stop b) Read and Write c) Request and Acknowledge d) Enable and Disable
2	In the strobe control method, how many control signals are used? a) Two b) One

	c) Three d) None
3	Which of the following is a disadvantage of asynchronous data transfer? a) Requires a shared clock b) Less flexible c) Slower due to control signaling d) Cannot support different device speeds
4	Which data transfer method is better suited for devices operating at different speeds? a) Synchronous b) Asynchronous c) Parallel d) Serial
Fill in the Blank	
5	In asynchronous transfer, devices do not share a common _____.
6	The strobe control method is _____ in direction and uses a single control signal.
7	Handshaking ensures data integrity by confirming that both sender and receiver are _____.
8	Asynchronous transfer is more _____ than synchronous transfer but generally slower.
True or False	
9	Handshaking requires a shared clock between sender and receiver.
10	Strobe control provides feedback from the receiver to the sender.
11	Asynchronous transfer supports communication between devices with different operating speeds.
12	Synchronous transfer is typically faster than asynchronous transfer.

4. MODES OF TRANSFER

Modes of transfer define how data moves between the CPU and I/O devices. Depending on system requirements and device speeds, different methods—such as Programmed I/O, Interrupt-Driven I/O, and Direct Memory Access (DMA)—are used to balance efficiency, speed, and processor involvement in I/O operations.

4.1. Programmed I/O

In a computer system, modes of data transfer determine how data moves between the CPU and I/O devices. One of the simplest and most common methods is Programmed I/O, where the CPU actively controls and monitors the data transfer process by executing explicit instructions.

What is Programmed I/O?

Programmed I/O is a method in which the CPU is responsible for initiating, controlling, and completing every data transfer operation with an I/O device. The processor communicates with the device by reading and writing to I/O registers, usually in a continuous polling loop.

How It Works

1. The CPU checks the status register of the I/O device to determine if it's ready (e.g., input is available or output buffer is empty).
2. If the device is not ready, the CPU keeps checking (polling).
3. When the device becomes ready, the CPU transfers data (read/write) to/from the I/O device.
4. The CPU continues program execution after the transfer is complete.

Illustrative Example

For an input device:

```
WAIT: IN STATUS_REGISTER ; Read status
      CMP STATUS, READY  ; Compare with ready signal
      JNE WAIT           ; If not ready, loop
      IN DATA_REGISTER  ; Read data when ready
```

The CPU is occupied during this entire process, continually checking the device until it is ready.

Characteristics

- CPU-Centric: The processor is fully in charge of I/O, which increases CPU workload.
- Polling-Based: The CPU repeatedly checks device status (no interrupts).
- Simple Implementation: Easy to design and program.
- Suitable for Simple or Low-Speed Devices.

Advantages

- Straightforward logic, especially in embedded or small systems.
- No complex hardware or interrupt mechanisms required.
- Useful when data transfer is infrequent or predictable.

Disadvantages

- Inefficient: The CPU wastes time polling, reducing overall system performance.
- Not scalable for systems with many devices or high-speed I/O.
- CPU Idle Time: The processor remains idle during waiting, even if it could perform other tasks.

Applications

- Basic microcontroller systems.
- Low-speed peripherals like keyboards, sensors, or switches.
- Educational systems for understanding I/O concepts.

Programmed I/O offers a simple but inefficient method for data transfer, especially when the CPU could be used for other tasks. It is suitable for systems where hardware simplicity is more important than performance, or when dealing with low-frequency I/O operations.

4.2. Interrupt-Driven I/O

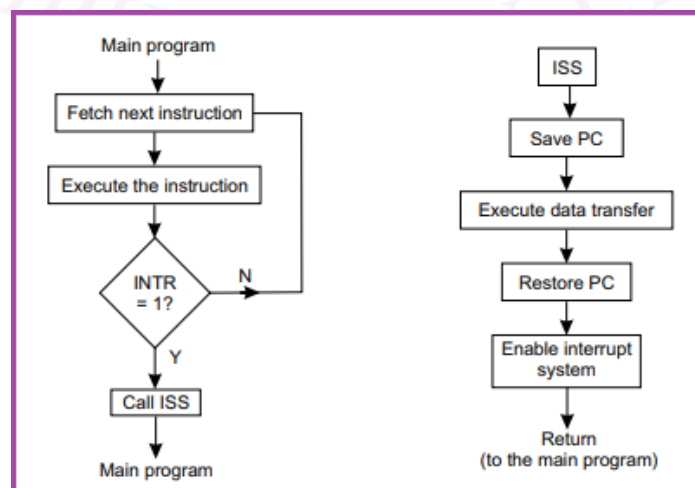


Figure 3: Interrupt Driven I/O

Interrupt-Driven I/O is a more efficient alternative to Programmed I/O, designed to minimise CPU idle time. Instead of continuously polling the device status, the CPU is notified by an interrupt signal whenever the I/O device is ready for communication. This allows the CPU to perform other tasks and only respond to I/O when necessary.

How It Works

1. The CPU initiates an I/O operation and continues with other instructions.
2. When the I/O device becomes ready (e.g., input received or output buffer available), it sends an interrupt signal to the CPU.
3. The CPU suspends its current execution and switches to a special routine called the Interrupt Service Routine (ISR).
4. The ISR handles the data transfer (read or write), then returns control to the original program.

Example

- CPU issues a request to read data from a keyboard.
- The keyboard sends an interrupt when a key is pressed.
- The CPU pauses its current task, reads the data from the keyboard, and then resumes its previous execution.

Key Characteristics

- Event-Driven: CPU responds only when an I/O event occurs.
- Efficient Use of CPU: No need for constant polling.
- Requires Interrupt Handling Logic: Includes interrupt vectors, priorities, and service routines.
- Faster Response than Programmed I/O in many cases.

Advantages

- Reduces CPU Idle Time: Frees the CPU to execute other tasks while waiting for I/O.
- Improved Efficiency: Particularly effective in multitasking systems.
- Better Performance in systems with frequent or time-sensitive I/O operations.

Disadvantages

- Requires complex interrupt control hardware and software.
- Interrupt Overhead: Context switching and handling may introduce small delays.
- Improper handling can lead to interrupt conflicts or missed signals.

Applications

- Keyboards, mice, and input devices.
- Timers, real-time clocks.
- High-speed communication interfaces (e.g., network cards, serial ports).
- Operating systems that support multitasking.

Interrupt-Driven I/O provides a more responsive and CPU-efficient mechanism for handling I/O operations. By allowing the processor to focus on other tasks until an I/O device signals readiness, this method significantly improves overall system performance—especially in environments with multiple devices or real-time requirements.

4.3. Direct Memory Access (DMA)

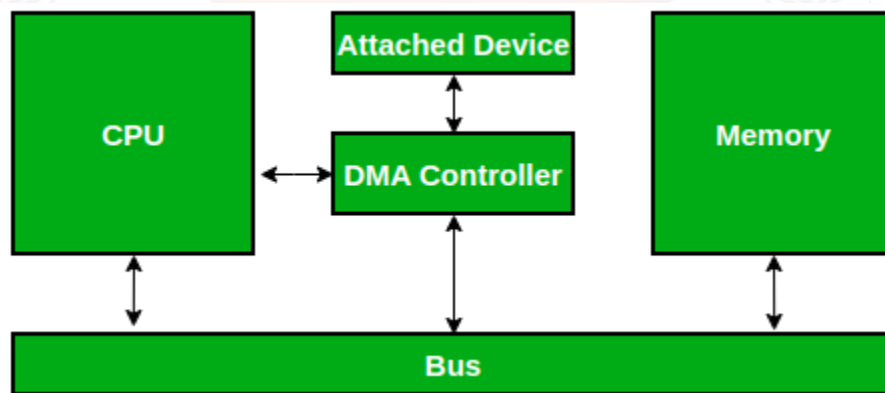


Figure 4: Direct Memory Access

Direct Memory Access (DMA) is an advanced data transfer technique that allows peripheral devices to directly read from or write to main memory without continuous CPU intervention. This significantly improves system efficiency, especially in high-speed data transfer applications, by offloading the CPU from routine I/O operations.

How DMA Works

1. The CPU initiates the DMA by providing the DMA controller with:
 - The memory address where data should be read or written.
 - The I/O device address.
 - The number of bytes to transfer.
2. Once setup is complete, the CPU releases control of the system buses.
3. The DMA controller handles the data transfer directly between the memory and the I/O device.
4. When the transfer is complete, the DMA controller sends an interrupt to notify the CPU.

DMA Components

- **DMA Controller (DMAC):** A dedicated hardware component that manages the DMA process.
- **Control Registers:** Store address, transfer count, and control signals.
- **Bus Arbitration Logic:** Allows the DMA controller to take control of the system bus from the CPU when needed.

Types of DMA Transfer Modes

1. Burst Mode (Block Transfer):
 - Transfers a block of data all at once.
 - CPU is halted during the entire transfer.
2. Cycle Stealing Mode:
 - DMA takes control of the bus for one cycle at a time.
 - CPU and DMA alternate access to the bus.
3. Transparent Mode:
 - DMA transfers occur only when the CPU is not using the system bus.
 - No noticeable performance impact on the CPU.

Advantages

- **High-Speed Data Transfer:** Ideal for large blocks of data (e.g., disk I/O, audio/video streams).
- **CPU Offloading:** Frees the CPU to perform other tasks during data transfer.
- **Efficient for Repetitive Tasks:** Reduces the overhead of instruction-based I/O operations.

Disadvantages

- **Increased Hardware Complexity:** Requires additional DMA controller circuitry.
- **Potential Bus Conflicts:** Needs bus arbitration to avoid clashes between CPU and DMA controller.
- **Less Control:** The CPU cedes temporary control of the bus during DMA operations.

Applications

- Hard disk data transfer
- Network interface cards (NICs)
- Sound and video cards
- Memory-to-memory copying in embedded systems

- Industrial control systems requiring high-speed I/O

DMA is an essential technique in modern computing systems for handling large and high-speed data transfers. By allowing peripherals to bypass the CPU and directly access memory, DMA greatly improves system performance, reduces latency, and ensures efficient bus utilisation.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
1	In Programmed I/O, the CPU: a) Waits for an interrupt b) Transfers data directly to memory c) Continuously checks device status d) Uses DMA controller
2	Which of the following is a key advantage of Interrupt-Driven I/O? a) Simple implementation b) No need for interrupt handling logic c) Reduces CPU idle time d) Requires constant polling
3	What component manages data transfer in Direct Memory Access (DMA)? a) CPU b) ALU c) DMA Controller d) Cache
4	Which DMA mode allows data transfer only when the CPU is not using the bus? a) Burst Mode b) Cycle Stealing c) Transparent Mode d) Interrupt Mode
Fill in the Blank	
5	In _____ I/O, the CPU is fully responsible for initiating and completing data transfers.
6	Interrupt-Driven I/O uses an _____ to notify the CPU when a device is ready.
7	DMA improves performance by allowing peripherals to access _____ directly.

8	In Cycle Stealing mode, the DMA controller takes control of the bus for _____ cycle(s) at a time.
True or False	
9	Programmed I/O is efficient for high-speed data transfer.
10	Interrupt-Driven I/O allows the CPU to perform other tasks while waiting for I/O.
11	DMA transfers always require CPU intervention during data movement.
12	Burst Mode DMA halts the CPU during the entire data transfer.



5. PRIORITY INTERRUPT

When multiple devices request the CPU's attention simultaneously, a priority interrupt system ensures that the most important request is handled first. This mechanism assigns priority levels to interrupts, allowing the CPU to respond to critical tasks promptly while temporarily deferring less urgent ones.

5.1. Interrupt Cycle

In systems with multiple I/O devices, several interrupts may occur simultaneously or in quick succession. To manage this efficiently, a priority interrupt system is used to determine which interrupt should be serviced first. This ensures orderly and predictable handling of multiple interrupt requests based on their urgency or importance.

Before understanding how priorities are assigned, it's essential to understand the Interrupt Cycle, which outlines how the CPU handles an interrupt from start to finish.

What is an Interrupt Cycle?

An Interrupt Cycle is a sequence of operations the CPU performs when it temporarily pauses the current program to service an interrupt request. It ensures that the current state of the processor is preserved, the appropriate Interrupt Service Routine (ISR) is executed, and the CPU resumes normal operation afterward.

Steps in the Interrupt Cycle

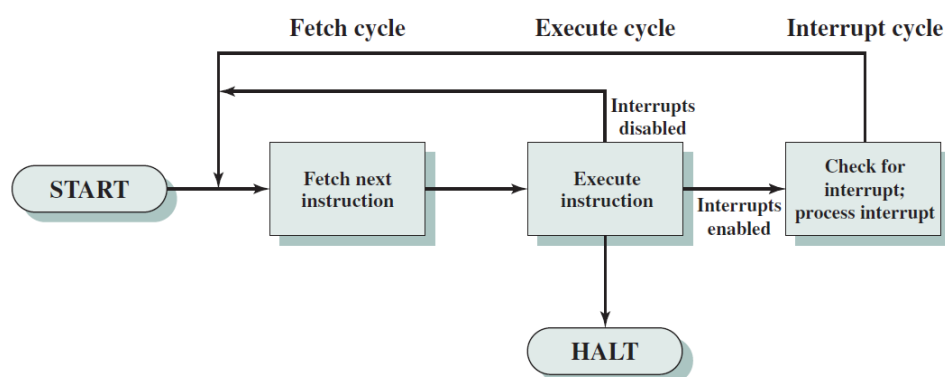


Figure 5: Interrupt Cycle

1. Interrupt Detection

- The CPU checks for interrupt signals after completing the execution of each instruction.

- If an interrupt request (IRQ) line is active and interrupts are enabled, the CPU begins the interrupt cycle.

2. Save Program State

- The CPU saves the Program Counter (PC) and possibly other registers to a stack or memory.
- This ensures that the interrupted program can resume from the exact point it was paused.

3. Identify the Interrupt Source

- If multiple devices can generate interrupts, the system must identify the highest-priority source.
- This may involve polling, priority encoders, or vectored interrupts.

4. Disable Further Interrupts (Optional)

- Some systems disable further interrupts temporarily to prevent nesting unless nested interrupts are supported.

5. Fetch and Execute the ISR

- The Interrupt Service Routine (or interrupt handler) corresponding to the device is fetched and executed.
- This routine handles the required I/O operation or event.

6. Restore Program State

- After the ISR completes, the CPU restores the saved register values and the PC.
- Execution of the original program resumes as if it was never interrupted.

Table 3: Interrupt Cycle vs Instruction Cycle

Feature	Instruction Cycle	Interrupt Cycle
Purpose	Execute the next instruction	Handle an external/internal request
Triggered By	Program counter	Interrupt signal (IRQ)
Regularity	Continuous	Occasional (event-driven)
Affects Program Flow	Follows normal sequence	Temporarily suspends program

The Interrupt Cycle is critical for enabling responsive, real-time interaction with external devices. It allows the CPU to efficiently handle urgent tasks like user input, hardware signals, or timer events, without losing progress on the currently running program.

5.2. Priority Schemes

In systems with multiple interrupt-generating devices, it's possible for two or more devices to request service at the same time. To determine which interrupt should be serviced first, priority schemes are implemented. These schemes define the order of precedence among different interrupt sources, ensuring that the most critical task gets CPU attention first.

Why Priority is Needed

Without a priority mechanism, the CPU might service interrupts in a random or first-come-first-served manner, which could delay important operations. A well-defined priority scheme:

- Improves system responsiveness to critical events.
- Prevents interrupt starvation of high-priority tasks.
- Allows nested interrupts, where a higher-priority interrupt can pre-empt a lower-priority one.

Types of Priority Schemes

1. Fixed Priority Scheme

Each device is assigned a static priority level. The CPU always services the highest-priority pending request.

- Example: Device A has higher priority than Device B. If both request interrupts, Device A is always serviced first.

Advantages:

- Simple to implement.
- Deterministic behaviour.

Disadvantages:

- Lower-priority devices may suffer from starvation if higher-priority devices interrupt frequently.

2. Rotating (or Round-Robin) Priority

Priority rotates among devices after each interrupt is serviced. The device that was just serviced moves to the lowest priority, and others move up.

- Ensures fairness among devices.
- Reduces the chance of any device being indefinitely delayed.

Example: After Device B is serviced, its priority is lowered, and the next interrupt (even from a lower-priority device) is handled next.

3. Dynamic Priority Scheme

Priority levels can change based on conditions, such as device activity, urgency, or system state.

- Often used in real-time systems.
- Priorities may be adjusted based on task deadlines, device load, or error conditions.

Example: A sensor generating emergency alerts may temporarily be assigned higher priority during specific conditions.

4. Software-Controlled Priority

The software or operating system determines which interrupt to service first by checking interrupt flags and status registers.

- Offers flexibility and customisation.
- Requires more CPU overhead to evaluate priority in software.

Hardware Implementation Tools

- Priority Encoder: Hardware circuit that outputs the highest-priority active input.
- Interrupt Vector Table: A memory table with addresses of ISRs arranged according to priority.

Priority schemes are essential in managing multiple simultaneous interrupt requests. Whether fixed or dynamic, a well-designed priority mechanism ensures timely responses to critical events while balancing the needs of all devices in the system.

5.3. Daisy-Chaining and Polling

When multiple devices can generate interrupt requests, the system needs a method to determine which device is requesting service and in what order. Two common hardware techniques for managing this are Daisy-Chaining and Polling. Both methods help the CPU identify and prioritise interrupt sources, especially when using a vectored interrupt system is not feasible.

1. Daisy-Chaining Method

Daisy-chaining is a hardware-based priority technique in which multiple devices are connected in a serial chain. The Interrupt Acknowledge (INTA) signal from the CPU is passed from one device to the

next in the chain. Each device has the opportunity to intercept the acknowledge signal and respond if it raised the interrupt.

How It Works:

1. All devices share a common interrupt request (IRQ) line.
2. When the CPU detects an interrupt, it sends an INTA signal.
3. The first device in the chain checks if it requested the interrupt:
 - If yes, it responds and stops the signal from passing further.
 - If no, it passes the INTA signal to the next device.
4. This continues until the requesting device is found.

Features:

- Simple and cost-effective.
- Priority is determined by physical position in the chain (closer = higher priority).
- No need for complex encoders or controllers.

Disadvantages:

- Devices farther down the chain have lower priority and longer response times.
- Failure of one device can disrupt the entire chain.
- Not scalable for large systems.

2. Polling Method

Polling is a software-based approach in which the CPU checks each device in a predefined order to determine which one requested service. This is done by reading status registers or interrupt flags of each device.

How It Works:

1. An interrupt is detected on a shared line.
2. The CPU enters a polling routine and sequentially queries each device to find the source.
3. When the interrupting device is found, its Interrupt Service Routine (ISR) is executed.
4. The CPU then resumes normal execution.

Types of Polling:

- Serial Polling: Devices are checked one by one in a fixed order.

- Parallel Polling: All devices send interrupt IDs simultaneously to a priority encoder (requires more hardware).

Advantages:

- Easy to implement and modify in software.
- Offers flexibility in changing priority order.

Disadvantages:

- Slower response time, especially for devices lower in the poll order.
- CPU spends time checking all devices—less efficient than hardware interrupt identification.

Table 4: Comparison Table

Feature	Daisy-Chaining	Polling
Method	Hardware	Software
Priority Control	Based on chain order	Based on polling sequence
Speed	Faster (hardware-driven)	Slower (software-driven)
Scalability	Limited	More flexible
Complexity	Low	Moderate (requires status checking)
Fault Tolerance	Lower (chain failure affects all)	Higher (independent checks)

Daisy-chaining and polling provide practical solutions for handling multiple interrupts in systems without complex interrupt controllers. Daisy-chaining is efficient for small, hardware-oriented setups, while polling offers greater flexibility and control in software-driven environments.

5.4. Software vs Hardware Interrupts

Interrupts in a computer system can be classified based on how they are triggered—either by hardware signals or software instructions. Understanding the difference between software and hardware interrupts is essential for designing efficient and responsive systems.

1. Hardware Interrupts

A hardware interrupt is triggered by an external device to signal the CPU that it requires immediate attention. These interrupts are generated asynchronously, meaning they can occur at any time during program execution.

Examples:

- Keyboard generates an interrupt when a key is pressed.
- Network card signals data arrival.
- Timer triggers a periodic interrupt.

Characteristics:

- Initiated by external hardware devices.
- Occur asynchronously with program execution.
- Typically used for real-time or event-driven tasks.
- Requires an Interrupt Service Routine (ISR) to handle the event.

2. Software Interrupts

A software interrupt is triggered by an instruction within a program to request a service from the operating system or interrupt handler. It is a way for software to interact with privileged system-level routines.

Examples:

- INT 21H in x86 assembly to access DOS services.
- Trap instructions to handle errors (e.g., divide-by-zero).
- System calls in modern OS kernels (e.g., Linux syscall).

Characteristics:

- Generated intentionally by programs.
- Occur synchronously as part of instruction execution.
- Commonly used for system services, debugging, or exception handling.
- Can be masked or controlled through software.

Table 5: Comparison Table

Feature	Hardware Interrupt	Software Interrupt
Triggered By	External hardware device	Program instruction
Timing	Asynchronous	Synchronous
Purpose	Handles I/O events, errors	Requests OS services, handles traps
Flexibility	Typically fixed (based on device)	Programmable by software
Examples	Keyboard input, disk I/O	System calls, divide-by-zero traps

Use in OS	Device drivers and real-time events	System-level service invocation
-----------	-------------------------------------	---------------------------------

Both hardware and software interrupts serve vital roles in system operation. Hardware interrupts improve responsiveness to real-world events, while software interrupts offer controlled access to system services. Together, they enable efficient multitasking, real-time processing, and user-program interaction with the underlying hardware.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
1	What is the first step in the interrupt cycle? a) Execute ISR b) Save program state c) Detect interrupt d) Restore program state
2	Which priority scheme assigns static priority levels to devices? a) Dynamic Priority b) Rotating Priority c) Fixed Priority d) Software-Controlled Priority
3	In daisy-chaining, the priority of devices is determined by: a) Software configuration b) Device type c) Physical position in the chain d) Interrupt vector table
4	Which type of interrupt is triggered by program instructions? a) Hardware Interrupt b) Software Interrupt c) External Interrupt d) Timer Interrupt
Fill in the Blank	
5	The _____ cycle temporarily suspends the current program to service an interrupt.
6	A _____ encoder is used in hardware to identify the highest-priority interrupt.
7	In polling, the CPU checks each device's _____ register to find the interrupt source.

8	Hardware interrupts occur _____ with program execution.
True or False	
9	In the interrupt cycle, the CPU resumes the original program after executing the ISR.
10	Rotating priority always favours the same device for servicing interrupts.
11	Daisy-chaining is a software-based method for handling interrupts.
12	Software interrupts are commonly used for system calls and exception handling.



6. SUMMARY

- This unit explores the mechanisms that enable a computer system to interact effectively with external devices through organised Input/Output (I/O) operations. It begins by examining the role of I/O interfaces, which act as communication bridges between the CPU and peripherals such as keyboards, printers, and sensors.
- The unit explains the differences between memory-mapped I/O and I/O-mapped (isolated) I/O, outlining their respective advantages in terms of flexibility, address space usage, and instruction handling. It then introduces asynchronous data transfer techniques, including the handshaking and strobe control methods, which facilitate communication between devices operating at different speeds. A comparison between synchronous and asynchronous transfers highlights their distinct timing mechanisms and use cases.
- The unit further explores three primary modes of data transfer: programmed I/O, where the CPU actively polls devices; interrupt-driven I/O, which allows devices to signal the CPU when ready; and Direct Memory Access (DMA), which enables peripherals to transfer data directly to memory without CPU intervention.
- To manage multiple simultaneous I/O requests, the concept of priority interrupts is introduced, along with various schemes such as fixed, rotating, dynamic, and software-controlled priorities. Techniques like daisy-chaining and polling are discussed as methods for identifying and servicing interrupt sources. Finally, the unit distinguishes between hardware interrupts, triggered by external devices, and software interrupts, initiated by program instructions, emphasising their roles in system responsiveness and service invocation.
- By the end of the unit, learners gain a comprehensive understanding of how I/O operations are structured and managed, equipping them with the knowledge to design efficient systems that integrate processing units with external hardware components.

7. GLOSSARY

I/O Interface	- A hardware or software component that facilitates communication between the CPU and peripheral devices.
Memory-Mapped I/O	- A technique where I/O devices are assigned addresses within the same space as memory, allowing standard memory instructions to access them.
I/O-Mapped (Isolated) I/O	- A method where I/O devices have a separate address space and require special instructions like IN and OUT for communication.
Peripheral Devices	- External hardware components such as keyboards, printers, and sensors that interact with the CPU.
Handshaking	- An asynchronous data transfer method using control signals (Request and Acknowledge) to synchronise communication between devices.
Strobe Control	- A simpler asynchronous transfer method using a single control signal to indicate data availability.
Synchronous Transfer	- Data transfer method where sender and receiver share a common clock for timing coordination.
Asynchronous Transfer	- Data transfer method without a shared clock, relying on control signals for synchronisation.
Programmed I/O	- A mode of data transfer where the CPU actively polls devices and manages the entire transfer process.
Interrupt-Driven I/O	- A mode where devices notify the CPU via interrupts when ready for data transfer, allowing the CPU to perform other tasks meanwhile.
Direct Memory Access (DMA)	- A high-speed data transfer method where peripherals directly access memory without CPU intervention.
Interrupt	- A signal that temporarily halts the CPU's current operations to service an I/O or system event.
Interrupt Cycle	- The sequence of operations the CPU performs to handle an interrupt, including saving state, executing the ISR, and resuming the program.

Priority Interrupt	- A system that assigns importance levels to interrupts to ensure critical tasks are serviced first.
Daisy-Chaining	- A hardware method for managing multiple interrupts by passing the interrupt acknowledge signal through a chain of devices.
Polling	- A software method where the CPU checks each device sequentially to identify the source of an interrupt.
Hardware Interrupt	- An interrupt triggered by an external device, such as a keyboard or timer.
Software Interrupt	- An interrupt initiated by a program instruction to request system-level services.
DMA Controller	- A dedicated hardware unit that manages DMA operations, including address and data transfer control.
Bus Arbitration	- The process of managing access to the system bus among multiple devices, especially during DMA.

8. TERMINAL QUESTIONS

1. What is the primary role of an I/O interface in a computer system?
2. Differentiate between memory-mapped I/O and I/O-mapped (isolated) I/O.
3. Explain the concept of handshaking in asynchronous data transfer.
4. How does the strobe control method work in asynchronous communication?
5. Compare synchronous and asynchronous data transfer methods with examples.
6. Describe the steps involved in the interrupt cycle.
7. What are the advantages and disadvantages of programmed I/O?
8. How does interrupt-driven I/O improve system efficiency compared to programmed I/O?
9. What is Direct Memory Access (DMA) and how does it benefit high-speed data transfer?
10. List and explain the different DMA transfer modes.
11. What is the purpose of priority interrupt handling in a multi-device system?
12. Compare fixed priority and rotating priority schemes in interrupt management.
13. How does daisy-chaining help in identifying interrupt sources?
14. What is polling, and how does it differ from daisy-chaining in interrupt handling?
15. Distinguish between hardware interrupts and software interrupts with suitable examples.

9. ANSWERS

Self-Assessment Questions

Self-Assessment Questions - 1

1. b) Shares address space with memory
2. c) IN and OUT instructions
3. b) Serial Interface
4. c) Identifying the target device
5. memory
6. parallel
7. Control
8. standard
9. False
10. True
11. True
12. False

Self-Assessment Questions - 2

1. c) Request and Acknowledge
2. b) One
3. c) Slower due to control signalling
4. b) Asynchronous
5. clock
6. unidirectional
7. ready
8. flexible
9. False
10. False
11. True
12. True

Self-Assessment Questions - 3

1. c) Continuously checks device status

2. c) Reduces CPU idle time
3. c) DMA Controller
4. c) Transparent Mode
5. Programmed
6. interrupt signal
7. memory
8. one
9. False
10. True
11. False
12. True

Self-Assessment Questions - 4

1. c) Detect interrupt
2. c) Fixed Priority
3. c) Physical position in the chain
4. b) Software Interrupt
5. interrupt
6. priority
7. status
8. asynchronously
9. True
10. False
11. False
12. True

Terminal Questions Answers

Answer 1: The I/O interface plays a vital role in enabling seamless communication between the CPU and external devices. It manages data exchange, handles speed mismatches, and ensures proper coordination through control signals and data formatting.

Refer Section 2.1 - Purpose and Types of Interfaces

Answer 2: Memory-mapped I/O uses the same address space as memory and allows standard instructions for device communication, offering flexibility. In contrast, I/O-mapped I/O uses a separate address space and requires special instructions, making it more efficient for simpler systems.

Refer Section 2.2 - I/O Mapped vs Memory Mapped I/O

Answer 3: Handshaking is an asynchronous data transfer method where the sender and receiver exchange control signals (Request and Acknowledge) to confirm readiness before data transmission, ensuring reliable and synchronised communication.

Refer Section 3.1 - Handshaking Method

Answer 4: In the strobe control method, the sender places data on the bus and activates a strobe signal to notify the receiver. The receiver reads the data while the strobe is active, making it a simple but less flexible method compared to handshaking.

Refer Section 3.2 - Strobe Control Method

Answer 5: Synchronous transfer relies on a shared clock between devices, enabling faster and predictable data exchange. Asynchronous transfer uses control signals and allows communication between devices with different speeds, offering greater flexibility but lower speed.

Refer Section 3.3 - Asynchronous vs Synchronous Transfer

Answer 6: The interrupt cycle involves detecting an interrupt, saving the current program state, identifying the interrupt source, executing the appropriate service routine, and restoring the program state to resume normal execution.

Refer Section 5.1 - Interrupt Cycle

Answer 7: Programmed I/O is simple and easy to implement, especially in small systems. However, it is inefficient because the CPU continuously polls the device, leading to wasted processing time and reduced overall performance.

Refer Section 4.1 - Programmed I/O

Answer 8: Interrupt-driven I/O enhances system efficiency by allowing the CPU to perform other tasks while waiting for I/O devices to signal readiness. This reduces idle time and supports better multitasking and responsiveness.

Refer Section 4.2 - Interrupt-Driven I/O

Answer 9: DMA enables high-speed data transfer by allowing peripherals to directly access memory without CPU intervention. This offloads the CPU, reduces latency, and improves system performance, especially for large data blocks.

Refer Section 4.3 - Direct Memory Access (DMA)

Answer 10: DMA transfer modes include Burst Mode (transfers a block of data at once), Cycle Stealing (alternates bus access between CPU and DMA), and Transparent Mode (transfers occur only when the CPU is idle), each offering different trade-offs in speed and CPU usage.

Refer Section 4.3 - Direct Memory Access (DMA)

Answer 11: Priority interrupt handling ensures that the most critical interrupt is serviced first when multiple requests occur simultaneously. This mechanism improves system responsiveness and prevents delays in handling urgent tasks.

Refer Section 5.2 - Priority Schemes

Answer 12: Fixed priority assigns static importance levels to devices, always servicing the highest-priority request first. Rotating priority changes the order after each interrupt, promoting fairness and preventing starvation of lower-priority devices.

Refer Section 5.2 - Priority Schemes

Answer 13: Daisy-chaining connects devices in a serial chain, passing the interrupt acknowledge signal from one device to the next until the requesting device responds. It is simple and cost-effective but less scalable.

Refer Section 5.3 - Daisy-Chaining and Polling

Answer 14: Polling is a software-based method where the CPU checks each device in sequence to identify the interrupt source. Unlike daisy-chaining, it offers flexibility but is slower and less efficient due to increased CPU involvement.

Refer Section 5.3 - Daisy-Chaining and Polling

Answer 15: Hardware interrupts are triggered by external devices like keyboards or timers and occur asynchronously. Software interrupts are initiated by program instructions to request system services and occur synchronously during execution.

Refer Section 5.4 - Software vs Hardware Interrupts

10. REFERENCES

BOOKS

1. Computer Architecture: A Quantitative Approach, John L. Hennessy, David A. Patterson, 6th Edition, 2017
2. Computer Organisation and Architecture: Designing for Performance, William Stallings, 11th Edition, 2018
3. Computer System Architecture, M. Morris Mano, 3rd Edition, 2006
4. Computer Organisation and Design: The Hardware/Software Interface, David A. Patterson, John L. Hennessy, 5th Edition, 2013
5. Digital Design and Computer Architecture, Sarah Harris, David Harris, 3rd Edition, 2021

E-REFERENCES

1. Logical and Shift Micro-operations Computer Organisation and Architecture, <https://care4you.in/logical-and-shift-micro-operations/>
2. Logic and Shift Micro Operations, <https://www.scribd.com/document/394026693/U-I-Logic-and-Shift-micro-operations>

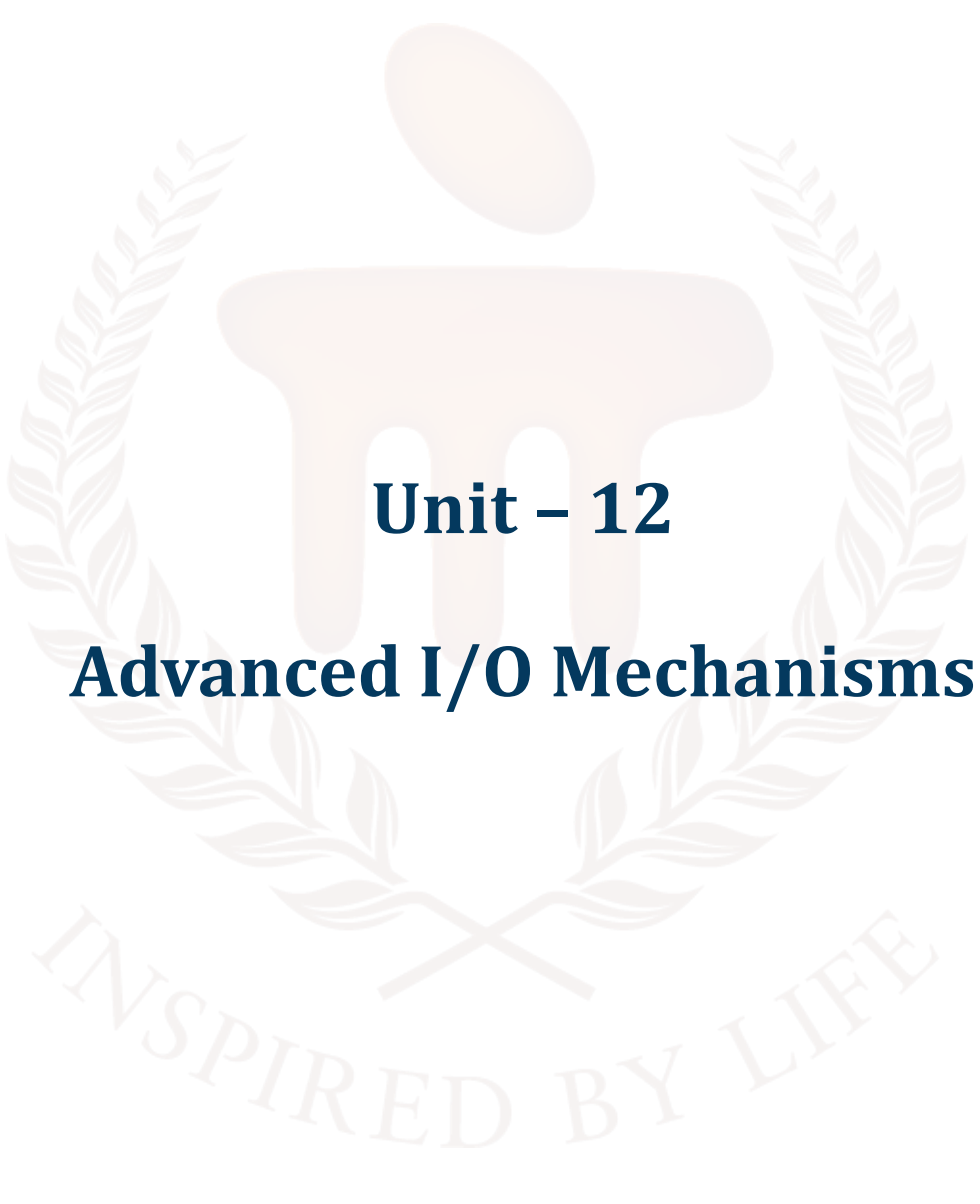
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 12

Advanced I/O Mechanisms

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Advanced I/O Mechanisms	-	-	6 - 11
2.1	Need for Advanced I/O	-	-	
2.2	Limitations of Traditional I/O Methods	-	-	
2.3	Comparison with Basic I/O Techniques	i, ii, iii	1	
3	Direct Memory Access (DMA)	-	-	12 - 19
3.1	DMA Operation	1	-	
3.2	DMA Transfer Modes	iv	-	
3.3	Advantages and Disadvantages of DMA	-	-	
3.4	Applications of DMA	-	2	
4	Input-Output Processor (IOP)	-	-	20 - 27
4.1	Architecture of IOP	2	-	
4.2	Role and Functioning of IOP	-	-	
4.3	IOP vs DMA	v	-	
4.4	CPU-IOP Communication	3	3	
5	Serial Communication	-	-	28 - 37
5.1	Basics of Serial Transmission	vi	-	
5.2	Asynchronous vs Synchronous Transmission	vii	-	
5.3	Serial Protocols: RS-232, USB, SPI, I2C	viii	-	
5.4	Comparison with Parallel Communication	ix, x, xi	4	
6	Summary	-	-	38
7	Glossary	-	-	39 - 40
8	Terminal Questions	-	-	41
9	Answers	-	-	42 - 46
10	References	-	-	47

1. INTRODUCTION

In the previous unit, we studied the foundational concepts of Input/Output (I/O) Organisation, focusing on how the CPU interacts with external devices. We explored the structure and function of the I/O interface, examined methods of asynchronous data transfer such as handshaking and strobe control, and analysed various modes of transfer including Programmed I/O, Interrupt-Driven I/O, and Direct Memory Access (DMA). Additionally, we discussed how systems handle multiple interrupt requests through priority interrupt mechanisms, including daisy-chaining, polling, and the distinction between hardware and software interrupts.

Building on that foundational knowledge, this unit delves into advanced I/O mechanisms that offer greater efficiency, flexibility, and scalability—especially in modern, high-performance systems. We begin by revisiting Direct Memory Access (DMA) in greater depth, analysing its various transfer modes, advantages, and practical applications in real-time systems.

The unit then introduces the Input-Output Processor (IOP), a specialised processor designed to manage complex I/O operations independently of the CPU. You will learn how an IOP offloads the CPU, enabling parallel processing and improving system throughput, particularly in multi-device environments.

Finally, we explore serial communication, a widely used method for data exchange over limited signal lines. You will understand the basic principles of serial transmission, distinguish between asynchronous and synchronous communication, and examine key serial protocols such as RS-232, USB, SPI, and I²C. We will also compare serial and parallel communication to highlight their respective use cases.

By the end of this unit, you will gain a clear understanding of advanced I/O techniques that are crucial for designing and managing efficient, modern computer systems, particularly those requiring high-speed and reliable communication with multiple external devices.

- Explain the need for advanced I/O mechanisms in modern computer systems.
- Discuss the operation and architecture of Direct Memory Access (DMA).
- Analyse the structure and functioning of an Input-Output Processor (IOP).
- Explain the basics of serial communication and how it differs from parallel communication.
- Select appropriate communication methods for different I/O scenarios in system design.



2. ADVANCED I/O MECHANISMS

As systems grow more complex and data-intensive, traditional I/O techniques often fall short in terms of speed and efficiency. Advanced I/O mechanisms like DMA, IOP, and serial communication protocols are designed to overcome these limitations by offloading work from the CPU and enabling faster, more efficient data transfer.

2.1. Need for Advanced I/O

As computer systems have evolved to handle increasingly complex tasks, the demand on Input/Output (I/O) subsystems has grown dramatically. Early I/O methods like Programmed I/O and Interrupt-Driven I/O are often insufficient for modern applications that require fast, reliable, and parallel communication with multiple peripheral devices. This has led to the development of advanced I/O mechanisms designed to improve system performance, reduce CPU workload, and support high-speed data transfer.

Limitations of Traditional I/O Methods

- 1. High CPU Involvement:**

In Programmed I/O, the CPU must constantly poll devices, wasting valuable processing time. Even in Interrupt-Driven I/O, frequent context switches and interrupt handling can affect CPU efficiency.

- 2. Low Throughput:**

Traditional methods are not suitable for large or high-speed data transfers, such as those required by hard disks, network interfaces, or real-time multimedia devices.

- 3. Inefficient Multitasking:**

When managing multiple I/O devices simultaneously, traditional methods often lead to bottlenecks, limiting system responsiveness and scalability.

- 4. Limited Flexibility:**

Basic I/O techniques do not easily support parallelism, device prioritisation, or independent I/O processing.

Emerging Requirements in Modern Systems

With the growth of real-time systems, high-speed networking, multimedia processing, and embedded applications, there is a growing need for:

- Faster data movement between memory and peripherals.
- Concurrent I/O operations with minimal CPU intervention.
- Dedicated I/O processing units to handle device communication independently.
- Efficient communication over long distances or limited pin counts (e.g., in embedded systems or mobile devices).

Why Advanced I/O Mechanisms Are Needed

To meet these demands, modern systems utilise advanced I/O mechanisms such as:

- **Direct Memory Access (DMA):** Enables fast data transfer directly between memory and I/O devices without burdening the CPU.
- **Input-Output Processor (IOP):** A separate processor dedicated to managing complex I/O operations independently of the CPU.
- **Serial Communication Protocols:** Allow data transfer over fewer lines, suitable for embedded, mobile, and peripheral devices.

These mechanisms help in:

- Reducing CPU overhead
- Increasing data throughput
- Enhancing system scalability and responsiveness
- Improving power and resource efficiency, especially in embedded environments

The limitations of basic I/O techniques in handling modern workloads necessitate the use of advanced I/O mechanisms. These methods not only enhance performance but also support the growing diversity and complexity of peripheral devices in today's computing environments.

2.2. Limitations of Traditional I/O Methods

Traditional I/O techniques—such as Programmed I/O and Interrupt-Driven I/O—were sufficient for early computing systems with limited peripherals and modest data requirements. However, in modern systems with high-speed devices, multitasking environments, and real-time constraints, these methods face significant limitations.

1. Excessive CPU Involvement

- In Programmed I/O, the CPU must constantly check the device status (polling), which wastes valuable processing time.

- Even with Interrupt-Driven I/O, the CPU must suspend ongoing tasks to handle interrupts, which can lead to frequent context switching.

2. Low Data Transfer Efficiency

- These methods are not suitable for large-volume or high-speed data transfers such as those from hard disks, network interfaces, or video devices.
- The CPU acts as a middleman for all data transfers, creating a bottleneck in performance.

3. Limited Parallelism

- Basic I/O techniques generally handle one device at a time, making it difficult to manage multiple I/O operations concurrently.
- This restricts the system's ability to scale and respond to simultaneous I/O requests.

4. High Latency

- The time it takes for the CPU to respond to interrupts or polling cycles can introduce delays in I/O processing.
- This is critical in real-time systems, where timely response is essential.

5. Inefficient Use of System Resources

- CPU cycles spent on managing I/O could be better utilised for computation or task execution.
- Peripheral devices often remain idle while waiting for CPU attention, reducing overall system efficiency.

6. Hardware Limitations

- Traditional I/O methods often require separate control logic for each device.
- They may not support complex or intelligent devices that need autonomous communication.

While traditional I/O methods are simple and easy to implement, they fail to meet the demands of modern computing. Their limitations in speed, scalability, and efficiency have led to the development of advanced I/O techniques like DMA, IOP, and serial protocols, which better suit today's high-performance systems.

2.3. Comparison with Basic I/O Techniques

To understand the value of advanced I/O mechanisms, it is important to compare them directly with basic techniques like Programmed I/O and Interrupt-Driven I/O. This comparison highlights how

advanced methods improve system performance, reduce CPU workload, and enable scalable, high-speed data handling.

1. CPU Involvement

Table 1: CPU Involvement

Technique	CPU Role
Programmed I/O	Actively controls and monitors every I/O transfer
Interrupt-Driven I/O	Responds to device interrupts; still executes ISR
DMA	CPU sets up transfer, DMA handles the rest
IOP	Offloads entire I/O control to a separate processor

2. Speed and Efficiency

- Programmed I/O: Slow due to polling; CPU is busy-waiting.
- Interrupt-Driven I/O: Faster, but frequent interrupts can disrupt execution.
- DMA: High-speed transfer directly between memory and device; minimal CPU use.
- IOP: Maximises throughput by handling multiple I/O devices independently.

3. Parallelism and Scalability

Table 2: Parallelism and Scalability

Feature	Basic Methods	Advanced Methods
Parallel Device Handling	Limited or sequential	Supports concurrent operations
System Scalability	Poor with many devices	High scalability with DMA/IOP

4. System Complexity

- Basic methods are easier to implement but offer limited performance.
- Advanced methods require additional hardware (DMA controller, IOP) and more complex configuration but significantly enhance efficiency and responsiveness.

5. Use Case Suitability

Table 3: Use Case Suitability

Scenario	Recommended Technique
Low-speed peripheral, simple system	Programmed I/O or Interrupts
High-speed disk or data stream	DMA

Multitasking or I/O-intensive OS	IOP
Embedded or mobile systems	Serial Communication (SPI, I ² C, USB)

While basic I/O techniques are adequate for simple systems or occasional data transfers, advanced mechanisms like DMA and IOP are essential for high-performance, multitasking environments. These methods offer faster data handling, lower CPU involvement, and greater flexibility, making them ideal for modern computing needs.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the primary goal of advanced I/O mechanisms in modern systems? a) Increase CPU usage b) Reduce memory size c) Improve data transfer efficiency d) Eliminate peripheral devices
2	Which of the following is a drawback of traditional I/O methods? a) High scalability b) Efficient multitasking c) Low throughput d) Parallel device handling
3	Which advanced I/O technique allows direct data transfer between memory and I/O devices? a) Interrupt-Driven I/O b) Programmed I/O c) DMA d) Serial Communication
4	What is a key benefit of using an Input-Output Processor (IOP)? a) Reduces memory size b) Handles I/O tasks independently c) Requires constant CPU polling d) Slows down system performance
Fill in the Blank	
5	Advanced I/O mechanisms are designed to reduce _____ involvement in data transfer.
6	Traditional I/O methods often lead to system _____ due to limited parallelism.

7	DMA and IOP are examples of _____ I/O techniques.
8	Interrupt-Driven I/O improves efficiency but still requires the CPU to execute the _____.
True or False	
9	Programmed I/O is highly efficient for multitasking systems.
10	Advanced I/O mechanisms help improve system scalability and responsiveness.
11	DMA transfers data only when the CPU is idle.
12	IOP can manage multiple I/O devices simultaneously.



3. DIRECT MEMORY ACCESS (DMA)

Direct Memory Access (DMA) is an advanced I/O technique that enables data to be transferred directly between memory and I/O devices without constant CPU involvement. It improves system performance by offloading routine data transfer tasks from the processor, making it ideal for high-speed and large-volume operations.

3.1. DMA Operation

Direct Memory Access (DMA) is a powerful data transfer technique that allows peripheral devices to transfer data directly to or from system memory, without continuous involvement of the CPU. This mechanism is especially useful in applications involving large or high-speed data transfers, such as disk access, audio/video streaming, and networking.

Basic Principle of DMA Operation

Under typical I/O methods, the CPU acts as an intermediary in data transfers between memory and I/O devices. However, in DMA, a dedicated hardware component called the DMA controller (DMAC) handles the transfer, freeing the CPU to perform other tasks.

Steps in DMA Operation

1. DMA Initialisation

- The CPU initiates the process by providing the DMA controller with the following:
 - Memory address where data will be read from or written to
 - I/O device address
 - Transfer size (number of bytes or words)
 - Direction of data transfer (read or write)

2. Bus Request

- The DMA controller sends a bus request signal to the CPU to gain control of the system buses (address, data, and control buses).

3. Bus Grant

- The CPU temporarily suspends its bus usage and grants control to the DMA controller.

4. Data Transfer

- The DMA controller performs the data transfer directly between memory and the I/O device, bypassing the CPU.

5. Bus Release and Interrupt

- After the transfer is complete, the DMA controller releases the bus and sends an interrupt signal to the CPU to notify that the transfer is done.

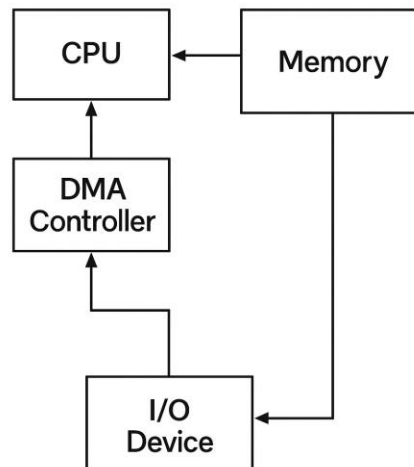


Fig 1: Direct Memory Access (DMA) operation

Modes of Operation

DMA can operate in various modes (discussed in detail in section 3.2), including:

- Burst Mode
- Cycle Stealing Mode
- Transparent Mode

Benefits of DMA Operation

- High Speed: Data transfer is much faster compared to Programmed or Interrupt-driven I/O.
- Low CPU Overhead: CPU is only involved during setup and completion.
- Efficient for Bulk Transfers: Ideal for scenarios that require movement of large data blocks.

Typical Applications

- Hard disk read/write operations
- Network data transmission
- Audio/video streaming

- Memory-to-memory transfer in embedded systems

The DMA operation significantly enhances system performance by eliminating the need for CPU-mediated data transfers. It allows the CPU to multitask efficiently while large volumes of data are moved in the background by the DMA controller.

3.2. DMA Transfer Modes

DMA Transfer Modes define how the DMA controller accesses the system bus and interacts with memory and I/O devices during data transfer. Each mode balances trade-offs between speed, CPU availability, and system complexity. Understanding these modes helps in selecting the most suitable method for a specific application or system design.

1. Burst Mode (Block Transfer)

In Burst Mode, the DMA controller gains complete control of the system bus and transfers an entire block of data in one continuous burst.

Features:

- CPU is halted during the entire data transfer.
- Fastest among all modes for large blocks.
- Efficient for high-speed data transfers like disk or memory dumps.

Advantages:

- High throughput due to uninterrupted transfer.
- Reduces overhead for frequent bus requests.

Disadvantages:

- CPU is completely blocked from using the bus during the transfer.
- Not ideal for multitasking environments.

2. Cycle Stealing Mode

In Cycle Stealing Mode, the DMA controller transfers one word (or byte) at a time, by temporarily “stealing” bus cycles from the CPU.

Features:

- CPU and DMA share the bus.
- CPU operation is only partially interrupted during the transfer.

- Maintains overall system responsiveness.

Advantages:

- Balances CPU usage and data transfer.
- Suitable for systems that require concurrent CPU and I/O activity.

Disadvantages:

- Slightly lower data transfer speed compared to burst mode.
- More complex bus control logic.

3. Transparent Mode

In Transparent Mode, the DMA controller only transfers data when the CPU is not using the bus, making the transfer “invisible” to the CPU.

Features:

- CPU is never interrupted.
- DMA operates in the background.
- Least disruptive to normal CPU operations.

Advantages:

- Maximum CPU availability for other tasks.
- Ideal for low-priority or background transfers.

Disadvantages:

- Slowest transfer rate among all DMA modes.
- Inefficient if CPU uses the bus frequently.

Table 4: Comparison Table

Feature	Burst Mode	Cycle Stealing	Transparent Mode
CPU Access to Bus	Blocked entirely	Partially available	Fully available
Transfer Speed	High	Moderate	Low
CPU Interruption	High	Minimal	None
Use Case	Fast block I/O	Balanced systems	Background transfers

Each DMA transfer mode serves a specific purpose, from maximum speed (burst mode) to maximum CPU availability (transparent mode). The choice of mode depends on system requirements, such as real-time responsiveness, data volume, and processor workload.

3.3. Advantages and Disadvantages of DMA

Direct Memory Access (DMA) significantly enhances system performance by enabling high-speed data transfers without overloading the CPU. However, like any technique, it comes with both benefits and limitations. Understanding these helps in making informed design choices in system architecture.

Advantages of DMA

1. Reduces CPU Workload

- The CPU is free from managing individual data transfers.
- It can focus on executing programs or handling other critical tasks.

2. Increases Data Transfer Speed

- Data moves directly between memory and peripherals, bypassing the CPU bottleneck.
- Especially effective for bulk transfers such as file loading, video/audio streaming, or sensor data.

3. Supports Multitasking

- Since the CPU is not busy with I/O operations, it can handle multiple processes simultaneously.
- Improves overall system responsiveness and throughput.

4. Efficient for Repetitive Tasks

- Ideal for repeated or scheduled data movement tasks (e.g., memory refresh, buffered I/O).

5. Works Well with High-Speed Devices

- Essential for devices like hard drives, network cards, and multimedia interfaces that generate large amounts of data.

Disadvantages of DMA

1. Increased Hardware Complexity

- Requires a DMA controller and additional bus control logic.
- Increases design and implementation effort in hardware and software.

2. Potential Bus Contention

- Since both CPU and DMA share the system bus, they may compete for access, causing performance trade-offs.
- Requires efficient bus arbitration mechanisms.

3. Limited Number of Channels

- Most systems have a fixed number of DMA channels, which may limit concurrent device support.

4. Security and Control Risks

- Improper configuration of DMA may lead to unauthorised memory access or data corruption.
- Needs strict control over which devices can perform DMA.

5. Debugging and Error Handling Complexity

- DMA errors (like incomplete transfers or wrong addresses) can be harder to trace than CPU-driven I/O operations.

While DMA offers significant advantages in terms of speed and CPU efficiency, it comes with added complexity in both hardware and software. Its use is highly beneficial in systems where high-volume, high-speed data transfer is required—provided that the system design accounts for its limitations.

3.4. Applications of DMA

Direct Memory Access (DMA) is widely used in systems that require high-speed, bulk, or real-time data transfers with minimal CPU intervention. Its ability to transfer data efficiently between memory and peripherals makes it essential in both general-purpose computing and specialised embedded applications.

1. Hard Disk Drives (HDDs) and Solid-State Drives (SSDs)

- DMA enables fast reading and writing of large blocks of data between storage devices and main memory.
- Reduces CPU overhead during file operations and system boot processes.

2. Network Interface Cards (NICs)

- Modern Ethernet and Wi-Fi adapters use DMA to transfer incoming and outgoing data packets directly to/from memory.

- Enhances data throughput in high-speed networking and reduces packet processing delays.

3. Audio and Video Streaming

- DMA supports real-time transfer of multimedia data between devices (e.g., microphones, speakers, cameras) and memory.
- Enables smooth playback and recording without glitches or delays.

4. Graphics Processing (GPU Data Transfer)

- Graphics cards use DMA to load textures, vertex data, and frame buffers from memory without burdening the CPU.
- Essential for high-performance gaming, simulations, and visual computing.

5. Embedded and IoT Devices

- Microcontrollers often use DMA for sensor data acquisition, memory-to-memory copy, or peripheral communication.
- Enables efficient data handling in low-power, resource-constrained systems.

6. Digital Signal Processing (DSP)

- In DSP applications like radar, sonar, and telecommunications, DMA transfers large blocks of sampled data between ADCs/DACs and memory at high speeds.

7. Memory Copy and Initialisation

- DMA is used for rapid block memory operations, such as loading firmware, initialising arrays, or copying buffers.

8. Industrial Automation

- PLCs and industrial controllers use DMA to continuously read data from sensors or write to actuators without interrupting control logic.

DMA is a cornerstone of modern computing and embedded systems where fast, repetitive, and large-volume data transfers are required. Its use improves system responsiveness, reduces CPU load, and enables real-time processing—making it vital in applications ranging from personal computers to mission-critical industrial systems.

SELF-ASSESSMENT QUESTIONS – 2

Multiple Choice Questions

1	What is the primary function of a DMA controller? a) Execute CPU instructions b) Manage memory allocation c) Transfer data between memory and I/O devices d) Handle interrupts
2	Which DMA mode allows the CPU and DMA to share the system bus? a) Burst Mode b) Transparent Mode c) Cycle Stealing Mode d) Interrupt Mode
3	Which of the following is a disadvantage of DMA? a) Low data transfer speed b) High CPU usage c) Increased hardware complexity d) No support for multitasking
4	DMA is commonly used in which of the following applications? a) Arithmetic operations b) File system formatting c) Audio and video streaming d) Instruction decoding
Fill in the Blank	
5	In DMA operation, the CPU is involved only during _____ and completion of the transfer.
6	Burst mode DMA transfers a _____ of data in one continuous operation.
7	Transparent mode DMA operates only when the CPU is _____.
8	DMA improves system performance by reducing _____ overhead.
True or False	
9	DMA allows data transfer without interrupting the CPU.
10	Cycle stealing mode completely blocks the CPU from accessing the bus.
11	DMA is not suitable for high-speed data transfers.
12	DMA controllers can introduce bus contention in a system.

4. INPUT-OUTPUT PROCESSOR (IOP)

An Input-Output Processor (IOP) is a dedicated processor designed to handle all I/O-related tasks independently of the CPU. It enables parallel processing of I/O operations, reduces CPU workload, and improves overall system efficiency—especially in environments with multiple high-speed devices.

4.1. Architecture of IOP

An Input-Output Processor (IOP) is a specialised processor designed to manage I/O operations independently of the CPU. In complex computer systems, where multiple devices generate high volumes of data, an IOP offloads the burden of I/O control and data transfer from the CPU, allowing it to focus on computation and logic.

What is an IOP?

- An IOP is a dedicated microprocessor or controller that interfaces with I/O devices.
- It has its own instruction set, memory, control logic, and often direct access to system buses.
- It can execute I/O-related tasks concurrently with the CPU, thus supporting parallelism in system operations.

IOP Architecture Components

The architecture of an IOP resembles a simplified version of a CPU, with specialised features tailored for I/O management:

1. Control Unit

- Directs the execution of I/O instructions.
- Interprets and processes commands received from the CPU or I/O devices.

2. Instruction Decoder

- Decodes I/O-specific instructions (different from general CPU instructions).
- Manages data transfer, device status checking, and control signaling.

3. Local Memory (Buffer Storage)

- Stores I/O instructions, temporary data, and device status.
- Acts as a buffer between memory and I/O devices during data transfer.

4. Registers

- I/O address registers: Identify connected devices.
- Data registers: Hold input/output data.
- Control/status registers: Manage device state and command execution.

5. I/O Channels or Ports

- Interfaces with multiple I/O devices simultaneously.
- Each channel may be dedicated to a specific device or type of data.

6. Bus Interface Unit

- Connects the IOP to the system's main memory and buses (address, data, control).
- Allows direct memory access or communication with the CPU.

IOP and System Buses

- The IOP typically shares the system bus with the CPU.
- It can operate autonomously, using DMA or bus arbitration to access memory.
- The CPU and IOP may communicate via shared memory, command/status registers, or interrupt signals.

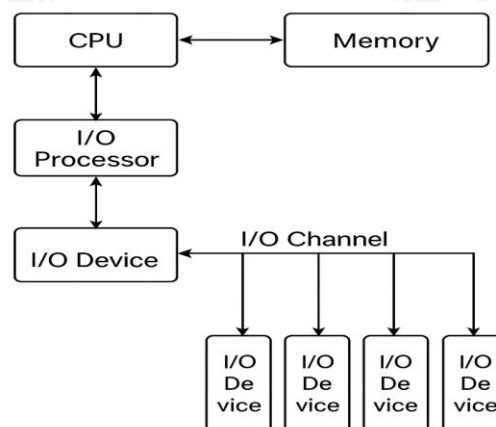


Fig 2: block diagram

The IOP architecture is tailored for managing I/O operations in complex systems. By having a dedicated processor for I/O, system performance is enhanced, CPU load is reduced, and data communication is more efficient—particularly in multitasking or real-time environments.

4.2. Role and Functioning of IOP

The Input-Output Processor (IOP) plays a vital role in managing I/O operations efficiently in systems where multiple I/O devices operate concurrently. It allows the CPU and I/O to work in parallel, increasing system throughput and enabling smooth multitasking.

Role of the IOP

1. Offloading I/O Tasks from the CPU

- The IOP handles routine and time-consuming I/O tasks such as device control, data transfer, and buffering, so the CPU can focus on core computation.

2. Coordinating I/O Devices

- The IOP manages multiple I/O channels, ensuring that various devices operate without interfering with each other.

3. Efficient Bus Utilisation

- It communicates directly with system memory using DMA, reducing data traffic on the CPU.

4. Interrupt Management

- The IOP can process device interrupts and notify the CPU only when necessary, reducing frequent context switches.

5. Parallel Execution

- While the CPU executes the main program, the IOP simultaneously manages I/O operations, allowing true parallel processing.

Functioning of the IOP

The IOP typically works in coordination with the CPU through a command-based system. Here's how the interaction occurs:

1. Command Issuance

- The CPU sends I/O instructions or commands to the IOP via shared memory or control registers.

2. Command Decoding

- The IOP decodes the instruction and identifies the I/O device and action (e.g., read, write,

status check).

3. Execution

- The IOP directly controls the specified I/O device, performs the requested operation, and manages data flow.

4. Status Reporting

- Once the operation is complete, the IOP updates the status or interrupts the CPU if necessary.

Example Use Case

- While the CPU is processing user input or calculations, the IOP may be:
 - Transferring a file from a storage device to memory,
 - Printing a document,
 - Receiving a data stream from a sensor.

This separation ensures both the CPU and IOP can operate simultaneously, increasing the efficiency of the entire system.

The IOP acts as an intelligent manager of I/O operations, enabling faster, more reliable, and parallel communication between the system and its peripherals. Its role is crucial in systems with high I/O demand, such as servers, embedded systems, and industrial automation setups.

4.3. IOP vs DMA

Both Input-Output Processor (IOP) and Direct Memory Access (DMA) are advanced I/O mechanisms designed to reduce CPU involvement in data transfer tasks. However, they differ significantly in complexity, capabilities, and use cases. Understanding the differences between them helps in selecting the appropriate method for a given system architecture.

Table 5: Key Differences Between IOP and DMA

Feature	DMA (Direct Memory Access)	IOP (Input-Output Processor)
Type of Component	Specialised hardware controller	Independent processor
Control	Controlled by the CPU (initially)	Executes I/O operations autonomously
Instruction Set	Does not have its own instruction set	Has a dedicated I/O instruction set

Processing Capability	Cannot process logic; only handles transfer	Can decode and execute I/O instructions
Device Management	Typically manages one device per channel	Can manage multiple devices simultaneously
Bus Access	Uses system buses during transfers	May share or arbitrate system bus with CPU
Communication with CPU	CPU sets up DMA and is notified on completion	CPU sends commands; IOP operates independently
Flexibility	Less flexible; best for simple transfers	Highly flexible; suitable for complex I/O control
Complexity and Cost	Simpler and less expensive	More complex and costly
Use Cases	High-speed transfers (e.g., disk, memory)	Multi-device systems, real-time and multitasking

When to Use DMA

- For fast, bulk data transfers between memory and a single I/O device.
- When CPU involvement must be minimised, but no complex I/O logic is needed.
- Example: Transferring a block of data from disk to RAM.

When to Use IOP

- When the system needs to handle multiple I/O devices concurrently.
- For tasks requiring intelligent I/O control, such as buffering, error checking, or command sequencing.
- Example: A server managing simultaneous network, disk, and peripheral I/O operations.

While both DMA and IOP improve system performance by offloading the CPU, DMA is ideal for high-speed data movement, whereas IOP is suited for more complex, multitasking I/O environments. The choice depends on system requirements, complexity, and the level of I/O management needed.

4.4. CPU-IOP Communication

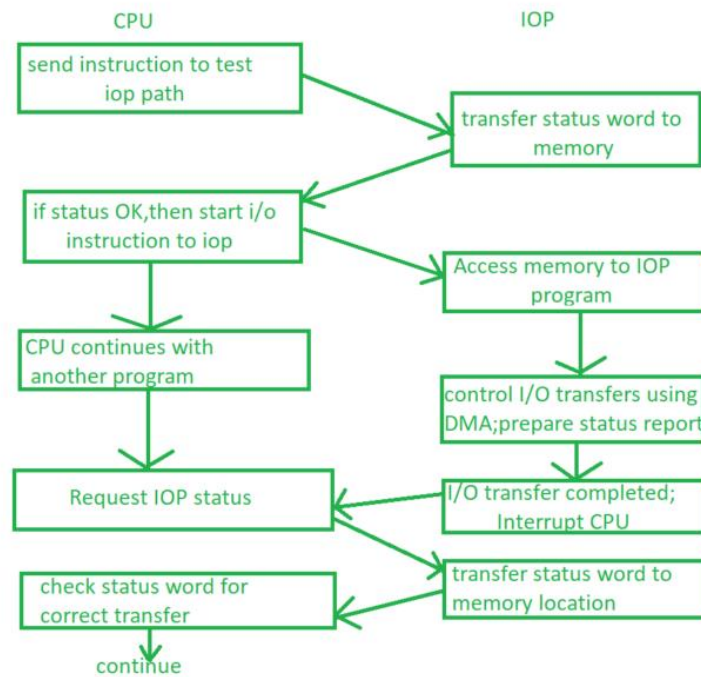


Fig 3: CPU-IOP Communication

In systems that include an Input-Output Processor (IOP), effective coordination between the CPU and IOP is essential for seamless and efficient I/O operations. Though the IOP operates independently, it still needs to receive commands from the CPU and report back the status or results of I/O tasks.

Why CPU-IOP Communication is Needed

- To delegate I/O tasks from the CPU to the IOP.
- To synchronise processing between computational and input/output activities.
- To ensure system stability, especially when both processors access shared resources like memory and buses.

Common Methods of CPU-IOP Communication

1. Command and Status Registers

- The CPU sends I/O commands to the IOP via dedicated command registers.
- The IOP updates status registers to indicate the completion or error status of operations.

2. Shared Memory

- Both CPU and IOP can access a common memory area to exchange commands, parameters, and data.
- Used for passing I/O job queues, buffer addresses, and control flags.

3. Interrupts

- The IOP can interrupt the CPU to signal the completion of an I/O operation or to report an error.
- Allows asynchronous communication and reduces CPU polling overhead.

4. Bus Arbitration

- When both CPU and IOP need access to memory, a bus arbitration mechanism ensures fair or prioritised access to the system bus.
- Prevents data conflicts and maintains system performance.

Typical Communication Sequence

1. Initialisation by CPU

- CPU sends a command (e.g., "read from disk") to the IOP along with parameters like device address, buffer location, and byte count.

2. Execution by IOP

- The IOP processes the command independently, handles the data transfer, and manages the device.

3. Completion Notification

- Upon completing the task, the IOP updates status information and may send an interrupt to the CPU.

4. CPU Response

- The CPU checks the status register or shared memory to retrieve the result or proceed with the next operation.

Advantages of CPU-IOP Coordination

- Parallelism: CPU and IOP can work simultaneously, improving system throughput.
- Efficiency: CPU doesn't need to manage every detail of I/O.
- Scalability: Supports complex systems with many I/O devices.

The communication between the CPU and IOP is central to the effective functioning of multi-processor systems. By sharing control and coordinating tasks through commands, memory, and interrupts, the system achieves better performance, higher responsiveness, and modular I/O management.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice questions

1	What is the primary role of an Input-Output Processor (IOP)? a) Execute arithmetic operations b) Manage memory allocation c) Handle I/O operations independently of the CPU d) Control the system clock
2	Which of the following is a key component of IOP architecture? a) Arithmetic Logic Unit b) Instruction Decoder c) Cache Memory d) Floating Point Unit
3	How does the CPU typically communicate with the IOP? a) Through the ALU b) Using DMA channels c) Via shared memory and command/status registers d) Through the GPU
4	Compared to DMA, IOP is: a) Less flexible and cheaper b) More complex and capable of executing I/O instructions c) Used only in embedded systems d) Limited to single-device communication
Fill in the Blank	
5	The IOP can execute I/O-related tasks _____ with the CPU.
6	The _____ unit in the IOP directs the execution of I/O instructions.
7	IOPs often use _____ to notify the CPU upon completion of an I/O task.
8	Unlike DMA, the IOP has its own _____ set for managing I/O operations.
True or False	
9	The IOP can manage multiple I/O devices simultaneously.
10	The CPU must remain idle while the IOP performs I/O operations.
11	IOPs are typically used in systems that require complex and multitasking I/O handling.
12	DMA is more suitable than IOP for managing multiple devices with complex control logic.

5. SERIAL COMMUNICATION

Serial communication is a widely used method of transferring data between devices using a single data line, transmitting bits one at a time. It is especially useful in systems where simplicity, long-distance communication, or minimal wiring is important—such as in embedded devices, sensors, and modern computer peripherals.

5.1. Basics of Serial Transmission

Serial communication is a method of transmitting data bit by bit over a single communication line or channel, making it highly efficient for long-distance and low-pin-count communication. Unlike parallel communication, which sends multiple bits simultaneously, serial transmission is simpler, cheaper, and better suited for compact or embedded systems.

Key Concepts of Serial Communication

1. Bit-by-Bit Transmission

- Data is sent one bit at a time, in a continuous stream.
- The transmission begins with a start bit, followed by the data bits, optional parity bit, and one or more stop bits.

2. Transmission Direction

- Simplex: Data flows in one direction only (e.g., keyboard to CPU).
- Half-duplex: Data flows in both directions, but only one direction at a time.
- Full-duplex: Data flows simultaneously in both directions (e.g., serial ports in networking).

Components of Serial Communication

- Transmitter: Converts parallel data into a serial stream.
- Receiver: Converts the incoming serial stream back to parallel form.
- Clocking Mechanism: Ensures that the sender and receiver are synchronised (synchronous/asynchronous).

Types of Serial Communication

- Synchronous Transmission
 - Sender and receiver share a common clock signal.

- More efficient for large data transfers.
- Asynchronous Transmission
 - Each data unit is framed with start and stop bits.
 - Suitable for low-speed, intermittent data.

Table 6: Common Serial Parameters

Parameter	Description
Baud Rate	Number of bits transmitted per second
Start/Stop Bits	Bits that mark the beginning and end of a transmission
Parity Bit	Used for simple error detection
Data Bits	The actual information being sent (usually 7 or 8 bits)

Advantages of Serial Transmission

- Fewer wires/pins required (cost-effective)
- Ideal for long-distance communication
- Less susceptible to noise and crosstalk than parallel communication
- Widely used in embedded systems, networking, and peripheral interfacing

Disadvantages

- Slower than parallel communication for short distances
- Requires precise synchronisation (especially in synchronous transmission)

Typical Applications

- Communication between microcontrollers and sensors
- Computer-to-modem connections
- USB, RS-232, SPI, I²C protocols

Serial transmission provides a practical, reliable, and efficient method of communication, particularly when pin count, space, and cost are constraints. Understanding its basics lays the foundation for working with various serial protocols used in modern digital systems.

5.2. Asynchronous vs Synchronous Transmission

Serial communication can be broadly classified into two types based on how data is timed and synchronised between the sender and receiver: Asynchronous and Synchronous transmission. Each

method has its own benefits, drawbacks, and typical use cases.

1. Asynchronous Transmission

In asynchronous communication, data is transmitted one character or byte at a time, each framed by start and stop bits. There is no shared clock between sender and receiver.

Key Features:

- Each data unit (usually a byte) starts with a start bit and ends with one or more stop bits.
- A parity bit may be included for simple error detection.
- Data can be sent intermittently, at irregular intervals.

Advantages:

- Simple and cost-effective implementation.
- Ideal for low-speed or irregular data transmission (e.g., keyboards, serial ports).
- No need to synchronise clocks between devices.

Disadvantages:

- Overhead due to extra start/stop bits.
- Lower efficiency for continuous or large data transfers.

Use Cases:

- RS-232 serial ports
- UART (Universal Asynchronous Receiver/Transmitter)
- Basic sensor communication

2. Synchronous Transmission

In synchronous communication, a continuous stream of data is sent without start/stop bits, and both sender and receiver are synchronised by a shared clock signal.

Key Features:

- Data is grouped into blocks or frames and transmitted in a steady stream.
- Sender and receiver must be timed precisely using either an external or embedded clock signal.

Advantages:

- High-speed data transfer with minimal overhead.
- Efficient for transmitting large amounts of data.

- Better suited for real-time and continuous communication.

Disadvantages:

- More complex setup and synchronisation.
- Requires both devices to remain in sync; otherwise, data corruption may occur.

Use Cases:

- SPI (Serial Peripheral Interface)
- I²C (Inter-Integrated Circuit)
- USB and high-speed networking protocols

Table 7: Comparison Table

Feature	Asynchronous Transmission	Synchronous Transmission
Clock Requirement	No shared clock	Requires shared/synchronised clock
Data Framing	Start and stop bits required	Sent in continuous blocks
Speed	Lower	Higher
Complexity	Simple	More complex
Overhead	Higher (extra bits per byte)	Lower (more data per frame)
Typical Use	UART, RS-232	SPI, I ² C, USB

Both asynchronous and synchronous transmission methods are essential in serial communication. While asynchronous is simple and effective for low-speed devices, synchronous communication offers greater speed and efficiency for high-performance systems. The choice depends on the system's timing requirements, complexity, and data volume.

5.3. Serial Protocols: RS-232, USB, SPI, I²C

In modern digital systems, serial communication protocols define how data is transmitted, received, and interpreted between devices. These protocols specify aspects such as timing, signaling, speed, and data format, ensuring reliable communication. The most commonly used serial protocols include RS-232, USB, SPI, and I²C, each suited to different applications and environments.

1. RS-232 (Recommended Standard 232)

This communication standard is one of the oldest and most widely used methods for asynchronous serial communication, primarily designed for point-to-point data exchange between devices such as computers and peripherals. It specifies both the electrical signaling and the physical connectors used,

typically DB9 or DB25. The system operates using voltage levels ranging from $\pm 3V$ to $\pm 15V$ to represent binary logic states, and supports full-duplex communication through separate transmit (TX) and receive (RX) lines. Although largely replaced in modern consumer electronics, it remains in use in legacy systems like modems, printers, and serial terminals, as well as in various types of industrial equipment where reliability and simplicity are essential.

Advantages:

- Simple and easy to implement.
- Widely supported.

Disadvantages:

- Low data rate (~ 115.2 kbps max).
- Not suitable for multi-device or long-distance networks.

2. USB (Universal Serial Bus)

This is a high-speed, plug-and-play communication protocol commonly used to connect peripherals to computers and embedded systems. It supports convenient features like hot-swapping (connecting/disconnecting devices without restarting), power delivery, and automatic device detection. USB standards offer varying data rates, such as 480 Mbps for USB 2.0 and 5 Gbps or more for USB 3.0 and newer versions. The protocol allows multiple devices to connect through a tiered-star topology and uses standardised connectors like USB-A and USB-C. USB is widely used in everyday devices including keyboards, mice, flash drives, cameras, and smartphones.

Advantages:

- High speed and versatility.
- Supports up to 127 devices per host.
- Provides power along with data.

Disadvantages:

- More complex protocol and hardware.
- Host-centric (one master device controls communication).

3. SPI (Serial Peripheral Interface)

This is a synchronous, full-duplex communication protocol designed for high-speed data exchange between microcontrollers and peripheral devices. It operates using four main lines: MISO (Master In

Slave Out), MOSI (Master Out Slave In), SCLK (Serial Clock), and SS/CS (Slave Select or Chip Select), and follows a master-slave configuration where the master controls the communication. SPI offers fast data transfer rates, often reaching tens of megabits per second, making it ideal for applications involving sensors, analog-to-digital and digital-to-analog converters (ADCs/DACs), and memory chips such as EEPROM and Flash.

Advantages:

- Simple and fast.
- Flexible data length and clock speed.

Disadvantages:

- Requires one SS/CS line per device → limited scalability.
- No built-in acknowledgment or error checking.

4. I²C (Inter-Integrated Circuit)

This is a synchronous communication protocol designed for short-distance data exchange within circuit boards, supporting multiple masters and slaves. It uses only two lines—SDA (Serial Data) and SCL (Serial Clock)—to transfer data, with each connected device assigned a unique address for identification. I²C supports various speed modes, including standard (100 kbps), fast (400 kbps), and high-speed (3.4 Mbps), making it suitable for connecting microcontrollers to peripherals such as sensors, real-time clocks (RTCs), and display modules.

Advantages:

- Very low pin count (2 wires for multiple devices).
- Good for compact and low-power applications.

Disadvantages:

- Slower than SPI.
- Shared bus can become congested with many devices.

Table 8: Comparison Table

Feature	RS-232	USB	SPI	I ² C
Type	Asynchronous	Synchronous	Synchronous	Synchronous

Max Devices	1-to-1	127	Limited by CS lines	100+ (addressed)
Speed	~115 kbps	Up to 5 Gbps	~50 Mbps	Up to 3.4 Mbps
Bus Lines	3+	4+	4	2
Use Case	Legacy I/O	Modern peripherals	High-speed sensor comm.	Compact, low-speed devices

Each serial protocol is optimised for specific needs—RS-232 for legacy devices, USB for versatile, high-speed connections, SPI for fast device-to-microcontroller links, and I²C for low-pin-count embedded applications. Understanding these protocols helps in selecting the most efficient communication method based on system requirements.

5.4. Comparison with Parallel Communication

Serial and parallel communication are two fundamental methods of data transmission between devices. While serial communication sends data bit by bit over a single channel, parallel communication transmits multiple bits simultaneously over multiple lines. Each has its own advantages, limitations, and ideal use cases depending on factors such as distance, speed, cost, and complexity.

1. Basic Concept

Table 9: Types of Communication

Communication Type	Transmission Method
Serial	One bit at a time, over a single wire or pair
Parallel	Multiple bits sent simultaneously on multiple wires

2. Speed and Distance

- Serial Communication
 - Generally slower than parallel for short distances, but more efficient over long distances.
 - Less interference and crosstalk due to fewer wires.
- Parallel Communication
 - Faster for short distances (due to simultaneous transmission).
 - Suffers from signal degradation, skew, and crosstalk over longer distances.

3. Hardware Complexity and Cost

- Serial
 - Requires fewer wires, connectors, and pins.
 - Simpler and more cost-effective for compact systems.
- Parallel
 - Requires more wires and synchronisation mechanisms.
 - Increases system cost and design complexity.

4. Synchronisation and Timing

- **Serial**
 Easier to synchronise, especially in asynchronous modes.
 In synchronous serial, clocking is shared or embedded.
- **Parallel**
 Requires precise timing alignment of all bits to avoid errors.
 Clock skew becomes a problem as speed or distance increases.

5. Data Integrity

- Serial
 More robust for long-distance communication.
 Less noise due to fewer signal lines.
- Parallel
 Higher chances of interference and bit distortion, especially when cable lengths increase.

5. Common Use Cases

Table 10: Use Cases

Serial Communication	Parallel Communication
USB, RS-232, SPI, I ² C	Internal CPU data buses, older printers (Centronics)
Long-distance networking	Short-distance, high-speed memory or peripheral buses
Embedded and compact systems	Motherboard-level parallel buses (e.g., PCI, older IDE)

Table 11: Comparison Table

Feature	Serial Communication	Parallel Communication
Data Lines	1 or 2	Multiple (e.g., 8, 16, 32)

Speed (short distances)	Moderate	High
Distance Capability	Excellent	Poor
Noise Susceptibility	Low	High
Cost	Low	Higher (more wiring)
Hardware Complexity	Simple	Complex
Synchronisation	Easier	Requires precise timing

Serial communication is preferred for long-distance, low-cost, and low-pin applications, while parallel communication excels in short-distance, high-speed data transfers within systems. The choice between them depends on the specific performance, distance, and resource requirements of the application.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice questions	
1	In serial communication, data is transmitted: a) In parallel over multiple lines b) Bit by bit over a single line c) Using DMA channels d) Only in one direction
2	Which of the following protocols is asynchronous? a) SPI b) I ² C c) RS-232 d) USB
3	What is a key advantage of serial communication over parallel communication? a) Higher speed over short distances b) Lower cost and fewer wires c) More complex synchronisation d) Requires multiple data lines
4	Which protocol supports multiple devices with unique addresses over two lines? a) SPI b) RS-232 c) USB

	d) I ² C
Fill in the Blank	
5	In asynchronous transmission, each data unit is framed with _____ and _____ bits.
6	USB is a _____ serial communication protocol that supports plug-and-play functionality.
7	SPI uses four main lines: MISO, MOSI, SCLK, and _____.
8	Parallel communication suffers from signal degradation and _____ over long distances.
True or False	
9	Serial communication is more suitable than parallel communication for long-distance data transfer.
10	Asynchronous transmission requires a shared clock between sender and receiver.
11	USB can support up to 127 devices connected to a single host.
12	I ² C is less scalable than SPI because it requires more wires per device.



6. SUMMARY

- This unit explores advanced Input/Output (I/O) mechanisms that enhance the efficiency, speed, and scalability of modern computer systems. Building on foundational I/O concepts, it introduces three key technologies: Direct Memory Access (DMA), Input-Output Processor (IOP), and Serial Communication.
- The unit begins by explaining the need for advanced I/O due to the limitations of traditional methods like Programmed I/O and Interrupt-Driven I/O, which involve high CPU overhead, low throughput, and limited multitasking capabilities. Advanced mechanisms address these issues by enabling faster data transfers and reducing CPU involvement.
- DMA is covered in detail, including its operation, transfer modes (burst, cycle stealing, and transparent), advantages (like high-speed transfer and low CPU usage), and applications in areas such as disk access, networking, and multimedia. DMA allows peripherals to communicate directly with memory, bypassing the CPU.
- The unit then introduces the IOP, a dedicated processor that manages I/O operations independently. It supports parallel processing, handles multiple devices, and communicates with the CPU through shared memory, interrupts, and command/status registers. A comparison between DMA and IOP highlights their differences in complexity, flexibility, and use cases.
- Finally, the unit delves into serial communication, explaining its principles, types (asynchronous and synchronous), and protocols such as RS-232, USB, SPI, and I²C. It contrasts serial with parallel communication, emphasising the former's suitability for long-distance, low-pin-count, and embedded applications.
- By the end of the unit, learners understand how advanced I/O mechanisms improve system performance, support multitasking, and enable efficient communication with a wide range of peripheral devices.

7. GLOSSARY

Advanced I/O Mechanisms	- Techniques like DMA, IOP, and serial communication that improve data transfer efficiency and reduce CPU workload.
Direct Memory Access (DMA)	- A method that allows peripherals to transfer data directly to or from memory without CPU intervention.
DMA Controller (DMAC)	- A hardware component that manages DMA operations, including bus requests and data transfers.
Burst Mode	- A DMA transfer mode where a block of data is transferred in one continuous burst, halting CPU access to the bus.
Cycle Stealing Mode	- A DMA mode where the controller temporarily takes control of the bus for one cycle at a time, allowing CPU and DMA to share the bus.
Transparent Mode	- A DMA mode where data transfer occurs only when the CPU is not using the bus, ensuring no CPU interruption.
Input-Output Processor (IOP)	- A dedicated processor that handles I/O operations independently of the CPU, enabling parallel processing.
IOP Architecture	- Includes control unit, instruction decoder, local memory, registers, I/O channels, and bus interface unit.
CPU-IOP Communication	- Interaction between CPU and IOP using shared memory, command/status registers, interrupts, and bus arbitration.
Serial Communication	- Data transmission method where bits are sent one at a time over a single line, suitable for long-distance and low-pin applications.
Asynchronous Transmission	- Serial communication without a shared clock, using start and stop bits for each data unit.
Synchronous Transmission	- Serial communication with a shared clock, allowing continuous data blocks to be sent efficiently.
RS-232	- An asynchronous serial protocol used for point-to-point communication, common in legacy systems.

USB (Universal Serial Bus)	- A high-speed serial protocol supporting multiple devices, plug-and-play functionality, and power delivery.
SPI (Serial Peripheral Interface)	- A synchronous serial protocol using multiple lines for fast communication between microcontrollers and peripherals.
I²C (Inter-Integrated Circuit)	- A synchronous serial protocol using two lines for communication between multiple devices on a circuit board.
Parallel Communication	- Data transmission method where multiple bits are sent simultaneously over multiple lines, faster but more complex than serial.
Bus Arbitration	- A mechanism to manage access to the system bus when multiple processors or controllers request usage.
Baud Rate	- The number of bits transmitted per second in serial communication.
Start/Stop Bits	- Bits used in asynchronous transmission to mark the beginning and end of a data unit.
Parity Bit	- A bit used for simple error detection in data transmission.

8. TERMINAL QUESTIONS

1. What are the key limitations of traditional I/O methods like Programmed I/O and Interrupt-Driven I/O?
2. Explain the need for advanced I/O mechanisms in modern computer systems.
3. Describe the basic operation of Direct Memory Access (DMA).
4. Compare the three DMA transfer modes: burst mode, cycle stealing, and transparent mode.
5. What are the main advantages and disadvantages of using DMA?
6. List at least four real-world applications where DMA is commonly used.
7. What is an Input-Output Processor (IOP), and how does it differ from a DMA controller?
8. Describe the architecture of an IOP and its key components.
9. How does the IOP communicate with the CPU during I/O operations?
10. Compare and contrast DMA and IOP in terms of control, flexibility, and use cases.
11. What is serial communication, and how does it differ from parallel communication?
12. Distinguish between asynchronous and synchronous serial transmission.
13. Explain the working principles and typical use cases of RS-232 and USB protocols.
14. What are the key differences between SPI and I²C serial communication protocols?
15. In what scenarios would you prefer serial communication over parallel communication, and why?

9. ANSWERS

Self-Assessment Questions

Self-Assessment Questions - 1

1. c) Improve data transfer efficiency
2. c) Low throughput
3. c) DMA
4. b) Handles I/O tasks independently
5. CPU
6. bottlenecks
7. advanced
8. interrupt service routine
9. False
10. True
11. False
12. True

Self-Assessment Questions - 2

1. c) Transfer data between memory and I/O devices
2. c) Cycle Stealing Mode
3. c) Increased hardware complexity
4. c) Audio and video streaming
5. setup
6. block
7. idle
8. CPU
9. True
10. False
11. False
12. True

Self-Assessment Questions - 3

1. c) Handle I/O operations independently of the CPU

2. b) Instruction Decoder
3. c) Via shared memory and command/status registers
4. b) More complex and capable of executing I/O instructions
5. concurrently
6. control
7. interrupts
8. instruction
9. True
10. False
11. True
12. False

Self-Assessment Questions - 4

1. b) Bit by bit over a single line
2. c) RS-232
3. b) Lower cost and fewer wires
4. d) I²C
5. start, stop
6. synchronous
7. SS/CS
8. crosstalk
9. True
10. False
11. True
12. False

Terminal Questions Answers

Answer 1: Traditional I/O methods like Programmed I/O and Interrupt-Driven I/O are limited by high CPU involvement, low throughput, and poor scalability. These methods require the CPU to either constantly poll devices or frequently handle interrupts, which can slow down overall system performance and reduce multitasking efficiency.

Refer Section 2.2 - Limitations of Traditional I/O Methods

Answer 2: Advanced I/O mechanisms are essential in modern systems to handle high-speed data transfers, reduce CPU workload, and support concurrent operations. They are particularly useful in real-time, multimedia, and embedded applications where responsiveness and efficiency are critical.

Refer Section 2.1 - Need for Advanced I/O

Answer 3: DMA operation allows peripheral devices to transfer data directly to or from system memory without continuous CPU intervention. The CPU initiates the transfer by configuring the DMA controller, which then manages the data movement independently, freeing the CPU for other tasks.

Refer Section 3.1 - DMA Operation

Answer 4: Burst mode transfers a large block of data in one go, temporarily halting CPU access to the bus. Cycle stealing allows the DMA controller to take control of the bus for one cycle at a time, enabling shared access. Transparent mode performs transfers only when the CPU is idle, ensuring no interruption to CPU operations.

Refer Section 3.2 - DMA Transfer Modes

Answer 5: DMA advantages include high-speed data transfer, reduced CPU load, and improved multitasking. Disadvantages involve increased hardware complexity, potential bus contention, and limited DMA channels, which may restrict simultaneous device support.

Refer Section 3.3 - Advantages and Disadvantages of DMA

Answer 6: DMA is commonly used in hard disk and SSD data transfers, network interface cards, audio/video streaming, GPU memory access, embedded systems, digital signal processing, memory initialisation, and industrial automation. These applications benefit from DMA's ability to handle large data volumes efficiently.

Refer Section 3.4 - Applications of DMA

Answer 7: An IOP is a dedicated processor that manages I/O operations independently of the CPU. Unlike DMA, which only handles data transfer, the IOP can decode and execute I/O instructions, manage multiple devices, and perform intelligent control tasks, making it suitable for complex systems.

Refer Section 4.3 - IOP vs DMA

Answer 8: IOP architecture includes a control unit, instruction decoder, local memory, registers, I/O channels, and a bus interface. These components enable the IOP to process I/O commands, manage device communication, and operate concurrently with the CPU.

Refer Section 4.1 - Architecture of IOP

Answer 9: The CPU communicates with the IOP using command and status registers, shared memory, interrupts, and bus arbitration. This coordination allows the CPU to delegate I/O tasks and receive updates or results from the IOP, enabling efficient parallel processing.

Refer Section 4.4 - CPU-IOP Communication

Answer 10: DMA is simpler and ideal for fast, bulk data transfers with minimal logic. IOP, on the other hand, is more flexible and capable of managing multiple devices and complex I/O tasks independently. DMA is best for speed; IOP is best for intelligent control and multitasking.

Refer Section 4.3 - IOP vs DMA

Answer 11: Serial communication sends data one bit at a time over a single line, making it suitable for long-distance and low-pin-count applications. Parallel communication transmits multiple bits simultaneously over multiple lines, offering higher speed but increased complexity and cost.

Refer Section 5.4 - Comparison with Parallel Communication

Answer 12: Asynchronous transmission uses start and stop bits to frame each data unit and does not require a shared clock. Synchronous transmission relies on a shared clock and sends data in continuous blocks, offering higher speed and efficiency for large data transfers.

Refer Section 5.2 - Asynchronous vs Synchronous Transmission

Answer 13: RS-232 is a legacy asynchronous protocol used for simple point-to-point communication, often in industrial and legacy systems. USB is a modern, high-speed protocol that supports multiple devices, plug-and-play functionality, and power delivery, making it ideal for peripherals.

Refer Section 5.3 - Serial Protocols: RS-232, USB, SPI, I²C

Answer 14: SPI is a fast, full-duplex protocol using multiple lines and a master-slave setup, suitable for high-speed sensor communication. I²C uses only two lines and supports multiple devices with addressing, making it ideal for compact, low-power embedded systems.

Refer Section 5.3 - Serial Protocols: RS-232, USB, SPI, I²C

Answer 15: Serial communication is preferred over parallel in scenarios requiring long-distance transmission, fewer wires, and lower cost. It is more robust against noise and easier to implement in embedded and mobile systems, whereas parallel is better for short-distance, high-speed internal transfers.

Refer Section 5.4 - Comparison with Parallel Communication



10. REFERENCES

BOOKS

1. Computer Architecture: A Quantitative Approach, John L. Hennessy, David A. Patterson, 6th Edition, 2017
2. Computer Organisation and Architecture: Designing for Performance, William Stallings, 11th Edition, 2018
3. Computer System Architecture, M. Morris Mano, 3rd Edition, 2006
4. Computer Organisation and Design: The Hardware/Software Interface, David A. Patterson, John L. Hennessy, 5th Edition, 2013
5. Digital Design and Computer Architecture, Sarah Harris, David Harris, 3rd Edition, 2021

E-REFERENCES

1. Logical and Shift Micro-operations Computer Organisation and Architecture, <https://care4you.in/logical-and-shift-micro-operations/>
2. Logic and Shift Micro Operations, <https://www.scribd.com/document/394026693/U-I-Logic-and-Shift-micro-operations>

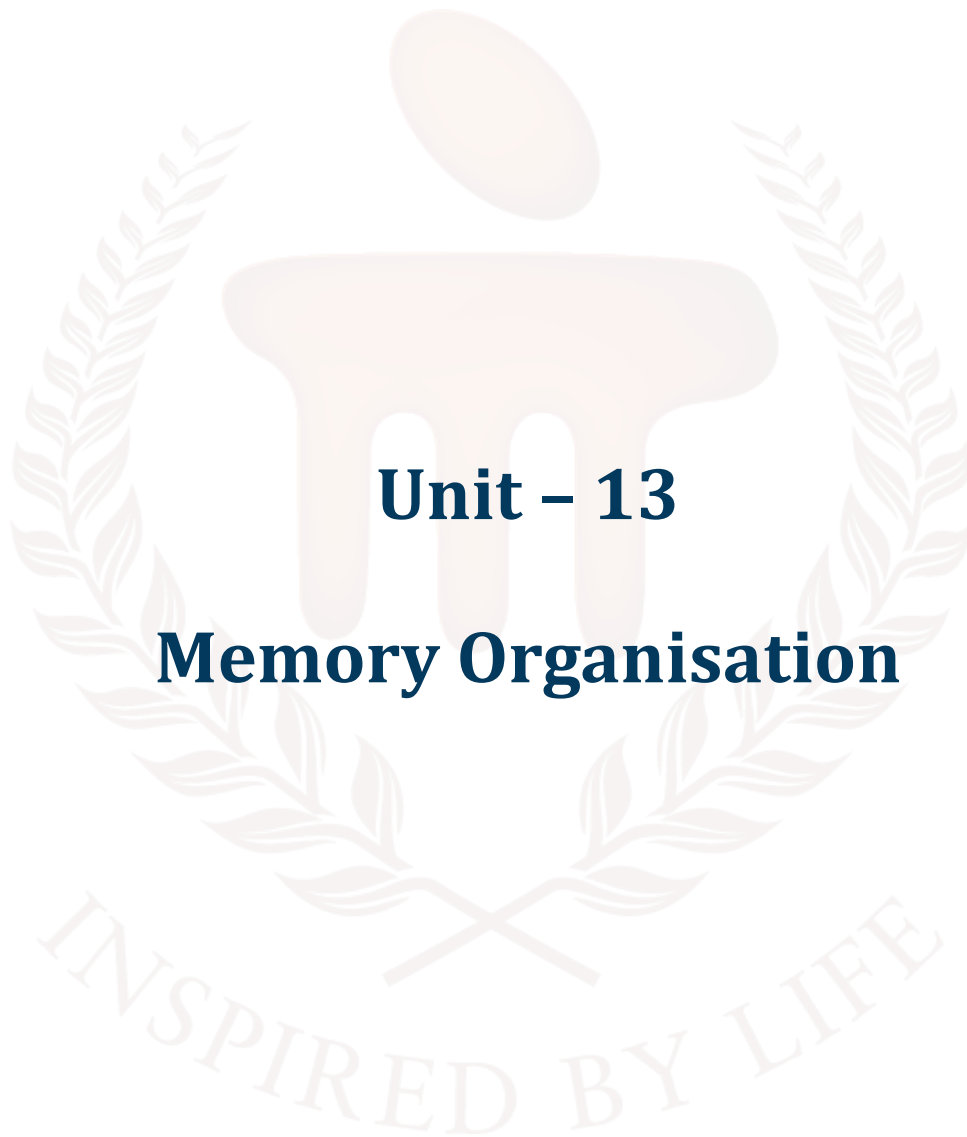
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 13

Memory Organisation

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4
1.1	Objectives	-	-	
2	Memory Hierarchy Overview	-	-	5 - 11
2.1	Memory Hierarchy Concept	-	-	
2.2	Characteristics of Memory	i	-	
2.3	Types of Memory (Primary, Secondary, Tertiary)	ii	1	
3	Associative Memory	-	-	12 - 17
3.1	Concept of Associative (Content Addressable) Memory	-	-	
3.2	Hardware Organisation	1	-	
3.3	Match Logic and Applications	-	2	
4	Cache Memory	-	-	18 - 24
4.1	Need for Cache Memory	-	-	
4.2	Cache Mapping Techniques (Direct, Associative, Set-Associative)	iii	-	
4.3	Cache Performance and Hit Ratio	iv	3	
5	Virtual Memory	-	-	25 - 36
5.1	Concept and Purpose of Virtual Memory	2	-	
5.2	Paging and Segmentation	3, 4, v	-	
5.3	Page Replacement Algorithms	5, 6, vi	-	
5.4	Advantages and Limitations	-	4	
6	Summary	-	-	37
7	Glossary	-	-	38 - 39
8	Terminal Questions	-	-	40
9	Answers	-	-	41 - 45
10	References	-	-	46

1. INTRODUCTION

In the previous unit, we explored various advanced I/O mechanisms that enhance system efficiency and data handling capabilities. Topics covered included Direct Memory Access (DMA), Input-Output Processor (IOP), and Serial Communication protocols such as RS-232, USB, SPI, and I²C. These mechanisms demonstrated how modern systems manage I/O operations with speed and minimal CPU involvement.

In this unit, we shift our focus to a critical component of computer architecture — Memory Organisation. Memory plays a central role in overall system performance, and its efficient organisation ensures faster data access, optimal utilisation, and support for large, complex applications.

We will begin by reviewing the memory hierarchy and its various types, followed by an in-depth discussion on associative memory (also known as content-addressable memory). We will then explore the structure and functioning of cache memory, which acts as a high-speed intermediary between the CPU and main memory. Finally, we will examine the concept of virtual memory, including its implementation through paging and segmentation, and techniques like page replacement algorithms.

By the end of this unit, learners will gain a comprehensive understanding of how different memory types work together to support efficient processing and system scalability.

1.1. Objectives

After studying this unit, you should be able to:

- Discuss the concept of memory hierarchy
- Explain the working and structure of associative (content-addressable) memory.
- Discuss the need for cache memory and compare various cache mapping techniques.
- Explain the concept of virtual memory and its implementation using paging and segmentation.
- Outline the page replacement algorithms used in virtual memory systems.



2. MEMORY HIERARCHY OVERVIEW

The memory hierarchy in computer systems is an organised structure of various memory types arranged by speed, cost, and size. This layered approach ensures that frequently accessed data is stored in faster memory, while less-used data is moved to slower, high-capacity storage, balancing performance and efficiency.

2.1. Memory Hierarchy Concept

In a computer system, different types of memory are used to store data and instructions. These memories vary in speed, cost, and capacity. To achieve an optimal balance between performance and cost, modern computer architectures are designed using a memory hierarchy.

What is Memory Hierarchy?

Memory hierarchy is the structured arrangement of storage systems in a computer based on access speed, storage capacity, and cost per bit. It organises memory components in a layered format where:

- Top levels are the fastest and most expensive (e.g., CPU registers, cache),
- Bottom levels are slower but larger and cheaper (e.g., hard disk, external drives).

This hierarchy ensures that frequently accessed data remains in faster memory, while less-used data is stored in slower memory.

Typical Memory Hierarchy Levels (Top to Bottom)

1. CPU Registers

- Smallest and fastest memory located within the processor.
- Stores immediate operands and results of calculations.

2. Cache Memory

- High-speed memory between the CPU and main memory.
- Stores recently or frequently used instructions and data.

3. Main Memory (RAM)

- Medium-speed, volatile memory used for executing programs.
- Stores current processes, instructions, and data in use.

4. Secondary Storage (Hard Disk, SSD)

- Non-volatile storage with high capacity but slower access.
- Stores OS, applications, and user data permanently.

5. Tertiary Storage (Optical Discs, Tape Drives, Cloud Storage)

- Used for backup, archival, and long-term storage.
- Slowest and typically accessed manually or by the system when needed.

Why Use a Memory Hierarchy?

- Performance Optimisation: Fast memory is expensive and limited in size. A hierarchy allows fast access to critical data while storing the rest economically.
- Cost Efficiency: Using a mix of high-speed and low-cost memory balances performance and affordability.
- Scalability: The layered approach supports growing data and system demands efficiently.

Working Principle

The memory hierarchy follows the principle of locality of reference:

- Temporal locality: Recently accessed data is likely to be accessed again soon.
- Spatial locality: Data near the currently accessed data is also likely to be accessed.

By keeping this frequently used data in faster memory layers, overall access time is minimised.

The memory hierarchy is a cornerstone of computer architecture, allowing systems to achieve high speed and large capacity at a reasonable cost. Understanding how these memory layers work together is essential for designing efficient computing systems.

2.2. Characteristics of Memory

Different types of memory in a computer system are distinguished by several key characteristics. These characteristics determine their suitability for specific roles in the memory hierarchy and influence overall system performance, cost, and design decisions.

Key Characteristics of Memory

1. Access Time

- The time it takes to read data from or write data to the memory.
- Faster memory (e.g., cache, registers) has shorter access times.

- Measured in nanoseconds (ns) or milliseconds (ms).

2. Memory Capacity

- The amount of data a memory unit can store.
- Higher-capacity memories (e.g., HDDs, SSDs) can store large amounts of data but are slower.
- Measured in bytes (e.g., KB, MB, GB, TB).

3. Cost per Bit

- Refers to the price of storing each bit of data.
- Faster and smaller memories are more expensive per bit than slower, larger ones.

4. Volatility

- Volatile memory loses its data when power is turned off (e.g., RAM, cache).
- Non-volatile memory retains data even when power is lost (e.g., ROM, SSD, HDD).

5. Location

- Memory may be located inside the CPU (e.g., registers), close to the CPU (e.g., cache), or external to the CPU (e.g., main and secondary memory).

6. Bandwidth

- The amount of data that can be transferred to or from memory in a given time.
- High bandwidth improves performance for large data operations.

7. Physical Size

- The actual size of the memory chip/module.
- Smaller form factors are important in embedded and portable systems.

Table 1: Comparison Table

Characteristic	Registers	Cache	Main Memory	Secondary Storage
Access Time	1 ns	5–20 ns	50–100 ns	1–10 ms
Capacity	Bytes	KB to MB	GB	GB to TB
Cost per Bit	Highest	High	Moderate	Lowest
Volatility	Volatile	Volatile	Volatile	Non-volatile
Location	CPU	Near CPU	Motherboard	External/Internal

Understanding the characteristics of different memory types helps in designing efficient systems that balance speed, capacity, cost, and power. These properties also guide the placement of memory types within the memory hierarchy, ensuring optimal system performance.

2.3. Types of Memory (Primary, Secondary, Tertiary)

Memory in a computer system is categorised into different types based on function, speed, and storage capacity. These types—primary, secondary, and tertiary memory—play distinct roles in ensuring data availability and efficient system operation. Understanding these types is crucial to appreciating how memory is organised and accessed in a hierarchical structure.

1. Primary Memory (Main Memory)

Primary memory is the main working memory of the computer. It holds data and instructions that the CPU needs during execution.

Types:

- RAM (Random Access Memory): Volatile memory used for temporary data and program storage.
- ROM (Read-Only Memory): Non-volatile memory that stores permanent system instructions (e.g., BIOS).

Characteristics:

- Fast access time
- Volatile (RAM loses data when power is off)
- Limited capacity
- Directly accessible by the CPU

Examples:

- DDR4/DDR5 RAM modules, Flash ROM

2. Secondary Memory (Storage Devices)

Secondary memory provides long-term data storage and is used to store programs, files, and the operating system.

Types:

- Hard Disk Drives (HDD)

- Solid State Drives (SSD)
- Optical Discs (CD/DVD)

Characteristics:

- Non-volatile
- Slower than primary memory
- High storage capacity
- Not directly accessed by the CPU (requires I/O operations)

Examples:

- 1 TB HDD, 512 GB SSD

3. Tertiary Memory (Backup/Archival Storage)

Tertiary memory refers to storage used for data backup, archival, and long-term storage. It is often used in data centers and enterprise systems.

Types:

- Magnetic Tape Drives
- Optical Jukeboxes
- Cloud Storage Services

Characteristics:

- Non-volatile
- Very high capacity
- Slow access time (often requires manual or automated mounting)
- Typically used infrequently

Examples:

- Tape libraries in data centers
- Cloud-based backup platforms (e.g., AWS Glacier)

Table 2: Comparison Table

Memory Type	Volatility	Speed	Capacity	Access Method	Usage
Primary	Volatile	Very Fast	Low–Medium	Direct (CPU)	Active programs, data

Secondary	Non-volatile	Moderate	High	Indirect (via I/O)	OS, files, applications
Tertiary	Non-volatile	Slow	Very High	Indirect/manual	Backups, archival storage

Each type of memory—primary, secondary, and tertiary—serves a specific function in the system. Together, they form the foundation of the memory hierarchy, supporting both the performance and storage needs of modern computing environments.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which memory type is located closest to the CPU and offers the fastest access? a) Cache b) RAM c) CPU Registers d) SSD
2	What is the primary reason for implementing a memory hierarchy in computer systems? a) To reduce power consumption b) To increase system size c) To balance performance and cost d) To eliminate the need for RAM
3	Which of the following is a characteristic of secondary memory? a) Volatile b) Low capacity c) Directly accessed by CPU d) Non-volatile
4	Which type of memory is used for long-term archival and backup purposes? a) Primary Memory b) Secondary Memory c) Tertiary Memory d) Cache Memory
Fill in the Blank	
5	The principle of _____ locality suggests that recently accessed data is likely to be accessed again soon.

6	_____ memory is used to store data and instructions currently being processed by the CPU.
7	The time taken to read or write data from memory is known as _____.
8	_____ storage devices are typically used for data backups and are accessed less frequently.
True or False	
9	Cache memory is slower than main memory.
10	RAM is a type of non-volatile memory.
11	Secondary memory is not directly accessed by the CPU and requires I/O operations.
12	The cost per bit of CPU registers is higher than that of hard disks.



3. ASSOCIATIVE MEMORY

Associative memory, also known as Content Addressable Memory (CAM), is a special type of memory that retrieves data based on content rather than address. It allows parallel searching of stored information, making it ideal for high-speed lookups in applications like cache memory, virtual memory systems, and networking.

3.1. Concept of Associative (Content Addressable) Memory

Associative Memory, also known as Content Addressable Memory (CAM), is a type of memory that allows data retrieval based on the content rather than a specific memory address. Unlike traditional memory systems, where data is accessed by referencing a unique address, associative memory searches all memory locations simultaneously and returns the data that matches a given key or pattern.

Working Principle

- Each memory word is stored with a unique tag or key.
- When a search key is presented, the memory compares the key in parallel with all stored keys.
- If a match is found, the corresponding data word is accessed or output.
- If multiple matches occur, the system can return all matches or use priority logic to select one.

Applications of Associative Memory

- Cache memory tag matching in CPU architectures.
- Translation Lookaside Buffer (TLB) for virtual-to-physical address mapping.
- Network routing (fast lookups in routing tables).
- Pattern recognition, machine learning, and database indexing.

Advantages

- Very fast search time due to parallel comparison.
- Efficient for applications requiring frequent, content-based lookups.

Disadvantages

- Expensive to implement due to parallel hardware structure.
- Typically has lower storage density than traditional RAM.

Associative (Content Addressable) Memory is a powerful tool when speed and pattern-based data retrieval are essential. Though not commonly used for general-purpose storage, it is critical in performance-sensitive subsystems like CPU caches and virtual memory management.

3.2. Hardware Organisation

The hardware organisation of associative memory is designed to enable parallel comparison of input data with all stored entries simultaneously. This makes it fundamentally different from traditional memory systems and allows for extremely fast data retrieval based on content.

Basic Structure of Associative Memory

An associative memory unit consists of the following key components:

1. Memory Array

- Contains n words, each of m bits.
- Each word is stored in a register, and all words are accessible simultaneously for comparison.

2. Key Register

- Holds the search data or pattern that is to be matched.
- The number of bits in the key register matches the word length (m bits).

3. Mask Register (optional)

- Used to select specific bits of the key for comparison.
- Allows partial matching (e.g., comparing only selected fields in a word).

4. Match Logic

- Each word is connected to a comparator circuit that performs bit-by-bit comparison between the key and stored word.
- If all compared bits match, a match signal is generated for that word.

5. Match Register

- Stores the results of comparisons from all words.
- Indicates which memory locations have matched the key.

6. Read/Write Logic

- Once a match is found, the corresponding word can be read or updated.

- Logic circuits control read/write selection, enabling lines, and data lines.

Block Diagram

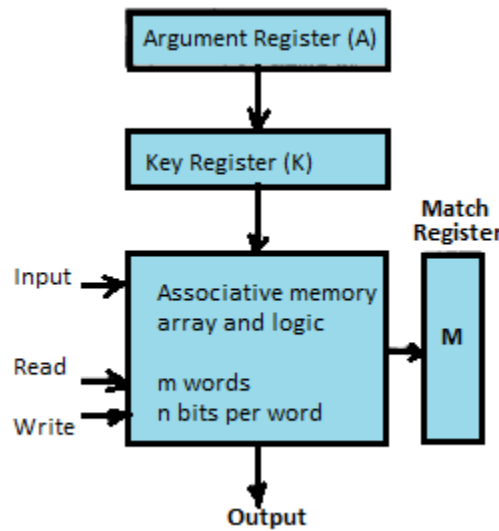


Figure 1: Associative Memory

Working Process

1. **Input:** Load the search key into the key register.
2. **Comparison:** All memory words are compared in parallel using the match logic.
3. **Match Detection:** If a word matches the key (fully or partially), a match signal is triggered.
4. **Action:** The system either retrieves the matched data or updates it, depending on the operation.

Benefits of This Hardware Design

- **Speed:** Enables real-time data lookup due to parallelism.
- **Efficiency:** Reduces search time significantly compared to sequential searching.
- **Flexibility:** With a mask register, it supports field-based comparisons.

Limitations

- **Hardware Complexity:** Requires more comparators and control logic.
- **Power Consumption:** Simultaneous comparison across all words consumes more power.
- **Scalability:** Less scalable for very large memory sizes compared to conventional RAM.

The hardware organisation of associative memory is tailored for fast content-based searches. Though complex and resource-intensive, it is crucial in scenarios where search speed is more important than storage capacity, such as in caches, TLBs, and networking hardware.

3.3. Match Logic and Applications

Match logic is the core component of associative memory that enables the system to compare a search key with stored words simultaneously. It determines whether a particular memory word matches the input pattern and activates the corresponding output line. This parallel comparison makes associative memory extremely fast and well-suited for specialised, high-speed applications.

Match Logic Mechanism

Each word in associative memory is associated with a comparator circuit that compares bit-by-bit values of:

- The input key from the key register
- The stored word in memory
- Optionally, a mask bit to indicate if the bit should be compared or ignored

Bit Match Logic:

For each bit position, the logic performs:

If (Mask Bit = 1) → Compare Key Bit with Stored Bit

If (Key Bit = Stored Bit) → Match

Else → Mismatch

Word Match Logic:

- A word is considered a full match only if all relevant bits match.
- Match lines for each word are set to 1 (true) or 0 (false), indicating the result.
- Priority Logic (Optional):
- If multiple words match, priority encoders or selection logic can determine which match to act on first.

Applications of Associative Memory

Associative memory's ability to search quickly by content rather than address makes it extremely useful in several domains:

1. Cache Memory (Tag Matching)

- Used to compare requested memory addresses with cache tags to determine cache hits/misses.

2. Translation Lookaside Buffer (TLB)

- In virtual memory systems, TLBs use associative memory to rapidly match virtual page numbers with their corresponding physical page frames.

3. Network Routing Tables

- Routers and switches use associative memory for high-speed lookups of IP addresses and routing paths.

4. Pattern Recognition

- Used in systems that require fast matching of input patterns, such as fingerprint recognition, face detection, or machine learning classifiers.

5. Database Search Engines

- Speeds up searching in key-value databases and indexing systems where data is retrieved by value rather than location.

The match logic in associative memory enables rapid, content-based data access through parallel comparison mechanisms. This makes it invaluable in real-time systems, memory management, and high-speed data routing, where performance depends on the ability to find information quickly and efficiently.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
1	What is the key feature of associative memory? a) Sequential data access b) Address-based retrieval c) Content-based retrieval d) Low-speed access
2	Which component holds the search pattern in associative memory? a) Match register b) Key register c) Mask register d) Memory array
3	What is the role of the mask register in associative memory? a) Stores matched data

	b) Controls read/write operations c) Selects specific bits for comparison d) Holds the search key
4	Which of the following is a common application of associative memory? a) Arithmetic operations b) Instruction decoding c) Cache tag matching d) File compression
Fill in the Blank	
5	Associative memory is also known as _____ memory.
6	The _____ logic performs parallel comparison between the key and stored words.
7	In associative memory, each word is stored with a unique _____ or key.
8	The _____ register stores the results of comparisons from all memory words.
True or False	
9	Associative memory retrieves data based on its physical address.
10	Match logic enables parallel comparison of input data with all stored entries.
11	Associative memory is commonly used for general-purpose data storage.
12	The mask register allows partial matching by ignoring selected bits during comparison.

4. CACHE MEMORY

Cache memory is a small, high-speed storage unit placed between the CPU and main memory to bridge the speed gap between them. It stores frequently accessed data and instructions, enabling faster access and improving overall system performance by reducing memory latency.

4.1. Need for Cache Memory

Modern computer processors operate at very high speeds, while access to main memory (RAM) is comparatively slower. This speed mismatch creates a performance bottleneck, where the CPU spends a significant amount of time waiting for data from memory. To reduce this delay and improve overall system performance, a cache memory is introduced.

What is Cache Memory?

Cache memory is a small, high-speed memory unit located between the CPU and main memory. It stores copies of frequently accessed instructions and data, enabling the processor to retrieve them much faster than from main memory.

Why Is Cache Memory Needed?

1. Bridging the Speed Gap

- CPU speed is much faster than main memory.
- Without cache, the CPU must wait for memory access, which slows down execution.

2. Exploiting Locality of Reference

- Temporal locality: Recently accessed data is likely to be accessed again soon.
- Spatial locality: Nearby data (in memory) is likely to be accessed soon.
- Cache takes advantage of both by keeping relevant blocks of data close to the processor.

3. Reducing Memory Latency

- Cache significantly reduces the average memory access time, improving CPU efficiency.
- Accessing cache is measured in nanoseconds, while RAM can take several times longer.

4. Boosting Instruction Execution Speed

- Frequently executed instructions (loops, branches) reside in cache.
- This allows the CPU to fetch instructions rapidly and maintain high throughput.

Real-World Analogy

Think of cache as a desk drawer where you keep your most-used tools. Instead of going to the storeroom (main memory) every time, you grab them from the drawer instantly—saving time and effort.

The need for cache memory arises from the desire to maximise CPU performance by minimising memory delays. By storing frequently used data close to the processor, cache plays a vital role in accelerating program execution and overall system responsiveness.

4.2. Cache Mapping Techniques (Direct, Associative, Set-Associative)

When data is stored in cache memory, the system needs a method to determine where in the cache a particular block of data from main memory should go. These methods are known as cache mapping techniques. The goal is to efficiently store, locate, and replace memory blocks in the cache to optimise speed and hit ratio.

The three primary cache mapping techniques are:

1. Direct Mapping

Concept:

- Each block of main memory maps to exactly one cache line.
- Simple and fast, but can lead to frequent conflicts if multiple blocks map to the same cache location.

Mapping Formula:

Cache Line = (Main Memory Block Number) MOD (Number of Cache Lines)

Structure:

- Each cache line stores:
- Tag (to identify the memory block)
- Data block
- Valid bit

Advantages:

- Simple hardware implementation
- Fast access time

Disadvantages:

- High conflict rate
- Poor performance if multiple frequently used blocks map to the same line

2. Associative Mapping**Concept:**

- A memory block can be placed in any cache line.
- Cache is searched fully (associatively) to find the block.

Structure:

- Each line stores:
- Tag (unique identifier for the memory block)
- Data block
- Valid bit

Advantages:

- Flexible placement
- Minimises conflicts

Disadvantages:

- Requires complex hardware for searching all tags in parallel
- Higher cost and access time

3. Set-Associative Mapping**Concept:**

- A compromise between direct and associative mapping.
- Cache is divided into several sets, and each memory block maps to a specific set, but can occupy any line within that set.

Example:

- In a 4-way set-associative cache, each set contains 4 cache lines.
- Block mapping formula:

$$\text{Set Number} = (\text{Main Memory Block Number}) \text{ MOD } (\text{Number of Sets})$$

Structure:

- Each set stores multiple lines, each with:
- Tag
- Data block
- Valid bit

Advantages:

- Balanced conflict reduction and hardware complexity
- Better performance than direct mapping
- Easier to implement than full associative mapping

Disadvantages:

- Slightly more complex than direct mapping
- Some possibility of conflict within sets

Table 3: Comparison Table

Mapping Type	Flexibility	Speed	Hardware Complexity	Conflict Miss Rate
Direct	Low	High	Low	High
Associative	High	Moderate	High	Low
Set-Associative	Moderate	Moderate	Moderate	Low-Moderate

Choosing the right cache mapping technique involves balancing speed, complexity, and conflict handling. Direct mapping is fastest but prone to conflicts, associative mapping offers flexibility but needs complex hardware, and set-associative mapping provides a practical compromise used in most modern processors.

4.3. Cache Performance and Hit Ratio

The effectiveness of cache memory is measured by how well it improves the speed of data access between the CPU and main memory. Two important metrics that determine cache performance are the hit ratio and miss penalty. A well-optimised cache system can significantly boost the performance of a computer by minimising the number of slower memory accesses.

1. Cache Hit and Miss

- **Cache Hit:** Occurs when the data requested by the CPU is found in the cache.

→ Result: Fast access.

- Cache Miss: Occurs when the requested data is not found in the cache and must be fetched from main memory.

→ Result: Slower access due to memory delay.

2. Hit Ratio

The hit ratio is a measure of how often the CPU successfully retrieves data from the cache.

$$\text{Hit Ratio} = \frac{\text{Number of Cache Hits}}{\text{Total Number of Memory Accesses}}$$

- Typically expressed as a percentage (%).
- A higher hit ratio means better cache performance.

Miss Ratio:

$$\text{"Miss Ratio"} = 1 - \text{"Hit Ratio"}$$

3. Miss Penalty

- The extra time required to access data from main memory when a cache miss occurs.
- Affects overall system performance, especially in low hit ratio situations.

4. Average Memory Access Time (AMAT)

A key performance formula that combines hit ratio and miss penalty:

$$\text{AMAT} = (\text{Hit Ratio} \times \text{Cache Access Time}) + (\text{Miss Ratio} \times \text{Miss Penalty})$$

- Lower AMAT indicates better performance.
- Used to evaluate and optimise cache efficiency.

Table 4: Factors Affecting Cache Performance

Factor	Impact on Performance
Cache Size	Larger cache can store more data → higher hit ratio
Block Size	Larger blocks may exploit spatial locality but increase miss penalty
Mapping Technique	Set-associative offers a good balance of speed and flexibility
Replacement Policy	Determines which block to evict (e.g., LRU, FIFO)
Write Policy	Write-through or write-back affect performance and consistency

6. Improving Cache Performance

- Use multi-level caches (L1, L2, L3) to reduce average access time.
- Optimise replacement algorithms to keep frequently used data in cache.
- Design instruction and data caches separately for efficiency.

Cache performance is crucial for overall system efficiency. By increasing the hit ratio and reducing miss penalty, systems can dramatically improve data access speed. Metrics like AMAT help designers evaluate how well the cache supports CPU processing needs.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
1	What is the primary purpose of cache memory in a computer system? a) To store backups b) To reduce CPU clock speed c) To bridge the speed gap between CPU and RAM d) To increase disk storage
2	In direct mapping, how is a block from main memory assigned to cache? a) Randomly b) To any cache line c) To a specific cache line using modulo operation d) Based on priority logic
3	Which cache mapping technique offers a balance between flexibility and hardware complexity? a) Direct Mapping b) Associative Mapping c) Set-Associative Mapping d) Random Mapping
4	What does a high cache hit ratio indicate? a) Frequent cache misses b) Efficient cache performance c) Increased memory latency d) Poor CPU utilisation

Fill in the Blank	
5	Cache memory stores _____ accessed data and instructions to improve processing speed.
6	In associative mapping, a memory block can be placed in _____ cache line.
7	The formula for calculating Average Memory Access Time (AMAT) includes hit ratio and _____.
8	Cache memory is typically located between the _____ and main memory.
True or False	
9	Cache memory is slower than RAM but faster than secondary storage.
10	Direct mapping is simple to implement but may lead to frequent cache conflicts.
11	A low hit ratio in cache memory results in faster data access.
12	Multi-level caches (L1, L2, L3) are used to further reduce average memory access time.



5. VIRTUAL MEMORY

Virtual memory is a memory management technique that allows a computer to compensate for physical memory (RAM) limitations by using a portion of the storage drive (usually the hard disk or SSD) as if it were additional RAM.

5.1. Concept and Purpose of Virtual Memory

Concept of Virtual Memory

Virtual memory works by abstracting physical memory into a large address space called the virtual address space. The operating system, with the help of the Memory Management Unit (MMU), translates virtual addresses used by programs into actual physical memory addresses.

When the required data is not present in RAM, it is fetched from secondary storage (e.g., hard disk) into memory. This process is called paging or swapping.

How Virtual Memory Works

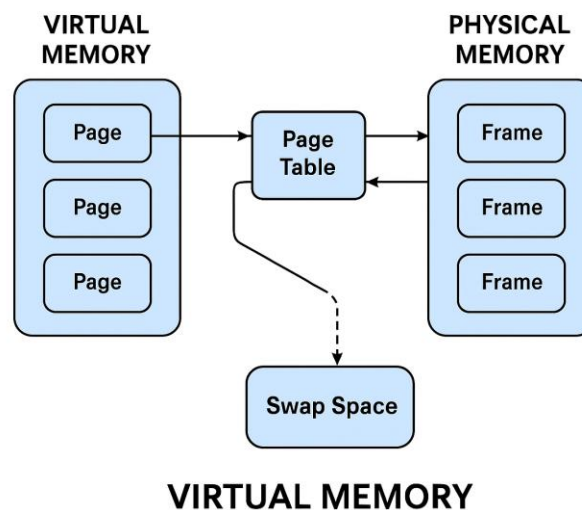


Figure 2: Working of Virtual Memory

1. Each process gets its own virtual address space, which is isolated from others.
2. The system divides virtual and physical memory into fixed-size blocks called pages and frames, respectively.
3. Page tables maintain the mapping between virtual pages and physical frames.
4. If a page is not in memory, a page fault occurs, and the OS loads it from disk.

Purpose of Virtual Memory

1. Execute Large Programs

- Programs can use more memory than what is physically available by swapping parts in and out of RAM.

2. Memory Isolation

- Each process runs in its own virtual space, improving security and stability by preventing one process from accessing another's memory.

3. Efficient Use of RAM

- Only the necessary portions of a program are loaded into RAM, making room for multitasking.

4. Simplified Programming Model

- Developers can code as if there is a large, continuous memory space, reducing complexity in memory handling.

Virtual memory is a fundamental technique that enhances system flexibility, security, and multitasking capabilities. It decouples the constraints of physical memory from program execution, allowing modern systems to run large and multiple applications efficiently.

5.2. Paging and Segmentation

1. Paging

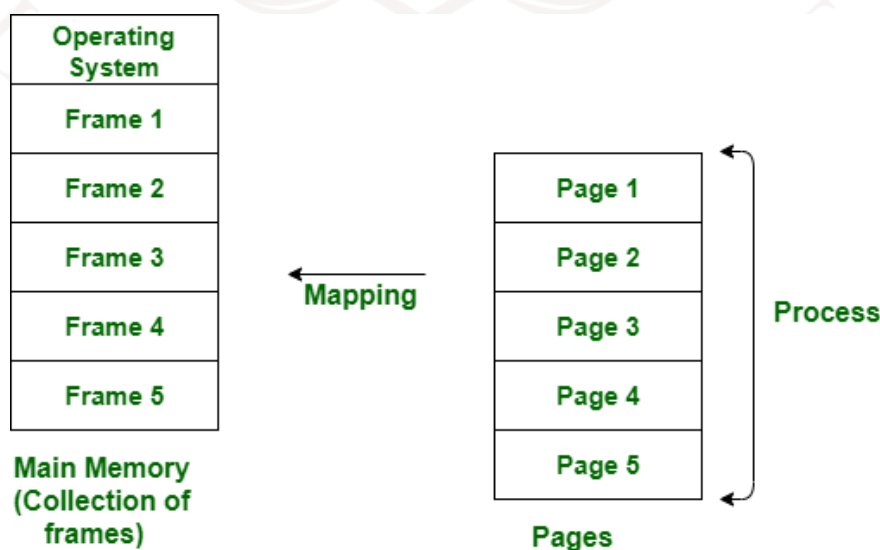


Figure 3: Paging

Concept:

- Paging divides both virtual and physical memory into fixed-size blocks:
 - Pages in virtual memory
 - Frames in physical memory
- Each virtual page is mapped to a physical frame using a page table.

Working:

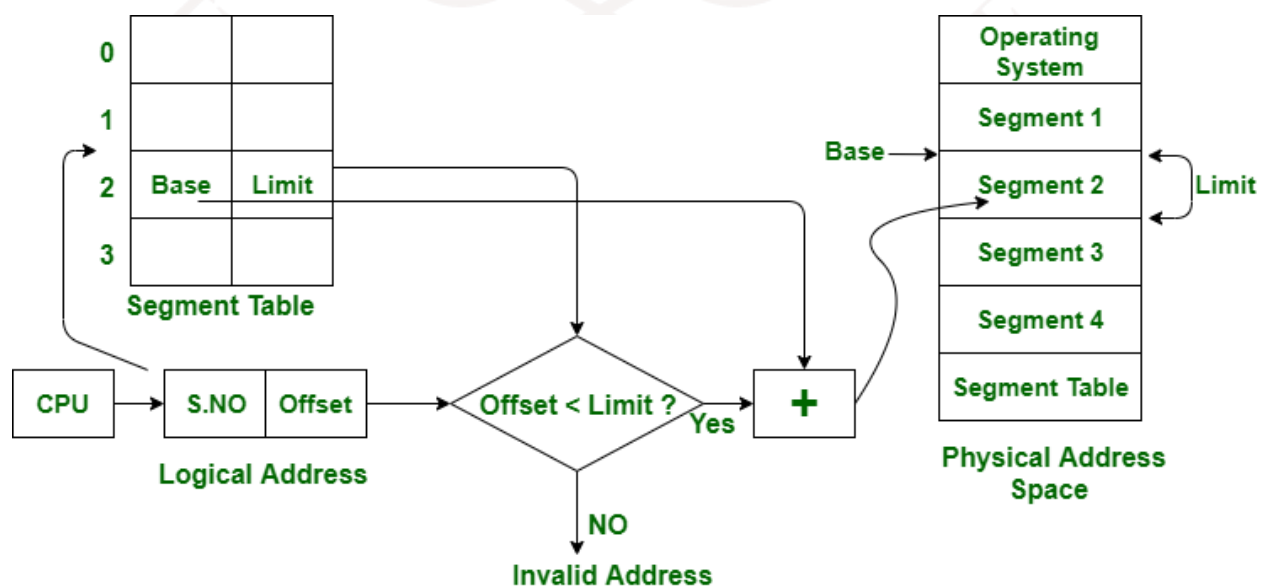
- When a program accesses data, the virtual address is split into:
 - Page number → used to index the page table
 - Offset → used to locate data within the page
- If the page is not in memory, a page fault occurs, and the OS loads it from disk.

Advantages:

- Eliminates external fragmentation
- Simplifies memory management
- Enables efficient virtual memory implementation

Disadvantages:

- Introduces page table overhead
- May cause internal fragmentation

2. Segmentation**Figure 4: Segmentation**

Concept:

- Segmentation divides the program's memory into variable-sized logical segments based on function, such as:
 - Code segment
 - Data segment
 - Stack segment
- Each segment has a base address and limit, stored in a segment table.

Working:

- A virtual address consists of:
 - Segment number → to access the segment table
 - Offset → location within the segment
- The OS checks whether the offset is within the segment's limit to ensure memory protection.

Advantages:

- Reflects logical program structure
- Provides memory protection and isolation
- Easier sharing of code/data between processes

Disadvantages:

- Suffers from external fragmentation
- Complex memory allocation and management

Table 5: Comparison Table: Paging vs. Segmentation

Feature	Paging	Segmentation
Division Type	Fixed-size pages and frames	Variable-size logical segments
Address Structure	Page number + offset	Segment number + offset
Fragmentation	Internal	External
Protection	Limited (per page)	Strong (per segment)
Logical View	Does not match program structure	Matches program's logical structure

Combined Paging and Segmentation

Some systems use a hybrid approach, where segmentation is applied at a high level, and each segment is further divided into pages. This combines the logical structure of segmentation with the efficient

memory management of paging.

Paging and segmentation are essential memory management techniques in virtual memory systems. While paging ensures efficient use of physical memory, segmentation supports logical organisation and protection. Together, they help modern operating systems manage complex memory demands effectively.

5.3. Page Replacement Algorithms

When a virtual memory system runs out of free page frames in physical memory and a page fault occurs, the operating system must choose an existing page to replace. This selection is made using a page replacement algorithm, which directly affects the system's performance, particularly the page fault rate.

Purpose of Page Replacement Algorithms

- To determine which memory page to evict when a new page needs to be loaded into RAM.
- Aim to minimise page faults and maintain efficient use of physical memory.

Common Page Replacement Algorithms

1. FIFO (First-In, First-Out)

- Replaces the oldest loaded page in memory.
- Easy to implement using a queue.

Advantages:

- Simple and fast

Disadvantages:

- Can lead to poor performance (known as Belady's Anomaly — more frames may increase page faults).

Example:

- Page reference string: 1, 2, 3, 4, 1, 2, 5
- Frame size: 3
- Steps:
 - Load 1 → [1]
 - Load 2 → [1, 2]

- Load 3 → [1, 2, 3]
- Load 4 → Replace 1 → [2, 3, 4]
- Load 1 → Replace 2 → [3, 4, 1]
- Load 2 → Replace 3 → [4, 1, 2]
- Load 5 → Replace 4 → [1, 2, 5]

Page Faults: 7

Page Reference String

1	2	3	4	1	1	5
---	---	---	---	---	---	---

1	2	3	→ Replace 1
4	3	4	→ Replace 2
3	4	1	→ Replace 3
1	2	5	→ Replace 4
1	2	5	→ Replace 4

Page Faults: 7

Figure 5: FIFO Representation

2. LRU (Least Recently Used)

- Replaces the page that has not been used for the longest time.
- Based on the idea that pages used recently are more likely to be used again soon.

Advantages:

- Generally performs better than FIFO

Disadvantages:

- Requires tracking recent usage (more complex and costly in hardware/software)

Example:

- Page reference string: 1, 2, 3, 4, 1, 2, 5
- Frame size: 3
- Steps:
 - Load 1 → [1]

- Load 2 → [1, 2]
- Load 3 → [1, 2, 3]
- Load 4 → Replace 1 (least recently used) → [2, 3, 4]
- Load 1 → Replace 2 → [3, 4, 1]
- Load 2 → Replace 3 → [4, 1, 2]
- Load 5 → Replace 4 → [1, 2, 5]

Page Faults: 7

Page Reference String

1	2	3	4	1	2	5	5
---	---	---	---	---	---	---	---

1	2	3	→ Replace 1
4	4	1	→ Replace 1 (least recently used)
3	4	1	→ Replace 2
4	2	5	→ Replace 3
1	2	5	→ Replace 4

Page Faults: 7

Figure 6: LRU Representation

3. Optimal Page Replacement

- Replaces the page that will not be used for the longest time in the future.

Advantages:

- Yields the lowest possible page fault rate

Disadvantages:

- Not practical, since it requires future knowledge of memory access patterns
- Used only for theoretical comparison

Example:

- Page reference string: 1, 2, 3, 4, 1, 2, 5
- Frame size: 3
- Steps:

- Load 1 → [1]
- Load 2 → [1, 2]
- Load 3 → [1, 2, 3]
- Load 4 → Replace 3 (used farthest in future) → [1, 2, 4]
- Load 1 → Already in memory
- Load 2 → Already in memory
- Load 5 → Replace 4 (not used again) → [1, 2, 5]

Page Faults: 5

4. Clock (Second-Chance) Algorithm

- Circular version of FIFO with a "use" bit for each page.
- If the use bit is 0, the page is replaced. If it's 1, the bit is cleared, and the algorithm checks the next page.

Advantages:

- Efficient approximation of LRU
- Low overhead

Disadvantages:

- Slightly more complex than FIFO

Example:

- Page reference string: 1, 2, 3, 4, 1, 2, 5
- Frame size: 3
- Initially all reference bits are 0.
- Pages are replaced based on the clock hand and reference bits.

Page Faults: Depends on reference bit updates, usually better than FIFO.

Table 6: Comparison Table

Algorithm	Basis	Complexity	Page Fault Rate	Practical Use
FIFO	Oldest page	Low	High (can vary)	Yes
LRU	Least recent use	Medium-High	Low	Yes
Optimal	Future knowledge	High	Lowest possible	No (theoretical)

Clock	Use bit reference	Medium	Low–Moderate	Yes
-------	-------------------	--------	--------------	-----

Choosing the Right Algorithm

- For simplicity: FIFO or Clock
- For better performance: LRU (if hardware allows)
- For comparison and benchmarking: Optimal

Page replacement algorithms are vital for ensuring smooth and efficient operation in virtual memory systems. By selecting which pages to evict during page faults, they directly affect the system's speed and responsiveness. An effective algorithm helps reduce page faults and maintain high CPU utilisation.

5.4. Advantages and Limitations

Virtual memory is a key innovation in memory management, enabling systems to run large and multiple applications efficiently. While it offers numerous benefits in flexibility and resource use, it also comes with certain limitations that affect system performance and complexity.

Advantages of Virtual Memory

1. Larger Logical Memory

- Programs can be larger than the available physical RAM.
- The system loads only the required parts of a program, increasing memory efficiency.

2. Efficient Multiprogramming

- Multiple programs can run simultaneously, as each gets its own isolated virtual address space.
- Enhances CPU utilisation and system responsiveness.

3. Memory Protection and Security

- Isolates processes from each other, preventing accidental or malicious access to another program's memory.
- Increases system reliability and safety.

4. Simplified Program Development

- Developers can write code without managing actual physical memory locations.
- Programs assume a large, contiguous memory space, simplifying memory handling.

5. Demand Paging Saves Resources

- Loads only the required pages into RAM, reducing the memory footprint.
- Idle or unused parts of a program remain on disk, freeing up RAM for active processes.

Limitations of Virtual Memory

1. Performance Overhead

- Page table lookups, address translation, and page faults introduce additional processing delays.
- Accessing data from disk (on page faults) is much slower than from RAM.

2. Thrashing

- If too many page faults occur in a short time, the system may enter a state of thrashing, where it spends more time swapping than executing.

3. Complex Implementation

- Requires sophisticated hardware (MMU) and operating system support.
- Managing page tables, replacement algorithms, and memory protection adds system complexity.

4. Disk Dependency

- Heavily relies on secondary storage; if the disk is slow or full, system performance can degrade sharply.

5. Increased Power Consumption

- Frequent disk I/O operations during paging can lead to higher power usage, especially in mobile devices.

Virtual memory offers a powerful solution for extending physical memory, improving multitasking, and providing process protection. However, it comes with trade-offs in terms of performance, complexity, and resource usage. A well-designed system must balance these factors to fully leverage the benefits of virtual memory.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
1	What is the main purpose of virtual memory in a computer system? a) To increase CPU speed b) To store backups c) To simulate a larger memory space than physically available d) To reduce power consumption
2	In paging, what are the fixed-size blocks in physical memory called? a) Segments b) Pages c) Frames d) Buffers
3	Which page replacement algorithm replaces the page that has not been used for the longest time? a) FIFO b) LRU c) Optimal d) Clock
4	Which of the following is a disadvantage of virtual memory? a) Simplified programming b) Increased multitasking c) Thrashing due to excessive paging d) Process isolation
Fill in the Blank	
5	Virtual memory allows each process to run in its own _____ address space.
6	In segmentation, memory is divided into _____ sized logical units.
7	A _____ occurs when the required page is not found in physical memory.
8	The _____ algorithm uses future knowledge to replace the page that will not be used for the longest time.
True or False	
9	Virtual memory enables execution of programs larger than the physical RAM.

10	Paging suffers from external fragmentation.
11	The Clock algorithm is a simple approximation of the FIFO algorithm.
12	Virtual memory increases system complexity and may introduce performance overhead.



6. SUMMARY

- Let's summarise this unit, We've explored how memory is organised in computer systems to improve performance and efficiency. The journey began with understanding the memory hierarchy, which arranges memory types from fastest (like CPU registers and cache) to slowest (like hard drives and cloud storage), balancing speed, cost, and capacity.
- We then looked at the characteristics of memory—such as access time, capacity, cost per bit, and volatility—which help determine where each type fits in the hierarchy. Memory is categorised into primary (RAM, ROM), secondary (HDD, SSD), and tertiary (tape drives, cloud storage), each serving different roles from active processing to long-term storage.
- Next, we learned about associative memory, also called Content Addressable Memory (CAM), which retrieves data based on content rather than address. It's used in high-speed applications like cache tag matching and network routing, though it's complex and costly to implement.
- We also studied cache memory, a fast storage layer between the CPU and main memory. It stores frequently accessed data to reduce latency. We explored three cache mapping techniques—direct, associative, and set-associative—and learned how cache performance is measured using hit ratio, miss penalty, and average memory access time (AMAT).
- Finally, we covered virtual memory, which allows systems to run large applications by using disk space as an extension of RAM. It's managed through paging and segmentation, and relies on page replacement algorithms like FIFO, LRU, Optimal, and Clock to decide which memory pages to evict. While virtual memory offers flexibility and protection, it also introduces performance overhead and complexity.
- In summary, this unit has shown how different memory types and techniques work together to support efficient computing, enabling systems to be faster, more scalable, and more reliable.

7. GLOSSARY

Memory Hierarchy	- A structured arrangement of memory types based on speed, cost, and capacity, designed to optimise performance and efficiency.
Access Time	- The time required to read from or write to a memory location.
Volatile Memory	- Memory that loses its contents when power is turned off (e.g., RAM).
Non-Volatile Memory	- Memory that retains data even when power is lost (e.g., ROM, SSD).
Primary Memory	- Fast, directly accessible memory used during active processing (e.g., RAM, ROM).
Secondary Memory	- Storage used for long-term data retention (e.g., HDD, SSD).
Tertiary Memory	- Backup and archival storage with very high capacity and slow access (e.g., tape drives, cloud storage).
Associative Memory (CAM)	- A type of memory that retrieves data based on content rather than address, enabling parallel search operations.
Key Register	- Holds the search pattern used in associative memory.
Mask Register	- Used to specify which bits of the key should be compared during a search.
Match Logic	- Circuitry in associative memory that performs parallel comparisons to detect matches.
Cache Memory	- High-speed memory located between the CPU and main memory that stores frequently accessed data to reduce latency.
Cache Mapping	- Techniques used to determine where data is stored in cache (Direct, Associative, Set-Associative).

Hit Ratio	- The percentage of memory accesses that are successfully found in the cache.
Miss Penalty	- The time delay incurred when data is not found in the cache and must be fetched from main memory.
Average Memory Access Time (AMAT)	- A performance metric combining cache access time and miss penalty.
Virtual Memory	- A memory management technique that uses disk space to simulate additional RAM, allowing execution of large programs.
Paging	- Divides memory into fixed-size blocks (pages and frames) for efficient management and address translation.
Segmentation	- Divides memory into variable-sized logical segments based on program structure.
Page Table	- A data structure used to map virtual pages to physical frames.
Page Fault	- Occurs when a requested page is not in physical memory and must be loaded from disk.
Page Replacement Algorithm	- Determines which memory page to evict during a page fault (e.g., FIFO, LRU, Optimal, Clock).
Thrashing	- A condition where excessive paging reduces system performance due to constant swapping.
Translation Lookaside Buffer (TLB)	- A cache used to store recent virtual-to-physical address translations

8. TERMINAL QUESTIONS

1. What is the purpose of organising memory into a hierarchy in computer systems?
2. Explain the difference between volatile and non-volatile memory with examples.
3. List and describe the three main types of memory in a computer system.
4. What is associative memory and how does it differ from conventional memory?
5. Describe the role of the key register and mask register in associative memory.
6. What are the main applications of associative (content-addressable) memory?
7. Why is cache memory necessary in modern computer systems?
8. Compare direct mapping, associative mapping, and set-associative mapping in cache memory.
9. Define hit ratio and explain its significance in cache performance.
10. What factors influence the average memory access time (AMAT) in a system?
11. How does virtual memory enhance the execution of large programs?
12. Differentiate between paging and segmentation in virtual memory management.
13. What is a page fault and how is it handled by the operating system?
14. Compare FIFO, LRU, Optimal, and Clock page replacement algorithms.
15. List two advantages and two limitations of using virtual memory in computing systems.

9. ANSWERS

Self-Assessment Questions

Self-Assessment Questions - 1

1. c) CPU Registers
2. c) To balance performance and cost
3. d) Non-volatile
4. c) Tertiary Memory
5. Temporal
6. Primary
7. Access Time
8. Tertiary
9. False
10. False
11. True
12. True

Self-Assessment Questions - 2

1. c) Content-based retrieval
2. b) Key register
3. c) Selects specific bits for comparison
4. c) Cache tag matching
5. Content Addressable
6. Match
7. Tag
8. Match
9. False
10. True
11. False
12. True

Self-Assessment Questions - 3

1. c) To bridge the speed gap between CPU and RAM

2. c) To a specific cache line using modulo operation
3. c) Set-Associative Mapping
4. b) Efficient cache performance
5. Frequently
6. Any
7. Miss penalty
8. CPU
9. False
10. True
11. False
12. True

Self-Assessment Questions - 4

1. c) To simulate a larger memory space than physically available
2. c) Frames
3. b) LRU
4. c) Thrashing due to excessive paging
5. Virtual
6. Variable
7. Page fault
8. Optimal
9. True
10. False
11. False
12. True

Terminal Questions Answers

Answer 1: Memory hierarchy is designed to optimise system performance by organising memory types from fastest to slowest. It ensures that frequently accessed data is stored in high-speed memory like cache or registers, while less-used data is moved to slower, larger storage like hard disks. This layered approach balances speed, cost, and capacity, allowing systems to operate efficiently.

Refer section 2.1 - Memory Hierarchy Concept

Answer 2: Volatile memory, such as RAM and cache, requires continuous power to retain data and loses all stored information when the system is turned off. Non-volatile memory, like ROM, SSDs, and HDDs, retains data even when power is lost, making it suitable for permanent storage of system files and user data.

Refer section 2.2 - Characteristics of Memory

Answer 3: Primary memory is the main working memory of the computer, including RAM and ROM, used for active data processing. Secondary memory provides long-term storage, such as HDDs and SSDs, and is used for storing the operating system, applications, and files. Tertiary memory, like tape drives and cloud storage, is used for backups and archival purposes, offering very high capacity but slower access.

Refer section 2.3 - Types of Memory (Primary, Secondary, Tertiary)

Answer 4: Associative memory, also known as Content Addressable Memory (CAM), allows data retrieval based on content rather than a specific address. Unlike conventional memory, which accesses data by location, associative memory compares input data with stored values in parallel and returns matching results instantly, making it ideal for high-speed search operations.

Refer section 3.1 - Concept of Associative (Content Addressable) Memory

Answer 5: In associative memory, the key register holds the search pattern or data to be matched. The mask register is used to specify which bits of the key should be considered during comparison, enabling partial matches. This allows flexible and efficient data retrieval based on selected fields.

Refer section 3.2 - Hardware Organisation

Answer 6: Associative memory is widely used in systems requiring fast lookups, such as cache memory for tag matching, Translation Lookaside Buffers (TLBs) for virtual-to-physical address mapping, network routing tables, pattern recognition systems, and database indexing engines. Its parallel search capability makes it highly effective in these domains.

Refer section 3.3 - Match Logic and Applications

Answer 7: Cache memory is essential because it reduces the time the CPU spends waiting for data from slower main memory. By storing frequently accessed instructions and data close to the processor, cache memory significantly improves execution speed and overall system responsiveness, especially in high-performance computing environments.

Refer section 4.1 - Need for Cache Memory

Answer 8: Direct mapping assigns each block of main memory to a specific cache line, making it simple and fast but prone to conflicts. Associative mapping allows any block to be placed in any cache line, reducing conflicts but requiring complex hardware. Set-associative mapping divides the cache into sets, allowing blocks to be placed in any line within a set, offering a balance between flexibility and complexity.

Refer section 4.2 - Cache Mapping Techniques (Direct, Associative, Set-Associative)

Answer 9: The hit ratio is the percentage of memory accesses that are successfully found in the cache. A high hit ratio means the CPU can retrieve data quickly from cache, reducing the need to access slower main memory and improving overall system performance.

Refer section 4.3 - Cache Performance and Hit Ratio

Answer 10: Average Memory Access Time (AMAT) is influenced by the hit ratio, cache access time, and miss penalty. Factors such as cache size, block size, mapping technique, replacement policy, and write strategy all affect AMAT. Lower AMAT indicates better cache efficiency and faster data access.

Refer section 4.3 - Cache Performance and Hit Ratio

Answer 11: Virtual memory enables systems to run programs larger than the available physical RAM by using disk space as an extension of memory. It allows multiple applications to run simultaneously, isolates processes for security, and provides a simplified programming model by presenting a large, continuous memory space to applications.

Refer section 5.1 - Concept and Purpose of Virtual Memory

Answer 12: Paging divides memory into fixed-size blocks called pages and frames, simplifying memory management and eliminating external fragmentation. Segmentation divides memory into variable-sized logical segments based on program structure, offering better protection and logical organisation but may suffer from external fragmentation.

Refer section 5.2 - Paging and Segmentation

Answer 13: A page fault occurs when a program accesses a page not currently in RAM. The operating system handles this by retrieving the required page from disk and updating the page table. This process ensures that the program continues execution without interruption, although it may introduce a delay.

Refer section 5.1 - Concept and Purpose of Virtual Memory

Answer 14: FIFO replaces the oldest page in memory, which is simple but may lead to poor performance. LRU replaces the page that hasn't been used for the longest time, offering better results but requiring more tracking. Optimal replacement uses future knowledge to choose the best page to evict, but it's impractical. Clock is a circular version of FIFO that uses a use-bit to approximate LRU with lower overhead.

Refer section 5.3 - Page Replacement Algorithms

Answer 15: Virtual memory offers advantages such as increased logical memory size, better multitasking, and process isolation. However, it also introduces limitations like performance overhead due to page faults and address translation, risk of thrashing when excessive paging occurs, and increased system complexity.

Refer section 5.4 - Advantages and Limitations

10. REFERENCES

BOOKS

1. Computer Architecture: A Quantitative Approach, John L. Hennessy, David A. Patterson, 6th Edition, 2017
2. Computer Organisation and Architecture: Designing for Performance, William Stallings, 11th Edition, 2018
3. Computer System Architecture, M. Morris Mano, 3rd Edition, 2006
4. Computer Organisation and Design: The Hardware/Software Interface, David A. Patterson, John L. Hennessy, 5th Edition, 2013
5. Digital Design and Computer Architecture, Sarah Harris, David Harris, 3rd Edition, 2021

E-REFERENCES

1. Logical and Shift Micro-operations Computer Organisation and Architecture, <https://care4you.in/logical-and-shift-micro-operations/>
2. Logic and Shift Micro Operations, <https://www.scribd.com/document/394026693/U-I-Logic-and-Shift-micro-operations>

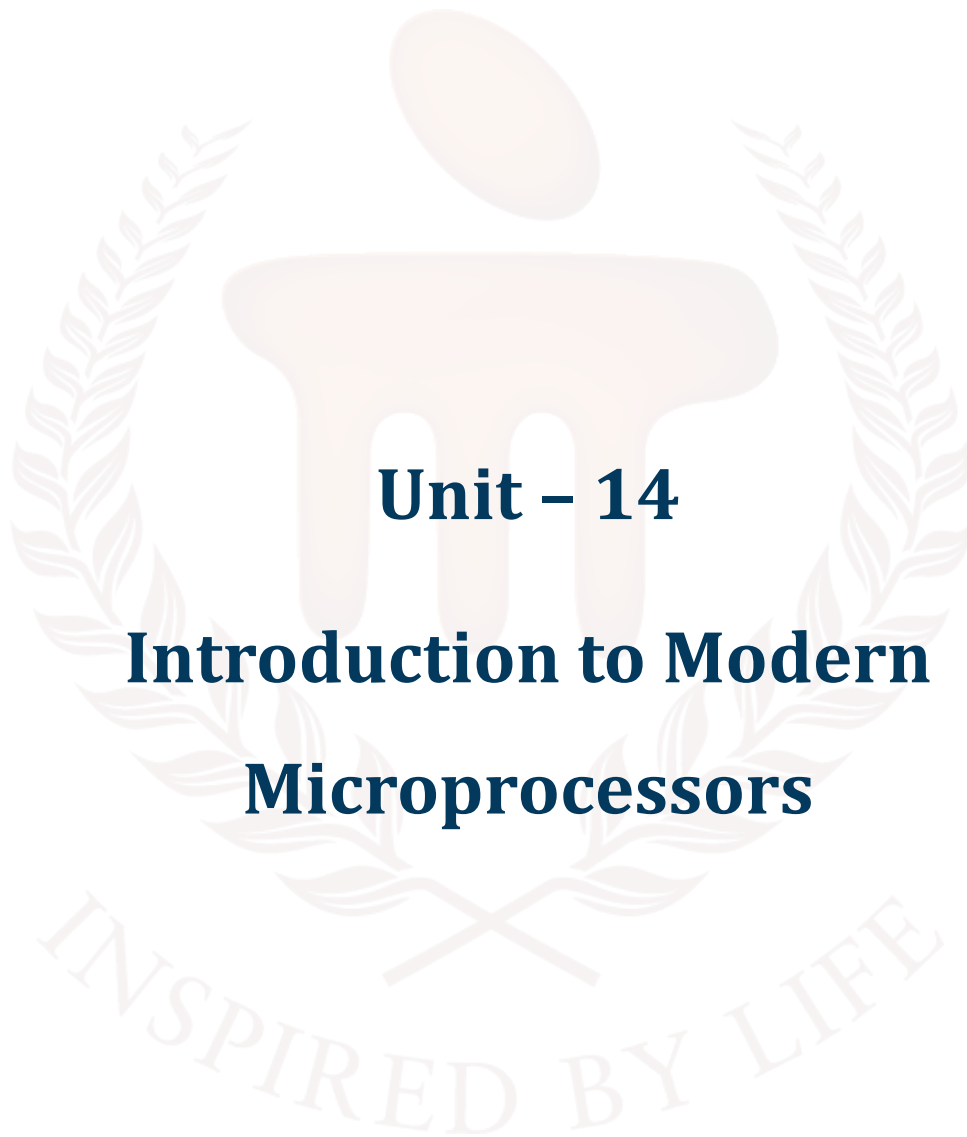
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2105

**COMPUTER ORGANIZATION AND
ARCHITECTURE**



Unit – 14

Introduction to Modern Microprocessors

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 6
1.1	Objectives	-	-	
2	Overview of Modern Microprocessors	-	-	7 - 13
2.1	Evolution of Microprocessors	i	-	
2.2	Characteristics of Modern Microprocessors	-	-	
2.3	Comparison with Traditional Microprocessors	-	1	
3	ARM Architecture	1	-	14 - 21
3.1	Features and Design Philosophy	-	-	
3.2	Instruction Set and Applications	ii	-	
3.3	Use Cases and Industry Adoption	-	2	
4	MIPS Architecture	2	-	22 - 29
4.1	Overview and Features	-	-	
4.2	Instruction Set Architecture	iii	-	
4.3	Educational and Industrial Applications	-	3	
5	x86 Architecture	3	-	30 - 36
5.1	Historical Development	iv	-	
5.2	Instruction Set and Design Considerations	-	-	
5.3	Role in Modern Computing	-	4	
6	Comparative Analysis	-	-	32 - 43
6.1	Performance and Efficiency	-	-	
6.2	Power Consumption and Scalability	-	-	
6.3	Market Adoption and Ecosystem Support	v	5	
7	Summary	-	-	44
8	Glossary	-	-	45 - 46
9	Terminal Questions	-	-	47

10	Answers	-	-	48 – 52
11	References	-	-	53



1. INTRODUCTION

In the previous unit, we explored the fundamental concepts of memory organisation, including memory hierarchy, associative memory, cache memory, and virtual memory. We examined how different memory types work together to optimise data access, storage efficiency, and overall system performance. The unit emphasised the importance of fast memory access and efficient memory management in modern computer architecture.

In this unit, we shift our focus to the modern microprocessors that drive contemporary computing systems. As technology has evolved, microprocessors have become more powerful, energy-efficient, and application-specific. This unit provides an overview of some of the most widely used modern microprocessor architectures: ARM, MIPS, and x86.

We will begin by understanding the evolution and features of modern microprocessors. The unit then delves into the design philosophies, instruction sets, and applications of ARM, MIPS, and x86 architectures. Finally, we will compare these architectures based on performance, power consumption, and market adoption, offering insights into their roles in embedded systems, mobile devices, and desktop computing.

This unit aims to provide a solid foundation for understanding how modern microprocessors differ from traditional designs and how they are shaping the future of computing.

1.1. Objectives

After studying this unit, you should be able to:

- Discuss the evolution and key characteristics of modern microprocessors.
- Identify the architectural features and design philosophies of widely used microprocessors.
- Explain the instruction sets and typical applications of ARM, MIPS, and x86 architectures.
- Analyse the differences between traditional and modern microprocessor designs.
- Compare modern microprocessors based on performance, power efficiency, scalability, and market usage.
- Outline the industry trends and use cases that drive the adoption of specific processor architectures in different computing environments.



2. OVERVIEW OF MODERN MICROPROCESSORS

Modern microprocessors have evolved significantly in terms of architecture, performance, and efficiency. They now feature multi-core designs, advanced instruction sets, and low power consumption, making them suitable for a wide range of applications—from smartphones and embedded systems to high-performance computing. This section explores their evolution, core features, and architectural distinctions.

2.1. Evolution of Microprocessors

Microprocessors have undergone tremendous evolution since their inception in the early 1970s. What began as simple, single-tasking processors with limited instruction sets and low clock speeds has transformed into highly complex, multi-core systems capable of executing billions of instructions per second. The evolution of microprocessors can be broadly categorised into generations based on architectural improvements, processing power, and technological advancements.

1. First Generation: 4-bit and 8-bit Processors

- Examples: Intel 4004 (1971), Intel 8008, Motorola 6800
- Characterised by very basic functionality and limited instruction sets.
- Used mainly in calculators, embedded systems, and early computers.

2. Second Generation: 16-bit Processors

- Examples: Intel 8086, Zilog Z8000
- Introduced more advanced instruction sets and addressing modes.
- Enabled the development of more sophisticated software and operating systems.

3. Third Generation: 32-bit Processors

- Examples: Intel 80386, Motorola 68020
- Supported multitasking, virtual memory, and advanced OS capabilities.
- Laid the foundation for personal computing and early graphical interfaces.

4. Fourth Generation: Introduction of RISC and Advanced Pipelining

- Rise of RISC (Reduced Instruction Set Computer) architectures like MIPS and ARM.
- Emphasis on simplified instruction sets, pipelined execution, and high-speed performance.
- Better efficiency and scalability compared to traditional CISC processors.

5. Fifth Generation and Beyond: 64-bit Multi-Core Processors

- Examples: Intel Core series, AMD Ryzen, Apple M1 (ARM-based), and newer ARM Cortex cores
- Incorporate multiple cores, integrated GPUs, advanced power management, and AI acceleration.
- Support complex computing tasks such as virtualisation, parallel processing, and deep learning.

Table 1: Key Trends in Microprocessor Evolution

Aspect	Early Generations	Modern Microprocessors
Word Size	4-bit, 8-bit	64-bit and beyond
Instruction Set	Simple, limited	Complex (x86) or highly optimised (ARM, MIPS)
Performance	Kilohertz to Megahertz	Gigahertz range, billions of instructions/sec
Architecture	Single-core, basic pipeline	Multi-core, out-of-order, superscalar
Integration	CPU only	System-on-Chip (SoC), integrated memory, GPU, AI engines
Power Efficiency	Low priority	Critical for mobile and embedded systems

The evolution of microprocessors reflects the ever-increasing demand for speed, efficiency, compactness, and versatility. Modern processors like ARM, MIPS, and x86 are the result of decades of innovation, each tailored to specific performance and application needs. Understanding this progression helps in appreciating the architectural decisions behind today's computing platforms.

2.2. Characteristics of Modern Microprocessors

Modern microprocessors are designed to deliver high performance, energy efficiency, and scalability. These characteristics enable them to meet the demanding requirements of diverse applications, from embedded systems and mobile devices to desktops, servers, and cloud computing platforms.

Key Characteristics of Modern Microprocessors

1. Multi-Core Architecture

- Modern CPUs often feature multiple processing cores on a single chip.

- Enables parallel execution of tasks, improving performance for multitasking and multithreaded applications.

2. High Clock Speeds

- Clock rates range from 2 GHz to 5+ GHz, enabling the execution of billions of instructions per second.
- Some processors feature dynamic frequency scaling to balance speed and power usage.

3. Pipelining and Superscalar Execution

- Use of instruction pipelining allows overlapping of instruction execution stages.
- Superscalar designs enable the execution of multiple instructions per clock cycle.

4. Advanced Instruction Sets

- Modern microprocessors support complex instruction sets (like x86) or optimised reduced sets (like ARM and MIPS).
- Support for SIMD (Single Instruction, Multiple Data), vector instructions, and cryptographic extensions enhances performance in specific tasks.

5. Integrated Components (System-on-Chip - SoC)

- Include not just the CPU but also GPU, memory controllers, I/O interfaces, and sometimes AI accelerators.
- Common in mobile and embedded devices for space and power efficiency.

6. Low Power Consumption

- Energy-efficient design is critical, especially in portable and battery-powered devices.
- Techniques like clock gating, power gating, and dynamic voltage scaling are employed.

7. Virtualisation Support

- Hardware-level support for running multiple operating systems or virtual machines concurrently (e.g., Intel VT-x, AMD-V).

8. Security Features

- Built-in protection mechanisms like hardware encryption, secure boot, trusted execution environments (TEE), and buffer overflow prevention.

9. Scalability and Customisability

- Microprocessors can be scaled across platforms—from low-power microcontrollers to high-performance server CPUs.
- Many designs allow for custom extensions, especially in open architectures like RISC-V.

Modern microprocessors combine performance, power efficiency, integration, and security to support the growing complexity of computing tasks. Their architectural advancements enable faster execution, better multitasking, and broader application support, making them central to nearly every modern computing device.

2.3. Comparison with Traditional Microprocessors

The transition from traditional to modern microprocessors marks a significant leap in processing capability, efficiency, and functionality. While traditional microprocessors laid the foundation for computing, modern architectures have built upon them with innovations suited for today's high-performance, energy-conscious, and multitasking environments.

1. Architectural Complexity

Architectural complexity in microprocessors has evolved significantly from traditional designs to modern implementations. Traditional microprocessors typically featured a single-core design, executing instructions sequentially with limited parallelism. Their instruction sets were predominantly CISC (Complex Instruction Set Computing), such as early x86 architectures, which aimed to perform complex tasks with fewer instructions. In contrast, modern microprocessors embrace multi-core or even many-core architectures, enabling simultaneous processing across multiple units. They utilise advanced techniques like pipelining, superscalar execution, and out-of-order processing to enhance performance and efficiency. Additionally, modern processors often employ optimised CISC or RISC (Reduced Instruction Set Computing) instruction sets, such as ARM or x86_64, balancing complexity and speed to meet the demands of contemporary computing tasks. This shift reflects a broader trend toward maximising throughput, energy efficiency, and scalability in processor design.

2. Performance and Speed

In terms of performance and speed, traditional CPUs were limited by their relatively low clock speeds, typically operating under 1 GHz, and were designed to handle basic single-threaded operations. This meant that they could execute only one instruction at a time, which constrained their ability to

perform complex or parallel tasks efficiently. In contrast, modern CPUs have seen a dramatic increase in clock speeds, commonly ranging from 2 to over 5 GHz, and are capable of executing multiple instructions per cycle. They also support hardware-level multitasking, allowing several threads or processes to run concurrently. This leap in performance is driven by architectural advancements such as multi-core designs, improved instruction pipelines, and enhanced parallelism, making modern processors vastly more powerful and responsive than their predecessors.

3. Power and Efficiency

In the realm of power and efficiency, traditional processors were primarily designed with a focus on maximising performance, often at the cost of higher power consumption relative to the computational output they delivered. These processors lacked sophisticated mechanisms for managing energy use, making them less suitable for power-sensitive applications. In contrast, modern processors place a strong emphasis on energy efficiency, especially in mobile and embedded systems where battery life and thermal constraints are critical. They incorporate advanced features such as dynamic power management, which adjusts power usage based on workload, and low-power sleep modes that conserve energy when the processor is idle. This shift reflects a broader industry trend toward sustainable computing and optimising performance-per-watt.

4. Integration and Functionality

In terms of integration and functionality, earlier microprocessors relied heavily on external components to perform various tasks, such as separate memory controllers, graphics processing units (GPUs), and I/O interfaces. This modular approach often led to increased complexity in system design and higher latency in communication between components. Modern processors, however, frequently adopt System-on-Chip (SoC) architectures, which consolidate multiple functionalities into a single chip. These integrated systems typically include CPU cores, GPU, memory and I/O controllers, AI engines, and networking and security modules, all working together seamlessly. This high level of integration enhances performance, reduces power consumption, and enables compact, efficient designs—especially crucial for mobile devices, embedded systems, and edge computing applications.

5. Application Scope

The application scope of microprocessors has expanded dramatically from their early days to the present. Traditional microprocessors were primarily used in desktops, early servers, and basic embedded systems, where computing demands were relatively modest and specialised. Their limited performance and integration made them suitable for straightforward, single-purpose tasks. In

contrast, modern microprocessors are deployed across a wide range of platforms, reflecting their enhanced capabilities and versatility. They power smartphones and tablets (e.g., ARM Cortex), laptops and desktops (e.g., Intel Core, AMD Ryzen), servers and data centers (e.g., x86_64 and ARM-based processors like AWS Graviton), and are increasingly found in IoT and edge computing devices. This broad deployment is enabled by advances in performance, energy efficiency, and integration, allowing modern processors to meet the diverse needs of everything from personal computing to large-scale cloud infrastructure.

6. Support for Modern Technologies

Traditional processors often lacked support for many of the advanced technologies that are now considered essential in modern computing environments. Features such as virtualisation, hardware-level security, cryptographic acceleration, and AI-specific processing capabilities were typically absent or had to be implemented through external components or software workarounds. In contrast, modern CPUs come with native support for these technologies, making them well-suited for demanding applications like cloud computing, secure financial transactions, and machine learning workloads. This built-in functionality not only enhances performance and security but also simplifies system design and deployment across a wide range of industries and use cases.

Modern microprocessors represent a significant advancement over traditional designs, offering enhanced performance, efficiency, and integration. These changes have enabled their use in a wide spectrum of applications, shaping the way technology impacts everyday life. Understanding this comparison highlights why architectures like ARM, MIPS, and x86 are central to today's computing systems.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which generation of microprocessors introduced RISC architecture and advanced pipelining? a) First generation b) Second generation c) Fourth generation d) Fifth generation
2	What is a key characteristic of modern microprocessors that improves multitasking performance? a) Single-core design

	b) Low clock speed c) Multi-core architecture d) External memory controllers
3	Which of the following is commonly integrated into modern System-on-Chip (SoC) designs? a) Only CPU b) CPU and RAM c) CPU, GPU, memory controller, and I/O interfaces d) CPU and hard disk
4	Compared to traditional microprocessors, modern microprocessors typically support: a) Sequential instruction execution b) Fixed instruction sets only c) Virtualisation and AI acceleration d) External GPU integration only
Fill in the Blank	
5	The first generation of microprocessors typically had a word size of _____ bits.
6	Modern microprocessors use _____ execution to process multiple instructions per clock cycle.
7	_____ is a technique used to reduce power consumption by disabling unused parts of the processor.
8	Traditional microprocessors were mostly used in _____ and early servers.
True or False	
9	Modern microprocessors often include integrated AI engines and support for secure boot.
10	Instruction pipelining is used only in traditional microprocessors.
11	Modern CPUs typically operate at clock speeds below 1 GHz.
12	Virtualisation support allows modern processors to run multiple operating systems simultaneously.

3. ARM ARCHITECTURE

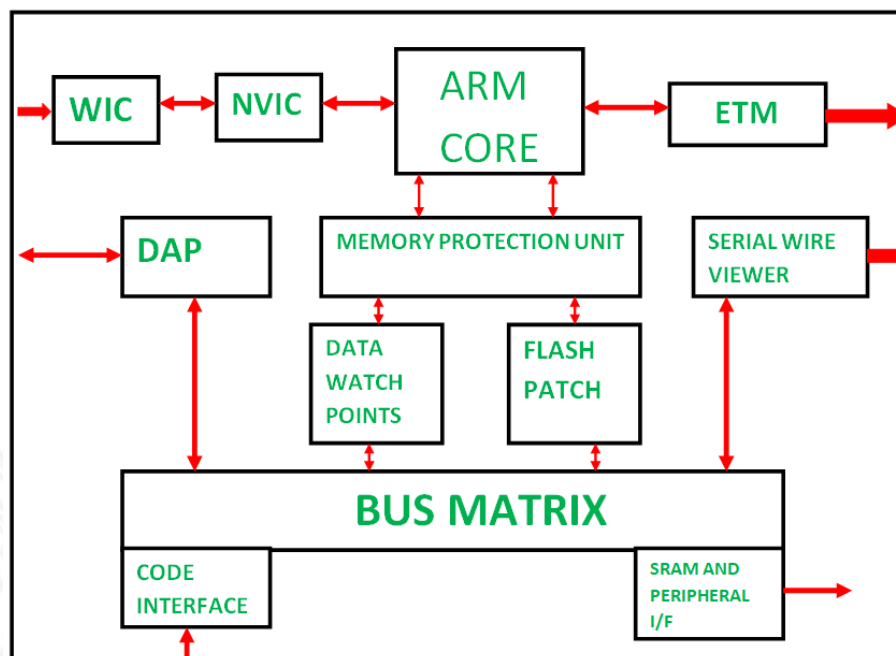


Figure 1: ARM Architecture

ARM Architecture is based on the Reduced Instruction Set Computing (RISC) model, designed for high efficiency and low power consumption. Widely used in smartphones, embedded systems, and IoT devices, ARM processors are valued for their simplicity, scalability, and strong performance-per-watt ratio, making them a leading choice in modern microprocessor design.

The diagram illustrates the internal architecture of an ARM Cortex-M processor. At the centre is the ARM Core, interfacing with the NVIC (Nested Vectored Interrupt Controller), WIC (Wake-up Interrupt Controller), and the ETM (Embedded Trace Macrocell) for debugging. The Bus Matrix connects the core to memory and peripherals, allowing communication with SRAM, flash, and interfaces. The Debug Access Port (DAP), Serial Wire Viewer, and other debug units like Flash Patch and Data Watchpoints facilitate real-time debugging and memory access. The Memory Protection Unit enhances system security by controlling access to memory regions.

3.1. Features and Design Philosophy

ARM (Advanced RISC Machine) is a family of microprocessor architectures based on the Reduced Instruction Set Computing (RISC) design philosophy. ARM processors are known for their simplicity, efficiency, and low power consumption, making them highly suitable for embedded systems, mobile devices, and increasingly, high-performance computing.

Design Philosophy of ARM

The ARM architecture is grounded in the RISC principles, which emphasise:

- Simplicity in instruction set: Fewer, fixed-length instructions that execute in a single clock cycle.
- Load/store architecture: Only load and store instructions access memory; all other operations are register-based.
- Efficient pipelining: ARM processors use deep pipelines for high instruction throughput.
- Orthogonality: A consistent instruction format and use of registers simplify decoding and execution.
- Minimised hardware complexity: Smaller die size, reduced cost, and lower power requirements.

Key Features of ARM Architecture

1. RISC-Based Instruction Set

- Simple and optimised instruction set improves execution speed and reduces decoding time.
- Supports both 32-bit (ARM) and 16-bit (Thumb) instruction sets for flexibility.

2. Low Power Consumption

- Designed with power efficiency as a core goal.
- Widely used in mobile phones, tablets, IoT devices, and wearables.

3. High Performance Efficiency

- Despite low power use, ARM processors deliver strong computational performance.
- Scales from simple microcontrollers to complex multi-core systems.

4. Register-Based Architecture

- Features a large set of general-purpose registers (e.g., 16–32), improving execution speed.
- Reduces the number of memory accesses.

5. Flexible Operating Modes

- Supports multiple modes (User, Supervisor, FIQ, IRQ, etc.) for better control and exception handling.

6. Pipelined Execution

- Most ARM CPUs use 3-stage to 8-stage pipelines for increased instruction throughput.

7. Support for Extensions

- Optional features like NEON SIMD, TrustZone for security, and FPU (Floating Point Unit).
- Extensions allow ARM cores to be tailored for specific applications.

8. System-on-Chip (SoC) Integration

- ARM cores are commonly integrated with memory controllers, I/O, GPUs, and other peripherals on a single chip.
- Makes ARM ideal for compact, embedded systems.

9. Licensing Model

- ARM Ltd. licenses its core designs (e.g., Cortex-A, Cortex-M, Cortex-R) to manufacturers like Qualcomm, Apple, and Samsung.
- Encourages broad industry adoption and customisation.

The ARM architecture is built for performance with efficiency, striking a balance that makes it dominant in mobile, embedded, and low-power environments. Its RISC-based simplicity, power-conscious design, and scalability have made ARM the most widely used processor architecture in the world today.

3.2. Instruction Set and Applications

Instruction Set of ARM Architecture

ARM processors are designed around a RISC (Reduced Instruction Set Computer) architecture, meaning they utilise a small, highly optimised set of instructions that execute quickly—typically in one clock cycle. This makes ARM processors fast, efficient, and easier to pipeline.

Key Features of ARM Instruction Set:

1. Fixed-Length Instructions

- Most ARM instructions are 32-bit wide, with a 16-bit Thumb mode available for increased code density in constrained environments.

2. Load/Store Architecture

- Only load and store instructions access memory.

- All other operations are performed between registers, improving speed and reducing complexity.

3. Conditional Execution

- Many instructions can be conditionally executed based on flag status, reducing branching and increasing performance.

4. Barrel Shifter and ALU Integration

- Allows shifting and arithmetic/logic operations in a single instruction, increasing computational power.

5. Support for DSP and SIMD

- NEON extension allows parallel processing for multimedia and signal processing tasks.

6. Security and Virtualisation

- ARMv8 includes TrustZone for secure execution and hardware virtualisation for OS isolation.

Table 2: Common Types of ARM Instructions

Type	Example Instructions	Purpose
Data Processing	ADD, SUB, AND, ORR	Arithmetic and logic operations
Load/Store	LDR, STR	Transfer data between memory and registers
Branching	B, BL, BX	Unconditional and conditional jumps
Move/Shift	MOV, LSL, ROR	Register and bit manipulations
Multiply/Divide	MUL, UDIV, SDIV	Integer multiplication and division
Floating Point (FPU)	VMUL.F32, VADD.F64	Floating-point arithmetic

Applications of ARM Architecture

Due to its versatility and efficiency, ARM is used in a wide range of domains:

1. Mobile and Embedded Devices

- Smartphones, tablets, and wearables predominantly use ARM-based SoCs (e.g., Qualcomm Snapdragon, Apple A-series).
- Offers the best trade-off between performance and battery life.

2. Microcontrollers and IoT

- Cortex-M series is widely used in low-power devices like smart home appliances, sensors, and industrial control systems.

3. Automotive Systems

- Used in infotainment systems, ADAS (Advanced Driver Assistance Systems), and vehicle networking.

4. Consumer Electronics

- ARM cores power TVs, gaming consoles, cameras, and smart gadgets.

5. Cloud and Edge Computing

- New ARM-based server processors (e.g., AWS Graviton) deliver energy-efficient performance in data centers.

6. Education and Research

- ARM's open documentation and flexible development tools make it ideal for teaching computer architecture and embedded systems.

The ARM instruction set is compact, efficient, and powerful, enabling a broad spectrum of devices from tiny IoT sensors to high-performance computing systems. Its modular and extensible design continues to drive innovation in both embedded and general-purpose processing domains.

3.3. Use Cases and Industry Adoption

The ARM architecture has seen widespread adoption across industries due to its scalability, low power consumption, and high performance. From consumer electronics to enterprise-level cloud computing, ARM processors are integrated into a wide variety of hardware platforms.

Key Use Cases of ARM Processors

Key Use Cases of ARM Processors span a wide range of industries and devices due to their efficiency, scalability, and flexibility.

1. Smartphones and Tablets are the most prominent domain, with nearly all modern devices—such as Android phones, iPhones, and iPads—powered by ARM-based SoCs like Apple's A-series, Qualcomm Snapdragon, and Samsung Exynos.

2. Embedded and IoT Devices also heavily rely on ARM, especially the Cortex-M and Cortex-R series, which are ideal for low-power applications like smart thermostats, wearable fitness trackers, and industrial monitoring systems.
3. Consumer Electronics such as smart TVs, set-top boxes, smart speakers (e.g., Amazon Echo, Google Nest), and gaming consoles like the Nintendo Switch also integrate ARM processors for their balance of performance and efficiency.
4. Automotive Industry, ARM chips are embedded in infotainment systems, ADAS, and vehicle control units (ECUs), with safety-certified cores like Cortex-R supporting real-time applications.
5. Data Centers and Cloud Computing are increasingly adopting ARM-based server processors like AWS Graviton and Ampere Altra, offering energy-efficient alternatives to x86 servers and growing support for Linux and cloud-native workloads.
6. Healthcare and Medical Devices benefit from ARM's low power consumption and compact size, making them suitable for portable diagnostic tools, wearable health monitors, and smart medical equipment.
7. Education and Research also leverage ARM's open architecture and accessible development tools, with platforms like Raspberry Pi providing affordable, ARM-based computing for students and developers.
8. Industry Adoption and Market Presence further highlight ARM's dominance. Apple has transitioned its entire Mac lineup to ARM-based Apple Silicon (M1, M2, M3), while companies like NVIDIA and Qualcomm design custom ARM cores for high-performance mobile and AI applications. Microsoft and Google actively support ARM on Windows and Android platforms, respectively. In the automotive sector, leaders like Tesla, BMW, and Toyota integrate ARM chips into their smart vehicle systems. In cloud computing, AWS leads with its custom Graviton processors, offering scalable, cost-effective ARM-based infrastructure.

ARM's wide industry adoption and broad use case coverage are a testament to its efficient, adaptable, and scalable design. Whether powering a smartwatch or a hyperscale server, ARM continues to be a dominant force in the microprocessor landscape.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
1	What is the core design philosophy of ARM architecture? a) Complex instruction set b) Load/store with variable-length instructions c) Reduced instruction set with fixed-length instructions d) Stack-based execution model
2	Which of the following is a key feature of ARM processors? a) High power consumption b) Complex decoding logic c) Deep pipelining and low power usage d) Exclusive use in desktop systems
3	What is the purpose of the Thumb instruction set in ARM architecture? a) To increase instruction complexity b) To reduce code density c) To provide 64-bit support d) To improve code density in constrained environments
4	Which industry has widely adopted ARM processors for real-time control and infotainment systems? a) Aerospace b) Automotive c) Agriculture d) Mining
Fill in the Blank	
5	ARM architecture uses a _____ model, where only specific instructions access memory.
6	The _____ extension in ARM enables parallel processing for multimedia and signal processing tasks.
7	ARM processors are commonly integrated into _____ designs, which include CPU, GPU, and other components on a single chip.
8	ARM's _____ mode allows secure execution of trusted code alongside normal applications.
True or False	

9	ARM architecture supports both 32-bit and 16-bit instruction sets.
10	ARM processors are rarely used in mobile or embedded systems.
11	ARM's licensing model allows manufacturers to customise and integrate ARM cores into their own chips.
12	ARM architecture is not suitable for cloud computing or server environments.



4. MIPS ARCHITECTURE

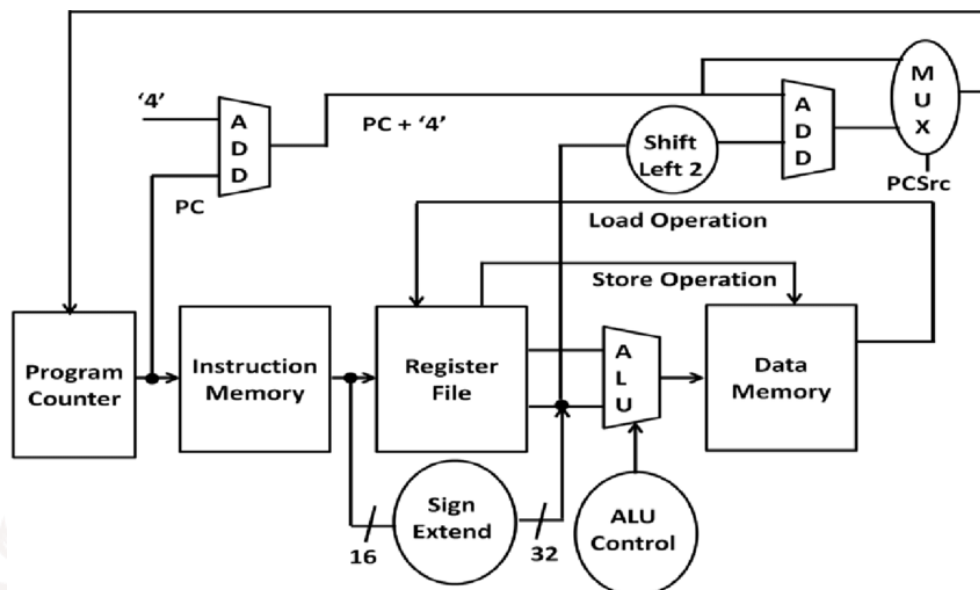


Figure 2: MIPS Architecture

MIPS Architecture is a classic RISC-based microprocessor design known for its simplicity, regular instruction format, and high-performance pipelining. Commonly used in embedded systems and academia, MIPS offers a clean, efficient model ideal for both implementation and learning computer architecture principles.

The diagram represents the datapath of a simplified MIPS processor used for executing instructions. The process begins at the Program Counter (PC), which provides the address to the Instruction Memory to fetch the instruction. The fetched instruction is decoded and used to read values from the Register File. Immediate values are sign-extended to 32 bits if needed. The ALU performs arithmetic or logical operations based on control signals from the ALU Control unit. For memory-related instructions, the Data Memory block either reads or writes data. The Shift Left 2 and Add blocks help calculate branch target addresses, while a MUX determines the next PC value based on control logic. This setup supports instruction fetch, decode, execute, memory access, and write-back phases of the MIPS instruction cycle.

4.1. Overview and Features

MIPS (Microprocessor without Interlocked Pipeline Stages) is a RISC (Reduced Instruction Set Computer) architecture known for its simplicity, high performance, and clean design. Originally

developed in the 1980s at Stanford University, MIPS has been widely adopted in embedded systems, academic environments, and networking devices.

Overview of MIPS Architecture

- **RISC-Based Design:** MIPS follows a pure RISC philosophy, using a small set of simple, fixed-length instructions that execute quickly and are easy to pipeline.
- **Register-Oriented Architecture:** Operations are performed primarily between registers, minimising memory access.
- **Simplicity and Regularity:** MIPS instructions have consistent formats, making them easy to decode and execute, which simplifies processor design.
- **Focus on Performance:** Designed with pipelining in mind to improve instruction throughput.
- **Widely Used in Education:** MIPS is often chosen for teaching computer architecture due to its clear, easy-to-understand design.

Key Features of MIPS Architecture

1. Fixed-Length Instructions

- All instructions are 32 bits in length.
- Simplifies instruction decoding and aligns well with pipelining.

2. Three-Address Instruction Format

- Most instructions use the format:
operation destination, source1, source2
Example: ADD R1, R2, R3

3. Load/Store Architecture

- Only LOAD and STORE instructions access memory.
- All arithmetic/logical operations occur between registers.

4. Large Number of General-Purpose Registers

- 32 general-purpose registers (R0 to R31) for fast and efficient computation.
- Register R0 is hardwired to zero.

5. Delayed Branching

- Uses a branch delay slot, where the instruction immediately following a branch is always executed, improving pipeline efficiency.

6. Support for Pipelining

- Designed to implement five-stage instruction pipelines:
Fetch → Decode → Execute → Memory → Write-back

7. Instruction Set Simplicity

- Reduced number of instruction types improves hardware performance and reduces power usage.

8. Variants for Embedded and Performance Markets

- MIPS processors are available in 32-bit (MIPS32) and 64-bit (MIPS64) versions.
- Found in routers, set-top boxes, printers, automotive electronics, and more.

The MIPS architecture emphasises speed, efficiency, and simplicity, making it ideal for embedded applications and educational use. Its clean RISC design, strong support for pipelining, and consistent instruction structure make MIPS both practical and easy to learn, contributing to its longevity in computer architecture studies and real-world systems.

4.2. Instruction Set Architecture (ISA) of MIPS

The MIPS Instruction Set Architecture (ISA) is designed around the RISC (Reduced Instruction Set Computer) philosophy, which emphasises simplicity, speed, and efficient pipelining. Its regular structure, fixed instruction length, and limited number of instructions make it ideal for both hardware implementation and educational use.

Key Features of MIPS ISA

1. Fixed-Length Instructions

- All instructions are 32 bits wide.
- Simplifies instruction decoding and pipeline stages.

2. Three Instruction Formats

MIPS uses three basic instruction formats:

Table 3: Instruction formats

Format	Fields	Example
R-type (Register)	opcode, rs, rt, rd, shamt, funct	add \$t0, \$t1, \$t2

I-type (Immediate)	opcode, rs, rt, immediate	addi \$t0, \$t1, 5
J-type (Jump)	opcode, address	j 10000

Common Instruction Categories

1. Arithmetic and Logic Instructions

- Operate between registers.
- Examples:
add, sub, and, or, slt (set less than)

2. Immediate Instructions

- Use a constant (immediate) value.
- Examples:
addi, andi, ori, slti

3. Memory Access Instructions

- Only way to access memory (load/store model).
- Examples:
lw (load word), sw (store word)

4. Control Transfer Instructions

- Used for branching and jumping.
- Examples:
beq (branch if equal), bne, j, jr

5. Shift Instructions

- Perform logical or arithmetic shifts.
- Examples:
sll, srl, sra

Registers in MIPS

- 32 general-purpose registers: \$zero, \$at, \$v0–\$v1, \$a0–\$a3, \$t0–\$t9, \$s0–\$s7, etc.
- Special-purpose registers:
HI/LO for multiplication/division results
Program Counter (PC) for instruction sequencing

Instruction Execution in Pipeline

MIPS ISA is optimised for a 5-stage instruction pipeline:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write Back (WB)

This structure supports high-speed execution with minimal hardware complexity.

The MIPS ISA is a prime example of a well-structured, efficient, and scalable instruction set. Its simplicity makes it ideal for teaching and implementation, while its performance benefits make it a reliable choice for embedded systems and custom processors.

4.3. Educational and Industrial Applications of MIPS Architecture

The MIPS architecture, due to its simplicity and efficiency, has found enduring use in both academic environments and industrial applications. While it may not dominate the mainstream computing market like ARM or x86, MIPS continues to thrive in specialised domains where its RISC-based design, low power requirements, and predictable performance are valued.

A. Educational Applications

1. Teaching Computer Architecture

- MIPS is widely used in university-level courses on computer organisation and processor design.
- Its clean instruction set, simple formats, and consistent pipeline model make it ideal for explaining key concepts like:
 - Instruction execution
 - Pipelining
 - Register usage
 - Assembly programming

2. Simulation and Development Tools

- Tools like SPIM, MARS, and QtSPIM allow students to simulate MIPS processors, write assembly programs, and visualise their execution.

- These simulators are lightweight and easy to use in educational settings.

3. Textbooks and Academic Resources

- Books like Computer Organisation and Design by Patterson and Hennessy use MIPS as the reference architecture, making it a global standard for learning.

B. Industrial Applications

1. Embedded Systems

- MIPS is commonly used in low-power embedded systems, including:
 - Network routers and gateways
 - Set-top boxes
 - Smart appliances
 - Industrial controllers

2. Networking Equipment

- MIPS processors are embedded in network devices due to their real-time processing capabilities, energy efficiency, and customisability.
- Examples: Products from companies like Microchip, MediaTek, and Broadcom.

3. Digital Consumer Devices

- MIPS CPUs have been used in devices such as:
 - Digital TVs
 - Blu-ray players
 - VoIP phones

4. Automotive Applications

- MIPS processors are found in in-vehicle infotainment systems, telemetry units, and vehicle control systems, where predictable performance and reliability are critical.

5. Licensable IP Cores

- Like ARM, MIPS designs are available as licensable IP, enabling companies to integrate MIPS cores into custom SoCs tailored to specific product needs.

MIPS architecture continues to hold relevance in education and specific industry niches where its simplicity, efficiency, and RISC-based performance offer tangible benefits. Whether for teaching

foundational computing concepts or powering compact embedded systems, MIPS remains a respected and practical microprocessor architecture.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
1	What does MIPS stand for in microprocessor architecture? a) Microprocessor with Integrated Power System b) Microprocessor without Interlocked Pipeline Stages c) Modular Instruction Processing System d) Multi-core Integrated Processing System
2	Which instruction format is used in MIPS for arithmetic operations between registers? a) I-type b) J-type c) R-type d) M-type
3	What is the purpose of the branch delay slot in MIPS architecture? a) To delay memory access b) To execute the instruction after a branch for pipeline efficiency c) To store temporary data d) To manage interrupts
4	In which domain is MIPS architecture most commonly used today? a) High-end gaming PCs b) Cloud computing c) Embedded systems and education d) Cryptocurrency mining
Fill in the Blank	
5	MIPS architecture follows the _____ philosophy, using a small set of simple instructions.
6	All MIPS instructions are _____ bits long, simplifying decoding and pipelining.
7	The _____ register in MIPS is hardwired to zero and cannot be modified.
8	MIPS processors are often used in _____ equipment due to their real-time processing capabilities.
True or False	

9	MIPS architecture uses variable-length instructions to increase flexibility.
10	Arithmetic and logical operations in MIPS are performed directly on memory locations.
11	MIPS architecture is widely adopted in academic settings for teaching processor design.
12	MIPS processors support pipelining, which improves instruction throughput.



5. X86 ARCHITECTURE

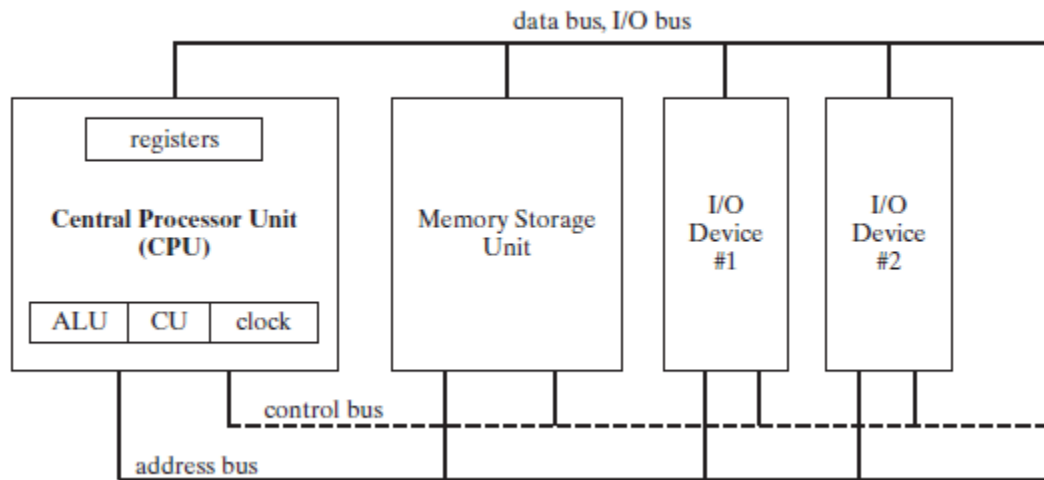


Figure 3: x86 Architecture

The x86 architecture, developed by Intel, is a widely adopted CISC-based microprocessor architecture that has powered personal computers and servers for over four decades. Known for its rich instruction set, backwards compatibility, and continuous evolution, x86 remains a cornerstone of modern desktop and enterprise computing.

The diagram illustrates the Von Neumann architecture, where the Central Processor Unit (CPU)—comprising the Arithmetic Logic Unit (ALU), Control Unit (CU), clock, and registers—interacts with the Memory Storage Unit and I/O Devices through three main buses. The address bus is used to specify memory locations, the data bus/I/O bus carries data between components, and the control bus manages commands and timing signals. The CPU uses this shared bus system to fetch instructions and data, perform operations, and communicate with I/O devices, forming the foundation of most modern computing systems.

5.1. Historical Development

The x86 architecture is one of the most widely used and historically significant instruction set architectures (ISAs) in computing. Developed by Intel, it has powered personal computers, servers, and workstations for decades. Known for its Complex Instruction Set Computing (CISC) design, the x86 architecture has evolved through multiple generations to support modern, high-performance computing.

Origin and Evolution

1. Intel 8086 (1978) – The Beginning

- Introduced as a 16-bit processor with a 20-bit address bus capable of addressing 1 MB of memory.
- Designed to be source-compatible with the 8-bit Intel 8080.
- Established the x86 naming convention (based on the 8086, 80186, etc.).

2. Intel 80286 (1982)

- Introduced protected mode, enabling advanced memory management and multitasking.
- Enhanced performance and allowed addressing of up to 16 MB of RAM.

3. Intel 80386 (1985)

- First 32-bit x86 processor.
- Supported virtual memory, hardware-level multitasking, and flat memory models.
- Marked a turning point in PC architecture and operating systems.

4. Intel 80486 (1989)

- Integrated floating-point unit (FPU) and improved pipelining, resulting in major speed improvements.
- Introduced on-chip L1 cache.

5. Pentium Series (1993 onwards)

- Added superscalar execution, allowing multiple instructions per clock cycle.
- Later versions introduced MMX, SSE, and hyper-threading technologies.

6. Intel x86-64 (2003) – 64-bit Era

- Also known as x86-64 or AMD64, initially developed by AMD and later adopted by Intel.
- Expanded address space to support 64-bit computing.
- Maintains backward compatibility with 16-bit and 32-bit x86 code.

Table 4: Milestones in x86 Evolution

Generation	Key Features	Year
8086	16-bit, segmented memory, 1MB addressable	1978
80286	Protected mode, 16MB memory	1982

80386	32-bit, virtual memory, multitasking	1985
80486	On-chip FPU, pipelining	1989
Pentium Series	Superscalar, MMX/SSE, cache improvements	1993–
x86-64	64-bit support, backward compatible	2003+

Widespread Adoption

- Microsoft Windows OS, as well as many enterprise and desktop applications, were originally developed for x86, securing its dominance.
- Intel and AMD have continuously refined and extended the x86 ISA.
- Even as ARM gains ground in mobile and cloud computing, x86 remains central to desktops, laptops, and servers.

The historical development of the x86 architecture illustrates how a processor family can evolve over decades while maintaining compatibility. From early PCs to modern high-performance servers, x86 has adapted to meet increasing demands, making it one of the most enduring and influential processor architectures in computing history.

5.2. Instruction Set and Design Considerations

The x86 instruction set architecture (ISA) is known for its complexity and versatility, supporting a wide range of instructions, data types, and addressing modes. It is a CISC (Complex Instruction Set Computer) architecture, designed to perform more work per instruction and provide high-level functionality at the hardware level.

Key Features of the x86 Instruction Set

The x86 instruction set architecture is known for its rich and complex design, offering several key features that have contributed to its longevity and widespread adoption.

1. **Variable-Length Instructions** allow instructions to range from 1 to 15 bytes, enabling a wide variety of operations and addressing modes, though this flexibility increases decoding complexity compared to fixed-length RISC instructions.
2. **Extensive Instruction Set** includes a broad range of operations such as arithmetic, logic, data transfer, string manipulation, bitwise operations, control flow (jumps, calls, loops), and advanced capabilities like floating-point and SIMD instructions (e.g., MMX, SSE, AVX).
3. **Rich Addressing Modes** are supported, including immediate, register-direct, register-indirect, base + offset, and scaled index, providing developers with versatile memory access options.

4. Backward Compatibility is a hallmark of x86, allowing legacy 16-bit and 32-bit software to run on modern systems. x86-64 processors support multiple operating modes—real mode (16-bit), protected mode (32-bit), and long mode (64-bit)—ensuring continuity across generations.
5. General-Purpose Registers have evolved from the classic 32-bit set (EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP) to include 64-bit versions (RAX, RBX, etc.) and an expanded set of 16 general-purpose registers in x86-64, enhancing performance and parallelism in modern applications.

Design Considerations of x86 Architecture

The design considerations of the x86 architecture reflect its complex yet powerful nature, shaped by decades of evolution.

1. CISC vs RISC Trade-offs are central to x86's design, where Complex Instruction Set Computing (CISC) enables more powerful instructions that reduce the number of lines of assembly code. However, this comes at the cost of increased hardware complexity, longer instruction decoding times, and lower energy efficiency compared to simpler RISC architectures.
2. Micro-Operations (μ ops) help mitigate this complexity in modern x86 CPUs by internally breaking down complex instructions into simpler micro-operations, enabling techniques like out-of-order execution and pipelining—similar to RISC processors.
3. Superscalar and Pipelined Execution allows modern x86 CPUs to execute multiple instructions per cycle using deep pipelines, branch prediction, and speculative execution, significantly boosting performance.
4. Power and Heat Considerations are important, as the intricate design and high transistor counts of x86 CPUs often lead to higher power consumption and heat generation compared to RISC-based alternatives.
5. Instruction Set Extensions have been added over time to enhance functionality, including MMX, SSE, AVX for multimedia and vector operations, Intel VT-x / AMD-V for virtualisation, and AES-NI for hardware-accelerated encryption, making x86 suitable for a wide range of modern computing tasks.

The x86 instruction set is one of the most robust and feature-rich architectures in use today. While complex, its backward compatibility, flexibility, and support for advanced execution features have allowed it to remain dominant in high-performance and general-purpose computing environments. Design considerations like internal micro-op translation and pipelining ensure that modern x86 processors deliver competitive performance despite their intricate ISA.

5.3. Role in Modern Computing

The x86 architecture continues to play a dominant role in modern computing, powering everything from personal desktops to enterprise-grade servers. Its combination of performance, compatibility, and scalability makes it a preferred choice for many general-purpose and high-performance applications.

1. Personal and Desktop Computing

- Most Windows and Linux PCs run on x86-based processors from Intel or AMD.
- x86 CPUs offer high single-threaded and multi-threaded performance, making them ideal for:
 - Office productivity
 - Multimedia editing
 - Gaming
 - Development environments

2. Enterprise and Server Systems

- x86-64 dominates data centers and cloud infrastructures, thanks to:
 - Broad support from enterprise software vendors
 - Advanced virtualisation features
 - Scalable multi-core architectures (e.g., Intel Xeon, AMD EPYC)
- Used in:
 - Cloud computing platforms (e.g., AWS EC2 x86 instances)
 - Virtualisation and containerisation (e.g., VMware, Docker)
 - Web hosting and large-scale data processing

3. Gaming and Graphics Workstations

- x86-based processors are the standard in gaming PCs and graphics workstations, offering:
 - High-speed processing
 - Integration with powerful GPUs
 - Support for advanced gaming and multimedia instruction sets (e.g., AVX, SSE)

4. Software Development and Compatibility

- x86's longevity ensures deep software compatibility, with countless applications, compilers, and tools optimised for it.
- Developers target x86 platforms due to:

- Mature ecosystems
- Reliable debugging tools
- Extensive documentation

5. Challenges and Competition

- **Power Efficiency:** Compared to ARM, x86 processors typically consume more power, making them less suitable for mobile and low-power environments.
- **Market Shift:** ARM is gaining traction in laptops (e.g., Apple Silicon) and cloud servers due to efficiency and customisability.
- Despite this, x86 still dominates legacy systems, powerful PCs, and data-centric computing.

6. Continued Innovation

- Modern x86 CPUs incorporate cutting-edge features like:
 - AI acceleration
 - Hardware security (Intel SGX, AMD SEV)
 - High core counts and multi-threading
 - Thermal and power management technologies

Despite increasing competition from ARM and RISC-based designs, x86 remains central to modern computing, especially in the areas of personal computing, enterprise systems, and developer tools. Its proven performance, mature ecosystem, and adaptability have allowed it to evolve with changing technology demands, sustaining its relevance for decades.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
1	Which company originally developed the x86 architecture? a) AMD b) ARM Ltd. c) Intel d) Motorola
2	What is a key feature of the x86 instruction set architecture? a) Fixed-length instructions b) Load/store-only operations c) Variable-length instructions

	d) Only 64-bit support
3	Which of the following is a major advantage of x86 architecture? a) Low power consumption b) Simple decoding logic c) Backward compatibility with older software d) Exclusive use in mobile devices
4	In which computing domain is x86 architecture most dominant? a) Embedded systems b) Mobile phones c) Desktop and server computing d) Wearable devices
Fill in the Blank	
5	The first x86 processor, Intel 8086, was introduced in the year _____.
6	x86 architecture is based on the _____ instruction set computing model.
7	The _____ mode in x86-64 architecture supports 64-bit computing.
8	x86 processors often translate complex instructions into simpler internal operations called _____.
True or False	
9	x86 architecture supports only 32-bit computing.
10	Superscalar execution allows x86 processors to execute multiple instructions per clock cycle.
11	x86 processors are not suitable for virtualisation or cloud computing.
12	The x86 instruction set includes support for SIMD and multimedia extensions like SSE and AVX.

6. COMPARATIVE ANALYSIS

6.1. Performance and Efficiency

The three major microprocessor architectures—ARM, MIPS, and x86—each offer unique advantages in terms of performance and efficiency, depending on their design goals and application domains. This section compares their strengths and trade-offs based on execution speed, processing capabilities, and energy consumption.

1. Performance Comparison

The x86 architecture is known for its high performance, especially in desktop and server-grade CPUs, supporting features like high clock speeds, hyper-threading, and out-of-order execution. However, it comes with trade-offs such as complex instruction decoding and higher power consumption, making it less ideal for energy-sensitive applications. In contrast, the ARM architecture excels in performance per watt, making it highly efficient for mobile and embedded systems. With recent advancements—such as Apple’s M-series and ARM Cortex-A chips—ARM now rivals x86 in general performance, though it has historically lagged in raw computing power compared to high-end x86 processors. Meanwhile, the MIPS architecture offers good performance for embedded and real-time systems, thanks to its clean RISC pipeline and simplicity. However, it is less optimised for high-end computing and has a limited presence in modern high-performance applications.

2. Power Efficiency

When it comes to power efficiency, ARM stands out as the most energy-conscious architecture, specifically designed for low power consumption, making it ideal for mobile and battery-operated devices. Its efficiency per watt has made it the dominant choice in smartphones, tablets, and embedded systems. MIPS also offers very good power efficiency, particularly in embedded and IoT applications where energy constraints are critical. Its clean RISC design contributes to minimal power usage in real-time systems. On the other hand, x86 architecture is traditionally less power-efficient, given its complexity and higher transistor counts. However, recent developments—such as Intel’s Core i-series U-line and AMD’s Ryzen Mobile processors—have significantly improved x86’s energy profile, making it more competitive in mobile and low-power environments.

3. Instruction Execution and Pipeline Design

Each architecture employs distinct pipeline designs to optimise instruction execution. ARM

processors use deep pipelines, efficient instruction decoding, and support out-of-order execution in higher-end cores, enabling strong performance with energy efficiency. MIPS, known for its simplicity, utilises a clean 5-stage pipeline structure, which is ideal for real-time and predictable performance, making it well-suited for embedded systems. Meanwhile, x86 architecture features a superscalar and highly pipelined design, where complex CISC instructions are internally translated into RISC-like micro-operations (μ ops). This allows modern x86 CPUs to leverage techniques like out-of-order execution, branch prediction, and speculative execution, enhancing throughput despite the complexity of the instruction set.

4. Application-Oriented Performance

In the realm of mobile devices, ARM is the preferred architecture due to its low power consumption and high efficiency, making it ideal for smartphones and tablets. For embedded systems and IoT devices, both MIPS and ARM are commonly used, thanks to their simple, predictable pipelines and energy-efficient designs, which are crucial for real-time and low-power applications. In desktops and laptops, x86 remains dominant because of its powerful CPUs and a mature software and operating system ecosystem, which supports a wide range of applications and workloads. In servers and data centers, x86 has traditionally been the go-to architecture for its robust performance and scalability, but ARM is rapidly gaining traction due to its energy efficiency and growing support in cloud-native environments.

Key note :

- x86 excels in raw performance, especially for compute-heavy tasks, but often at the cost of higher power usage.
- ARM offers a strong balance of performance and efficiency, making it dominant in mobile and increasingly competitive in servers and desktops.
- MIPS provides a simpler alternative, excelling in embedded environments with predictable performance and low power draw.

Each architecture's suitability depends on the specific needs of the application, whether it be power-sensitive mobile devices or high-throughput computing platforms.

6.2. Power Consumption and Scalability

When choosing a microprocessor architecture for a particular application, power consumption and scalability are critical factors—especially in domains like mobile computing, embedded systems, and

data centers. This section compares ARM, MIPS, and x86 based on how efficiently they use power and how well they scale across performance tiers and application types.

1. Power Consumption

The power profile of each architecture reflects its design priorities and typical use cases. ARM is engineered with power efficiency as a core principle, making it the dominant choice for smartphones, tablets, and IoT devices where ultra-low power usage is essential. Its architecture enables long battery life and minimal heat generation, ideal for mobile environments. MIPS also offers good power efficiency, particularly in simple embedded and real-time systems where minimal complexity and predictable performance are key. Its streamlined RISC design helps conserve energy in constrained environments. In contrast, x86 has historically exhibited higher power draw due to its complex instruction set and high-performance capabilities. However, recent innovations—such as Intel’s hybrid architecture and AMD’s Zen cores—have significantly improved x86’s energy efficiency, especially in laptops and low-power servers, making it more competitive in power-sensitive applications.

Note:

- ARM leads in ultra-low power use, making it ideal for battery-powered devices.
- MIPS is a solid choice for power-sensitive embedded applications.
- x86 still consumes more power in high-performance scenarios but has improved significantly in mobile and thin client designs.

2. Scalability

Each architecture serves distinct roles across the computing landscape. ARM offers exceptional scalability, ranging from microcontrollers (e.g., Cortex-M) to server-grade processors (e.g., ARM Neoverse), making it highly versatile. Its high customisability, enabled by licensable cores, allows vendors to tailor System-on-Chip (SoC) designs for specific applications. ARM also supports efficient multi-core configurations, including big.LITTLE architectures, and enjoys rapid market growth in mobile, laptops, and cloud computing. MIPS, while also licensable and moderately customisable, is primarily used in embedded systems and real-time applications, with limited presence in high-end markets. It typically supports single or dual-core setups, and maintains a steady niche in embedded networking. x86, on the other hand, scales from consumer laptops to high-performance servers, with limited customisability due to proprietary control by Intel and AMD. It supports high core counts—

up to 16+ cores in consumer CPUs and 64+ in server-grade chips—and has a mature and widespread market presence in desktops, enterprise systems, and gaming platforms.

Key Observations

- ARM shows the highest flexibility and power scalability, adapting well to both low-power embedded systems and high-performance computing.
- MIPS is more limited in scalability, largely focused on fixed-function or narrowly scoped embedded environments.
- x86, though more power-hungry, offers impressive scalability from entry-level to enterprise-class multi-core systems.

In terms of power consumption, ARM is the clear leader, followed by MIPS, with x86 trading efficiency for raw performance. When it comes to scalability, x86 and ARM both shine—x86 in performance-centric systems and ARM in power-to-performance balanced platforms. MIPS remains a niche but efficient option in predictable, low-power applications.

6.3. Market Adoption and Ecosystem Support

The success of a microprocessor architecture depends not only on its technical strengths but also on its industry adoption and ecosystem support—including tools, operating systems, developer communities, and market presence. This section compares ARM, MIPS, and x86 in terms of their real-world adoption and ecosystem maturity.

1. ARM Architecture

Market Adoption

- Dominant in mobile and embedded markets.
- Used in 95%+ of smartphones, tablets, wearables, and IoT devices.
- Gaining traction in laptops (e.g., Apple M-series) and cloud servers (e.g., AWS Graviton).

Ecosystem Support

- Supported by major operating systems: Android, iOS, Linux, and now macOS.
- Broad toolchain support: GCC, LLVM, Keil, ARM DS, etc.
- Strong community, open resources, and hardware partners (Qualcomm, NVIDIA, Broadcom, etc.).
- Licensable core model fosters innovation and broad adoption across industries.

2. MIPS Architecture

Market Adoption

- Widely used in embedded systems, especially in networking equipment, digital appliances, and low-cost SoCs.
- Once popular in set-top boxes, routers, and educational platforms, but declining in favour of ARM and RISC-V in recent years.

Ecosystem Support

- Tool support: GCC, MIPS toolchain, MARS, SPIM.
- Common in academia for teaching computer architecture.
- Limited recent updates and reduced commercial presence in high-growth sectors.

3. x86 Architecture

Market Adoption

- Dominant in desktops, laptops, workstations, and data centers.
- Powers most Windows and Linux PCs, gaming systems, and high-performance servers (e.g., Intel Xeon, AMD EPYC).

Ecosystem Support

- Extremely mature software ecosystem: Windows, Linux, BSD, all natively supported.
- Extensive development tools: Intel VTune, AMD uProf, Visual Studio, etc.
- Strong backward compatibility—can run software developed decades ago.
- Major players: Intel, AMD, with robust support for virtualisation, security, and AI workloads.

Table 5: Market and Ecosystem Comparison

Architecture	Market Focus	Ecosystem Maturity	Notable Use Cases
ARM	Mobile, embedded, cloud servers	Very strong	Smartphones, Apple M-series Macs, AWS Graviton
MIPS	Embedded, academic	Moderate (declining)	Routers, network devices, educational tools
x86	Desktops, laptops, servers	Extremely strong	PCs, gaming, cloud infrastructure

Note:

- ARM has the strongest growth trajectory and broadest adoption, backed by an expansive ecosystem.
- x86 remains a powerhouse in traditional computing with unmatched software and legacy support.
- MIPS, though less prevalent in commercial products today, continues to have value in academic and select embedded use cases.

Together, these ecosystems shape the diversity and capability of today's processor landscape.

SELF-ASSESSMENT QUESTIONS - 5

Multiple Choice Questions	
1	Which architecture is known for offering the best performance-per-watt ratio in mobile devices? a) x86 b) ARM c) MIPS d) RISC-V
2	Which architecture is most commonly used in high-performance desktop and server systems? a) ARM b) MIPS c) x86 d) SPARC
3	What is a key reason for MIPS being preferred in embedded systems? a) High clock speed b) Complex instruction set c) Predictable performance and low power usage d) Advanced virtualisation support
4	Which architecture has the broadest ecosystem support in mobile and cloud platforms? a) x86 b) MIPS c) ARM

	d) PowerPC
Fill in the Blank	
5	ARM architecture is widely adopted in smartphones due to its _____ power consumption and scalability.
6	x86 processors are known for their strong support in _____ computing environments.
7	MIPS architecture is commonly used in _____ and academic settings.
8	ARM and x86 architectures both support _____ computing, allowing multiple OS instances to run simultaneously.
True or False	
9	ARM architecture is less scalable than x86 in high-performance computing.
10	MIPS architecture is declining in commercial use but remains relevant in education.
11	x86 processors are not suitable for cloud computing due to lack of ecosystem support.
12	ARM architecture is gaining traction in laptops and cloud servers.



7. SUMMARY

This unit introduces us to the world of modern microprocessors, focusing on three major architectures: ARM, MIPS, and x86. It begins by tracing the evolution of microprocessors from simple 4-bit and 8-bit designs to today's powerful 64-bit multi-core systems. Over time, processors have become faster, more efficient, and more integrated, supporting advanced features like virtualisation, AI acceleration, and secure execution environments.

We then explore the defining characteristics of modern microprocessors, such as multi-core architecture, high clock speeds, pipelining, superscalar execution, and low power consumption. These features enable modern CPUs to handle complex tasks across a wide range of devices—from smartphones and embedded systems to desktops and cloud servers.

The unit dives deeper into the ARM architecture, highlighting its RISC-based design, energy efficiency, and widespread use in mobile and embedded devices. ARM's instruction set is simple yet powerful, and its flexible licensing model has led to broad industry adoption. Next, we examine MIPS architecture, known for its clean design and educational value. MIPS is widely used in embedded systems and academic settings due to its straightforward instruction set and efficient pipelining.

The x86 architecture, developed by Intel, is presented as a cornerstone of desktop and enterprise computing. With its complex instruction set and strong backward compatibility, x86 has evolved to support high-performance applications, making it dominant in PCs, servers, and gaming systems.

Finally, the unit compares these architectures in terms of performance, power efficiency, scalability, and market adoption. ARM stands out for its energy efficiency and flexibility, x86 for its raw performance and mature ecosystem, and MIPS for its simplicity and reliability in embedded environments.

In summary, this unit provides a comprehensive understanding of how modern microprocessors are designed, how they differ from traditional ones, and how they are applied across various computing platforms. It equips learners with the knowledge to appreciate the architectural choices behind today's most influential processors.

8. GLOSSARY

Microprocessor	- A central processing unit (CPU) on a single integrated circuit chip that performs computation and control tasks in a computer system.
RISC (Reduced Instruction Set Computer)	- A processor architecture that uses a small, highly optimised set of instructions for fast execution and simplified hardware design.
CISC (Complex Instruction Set Computer)	- A processor architecture with a large set of instructions, allowing complex operations to be performed with fewer lines of code.
ARM Architecture	- A family of RISC-based microprocessor architectures known for low power consumption and high efficiency, widely used in mobile and embedded systems.
MIPS Architecture	- A RISC-based microprocessor architecture known for its simplicity and use in embedded systems and academic environments.
x86 Architecture	- A CISC-based microprocessor architecture developed by Intel, dominant in desktop, laptop, and server computing.
Instruction Set Architecture (ISA)	- The set of instructions that a processor can execute, defining how software communicates with hardware.
System-on-Chip (SoC)	- An integrated circuit that combines a CPU with other components like GPU, memory controller, and I/O interfaces on a single chip.
Pipelining	- A technique where multiple instruction stages are overlapped to improve CPU throughput.
Superscalar Execution	- A processor design that allows multiple instructions to be executed simultaneously in one clock cycle.
Multicore Processor	- A CPU with two or more independent cores that can execute instructions simultaneously, improving performance.
Virtualisation	- Technology that allows multiple operating systems or applications to run on a single physical machine using virtual machines.

Thumb Instruction Set	- A 16-bit compressed instruction set used in ARM processors to reduce code size and improve efficiency.
NEON SIMD	- An ARM extension for Single Instruction, Multiple Data operations, used in multimedia and signal processing.
TrustZone	- A security extension in ARM architecture that enables secure execution environments for sensitive operations.
Branch Delay Slot	- A technique used in MIPS architecture where the instruction following a branch is executed regardless of the branch outcome.

9. TERMINAL QUESTIONS

1. What are the key differences between RISC and CISC architectures?
2. Explain the evolution of microprocessors from first-generation to modern multi-core systems.
3. List three characteristics that define modern microprocessors.
4. What is the role of pipelining in improving processor performance?
5. Describe the design philosophy of ARM architecture.
6. What are the advantages of using ARM processors in mobile and embedded systems?
7. Explain the concept of load/store architecture in ARM and MIPS.
8. What are the three instruction formats used in MIPS architecture?
9. How does delayed branching improve pipeline efficiency in MIPS processors?
10. Discuss the historical development of x86 architecture from Intel 8086 to x86-64.
11. What are the key features of the x86 instruction set?
12. Compare the power efficiency of ARM, MIPS, and x86 architectures.
13. In which application areas is ARM architecture most widely adopted?
14. Why is x86 architecture still dominant in desktop and server computing?
15. What factors influence the market adoption and ecosystem support of a microprocessor architecture?

10. ANSWERS

Self-Assessment Questions

Self-Assessment Questions – 1

1. c) Fourth generation
2. c) Multi-core architecture
3. c) CPU, GPU, memory controller, and I/O interfaces
4. c) Virtualisation and AI acceleration
5. 4 or 8
6. Superscalar
7. Power gating
8. Desktops
9. True
10. False
11. False
12. True

Self-Assessment Questions - 2

1. c) Reduced instruction set with fixed-length instructions
2. c) Deep pipelining and low power usage
3. d) To improve code density in constrained environments
4. b) Automotive
5. Load/store
6. NEON
7. System-on-Chip (SoC)
8. TrustZone
9. True
10. False
11. True
12. False

Self-Assessment Questions - 3

1. b) Microprocessor without Interlocked Pipeline Stages

2. c) R-type
3. b) To execute the instruction after a branch for pipeline efficiency
4. c) Embedded systems and education
5. RISC
6. 32
7. R0
8. Networking
9. False
10. False
11. True
12. True

Self-Assessment Questions - 4

1. c) Intel
2. c) Variable-length instructions
3. c) Backward compatibility with older software
4. c) Desktop and server computing
5. 1978
6. Complex (CISC)
7. Long
8. Micro-operations (μ ops)
9. False
10. True
11. False
12. True

Self-Assessment Questions - 5

1. b) ARM
2. c) x86
3. c) Predictable performance and low power usage
4. c) ARM
5. low
6. desktop and server
7. embedded systems

8. virtualisation
9. False
10. True
11. False
12. True

Terminal Questions Answers

Answer 1: RISC architectures use a small, optimised set of instructions for fast execution and simplified hardware, while CISC architectures like x86 use complex instructions that perform multiple operations, offering more functionality per instruction but requiring more decoding.

Refer section 2.3 - Comparison with Traditional Microprocessors

Answer 2: Microprocessors evolved from basic 4-bit and 8-bit designs to powerful 64-bit multi-core systems. Each generation introduced improvements in instruction sets, performance, and integration, leading to modern processors capable of handling complex tasks like AI and virtualisation.

Refer section 2.1 - Evolution of Microprocessors

Answer 3: Modern microprocessors are defined by multi-core architecture, high clock speeds, energy efficiency, advanced instruction sets, and integrated components like GPUs and memory controllers.

Refer section 2.2 - Characteristics of Modern Microprocessors

Answer 4: Pipelining improves performance by allowing multiple instruction stages to execute simultaneously, increasing throughput and reducing execution time per instruction.

Refer section 2.2 - Characteristics of Modern Microprocessors

Answer 5: ARM architecture follows the RISC philosophy, emphasising simplicity, low power consumption, and efficient pipelining. It uses a load/store model and fixed-length instructions for streamlined execution.

Refer section 3.1- Features and Design Philosophy

Answer 6: ARM processors are ideal for mobile and embedded systems due to their low power usage, scalability, and high performance-per-watt ratio, making them dominant in smartphones, tablets, and IoT devices.

Refer section 3.1 - Features and Design Philosophy

Answer 7: In load/store architecture, only specific instructions access memory, while all other operations occur between registers. This model simplifies execution and improves speed.

Refer section 3.2 - Instruction Set and Applications

Answer 8: MIPS uses three instruction formats: R-type for register operations, I-type for immediate values, and J-type for jump instructions. Each format is 32 bits wide and supports efficient pipelining.

Refer section 4.2 - Instruction Set Architecture (ISA) of MIPS

Answer 9: Delayed branching in MIPS ensures that the instruction following a branch is executed regardless of the branch outcome, improving pipeline efficiency and reducing stalls.

Refer section 4.1 - Overview and Features

Answer 10: x86 architecture began with the Intel 8086 and evolved through generations like 80286, 80386, and Pentium, eventually reaching x86-64, which supports 64-bit computing and backward compatibility.

Refer section 5.1 - Historical Development

Answer 11: The x86 instruction set is complex and variable-length, supporting a wide range of operations including arithmetic, logic, control flow, and SIMD. It also offers rich addressing modes and legacy support.

Refer section 5.2 - Instruction Set and Design Considerations

Answer 12: ARM is the most power-efficient, ideal for mobile devices. MIPS is also efficient in embedded systems. x86 traditionally consumes more power but has improved with newer mobile-focused designs.

Refer section 6.2 - Power Consumption and Scalability

Answer 13: ARM is widely adopted in smartphones, tablets, IoT devices, automotive systems, and increasingly in cloud computing due to its efficiency and scalability.

Refer section 3.3 - Use Cases and Industry Adoption

Answer 14: x86 remains dominant in desktops, laptops, and servers because of its high performance, mature software ecosystem, and strong backward compatibility.

Refer section 5.3 - Role in Modern Computing

Answer 15: Market adoption depends on factors like performance, power efficiency, scalability, and ecosystem support including tools, operating systems, and developer communities.

Refer section 6.3 - Market Adoption and Ecosystem Support



11. REFERENCES

BOOKS

1. Computer Architecture: A Quantitative Approach, John L. Hennessy, David A. Patterson, 6th Edition, 2017
2. Computer Organisation and Architecture: Designing for Performance, William Stallings, 11th Edition, 2018
3. Computer System Architecture, M. Morris Mano, 3rd Edition, 2006
4. Computer Organisation and Design: The Hardware/Software Interface, David A. Patterson, John L. Hennessy, 5th Edition, 2013
5. Digital Design and Computer Architecture, Sarah Harris, David Harris, 3rd Edition, 2021

E-REFERENCES

1. Logical and Shift Micro-operations Computer Organisation and Architecture, <https://care4you.in/logical-and-shift-micro-operations/>
2. Logic and Shift Micro Operations, <https://www.scribd.com/document/394026693/U-I-Logic-and-Shift-micro-operations>